

GCP — in — **Action**

A practical guide to building and deploying secure, scalable applications using Google Cloud Platform



Raymond Blum



GCP — **in** — **Action**

A practical guide to building and deploying secure,
scalable applications using Google Cloud Platform



Raymond Blum



GCP en Acción

Una guía práctica para crear e
implementar ,
aplicaciones escalables y seguras utilizando
Plataforma de Google Cloud

Raymond Blum



www.bpbonline.com

Primera edición 2025

Derechos de autor © BPB Publications, India

ISBN: 978-93-65897-500

Todos los derechos reservados. Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en ninguna forma o por ningún medio ni almacenada en una base de datos o sistema de recuperación, sin el permiso previo por escrito del editor, con excepción de los listados de programas que pueden ingresarse, almacenarse y ejecutarse en un sistema informático, pero no pueden reproducirse por medio de publicación, fotocopia, grabación o por cualquier medio electrónico o mecánico.

LÍMITES DE RESPONSABILIDAD Y EXENCIÓN DE GARANTÍA

La información contenida en este libro es veraz y correcta según el leal saber y entender del autor y la editorial. El autor ha hecho todo lo posible por garantizar la exactitud de estas publicaciones, pero la editorial no se responsabiliza de ninguna pérdida o daño derivado de la información contenida en este libro.

Todas las marcas comerciales a las que se hace referencia en el libro se reconocen como propiedad de sus respectivos dueños, pero BPB Publications no puede garantizar la exactitud de esta información.

**To View Complete
BPB Publications Catalogue
Scan the QR Code:**



www.bpbonline.com

Dedicado a

[Stephen McHenry](#), quien siempre me trató como a un igual.

y no esperaba menos de mí, .

Eva, que me animó durante muchos largos días.

y noches de investigación y escritura.

[Joëlle Skaf](#), la mejor y más auténtica compañera de equipo de todos los tiempos .

Acerca del autor

Raymond Blum es actualmente ingeniero sénior en Google DeepMind, donde trabaja en el cumplimiento de las políticas de Gemini. Anteriormente, fue gerente de ingeniería y líder técnico en Google, en diversos equipos de infraestructura técnica y confiabilidad de sitios. Lideró y promovió los equipos de recuperación e integridad de datos en Google durante varios años y ha publicado artículos sobre la permanencia y confiabilidad de los datos digitales para la Asociación de Maquinaria de Computación (ACM) y con Google.

Previamente, creó y dirigió una empresa de servicios de internet durante varios años, tras una trayectoria como desarrollador de software en empresas de medios y financieras. Durante este tiempo, impartió talleres de informática y habilidades profesionales a estudiantes de grado en la Universidad de la Ciudad de Nueva York.

Raymond es licenciado en informática por la Universidad de la Ciudad de Nueva York. Dedicó su tiempo y su asesoramiento a organizaciones que promueven y brindan igualdad de acceso a carreras profesionales para todos.

Acerca de los revisores

Artiom Kuciuk es un experimentado arquitecto de sistemas backend y líder técnico con amplia experiencia profesional en infraestructuras de pago escalables, desarrollo backend, integración de sistemas y arquitecturas basadas en microservicios. Sus áreas de especialización incluyen el diseño de sistemas de alto rendimiento, el procesamiento seguro de transacciones, las tecnologías nativas de la nube y la optimización backend.

Artiom domina las tecnologías de AWS, Azure y GCP, y ha trabajado extensamente con plataformas multicloud para ofrecer aplicaciones confiables y con tolerancia a fallos. Cuenta con una trayectoria comprobada en la creación de ecosistemas de pago compatibles con decenas de proveedores globales, cumpliendo con estándares del sector como PCI DSS y aprovechando protocolos de cifrado como TLS.

Se especializa en escalabilidad de backend y arquitectura nativa de la nube, y le apasiona construir infraestructuras resilientes que impulsen el crecimiento y la seguridad empresarial. Colabora frecuentemente en hackatones y comunidades de ingeniería, donde comparte sus conocimientos sobre diseño de sistemas y optimización del rendimiento.

Actualmente trabaja como líder técnico en Greentube GmbH, un proveedor líder de soluciones de juegos en línea, y es parte del equipo central que impulsa la innovación y la arquitectura de backend.

František Krajňák es un apasionado especialista en DevSecOps con más de 10 años de experiencia profesional en seguridad en la nube, automatización de infraestructura y soluciones de Google Cloud Platform. Posee múltiples certificaciones de la industria, incluida Google Cloud.

Ingeniero de seguridad profesional, arquitecto profesional e ingeniero de datos.

Se especializa en diseño de sistemas y prácticas de DevSecOps, con especial experiencia en optimización de GKE, gestión de infraestructura de Terraform y gestión de vulnerabilidades. Disfruta trabajando en migraciones complejas a la nube y desarrollando soluciones innovadoras que conectan a los equipos de desarrollo y operaciones.

Actualmente trabaja como ingeniero senior en la nube, especialista en GCP, en Deutsche Börse en Praga, donde se centra en soluciones impulsadas por IA, incluidos chatbots basados en RAG y LLM, junto con implementaciones en la nube a escala empresarial y marcos de cumplimiento.

Reconocimiento

Me gustaría expresar mi sincero agradecimiento a todos aquellos que contribuyeron a la finalización de este libro.

Ante todo, quiero expresar mi más sincero agradecimiento a mi familia y amigos por su inquebrantable apoyo y aliento durante este camino. Su amor y ánimo me han motivado a seguir adelante.

Quisiera expresar mi especial agradecimiento a Alberto Rojas por su valiosa contribución a este proyecto. Sus ideas y comentarios han sido fundamentales para dar forma al contenido y mejorar la calidad de este libro. Gracias por su atención y su invaluable apoyo.

Estoy inmensamente agradecido a BPB Publications por su guía y experiencia para hacer realidad este libro. Su apoyo y asistencia fueron invaluable para sortear las complejidades del proceso editorial.

También quisiera agradecer a los revisores técnicos y editores que brindaron valiosos comentarios y contribuyeron al perfeccionamiento de este manuscrito. Sus ideas y sugerencias han mejorado significativamente la calidad del libro.

Finalmente, quiero expresar mi gratitud a los lectores que han mostrado interés en nuestro libro. Agradezco profundamente su apoyo y aliento.

Gracias a todos los que han contribuido a hacer de este libro una realidad.

Prefacio

Comprender la función y el valor de la infraestructura en la nube es crucial para la entrega de sistemas de aplicaciones distribuidas eficientes, seguros y sostenibles. Este libro aborda las necesidades emergentes de los sistemas actuales, altamente escalables y disponibles, y los productos específicos que ofrece Google Cloud Platform (GCP) para satisfacer dichas necesidades. También se exploran los productos y tecnologías utilizados en funciones complementarias a los servicios en la nube. El libro ofrece a los ingenieros de software con experiencia una guía completa de prácticas de desarrollo en la nube y patrones de arquitectura que utilizan los productos de GCP.

Compuesto por catorce capítulos centrados en cuatro secciones, este libro cubre una amplia gama de temas esenciales para comprender la estructura de un sistema distribuido y el diseño de una aplicación exitosa.

La primera sección presenta al lector una clasificación de los productos en la nube según su responsabilidad (el aspecto del sistema informático que representan). Se detallan la potencia de procesamiento, la persistencia de datos y la autenticación de usuarios, y se analizan las diversas ventajas y desventajas de varias opciones de GCP. La segunda sección revisa la arquitectura completa de un sistema de aplicaciones: servicios en la nube, clientes frontend y pipelines backend.

Se presentan patrones de diseño y principios de ingeniería que han demostrado su eficacia a través de su uso en el desarrollo de productos exitosos para cada parte del sistema de aplicación general.

La tercera sección del libro presenta un estudio de caso: el diseño y desarrollo de un sistema de votación global distribuido. El lector es...

Se mostraron métodos para pronosticar la demanda y se presentaron prácticas de diseño para brindar un servicio confiable y de alta disponibilidad. Las prácticas de monitoreo e implementación son tan importantes para la experiencia del usuario como un código base bien desarrollado, por lo que se exploran en detalle las consideraciones y prácticas de confiabilidad del sitio que utilizan productos de GCP.

La cuarta y última sección del libro aplica tanto las lecciones aprendidas como los resultados del caso práctico a una amplia gama de aplicaciones empresariales. Una visión general del sistema de votación puede utilizarse para diseñar sistemas en otros ámbitos. Los componentes de la aplicación de votación sirven como modelos o pueden utilizarse directamente en sistemas independientes con requisitos funcionales similares. Se recapitulan los procesos de diseño que permiten dicha reutilización y se refuerza su valor mediante varios ejemplos.

Sección **1** Servicios en la nube por responsabilidad

Capítulo 1 Responsabilidades y utilidad de una plataforma en la nube: Este capítulo introductorio revisará la evolución del desarrollo de aplicaciones que ha impulsado la necesidad de plataformas de servicios en la nube. Se describirán los requisitos básicos de una aplicación distribuida y, a continuación, exploraremos cómo un servicio en la nube aporta valor a una aplicación. Se describirán las consideraciones arquitectónicas de una aplicación distribuida y los tipos de servicios que ofrece una plataforma en la nube para abordarlas. A continuación, el capítulo se centra en los productos de GCP para cada uno de estos tipos de servicio. Concluiremos con una revisión de las características que no se adaptan bien a los servicios de GCP y cuáles son sus plataformas óptimas.

Capítulo 2 División de las características de una aplicación en la nube: En este capítulo, se muestra al lector una metodología para descomponer las características de una aplicación con un propósito específico. El conjunto de características resultante se evalúa para identificar los aspectos de la aplicación relevantes para la selección de una plataforma de infraestructura en la nube.

El capítulo analiza y ejemplifica un proceso para diseñar una arquitectura de sistema que se centra en el cumplimiento óptimo de

Requisitos y cómo estos requisitos impulsan la selección de servicios de soporte. Se ofrecen al lector explicaciones claras de términos que a menudo se presentan de forma demasiado vaga o exagerada, desmitificando las características deseables de un marco de sistemas distribuidos como GCP.

Capítulo 3 **Determinación de la infraestructura sobre la que se construirán las funciones:**

Este capítulo explora algunos requisitos técnicos comunes, y a menudo malinterpretados, de las funciones de la aplicación, como la latencia y los límites de transacción. Al analizar estos requisitos en sus elementos principales, el lector podrá identificar los efectos que realmente requiere una aplicación.

Tener requisitos con un nivel de granularidad más preciso aumenta la flexibilidad que ofrece al arquitecto para cumplirlos. Este capítulo analiza los servicios en la nube que ofrecen soluciones a estos requisitos fundamentales y las maneras de organizarlos para que cooperen con los objetivos generales.

Capítulo 4 **Elección entre opciones de GCP:** Este capítulo desambigua productos de GCP populares y frecuentemente confundidos en cuatro dominios principales: potencia de cómputo, almacenamiento de datos, identidad y autenticación, y procesamiento asíncrono. El lector aprende a determinar cuál de los productos aparentemente equivalentes en cada uno de estos dominios es la opción más adecuada para su aplicación, considerando tanto las fortalezas del producto como sus posibles limitaciones.

Sección 2 **Arquitectura de aplicaciones respaldadas por la nube**

Capítulo 5 **Opciones de tecnología del lado del cliente:** En este capítulo, resumiremos las tecnologías del lado del cliente, tanto populares como emergentes, que ofrecen una perspectiva del sistema desde el punto de vista del usuario final. Se mostrarán al lector los costos de diferentes enfoques para el desarrollo inicial y el mantenimiento de una aplicación en múltiples plataformas. Posteriormente, el capítulo guiará al lector en la consideración del desarrollo de una interfaz de usuario como una aplicación cliente nativa de la plataforma, una aplicación cliente multiplataforma, una aplicación web basada en navegador o una combinación de estas.

Capítulo 6 **Servicios en la nube en la frontera del backend:** este capítulo presentará las mejores prácticas que resuelven la ambigüedad y la confusión que

Resultan de la superposición de características entre productos similares de la suite de productos GCP. El lector aprenderá a diseñar componentes que encapsulen los detalles de su implementación y dependencias.

El capítulo mostrará cómo diseñar sistemas que puedan adaptarse a los cambios en el panorama de la infraestructura.

Capítulo 7. Frontends en la frontera del cliente: En este capítulo, el lector se familiarizará con el desglose de la plataforma de alto nivel de una aplicación distribuida. Aprenderá los criterios para aislar la funcionalidad y asignarla a un nivel arquitectónico específico. Se expondrán los riesgos para el rendimiento y la integridad de la aplicación dentro de cada nivel y en sus fronteras, y se explorarán y explicarán las técnicas para prevenirlos o mitigarlos.

Sección 8. De extremo a extremo: un sistema de votación global

Capítulo 8. Lograr que un sistema de votación sea disponible y confiable: Este capítulo explorará las fuerzas del mundo real que desafían la confiabilidad y disponibilidad de un sistema distribuido, específicamente en el contexto de nuestra aplicación de votación. Identificaremos aspectos comunes de estos desafíos, como la distribución geográfica y temporal de la demanda.

El capítulo examinará análisis detallados de la demanda y luego identificará formas de mitigar el riesgo a la integridad del sistema expuesto por estos análisis.

Capítulo 9. Identificación de los puntos de estrés de nuestro sistema de votación. En este capítulo, se guiará al lector sobre las formas en que un sistema puede fallar y las maneras de mitigarlas. Este capítulo identificará las dimensiones de una interrupción del servicio, describiendo las técnicas e indicadores utilizados para categorizar un incidente específico. El lector finalizará este capítulo con una metodología para categorizar los modos de fallo y medidas de mitigación, aplicable a cualquier sistema distribuido de alta disponibilidad.

Capítulo 10. Protección de todos los votos de los usuarios: Este capítulo revisará las funciones de Google Cloud Platform que abordan los cambios en el tráfico de usuarios. Tanto las funciones específicas del producto, como el escalado automático y

Se discutirán patrones de diseño como medios para dar forma dinámica a la capacidad del sistema para que coincida con la demanda.

Capítulo 11 Garantizar la permanencia de los datos: En este capítulo, el lector explorará los diferentes tipos de datos almacenados en una aplicación empresarial y los distintos tipos de acceso. El capítulo categorizará el comportamiento de las aplicaciones según patrones de acceso típicos.

Luego, el capítulo guiará al lector a través de la evaluación de la necesidad de permanencia de los datos durante la vida útil de la aplicación y del usuario.

Se identificarán y explicarán las prácticas y mecanismos de implementación para garantizar la permanencia y disponibilidad necesaria de los datos.

Capítulo 12 Desarrollo de Frontends de Votación: Este capítulo examinará la composición de una función visible para el usuario, explorando los detalles específicos de implementación de su funcionalidad. Se guiará al lector en la evaluación de un nivel óptimo para cada una de estas funciones. El resultado será un diseño detallado de las funciones visibles para el usuario que identifica claramente el alcance de las responsabilidades y dependencias de la aplicación cliente. El diseño y el código base se refactorizarán para facilitar el desarrollo independiente de múltiples productos de software relacionados.

Sección 4 Resumen: Aplicación del estudio de caso

Capítulo 13 Reemplazo de componentes de forma aislada: En este capítulo, se describirán los principios de ingeniería para maximizar la reutilización del software. El capítulo mostrará cómo interactuar con los servicios de GCP de forma que se maximice la independencia de los diferentes componentes.

El lector examinará un caso práctico sobre la sustitución de un componente de infraestructura en un sistema en funcionamiento. Este ejercicio de migración destacará el valor de las mejores prácticas descritas y seguidas. Para evitar posibles daños al negocio, también elaboraremos planes para reducir y mitigar los riesgos restantes de la migración.

Capítulo 14: Sistema de votación como implementación de referencia. El capítulo final examinará dos extensiones de nuestro sistema de votación: la adaptación del sistema a otro ámbito empresarial y la aplicación de componentes aislados a un producto independiente con requisitos de infraestructura similares. Se presentará al lector una perspectiva de cómo se conectan problemas generales menores para resolver un problema empresarial específico. Al final de este capítulo y al final del libro, se mostrará al lector la posibilidad de reutilizar la aplicación de votación, así como la utilidad de los principios y metodologías empleados en proyección.

Paquete de códigos e imágenes en color

Por favor siga el enlace para descargar el
Paquete de códigos e imágenes en color del libro:

<https://rebrand.ly/xr0t33q>

El paquete de código del libro también está alojado en GitHub en
<https://github.com/bpbpublications/GCP-in-Action>. En caso de que haya una
actualización del código, se actualizará en el repositorio de GitHub existente.

Tenemos paquetes de códigos de nuestro amplio catálogo de libros y videos
disponibles en <https://github.com/bpbpublications>. ¡Échales un vistazo!

Errata

En BPB Publications, nos enorgullecemos enormemente de nuestro trabajo y
seguimos las mejores prácticas para garantizar la precisión de nuestro contenido
y ofrecer una experiencia de lectura placentera a nuestros suscriptores. Nuestros
lectores son nuestros espejos, y utilizamos sus aportaciones para reflexionar
y mejorar los errores humanos, si los hubiera, que pudieran haberse producido
durante el proceso de publicación. Para mantener la calidad y ayudarnos a
contactar a los lectores que puedan tener dificultades debido a errores
imprevistos, escríbanos a: errata@bpbonline.com

La familia de Publicaciones

BPB agradece enormemente su apoyo, sugerencias y comentarios.

¿Sabías que BPB ofrece versiones en formato eBook de todos los libros publicados, con
archivos PDF y ePub disponibles? Puedes actualizar a la versión en formato eBook en

www.bpbonline.com Como cliente de la versión impresa, tiene derecho a un descuento en la copia del eBook. Contáctenos en:

negocios@bpbonline.com Para más detalles.

En www.bpbonline.com,. También puede leer una colección de artículos técnicos gratuitos, suscribirse a una variedad de boletines gratuitos y recibir descuentos y ofertas exclusivas en libros y libros electrónicos de BPB.

Piratería.

Si encuentra copias ilegales de nuestras obras en internet, en cualquier formato, le agradeceríamos que nos proporcionara la dirección o el nombre del sitio web. Por favor, contáctenos en business@bpbonline.com. con un enlace al material.

Si estás interesado en convertirte en autor

Si hay un tema en el que usted es experto y está interesado en escribir o contribuir a un libro, visite www.bpbonline.com. Hemos trabajado con miles de desarrolladores y profesionales de la tecnología, como tú, para ayudarles a compartir sus conocimientos con la comunidad tecnológica global. Puedes enviar una solicitud general, postularte para un tema de actualidad específico para el que buscamos autores o enviar tu propia idea.

Reseñas

Por favor, deja una reseña. Una vez que hayas leído y usado este libro, ¿por qué no dejas una reseña en el sitio web donde lo compraste? Los lectores potenciales podrán ver y usar tu opinión imparcial para tomar decisiones de compra. En BPB entendemos tu opinión sobre nuestros productos, y nuestros autores pueden ver tus comentarios sobre su libro. ¡Gracias!

Para obtener más información sobre BPB, visite www.bpbonline.com.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com>



Tabla de contenido

Sección 1: Servicios en la nube por responsabilidad

1. Responsabilidades y utilidad de una plataforma en la nube

Introducción

Estructura

Objetivos

Consideraciones generales sobre infraestructura

Identidad y autenticación

Escalabilidad

Procesamiento asincrónico

Permanencia de los datos

Entrega de respuesta

Análisis de consideraciones de infraestructura

Interdependencia de las responsabilidades de la plataforma

Asignación de roles por plataforma

Descripción general del producto Google Cloud Platform

Categorías de nivel superior de los servicios de Google Cloud

potencia de cálculo

Bases de datos

Almacenamiento

Redes

Análisis de datos

Seguridad e identidad

Herramientas para desarrolladores

Inteligencia artificial y aprendizaje automático

Componentes fuera de los servicios en la nube

Conclusión

2. División de las características de una aplicación respaldada en la nube

Introducción

Estructura

Objetivos

Enumeración de responsabilidades y dependencias

Diferentes puntos de vista válidos

Agrupación por localidad topológica

Agrupación por afinidad de características

Determinación de la afinidad y proximidad de las características

Agrupación por consideraciones de rendimiento de funciones

Comprender los requisitos de las funciones

Definición de la composición de las características de la aplicación

Cualidades pertinentes de un servicio o componente de infraestructura

Descripción de las opciones de infraestructura

Conclusión

3. Determinar la infraestructura sobre la que se construirán las características

Introducción

Estructura

Objetivos:

¡Solicitar canalización para escalar a $O(N)$!

Urgencia versus importancia

Consistencia eventual en sistemas distribuidos Un estudio de caso sobre consistencia eventual

Garantizar una capacidad de respuesta visible para el usuario

Escalado horizontal mediante Pub/ Sub

Garantías de transacción

Más allá del modelo de transacción base

Seguridad, privacidad y permanencia

Ciberseguridad

Seguridad en Google Cloud Platform

Privacidad de datos

Permanencia de los datos

Conclusión

4. Elección entre opciones de GCP

Introducción

Estructura

Objetivos

Elegir una plataforma de ejecución del lado del servidor

El impacto de la selección de la plataforma informática

Motor de aplicaciones de Google

Google Cloud Run

Motor de cómputo

Máquinas virtuales

Calcular comparación de productos

Gestión de recursos informáticos de GCP

Recursos computacionales ofrecidos

Cómo determinar dónde es mejor almacenar sus datos

Firestore en la nube

Lenguaje de consulta estructurado en la nube

Mesa grande

Rendimiento de Bigtable

Llave

Almacenamiento en la nube

Identidad y autenticación

Autenticación de Firebase

Funciones de autenticación de Firebase

Gestión de identidades y accesos (IAM) de Google Cloud

Roles versus usuarios

Plataforma de identidad

Identificar las características de autenticación de la plataforma

Manejo asincrónico de mensajes

Pub / Sub

Escalabilidad de una arquitectura de mensajería

Tareas en la nube

Conclusión

Sección 2. Arquitectura de aplicaciones respaldadas por la nube

5. Opciones tecnológicas del lado del cliente

Introducción

Estructura

Objetivos

Aplicaciones móviles nativas

Herramientas de desarrollo de Android

Herramientas de desarrollo de iOS

Marcos móviles multiplataforma

Aleto

Reaccionar nativo

Kotlin Multiplataforma

Aplicaciones web

Aplicaciones cliente de escritorio

Desarrollo de escritorio nativo

Desarrollo de escritorio multiplataforma

Aplicaciones cliente en el navegador

Lenguajes de programación de aplicaciones web

Angular

React y ReactDOM

Herramientas de desarrollo de aplicaciones web

Compatibilidad de GCP con clientes de aplicaciones web

Chatbots y agentes de voz

Chatbots y GCP

Interfaces de voz de chatbot

Conclusión

6. Servicios en la nube en la frontera del backend

Introducción

Estructura

Mantenimiento de clientes independientes del backend

Principios generales de diseño de API

Consideraciones especiales para interfaces de cliente o de nube

Niveles de visibilidad para el cliente

Planificación de la migración a servicios de nube sustitutos o equivalentes

Bloqueo del proveedor

Obsolescencia del producto

Diseño para la migración

Visibilidad entre servicios en la nube

Dependencias entre productos de servicios en la nube

Aislamiento de productos de servicios en la nube

Identificación y empaquetado de servicios estrechamente acoplados

Conclusión

7. Frontends en la frontera del cliente

Introducción

Estructura

Objetivos

Lógica del lado del cliente

- Evite múltiples fuentes de verdad

- Presentación de la interfaz de usuario

- Captura de la entrada y selección de datos del usuario

- Ocultar la lógica sensible a la observación

- Hacer que el código que se modifica con frecuencia sea fácil de implementar

Definición de solicitudes de clientes al backend

- Backends para frontends

Respuestas de solicitud bloqueantes y no bloqueantes

- Identificación de métodos asincrónicos

- Tecnologías de cliente para el procesamiento asincrónico

Rechazo y manejo de respuestas inesperadas

Definición y gestión de acciones no iniciadas por el usuario

- Casos de uso para acciones iniciadas por el servidor

- Mecanismos para recibir tráfico generado por el servidor

- Consideraciones especiales para interfaces de cliente o de nube

Conclusión

Sección **De extremo a extremo: un sistema de votación global**

8. Hacer que un sistema de votación esté disponible y sea confiable

Introducción

Estructura

Objetivos

Descripción general del sistema de votación digital

Descripción general de un concurso de talentos en línea

Proceso de votación

Requisitos de escalabilidad

Requisitos de rendimiento del sistema de votación

Modelos de uso

Límites regionales de Google Cloud Platform

Demanda de sistemas de votación por continente

Agregación de datos

Agregación de votos distribuidos en un sistema global

Accesibilidad

Reubicación estratégica de recursos

Optimizar la aplicación para que se ajuste a los recursos disponibles

Rediseño de la aplicación

Creación del paquete y la estructura de directorios del codelab

Componentes de la plataforma y archivos iniciales

Backends para frontends

API de votación

Conclusión

9. Identificar los puntos de estrés de nuestro sistema de votación

Introducción

Estructura

Objetivos

Tipos de fallos

Alcance de un fallo

Progresión

Causas de un fallo

Categorización de fallos

Evaluación del impacto

Determinación del costo a corto plazo de una interrupción

El costo a largo plazo de una interrupción del servicio

Monitoreo, detección y notificación

Observar primero y luego controlar

Terminología de monitorización

Tipos de métricas

Métricas clave de confiabilidad

Monitoreo de sistemas distribuidos

Monitoreo de aplicaciones de GCP

Puntos de observación del rendimiento en la aplicación de votación

Respuesta a incidentes

Matriz de modos de fallo y medidas apropiadas

Implementación del laboratorio de código de la API de ingesta de votos

Consideraciones del proyecto al configurar el repositorio

Cliente web y código común

Código basado en la nube

Configuración del proyecto de Google Cloud

Configuración del proyecto de Firebase

Monitoreo de GCP

Conclusión

10. Asegurar todos los votos de los usuarios

Introducción

Estructura

Objetivos

Modelado de la demanda

Ciclos independientes de demanda

Equilibrio de carga

Tipos de balanceadores de carga

Algoritmos y consideraciones de equilibrio de carga

Equilibrio de carga en Google Cloud Platform

Selección de balanceadores de carga del sistema de votación

Uso de colas de mensajes para agrupar solicitudes

Creando Pub / Subrecursos para votos

Publicación de votos para Pub/ Sub

Procesamiento asincrónico de solicitudes

Procesamiento de votos publicados en Pub/ Sub

Escalado del procesamiento backend

Aplicación del escalado de GCP

Implementación del procesador de votos

Monitoreo del rendimiento del backend

Envío de solicitudes de usuarios de forma asincrónica a través del laboratorio de código Pub/Sub

Componentes de procesamiento de votos asincrónicos

Directorios de nivel superior con nuevo código en la nube

Adiciones de código al procesador de votos

Pub / Subconfiguración

Código del procesador de votos

Configuración e implementación de Cloud Run

Configuración del escalado automático del procesador de votos

Monitoreo del uso de recursos del servicio

Monitoreo para Pub / Subtemas atrasados

Monitoreo para Pub / Sub mensajes antiguos

Monitoreo de la latencia del frontend

Respondiendo a notificaciones

Conclusión

11. Garantizar la permanencia de los datos

Introducción

Estructura

Objetivos

Tipos de datos

Datos estructurados y no estructurados

Datos profundos vs. datos planos

Definiciones de datos de votación

Bases de datos relacionales y no relacionales

Estructuras no relacionales

Modelos de almacenamiento para datos de votación

Objetos binarios grandes

Patrones de acceso

Garantizar la disponibilidad de los datos

Objetivos de permanencia de instantáneas y archivos

Copias de seguridad para recuperación

Instantáneas para la continuidad

Archivos para la posteridad

Instantáneas y archivos de las votaciones

Laboratorio de código sobre persistencia de votos y actividad electoral

Componentes de Codelab

Habilitación de la facturación para el proyecto

Interfaces de administración de GCP

Definición de los componentes de Spanner

Confirmar los recursos de Spanner en la consola

Deshabilitar la facturación del proyecto

Código para conectarse a los recursos de Spanner

Adiciones de código al procesador de votos

Adiciones de código a la interfaz de votación

Código en el motor de políticas de votación

Código en la página del cliente web

Implementación del servicio Cloud Run

Inspeccionar el programa de copias de seguridad

Agregar una programación de copias de seguridad

Conclusión

12. Desarrollo de interfaces de votación

Introducción

Estructura

Objetivos

Responsabilidades de la aplicación cliente

- Identificación de plataformas de aplicaciones cliente

- Responsabilidades de la aplicación cliente

Fuera del alcance

- Responsabilidades de los niveles backend

Nuestra aplicación para clientes votantes

- Requisitos de la solicitud de cliente de votación

- API de interfaz de votación

Uso de la API del lado del servidor

- Rendimiento del cliente de votación

- Reglas de política del lado del cliente

- Resumen de requisitos

Interfaz web del votante

- Lenguaje y marco de cliente web

- Inicio y cierre de sesión del cliente web

- Manejo de errores

- Accesibilidad del cliente web

- Reglas de política del lado del cliente

Aplicación móvil para votantes

- Herramientas de desarrollo de aplicaciones móviles

- Aplicaciones móviles BeeWare

- Reglas de política del lado del cliente móvil

Refactorización del código base

- Portabilidad de paquetes de Python

Aislar la fuente

Definiendo el paquete

Identificación de un repositorio de paquetes

Crear o construir un paquete

Distribución de paquetes

Instalación de paquetes

 Codelab para la creación de múltiples aplicaciones cliente

 Refactorización de la jerarquía de paquetes

 Construir y distribuir paquetes de Python

Nombre del paquete

Metadatos y configuración del paquete

 Distribución de paquetes mediante un registro de artefactos

 Instalación del paquete Local Artifact Registry

 Implementación en la nube mediante paquetes de Artifact Registry

 Desarrollo con BeeWare

Descripción general de BeeWare

Instalación de BeeWare

 Cómo escribir una aplicación BeeWare

 Implementación de la aplicación en iOS

Conclusión

Sección 4. Resumen: Aplicación del estudio de caso

13. Reemplazo de componentes de forma aislada

 Introducción

 Estructura

 Objetivos

 Diseño de abstracciones para responsabilidades de infraestructura

 Evaluación de infraestructura basada en requisitos

 Una plantilla para el análisis de requisitos

Mapeo de la funcionalidad de la base en interfaces de infraestructura

Encontrar niveles adecuados de especificidad para los requisitos

Contener dependencias mediante la convergencia a cajas negras

Requisitos como especificaciones de caja negra

Gramática para especificaciones de caja negra

Ventajas de las especificaciones de caja negra

Planificación de una migración

Determinar la necesidad de migrar

Recopilación de candidatos sustitutos

Evaluación de las alternativas

Cálculo del coste del servicio de base de datos

Cálculo de valores para los factores de costo de AlloyDB

Calcular los costos de todas las alternativas

Cálculo de los costes de todas las alternativas

Reevaluación de la decisión migratoria

Construcción de un plan migratorio

Cambiar los neumáticos mientras el autobús está en movimiento

Diseño de la migración de un laboratorio de código de almacén de datos

El script de migración

Estar preparado para lo peor

Conclusión

14. Sistema de votación como implementación de referencia

Introducción

Estructura

Objetivos

Sistemas de votación en las empresas

Solicitudes de votación no reconocidas

Aplicaciones de votación comúnmente vistas

Derivando una aplicación de votación similar

Reutilización de componentes del sistema de votación

Reutilización de patrones de diseño

Necesidad de clasificadores y procesadores específicos de clase

Optimización del diseño para el rendimiento

Estudio de caso de un procesador de solicitudes altamente escalable

Patrón de diseño de trabajador genérico

Aplicaciones del procesamiento de solicitudes de alto rendimiento

Evolución de los sistemas de procesamiento de grandes solicitudes

Ventajas de la infraestructura en la nube

Reflexiones finales sobre los roles genéricos

El sistema de votación completo

Comparación de la solicitud de votación

Google Cloud Platform para aplicaciones distribuidas generales

Conclusión

Índice

Sección **1:**

Servicios en la nube por Responsabilidad

CAPÍTULO 1

Responsabilidades y utilidad de un Plataforma en la nube

Introducción

Este libro ofrece una explicación integral y una guía de uso de Google Cloud Platform como capa fundamental para aplicaciones informáticas distribuidas. Se analizará y evaluará tanto un modelo abstracto de una aplicación como de un servicio en la nube, así como una instancia concreta de cada uno. Al finalizar el libro, el lector tendrá una comprensión conceptual y una implementación de referencia real para diseñar sus propias arquitecturas distribuidas.

En la primera parte de este libro, examinaremos las consideraciones arquitectónicas de cualquier sistema de software distribuido y escalable, sin depender de ningún proveedor. A continuación, dividiremos los requisitos funcionales de una aplicación en componentes, según las características de nuestra plataforma en la nube.

Tras la planificación, en la segunda parte nos centramos en el cliente. Nuestros usuarios no están en la nube; están conectados a un teléfono, una tableta o una computadora. Se describirán las plataformas del cliente, incluyendo las razones para ofrecer soporte a cada una.

La frontera entre el cliente y la nube es la siguiente: consideraremos la

fortalezas y debilidades de los entornos de cliente y de nube y diseñar interfaces bien definidas para cada uno de ellos.

Luego, ponemos los principios en práctica. La tercera parte de este libro construye un sistema distribuido donde la escalabilidad y la confiabilidad son cruciales: un sistema de votación para programas de talentos televisados en las democracias más grandes del mundo.

Con el sistema de votación implementado, la cuarta parte del libro resume una implementación exitosa. Describiremos una tarea común: migrar nuestra aplicación a un nuevo servicio en la nube. Un sistema bien diseñado permite migrar componentes de nuestro sistema con un impacto mínimo en el resto de la aplicación.

Finalizamos con una descripción de nuestro sistema de votación como modelo de referencia para sistemas con requerimientos técnicos similares.

Estructura

Este capítulo cubre los siguientes temas:

- Consideraciones genéricas sobre infraestructura
- Identidad y autenticación
- Escalabilidad
- Procesamiento asíncrono
- Permanencia de los datos
- Entrega de respuesta
- Análisis de responsabilidades de infraestructura
- Descripción general del producto Google Cloud Platform

Objetivos

Este capítulo revisará la evolución del desarrollo de aplicaciones que ha impulsado la necesidad de plataformas de servicios en la nube. A continuación, revisaremos las consideraciones arquitectónicas de una aplicación distribuida y los tipos de servicios que ofrece una plataforma en la nube para abordarlas.

A continuación, el capítulo se centra en algunos de los productos de Google Cloud Platform (GCP) en cada una de estas categorías de servicios. Concluimos el capítulo.

Capítulo introductorio con una revisión de qué características no encajan en los servicios de GCP y cuáles son cada una de sus plataformas óptimas.

Se describirán los requisitos básicos de una aplicación distribuida típica y exploraremos cómo un servicio en la nube puede agregar valor a la solución de una aplicación para cada uno de esos requisitos.

Consideraciones generales sobre infraestructura

Existen algunas consideraciones comunes que se aplican a todas las partes de un sistema distribuido y que deben tenerse en cuenta al elegir un proveedor de infraestructura y al diseñarlo en general. En primer lugar, ¿cuál es la interfaz de cliente? Bibliotecas de lenguaje de cliente, una API REST, llamadas a procedimientos remotos basadas en sockets. ¿Cómo se invoca una operación y se obtiene una respuesta? Asegúrese de que su proveedor de infraestructura sea compatible con otras consideraciones: la base de conocimientos de sus ingenieros, las plataformas a las que se dirige y cualquier producto de software específico del dominio con el que deba interactuar. Otras consideraciones generales incluyen la disponibilidad regional, el modelo de facturación (capacidad aprovisionada o basada en la demanda) y los planes de soporte técnico y de otro tipo que ofrece el proveedor o terceros.

Identidad y autenticación

"Esto es lo que soy" y "Puedo hacer esto" son dos de las operaciones más fundamentales y sometidas a pruebas de estrés que un sistema de aplicaciones debe soportar. Fundamentales porque todos los sistemas multiusuario con estado tienen la noción de identificar a un usuario conocido y la necesidad de autenticarlo para evitar la suplantación de identidad. Estas funciones son sometidas a pruebas de estrés por parte de muchas personas, sistemas y agentes bot dedicados a engañar, eludir o romper el mecanismo de autenticación con el objetivo de gastar el dinero, la autoridad o la reputación de otros. Los delincuentes son astutos y persistentes, por lo que se requiere un esfuerzo considerable para ir un paso por delante de ellos.

Escalabilidad

La necesidad de escalar es el mejor problema que se puede tener. Esta es una verdad sorprendente que a menudo se encuentra al lanzar por primera vez un sistema popular a gran escala. Por ejemplo, después de pasar meses mejorando la calidad y el impacto de un nuevo sistema interno, los usuarios aún se quejan de la latencia y la capacidad. En lugar de desanimarse, esto es motivo de celebración. ¡Esto significa que los usuarios no se cansan de usar su servicio!

Un sistema impopular sin crecimiento de usuarios no presenta desafíos de escalabilidad. En la mayoría de los escenarios empresariales, tampoco tiene futuro.

Una de las razones más convincentes para alojar un sistema en la nube es su elasticidad. El número de trabajadores no está limitado por la cantidad de RAM de sus servidores, el tamaño de su sala de servidores, la amplitud de su bloque de IP ni la cantidad de vatios que permitan sus disyuntores. Existen límites, por supuesto, pero son límites ajenos, y ese alguien ha hecho un negocio rentable al poder proporcionar más capacidad informática bajo demanda a cambio de un precio. El trabajo de su proveedor de nube es llenar los centros de datos, comprar conmutadores de red y pagarles cuando necesite usar cualquier cantidad.

Procesamiento asincrónico

Las personas no necesitan respuestas a sus preguntas tan inmediatas como suelen creer. Lo que realmente desean es un reconocimiento de que la pregunta fue escuchada y atendida. Tomemos como ejemplo un sistema de banca en línea: se puede usar la cuenta corriente para pagar el saldo de la tarjeta de crédito. La solicitud tarda unos segundos en formularse: se hace clic en un botón, se selecciona una fuente de pago, se introduce un importe y se envía. En realidad, los sistemas bancarios son amplios y complejos: una transacción como esta puede tardar varios segundos o minutos en ejecutarse por completo y propagarse a través de los procesos y almacenes de datos de los bancos. ¿Ve el usuario una pelota de playa girando o espera el tiempo necesario? Probablemente cerraría la ventana o su navegador se bloquearía mientras espera. En cambio, se le dice inmediatamente:

Transacción pendiente, con algunos detalles para confirmar que la transferencia
De hecho, la solicitud fue escuchada y los sistemas backend del banco...
procesar esta transferencia de forma asincrónica, desacoplada del usuario
Frontend visible. Finalmente, el efecto de la transacción completada es
se hace visible para el usuario y el pago pendiente se muestra como
completo, con saldos y límites todos actualizados en sus respectivos
Cuentas. El medio para poner en cola solicitudes pendientes como esta para más adelante.
El procesamiento sin bloquear al solicitante es una capacidad importante
ofrecidos por plataformas en la nube, generalmente en forma de cola de mensajes
servicio. Una característica complementaria a esto es un grupo de trabajo elástico, donde
Se pueden crear nuevas tareas según sea necesario para procesar una cola creciente de pendientes.
solicitudes.

Permanencia de los datos

La permanencia de los datos se puede resumir en una simple invariante: ¿Qué
Lo que lees es lo que escribiste. Punto. Cuando quieras leer algo.
valor almacenado, el único resultado aceptable es el valor que almacenó.
No hay resultados parciales, ni valores ilegibles o incorrectos, ni
tiempos de espera. Por ejemplo, después de almorzar con un compañero de trabajo, podrías escribir
Envíame una nota. Joelle me debe \$ y cuando quieras, más tarde.
Si te preguntas sobre el saldo de Joelle, calcularás -10.00. Después de tu
amigo te paga, escribirías, Joelle ahora me debe \$ Cuando posteriormente calculas 0.00 .
el saldo de Joelle, llegas a un
balance de 0 y sin momentos incómodos la próxima vez que hables. Esto
Sólo puede suceder si tienes confianza en la permanencia de los datos
Garantizado por su servicio de almacenamiento, en este caso, su aplicación para tomar notas.
o cuaderno. Este es el aspecto más impactante y crítico de un
Confiabilidad de la aplicación: una consulta de usuario descartada se puede volver a intentar, pero se pierde
Los datos (y un amigo desairado) se han ido para siempre.

Entrega de respuesta

La entrega de respuestas es muy similar a la disociación de nuestra tarjeta de crédito.
solicitud de pago desde el procesamiento que desencadena. La entrega de un

La respuesta al cliente suele estar desvinculada de la finalización de una tarea. Cuando nuestro sistema de pago con tarjeta de crédito completa la compleja transacción de solicitar un pago, ¿debería detenerse y esperar a que se envíe el estado actualizado al usuario? Eso sería un problema. Supongamos que están desconectados o que la latencia de la red retrasa el mensaje varios segundos. Otra aplicación del procesamiento asíncrono, que a veces utiliza los mismos servicios de cola de mensajes, permite enviar respuestas a los destinatarios, quienes deben recuperarlas cuando estén disponibles. Una variante de esto es la compatibilidad con notificaciones push, donde el servicio de mensajería invoca un punto final designado en el cliente, en lugar de que el cliente consulte los mensajes pendientes. Cada modelo tiene sus usos y se explorará a lo largo de este libro.

Análisis de consideraciones de infraestructura

La plataforma de infraestructura que utilizamos ofrece características distintivas para la implementación de nuestro producto final. Estas características no siempre están claramente definidas, y las capacidades pueden solaparse o parecer redundantes. La siguiente sección explica cómo y, sobre todo, por qué una aplicación puede descomponerse en componentes funcionales.

Interdependencia de las responsabilidades de la plataforma.

Ahora que podemos consultar las definiciones prácticas de estos temas, podemos analizar la topografía de una aplicación y ver cómo se superponen estas consideraciones. El siguiente diagrama de Venn nos guiará en esta exploración:



Cifra **1.1:** Responsabilidades críticas dentro de una aplicación distribuida

Sería ingenuo considerar cualquiera de estos temas de forma aislada.

Por ejemplo, las solicitudes de los usuarios no son la única fuente de demanda que tenemos.

Se espera que aumente la escala. Los inicios de sesión de los usuarios, que satisfacen la demanda de identidad, y el procesamiento de autenticación introducirán una carga cada vez mayor en

nuestro sistema y cualquier mecanismo que usemos para mantenernos al día con un

El aumento en la tasa de solicitudes de los usuarios podría incidir en el número cada vez mayor de inicios de sesión en nuestro popular producto.

¿Qué hace que un sistema de aplicación sea digno o, mejor, adecuado para...

¿Incorporar un servicio en la nube en su implementación técnica? La respuesta es la misma que para muchas otras preguntas que surgen cuando se tiene la presión de entregar un producto con un coste o plazo fijos.

Debería ser la opción con el mayor valor neto positivo: el valor siempre que los costos superen los de las alternativas sin nube.

Al observar los laboratorios de código o las guías de inicio rápido (por una definición más comprensiva de "rápido") para GCP, resulta abrumador cuánto hay que aprender antes de poder escribir un "Hola Mundo" exitoso desde la aplicación en la nube. Este proceso acelerado supone una gran barrera de entrada, y es muy tentador creer que estos servicios son innecesariamente complejos o que la documentación técnica es insuficiente. Muchos ingenieros han terminado tras horas o días de intentos frustrantes por lograr que una solución sencilla funcione.

Cualquier plataforma de servicios en la nube, incluido GCP, es una máquina extensa y compleja. Existen numerosos engranajes, ejes y resortes internos. Si bien los diseñadores de estas máquinas tan complejas se esfuerzan por ocultar los detalles de implementación, la reducción de la complejidad visible públicamente solo puede llegar hasta cierto punto sin bloquear el acceso a gran parte de las capacidades inherentes de la plataforma. Un componente de software capaz de dar servicio a docenas o cientos de casos de uso diferentes no puede hacerlo a través de una API con una sola llamada a un método sin parámetros. En la práctica, si un componente puede abordar $N = 100$ escenarios diferentes, su interfaz pública podría constar de 100 métodos independientes, 50 métodos independientes, cada uno con un argumento booleano, y 10 métodos con 10 secuencias de invocación significativas diferentes. O bien, un método con 100 combinaciones de valores de parámetros diferentes, pero habrá 100 elementos invocables visibles para los usuarios de ese componente.

[Asignación de roles por plataforma.](#) Un

concepto de ingeniería útil y entretenido es el de los antipatrones. Se trata de una categoría de patrones de diseño que se observa con frecuencia en grandes proyectos de desarrollo que han fracasado. Como complemento perfecto al concepto y catálogo de patrones de diseño aceptados, había aspectos que...

emular en el diseño de software, y había cosas que evitar.

Uno de estos antipatrones se denomina Clase Dios, una clase con demasiada responsabilidad y funcionalidad. Una característica común de las clases Dios es el método `doItAll`, que lo hace todo. Todo lo que la clase hace se encuentra en un cuerpo de método extenso. Esto es la antítesis de las prácticas de diseño de una aplicación respaldada por un servicio en la nube. En una aplicación respaldada por la nube, las responsabilidades se especifican tanto para la función como para la forma ideal: la función es el efecto deseado y la forma es el entorno de ejecución particular en el que esa función se ejecuta de forma óptima. Algunas cosas pertenecen al usuario; otras están más estrechamente vinculadas a un almacén de datos, caché global u otro sistema de software con el que la aplicación interactúa.

Analicemos algunas razones por las que no deberías escribir monolitos grandes y torpes:

- Diferentes operaciones tienen diferentes tasas de solicitud: No todas las operaciones de un sistema tienen los mismos impulsores de demanda. Una operación de inicio de sesión ocurre una vez por sesión de usuario y quizás con menos frecuencia si un cliente permite que el usuario permanezca conectado durante períodos prolongados. Por el contrario, la visualización de la puja más alta actual en una subasta ocurre constantemente a intervalos cortos durante alguna etapa de la sesión de usuario. Algunos recursos se asignan para autenticar a un usuario, pero no son necesarios una vez que se conoce su identidad. Aislar las operaciones de inicio de sesión nos permite liberar recursos que deberían estar fuera del alcance después de que hayan cumplido su propósito.
- Los métodos largos son difíciles de entender: La legibilidad y la comprensión por parte de los ingenieros son los factores que determinan la precisión de un programa informático, no la comprensión de un compilador o microprocesador. Se han detectado errores notables en los traductores de lenguajes (también conocidos como compiladores, intérpretes e intérpretes de código de bytes en tiempo de ejecución) y procesadores, pero son notables porque son poco frecuentes. Por otro lado, existen errores debidos a errores de programación, específicamente la interpretación errónea del efecto de algún fragmento de código en todos los productos de software, y se observan más.

Se introducen a diario. Un factor fácil, obvio y de aplicación global para determinar la comprensión del método que lee y escribe un desarrollador de software es la longitud del cuerpo del método. Una vez que una parte del método desaparece de la vista en un editor de código fuente de tamaño típico (unas 24 líneas), desaparece.

Los lenguajes de programación son densos, y mantener la conciencia de 20, 30 o 100 líneas de código es más de lo que la conciencia humana está equipada para hacer. • La falta de

aislamiento de valores invita a efectos secundarios no deseados y exploits: la mayoría de los lenguajes de programación proporcionan algunas opciones para el alcance de una variable o función. Global, de clase o local a un método (o función). En realidad, hay alcances más finos disponibles, como el nivel de bloque, pero estos son desconocidos y rara vez se usan intencionalmente porque la mayoría de los bucles for los incluyen como índice, pero los desarrolladores que los escriben no los piensan como un ejemplo de una variable de nivel de bloque, sino más bien como algo específico del bucle for.

Este es el problema: un método grande que encapsula muchas operaciones diferentes incluirá variables locales para esas operaciones dispares pero sin nada que proteja a la operación A de acceder a una variable declarada y destinada a ser utilizada por la operación hermana B.

Este tipo de efecto secundario indeseable puede ser involuntario, como en el siguiente fragmento de código omitido: **int**

```
x,y;  
...  
x = alto/2; y =  
ancho/2;  
...  
para (x=0; x<número_de_formas; x+  
+) { reposicionar_forma(x); }  
...
```

```
Mientras (rect)
{ rect.resize_to_frame(x,y); rect
  = rect.nextShape;
  número_de_formas
  += 1; }
```

Busque el error latente en el código anterior, sintácticamente correcto pero claramente un error.

La x en el bucle for es la misma variable de método que la x asignada anteriormente y restablece ese valor, que se había establecido en la mitad de la altura.

Por supuesto, mejores nombres de variables también serían útiles en este caso; ¿por qué x es un nombre de índice tan común? Sin embargo, esa es otra cuestión.

Peor aún que el error imprevisto ilustrado anteriormente, este tipo de efecto secundario puede ser latente. Esto significa que no se presenta en situaciones esperadas, pero está ahí para activarse en algunos casos extremos. Esto crea un terreno fértil para exploits como los ataques de desbordamiento de búfer basados en la pila, que suelen lanzarse cuando un lector malicioso del código fuente del programa detecta la posibilidad de secuestrar un fragmento de código y una forma de modificar su comportamiento para obtener un efecto no deseado y probablemente dañino.

Una práctica que minimiza esta clase general de defecto es simple: mantener las responsabilidades de los métodos bien definidas y estrechamente enfocadas, con solo el código que necesitan para realizar esa tarea muy bien definida.

Reducir el alcance compartido reduce el potencial y el impacto de efectos secundarios destructivos:

Un cierto grado de paralelismo puede ser ideal para algunas operaciones, pero no para otras: esta consideración es más relevante para un sistema distribuido que para una aplicación local con una única plataforma de ejecución. Algunas tareas se benefician de la asignación de más trabajadores, mientras que otras no. Hay un viejo dicho que dice que no se puede tener un bebé en un mes, pero podemos ser más relevantes. Se puede convertir una imagen grande a escala de grises en aproximadamente una centésima parte del tiempo asignando a cada una de las 100...

Segmentos del mapa de bits a un proceso diferente. Por otro lado, la tarea previa de capturar la imagen desde una cámara es una operación que requiere un solo trabajador.

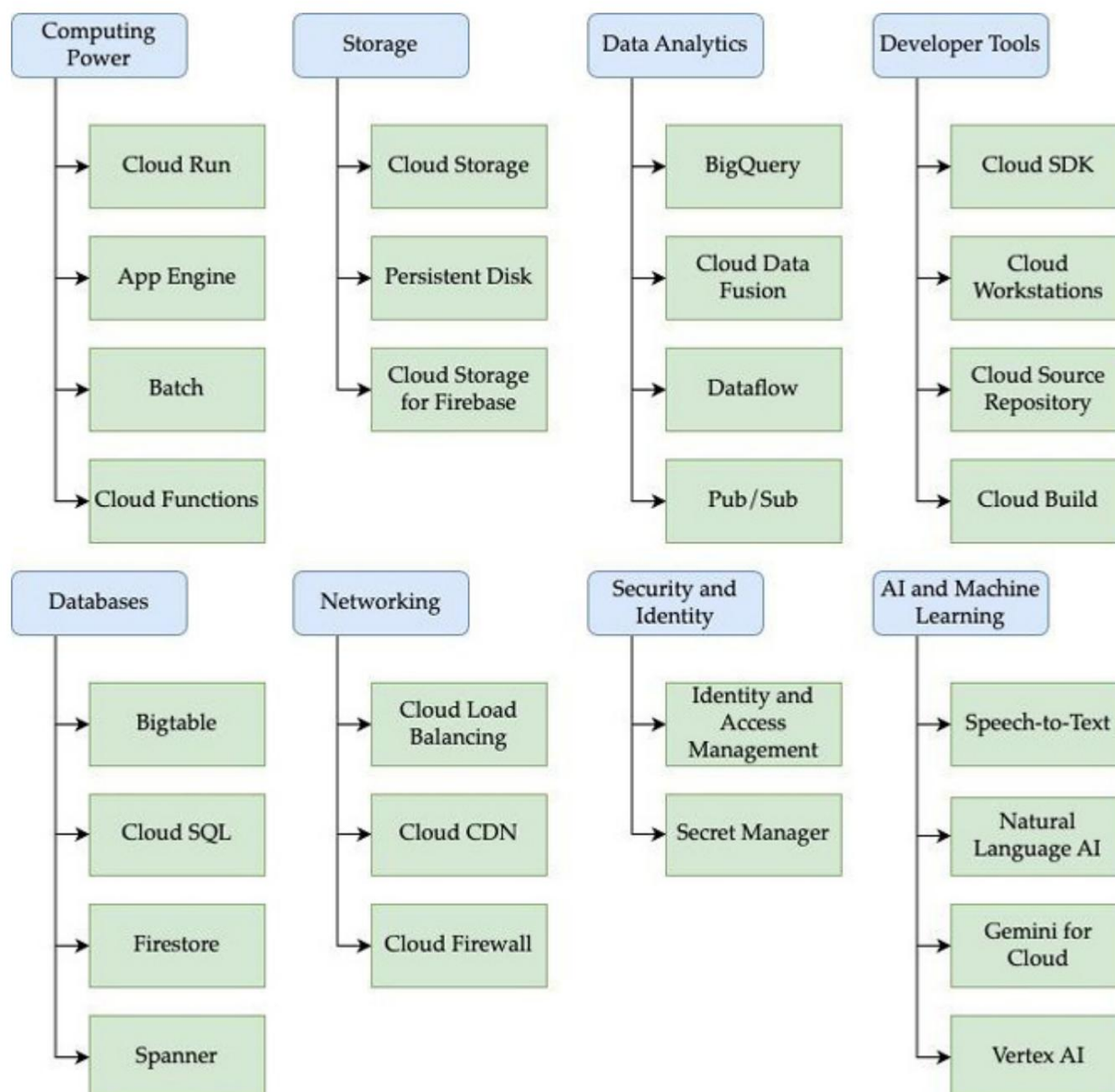
Todo trabajo requiere el producto de (tiempo * espacio), y se desea decidir cuánto aumentar el espacio para disminuir el tiempo de forma dinámica y adecuada a la tarea específica en cuestión. En la operación de escala de grises de la imagen presentada anteriormente, procesar un mapa de bits de baja resolución de 340 x 240 píxeles no requeriría tantos trabajadores como una imagen de ultraalta definición de 10 megapíxeles. La imagen más grande tiene 130 veces más píxeles para iterar, por lo que si el objetivo es mantener el tiempo de procesamiento casi constante, se requerirían más de 100 trabajadores para procesar la imagen en paralelo. La dinámica y adecuada mencionada anteriormente es uno de los puntos fuertes de un servicio en la nube; se destacan por proporcionar trabajadores bajo demanda y distribuir el trabajo equitativamente entre los trabajadores disponibles en el grupo.

Esperamos que a estas alturas se haya dado cuenta de lo peligroso que puede ser crear un sistema grande sin seguir las mejores prácticas. Sin embargo, no se rinda. No es tan grave: probablemente ha estado produciendo software hasta ahora, y esas aplicaciones han cumplido su función.

Desarrollar aplicaciones sin un servicio en la nube en su infraestructura no lo hace incompetente, al igual que incluir uno no es garantía de éxito. Aprender sobre los servicios en la nube y utilizarlos en sus sistemas aumentará la velocidad de desarrollo y reducirá el esfuerzo necesario para producir sistemas con mayor confiabilidad, capacidad de respuesta y, en definitiva, satisfacción del usuario.

Descripción general del producto Google Cloud Platform

Comencemos explorando los servicios específicos que ofrece GCP y organizándolos según su tema principal. Analicemos primero otro diagrama, una taxonomía parcial de los servicios de GCP, agrupados por tema general:



Cifra 1.2 Taxonomía de los principales servicios de GCP

Nota: A partir del verano 2024, GCP incluye más de 100 servicios, muchos de los cuales existen para casos de uso altamente especializados, como la migración de cargas de trabajo de VMware existentes o la gestión de nodos de hardware totalmente dedicados que podrían requerirse para cumplir con los requisitos específicos de una industria regulada. El diagrama anterior y el contenido general de este libro se centran en un servicio amplio y de aplicación más común. Para obtener una lista completa de los servicios de GCP, consulte siempre el producto publicado por

Descripción general en <https://cloud.google.com/products>.

Categorías de nivel superior de los servicios de Google Cloud. GCP ofrece numerosos servicios que abordan las mismas necesidades básicas. De nuevo, "¿Por qué no lo simplifican?" es una respuesta común al ver la extensa lista de servicios disponibles.

Hay dos razones principales para la aparente duplicación: la especialización y la evolución.

Por especialización, nos referimos a la amplia gama de algunas de estas categorías. ¿Desea almacenar muchos archivos pequeños, menos archivos de gran tamaño o mantener sus propios sistemas de archivos POSIX? En lugar de reducir los múltiples casos de uso posibles al máximo común denominador, lo que dejaría sin cumplir muchos requisitos específicos de cada escenario, GCP ofrece diferentes servicios de almacenamiento para abordar diferentes conjuntos comunes de requisitos.

La evolución se refiere a las características cambiantes de una solución a lo largo del tiempo, ya sea debido a cambios en el mercado o a la retroalimentación y las lecciones aprendidas de soluciones anteriores. Puede haber nuevos paradigmas o arquitecturas mejoradas que ofrecer, pero sin eliminar los servicios existentes con los que miles de clientes cuentan y de los que dependen. La forma equilibrada de actualizar las capacidades de esta manera es introducir servicios nuevos y diferenciados, manteniendo los antiguos, manteniéndolos y dándoles soporte hasta que la base de usuarios haya migrado a sus contrapartes más nuevas, más avanzadas y, con suerte, más valiosas.

Repasemos brevemente la misión de las categorías y servicios representados en [la Figura 1.2](#).

Potencia de cómputo. Esta

clase de servicios proporciona la capacidad de ejecutar un programa. Esto incluye memoria de acceso aleatorio, un procesador y algunos componentes complementarios generalmente ignorados. Las características de rendimiento relevantes son la velocidad, el número de instrucciones por segundo y la memoria, es decir, la cantidad de bytes (megabytes, gigabytes) de código y datos que pueden almacenarse.

Los servicios que describiremos ofrecen potencia de procesamiento optimizada para diferentes tipos de cargas de trabajo. Conozcamos los servicios:

- **Cloud Run:** Cloud Run te permite ejecutar una carga de trabajo escrita en el lenguaje y los frameworks que elijas, tal como lo implementarías y ejecutarías en tus propias máquinas. Tu aplicación implementable se empaqueta en una imagen de contenedor, ya sea por ti o por la herramienta Cloud Build, y está lista para ejecutarse en producción. Cloud Run inicia automáticamente nuevas instancias de tu contenedor para gestionar las solicitudes entrantes y retira los trabajadores inactivos e innecesarios a medida que disminuye la demanda.

Cloud Run puede ejecutar trabajadores que procesan solicitudes de usuarios, pero también puede administrar trabajos no interactivos que se ejecutan hasta por 24 horas.

- **App Engine:** App Engine es un servicio administrado ideal para ejecutar sitios web renderizados del lado del servidor escritos en uno de los lenguajes compatibles: Node.js, Java, Ruby, C#, Go, Python o PHP. Al ser un entorno completamente administrado, App Engine oculta toda la infraestructura y el sistema operativo subyacentes; simplemente crea e implementa tu aplicación, que luego se ejecuta en un entorno aislado con características de rendimiento computacional y límites de almacenamiento para el tipo de instancia que selecciones de un conjunto disponible. App Engine escala el número de servidores que ejecutan tu aplicación para satisfacer las demandas de tráfico.

App Engine cuenta con dos tipos de entorno: el entorno estándar de App Engine y el entorno flexible de App Engine. El entorno flexible admite un mayor número de lenguajes y frameworks, pero requiere que el usuario realice previsiones y planificación sobre cómo escalará el tráfico de su aplicación, mientras que el entorno estándar gestiona los cambios rápidos y dinámicos de la demanda.

- **Lote:** Lote es el equivalente de GCP de un trabajo cron clásico de Unix. El servicio por lotes programa, pone en cola y ejecuta trabajos y cargas de trabajo no interactivos que no se activan mediante un evento de usuario. El servicio por lotes pone en cola instancias de sus trabajos por lotes.

Según los parámetros de creación que especifique. Cuando los requisitos de recursos especificados estén disponibles, los trabajos en cola se ejecutan con parámetros de programación que puede anular a partir de los valores predeterminados calculados.

Batch está integrado con Cloud Run y otros servicios de computación, y es similar a ellos, pero ofrece opciones que lo convierten en una opción atractiva para tareas que se ajustan a sus limitaciones. Dado que los trabajos ejecutados por Batch no son interactivos y se supone que tienen entradas existentes bien definidas, tolerancia a fallos y capacidad de reinicio, los trabajos por lotes pueden ejecutarse en máquinas virtuales de Spot con los mismos perfiles de hardware que otros servicios de computación, pero sin garantías de disponibilidad continua. En otras palabras, la máquina virtual en la que se ejecuta el trabajador por lotes podría apagarse o reiniciarse, y la tarea se iniciará en otra máquina virtual cuando esté disponible. Ejecutar en estas instancias, potencialmente de corta duración, y gestionar la semántica de reinicio ofrece una ventaja: máquinas entre un 60 % y un 91 % más económicas.

- **Funciones de Cloud Run:** Las funciones de Cloud Run son una implementación del paradigma abstracto de infraestructura definitivo: función como servicio (FaaS). Para usar las funciones de Cloud Run, se proporciona una función de propósito único en un lenguaje compatible y se asocia a un evento desencadenante. Cuando se produce dicho evento, FaaS invoca la función y le envía los parámetros del evento desencadenante.

Las funciones en la nube están escritas en un entorno de ejecución compatible: muchas versiones de Node.js, Python, Go, Java, Ruby, PHP o .NET Core.

Esta lista puede resultar familiar a partir de la descripción anterior de App Engine; el lector puede preguntarse por qué puede ser así.

Bases de datos

Los servicios de base de datos son un conjunto de productos que proporcionan acceso a bases de datos relacionales y no relacionales, alojadas e integradas con otros servicios de GCP. Los productos de base de datos de GCP pueden residir con las mismas especificaciones regionales, identificarse y accederse mediante direcciones de red de Google, y estar protegidos por los servicios de seguridad de Google.

Existen bases de datos relacionales y no relacionales, y tienen el mismo rango: el modelo relacional ha sido el estándar de la industria durante décadas, y el lenguaje de consulta estructurado (SQL) puede describir y extraer modelos de datos complejos. Sin embargo, las bases de datos no relacionales ofrecen gran escalabilidad y mayor flexibilidad para admitir datos menos estructurados. Los siguientes productos de bases de datos representan ambos paradigmas:

- **Cloud SQL:** Cloud SQL es la interfaz genérica de GCP para muchos productos de bases de datos relacionales populares: PostgreSQL, MySQL y SQL Server. CloudSQL proporciona una capa de abstracción común a todos estos productos subyacentes, lo que permite que su aplicación sea independiente del proveedor. Además de esta API común, Cloud SQL proporciona funciones de gestión de bases de datos; las copias de seguridad específicas del proveedor, la replicación de bases de datos, la gestión de la capacidad (escalado del almacenamiento) y la aplicación de parches de software se realizan en su base de datos subyacente según lo especifique en una especificación de configuración independiente del proveedor.
- **Bigtable:** Bigtable es la base de datos NoSQL original, desarrollada y utilizada internamente en Google para sus propias necesidades de soporte de productos como la Búsqueda y el Acceso Distribuido mediante software como MapReduce. El esquema de estos productos era relativamente simple. Se necesitaba velocidad y la capacidad de leer y escribir filas extremadamente grandes. Los datos de BigTable no son una base de datos relacional; el modelo de datos es simple.

Claves=[Valor@Marca de tiempo] mapa.

Las bases de datos de Bigtable pueden distribuirse en cientos de miles de máquinas y almacenar petabytes de datos. El modelo de acceso de Bigtable puede requerir cierta adaptación para los desarrolladores acostumbrados a bases de datos relacionales, pero para un ajuste adecuado de los datos, el esquema se puede expresar de forma sencilla y el rendimiento es inigualable.

- **Firestore:** Firestore es una base de datos documental administrada, escalable y sin servidor. Sus documentos son compatibles con JSON y altamente consultables con índices de gran volumen. Firestore admite reglas de seguridad y validación, así como transacciones de Atomicidad, Consistencia, Aislamiento y Durabilidad (ACID), lo que garantiza la seguridad de los datos.

integridad.

El servicio Firestore es parte de la plataforma de desarrollo Firebase, que proporciona acceso directo a la base de datos de Firestore desde aplicaciones cliente en dispositivos móviles y la web.

Un producto sin servidor completamente administrado, el escalamiento y el mantenimiento de la base de datos de Firestore se realizan internamente, sin ninguna carga para el cliente.

- **Spanner:** Spanner es la base de datos más reciente de Google, que incorpora funciones y lecciones aprendidas de productos de generaciones anteriores como Bigtable. Spanner ofrece alta disponibilidad y un control excepcionalmente flexible sobre el escalado para adaptar la demanda de acceso y los requisitos de almacenamiento por separado. Las tareas de administración de la base de datos, como la fragmentación y el escalado de los recursos informáticos, se realizan automáticamente. Los cambios dinámicos de esquema se procesan en línea y se propagan mientras se atiende el tráfico, sin incurrir en tiempo de inactividad.

Las transacciones Spanner brindan fuertes garantías ACID y ordenamiento temporal independiente de la región (es decir, universal).

Se puede acceder a las tablas de Spanner usando GoogleSQL, un dialecto compatible con Google del lenguaje de consulta estructurado (SQL), pero Spanner también proporciona una interfaz PostgreSQL para facilitar la migración a Spanner y para soportar la migración de aplicaciones y experiencia de los desarrolladores.

Los productos

de **almacenamiento** ofrecen lo que tradicionalmente conocemos como espacio en disco.

Un servicio de almacenamiento permite guardar, catalogar y recuperar bloques u objetos independientes sin una estructura general o impuesta. Analicémoslos a continuación:

- **Almacenamiento en la nube:** El almacenamiento en la nube es un servicio administrado que almacena datos no estructurados en buckets con diferentes frecuencias de acceso. La frecuencia de acceso esperada y el tipo de almacenamiento en el almacenamiento en la nube determinan el costo y las garantías del servicio.

Las opciones de almacenamiento en la nube pueden ser de corta duración o muy duraderas y tienen vidas útiles máximas para los objetos.

Los depósitos de almacenamiento en la nube, aproximadamente análogos a los volúmenes, se pueden replicar para optimizar el rendimiento a través del acceso localizado o cercano, admitir la conmutación por error automática e invisible entre réplicas o garantizar la disponibilidad de los datos.

Para facilitar el desarrollo de aplicaciones y la manipulación directa del usuario, Cloud Storage FUSE puede hacer que sus buckets estén disponibles como un sistema de archivos local.

- Disco persistente: el servicio de disco persistente proporciona una interfaz de disco simple para almacenamiento en bloque redundante y de tamaño automático.

Los discos persistentes ofrecen opciones de gestión similares a las del paradigma de dispositivo de disco estándar, y a menudo intercambiables con este. Los PDD se pueden clonar, desconectar y conectar a diferentes máquinas, y se pueden cifrar con claves del sistema o proporcionadas por el usuario.

Los discos persistentes pueden estar respaldados por unidades de disco duro (HDD) o unidades de estado sólido (SSD) para adaptarse a los requisitos de rendimiento y costos del cliente.

- Almacenamiento en la nube para Firebase: El servicio Almacenamiento en la nube para Firebase facilita el almacenamiento y la distribución de contenido generado por el usuario. Orientado a la creación y entrega de clientes, Almacenamiento en la nube para Firebase proporciona kits de desarrollo de software (SDK) de cliente con todas las funciones necesarias para que los clientes móviles y web almacenen y recuperen directamente objetos binarios como vídeos, fotos y audio. Para minimizar el impacto de los problemas de conectividad del cliente, las operaciones de almacenamiento y recuperación son reanudables, reintentando una operación interrumpida desde el punto donde se interrumpió.

Para el procesamiento en segundo plano o asincrónico de sus objetos de Firebase, se puede usar la interfaz de programación de aplicaciones (API) de Google Cloud Storage para acceder a los objetos desde los componentes de GCP del lado del servidor.

Esto es útil cuando una tarea puede requerir más potencia computacional de la que se espera que tenga una plataforma cliente, como una transcodificación a gran escala o una preparación y escalado de imágenes.

Redes Los

productos de Redes de Google son una combinación de servicios y herramientas de conectividad que respaldan la amplitud y el rendimiento de la red que ofrece dicha conectividad. El despacho, el enrutamiento y el balanceo de carga informados, la localización de solicitudes y la reducción del consumo de ancho de banda son fundamentales para que una aplicación utilice la red de forma rentable si presta servicio a clientes en cualquier lugar de Internet, sin penalizaciones para ningún usuario según su ubicación. Google describe los servicios de red de Google Cloud como escalables a escala de One networkplanet. Analicémoslos en detalle:

- Balanceo de carga en la nube: El balanceo de carga en la nube le permite elegir el método mediante el cual los frontends de Google reenviarán las solicitudes a su aplicación. Dispone de varias opciones según el tipo de tráfico que recibe su aplicación, la ubicación de sus clientes y la amplitud con la que desea distribuir los trabajadores que reciben estas solicitudes.

Independientemente del tipo de balanceador de carga, el balanceo de carga de Google Cloud escala para procesar más de un millón de consultas por segundo (QPS). Los frontends y los balanceadores de carga utilizan las mismas tecnologías que la red de producción de Google, que da servicio a la Búsqueda, los Anuncios y YouTube.

- Redes de distribución de contenido (CDN) en la nube y multimedia : las CDN, también conocidas como redes Edge, son una capa de software por encima de las redes IP estándar y están especializadas para optimizar la distribución de contenido, como archivos de video y audio.

Ambos servicios CDN de GCP comparten el mismo principio operativo: identificar el contenido que se obtiene de una fuente remota en internet y crear una copia de dicho contenido lo más cerca posible del mayor número posible de clientes solicitantes. A medida que se detectan solicitudes posteriores de este mismo contenido, se interceptan y se entregan desde la copia cercana, evitando así la repetición de búsquedas redundantes desde ese punto de origen remoto. El resultado es una menor latencia para los usuarios y menores costos de red para la aplicación.

Media CDN utiliza la misma infraestructura que YouTube para acercar las transmisiones de video y las descargas de archivos grandes al usuario.

Cloud CDN utiliza la red perimetral de Google para almacenar y servir contenido desde una caché de Cloud CDN. La caché de la CDN utiliza el balanceador de carga en la nube de la aplicación para abastecerse de la fuente más cercana del contenido que ofrece. Cloud CDN se puede usar para cualquier respuesta almacenable en caché a solicitudes GET, según varios criterios. Normalmente, los tipos de contenido estático, como imágenes, vídeos, audio, archivos PDF y fuentes, son buenos candidatos para servir en la CDN.

- Firewall en la nube: GCP Cloud Firewall es un firewall de nueva generación que forma parte intrínseca del tejido de red de Google y está siempre presente. Es un firewall con modelo de inspección de estado que aplica reglas que usted selecciona o programa para inspeccionar todo el tráfico y evitar que el tráfico sospechoso o malicioso llegue a su aplicación.

Google selecciona y publica un conjunto de políticas globales que puedes ejecutar desde tus propias reglas de firewall para mantenerte al día con las amenazas emergentes y en evolución.

[Análisis de datos](#) La

suite de análisis de datos de Google incluye productos para realizar operaciones de análisis de datos en el modo GCP y varios productos de soporte que proporcionan infraestructura para dicha infraestructura.

Retrocedamos un poco y separemos estas dos capas de producto.

El análisis de datos, según la teoría, es el proceso general de examinar conjuntos de datos para identificar tendencias y luego sacar conclusiones sobre los datos que contienen, con el objetivo de explicar y/o pronosticar movimientos y tendencias dentro de los datos.

Aquí se presenta un modelo mental muy breve de una tarea de análisis de datos: derivadas multinivel en cálculo. Si su conjunto de datos de puntos de sincronización de geolocalización es una serie de valores de marca de tiempo y ubicación, puede derivar la velocidad y, a partir de esta serie de valores, calcular su segundo nivel.

derivada: aceleración.

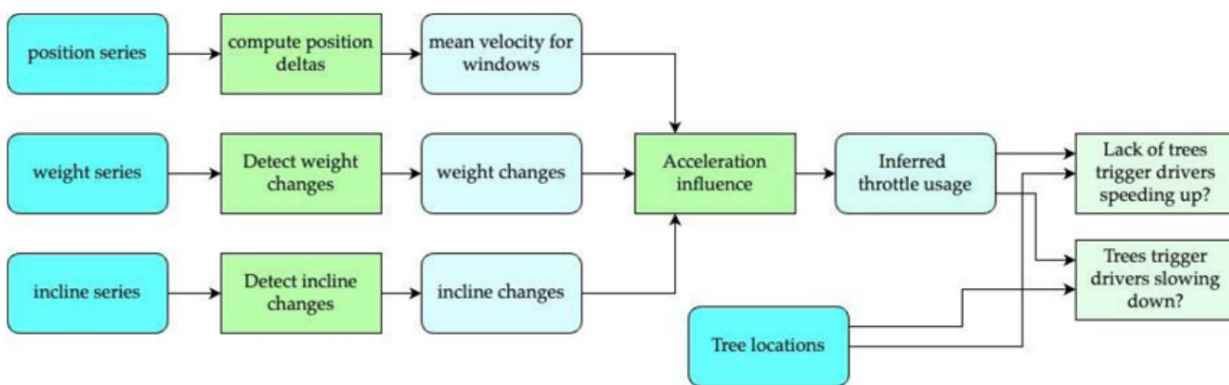
Imagine un proceso similar pero para el gasto del consumidor en un lugar o en un día específico de la semana, y luego encontrar la correlación entre las compras de dos artículos aparentemente independientes, cada uno en un día específico de la semana.

Este tipo de cálculo iterativo requiere una gran cantidad de cálculos. En las últimas dos décadas, este tipo de análisis ha pasado de libros de Excel aislados, quizás con algunas macros, a sistemas y lenguajes de programación altamente especializados.

Estos modelos de procesamiento no tienen una escala que permita ejecutarse bien incluso en las estaciones de trabajo más rápidas: tanto la potencia informática necesaria como el tamaño de los conjuntos de datos y el almacenamiento transitorio los hacen muy poco prácticos.

Otra cosa a tener en cuenta antes de la gran revelación que se avecina es que estas operaciones son iterativas: una secuencia de pasos que se basan en los resultados de los pasos anteriores.

Para ilustrar una versión ampliada del modelo anterior para ser utilizado en estudios de planificación vial y seguridad urbana, consulte la siguiente figura:



Cifra 1.3 Tubería para calcular el efecto de los árboles en las carreteras sobre el uso del acelerador

Cada uno de estos cálculos tiene dependencias y, por lo tanto, la secuencia es necesaria. Al describir un conjunto de operaciones dependientes como esta, se utiliza el término "tubería", en el que los datos que fluyen a través de los procesos sustituyen al agua.

Volviendo a los detalles del análisis de datos de GCP. Algunos productos de esta suite...

Se utilizan herramientas como BigQuery y Dataflow para especificar y realizar los cálculos de los valores objetivo: aceleración, uso del pedal del acelerador y ubicación del árbol, en comparación con el uso del pedal del acelerador. Otros productos, como Cloud Data Fusion, se utilizan para construir, ejecutar y gestionar las tuberías; son los encargados de instalar y mantener la tubería.

- **BigQuery:** BigQuery es una plataforma de análisis de datos que te ayuda a crear consultas que revelan información inherente pero oculta en tu conjunto de datos. Ofrece lenguajes de consulta, conectores para tus datos con lenguajes como Python y frameworks como Apache Spark, y operaciones de metadatos como catalogación de datos, seguimiento de linaje y creación de perfiles de datos.

Gemini en BigQuery proporciona asistencia basada en LLM para muchas de las tareas que realizará con las herramientas en BigQuery.

- **Fusión de datos en la nube:** La fusión de datos en la nube actúa como un nexo práctico, o quizás un mediador, entre las herramientas de análisis de datos como BigQuery y los grandes conjuntos de datos que procesan. Los datos de origen, esas grandes colecciones de puntos de datos recopilados, se almacenan, con suerte, en algún servicio disponible y duradero, como Cloud Storage o tablas de Cloud SQL. Los datos que la canalización calcula en cada paso intermedio, así como el resultado final, deben almacenarse en algún lugar.

Esa es la función de un producto de extracción, transformación y carga (ETL), como la fusión de datos en la nube. Fusion ofrece transformaciones y conexiones predefinidas y personalizadas que se integran en las canalizaciones que describimos anteriormente. Además de los componentes centrados en los datos, como los almacenes de datos de origen y de resultados, Fusion también especifica y gestiona los motores que realizan el trabajo: la capacidad de cómputo utilizada para realizar los cálculos que transforman los datos en cada paso de la canalización.

- **Flujo de datos:** El servicio Dataflow está especializado en brindar soporte de análisis de transmisión de datos en tiempo real y de baja latencia para que pueda detectar y reaccionar ante eventos a gran escala a medida que ocurren: las personas en Europa preguntan sobre los precios de las camisetas de fútbol cada vez más

Lo que pasó hace una hora es un ejemplo simple de algo que un minorista en línea debe tener en cuenta cuando sucede.

- Pub/Sub: Pub/Sub es el servicio de cola de mensajes de Google, y su clasificación como producto de análisis de datos refleja más su uso popular que su naturaleza. Pub/Sub en sí no se utiliza para realizar operaciones de análisis de datos, pero es una fuente muy práctica y común de la gran cantidad de datos que alimentan los canales de análisis de datos. Nuestro análisis de Pub/Sub se centrará más en sus capacidades generales y no en su uso con herramientas de análisis de datos.

Pub/Sub es una abreviatura simple de Publicar y Suscribirse, las dos operaciones principales que admite un servicio de mensajería.

Piensa en una lista de correo electrónico como un modelo de referencia para Pub/Sub. Te suscribes a una lista titulada "Acorazado espacial-Yamato-fandom" usando el botón de suscripción en tu sitio web favorito de anime clásico. Unas horas después, alguien más envía un correo electrónico a la lista contándoles a todos sobre un increíble nuevo diagrama técnico del Yamato que ha visto en un archivo histórico: acaban de publicar un mensaje.

Como suscriptor del tema de Space-Battleship-Yamato-fandom, recibirás este mensaje en tu bandeja de entrada. Probablemente no llegó inmediatamente después de su envío, ni lo esperabas; pero te llegará , junto con otras decenas de miles de personas felices con un gusto exquisito por el entretenimiento. Quien envió el mensaje, el editor, no se quedó a comprobar si todos lo recibieron; confía en que la infraestructura del correo electrónico funciona.

Ese es el modelo de roles y responsabilidades que implementa Pub/Sub, pero para datos más estructurados y acceso programático en lugar de correspondencia entre felices fanáticos del anime.

Pub/Sub define temas en un registro y permite que las aplicaciones cliente se suscriban a ellos. Los clientes publicadores publican mensajes dirigidos a un tema. Los suscriptores pueden enviar sus mensajes de interés de diferentes maneras: Push, Pull y

Exportar:

o Las suscripciones Push son aquellas para las que el suscriptor proporciona una devolución de llamada, un punto final para que Pub/Sub llame cuando hay un mensaje de interés.

Las suscripciones de extracción conservan sus mensajes hasta que se solicitan. Cuando están listas, el suscriptor se conecta a su tema de Pub/Sub y recupera todos los mensajes enviados a ese tema.

Las suscripciones de exportación sirven para gestionar por lotes grandes cantidades de mensajes. Los mensajes se añaden a un conjunto de datos de BigQuery o a un bucket de Cloud Storage para el procesamiento posterior que realice la aplicación. Esta es la superposición entre Pub/Sub y el análisis de datos mencionada anteriormente: Pub/Sub es una forma rápida y fiable de incorporar datos a una canalización de Analytics.

[Seguridad e identidad](#). La suite de

seguridad e identidad es una amplia colección de productos con la amplia misión de ayudarle a alcanzar sus objetivos políticos, regulatorios, y comerciales. Las ofertas en este ámbito se expanden y evolucionan constantemente debido a dos fuerzas externas irresistibles e imprevistas que impulsan este espacio de soluciones: los reguladores gubernamentales y la actividad delictiva.

Utilizados en conjunto, estos dos productos proporcionan los medios para un control de acceso seguro y para una gestión responsable de los controles utilizados para gestionar dicho acceso:

- Gestión de identidad y acceso: la gestión de identidad y acceso proporciona un control de acceso detallado a todos los recursos en la nube que componen su aplicación.

Hay dos clases principales de características: control de acceso y definición de roles y grupos.

Funciones de control de acceso para controlar quién puede acceder a qué y qué formas puede adoptar dicho acceso. Por ejemplo, muchas tareas de nuestra aplicación pueden escribir en los registros, pero no tienen una buena razón para leerlos, por lo que el acceso debe controlarse de esta manera. Acceso

Se puede registrar, bloquear o permitir el acceso a un recurso, según las reglas de acceso definidas para el recurso cuando se aplican a los usuarios que intentan acceder a él y a los roles y grupos definidos.

El apoyo a roles y grupos se basa en la práctica recomendada, de larga data y probada: no depender de un solo individuo, sino de un rol específico con las responsabilidades y privilegios adecuados, para el cual se han designado ciertas personas. Esto permite la redundancia: los suplentes, sustitutos y personas o equipos sustitutos pueden prevenir puntos de éxito únicos y aumentar los factores de lotería de un sistema. Esto también proporciona seguridad ante la movilidad: las personas pueden transferirse, jubilarse o ser despedidas, pero sus responsabilidades críticas se asignarán a alguien.

- Secret Manager: El servicio Secret Manager almacena sus datos más confidenciales: contraseñas, certificados, claves API y otros artefactos digitales críticos. Perderlos o revelarlos inadvertidamente es posiblemente peor que perder o revelar accidentalmente la información que protegen.

Los datos cifrados contienen muy poca información sin los medios para descifrarlos. Es muy poca, en lugar de ninguna, ya que aún pueden existir metadatos visibles, como el tamaño aproximado de un archivo cifrado. Las técnicas de cifrado robustas gozan de la suficiente confianza como para que un método común y aceptado de eliminación de datos sea simplemente destruir las claves necesarias para descifrarlos. Esta es una forma extraordinariamente eficiente de destruir, por ejemplo, todos los registros escritos en todos nuestros archivos en una semana determinada si rotamos nuestras claves de cifrado semanalmente. Ahora podemos ejecutar una operación de eliminación en millones de registros dentro de miles de archivos con solo eliminar un pequeño elemento: la clave de descifrado de esa semana. Esta también es una forma extraordinariamente fácil de perder todos nuestros datos comerciales de esa semana con esa pe

Por lo tanto, debemos ser particulares en dos cosas:

- No perder las llaves.
- Si tiras esas llaves intencionadamente, asegúrate de que realmente desaparezcan.

Hay una declaración adicional que hacer:

- ¡Sepa quién tiene las llaves!

Por eso utilizas Secret Manager.

La replicación, el control de acceso, el registro de auditoría y el cifrado de sus secretos son algunas de las características que mantienen sus secretos seguros.

Herramientas para

desarrolladores. Los productos de herramientas para desarrolladores de GCP eliminan un obstáculo importante para el desarrollo de proyectos grandes y ambiciosos: nuestras estaciones de trabajo de ingeniería. Las tareas de desarrollo, depuración e implementación de sistemas con una gran cantidad de tecnologías, componentes y entornos de destino de una aplicación en la nube sobrecargan las estaciones de trabajo más rápidas y saturan la conexión de red más rápida.

Desarrollar allí, en la nube, para la nube. Eso es lo que permiten las herramientas para desarrolladores en la nube. El entorno de desarrollo debe ser similar en rendimiento y escala al objetivo de implementación; de lo contrario, se estaría intentando construir una nave espacial con cuchillos de piedra y pieles de oso.

Desarrollar en la nube también democratiza la experiencia de desarrollo: un portátil modesto y ligero ahora es tan bueno como cualquier estación de trabajo de gama alta y mucho más fácil de llevar a un espacio de coworking, una cafetería o un parque.

Analicemos estas herramientas: SDK de la nube:

- El SDK de la nube proporciona un conjunto de herramientas de línea de comandos que permiten acceder a los recursos de GCP de forma similar a cómo las herramientas de línea de comandos existentes acceden a recursos locales como archivos, bases de datos y shells interactivos que se ejecutan en máquinas virtuales de cómputo.

Cloud SDK también incluye bibliotecas de cliente para muchos productos de GCP que le permiten desarrollar aplicaciones que acceden a servicios como administración de App Engine, Batch, Bigtable y Cloud Storage utilizando lenguajes familiares: clases y métodos en lenguajes como Python, Go y Java, en lugar de, por ejemplo, construir y analizar llamadas a API REST a través de HTTP.

La última categoría de características del SDK de Cloud sigue un modelo que se encuentra en

En cierto modo, una alternativa a las otras categorías: emuladores locales para servicios de datos, como Bigtable, Spanner, Pub/Sub, Cloud Firestore y Firestore. Esto permite un desarrollo y pruebas rápidos sin necesidad de configurar y rellenar previamente los datos reales de la nube.

- Estaciones de trabajo en la nube: las estaciones de trabajo de Google Cloud son máquinas virtuales configuradas con herramientas de desarrollo actuales, incluidos productos de GCP y otros, que residen en su entorno de GCP.

Son seguros, totalmente administrados y locales para sus otros recursos de GCP, lo que reduce la latencia y las vulnerabilidades introducidas por la red entre una estación de trabajo local y los recursos que utiliza. acceso.

Las estaciones de trabajo en la nube se gestionan y configuran completamente con opciones conocidas, y los entornos de los desarrolladores son reproducibles y consistentes. Esto minimiza un síndrome común, cómico pero frustrante: «Pero funciona en mi equipo».

- Integración de GitLab con Google Cloud: Los servicios de Google Cloud están completamente integrados con GitLab, lo que permite que un proyecto de GitLab se implemente directamente en Google Cloud. GitLab es una plataforma de desarrollo de software colaborativo de código abierto que incluye almacenamiento de código fuente. Si bien utilizan gran parte de la misma tecnología subyacente de Git, GitLab no es GitHub, sino que ofrece funciones de gestión de proyectos más completas.

Los desarrolladores que están familiarizados con GitHub encontrarán familiares los conceptos y características de control de versiones de GitLab.

- Cloud Build: Google Cloud Build es un sistema distribuido de integración continua/entrega continua.

Cloud Build integra decenas, cientos o miles de trabajadores de cómputo en una compilación específica, ejecutando cadenas de dependencia independientes en paralelo, lo que acelera considerablemente el tiempo de compilación e implementación. Cloud Build almacena en caché los artefactos de compilación intermedios, de modo que las compilaciones redundantes a menudo no son más que la recuperación de una referencia a un objeto almacenado en caché.

Cloud Build proporciona trazabilidad de linaje y otras garantías para

Lanzamiento de paquetes que permiten el cumplimiento de SLSA Nivel 3 y aumentan las recomendaciones o requisitos para industrias y casos de uso de alto impacto.

Consulte <https://slsa.dev> para obtener más información sobre los niveles de la cadena de suministro de artefactos de software.

Inteligencia artificial y aprendizaje automático. Los productos de inteligencia

artificial (IA) y aprendizaje automático (AA) son una combinación de aquellos que ofrecen funciones de IA/AA como Gemini, aquellos que dependen en gran medida de la funcionalidad de IA/AA como la IA de lenguaje natural y la conversión de voz a texto, y aquellos que se utilizan para gestionar artefactos de IA/AA; también conocidos como modelos. Aprendamos sobre estos: Conversión de voz a texto: El servicio de conversión de

- voz a texto proporciona una solución rápida, de alta calidad y ampliamente aplicable a un problema que ha sido desafiante desde la década de 1980: reconocer y transcribir el habla humana a partir de audio codificado.

Google Cloud Speech to Text envía un archivo de audio o una transmisión en tiempo real, como un micrófono en vivo, desde un cliente al servicio en la nube, que puede reconocer y transcribir el habla en más de 100 idiomas. La función de voz a texto puede identificar el idioma hablado con gran precisión tras escuchar los primeros segundos del audio.

- IA en lenguaje natural: la IA en lenguaje natural de Google es un conjunto de tres productos relacionados: Natural Language API, AutoML y Healthcare Natural Language AI.

La API de Lenguaje Natural de GCP aplica modelos de aprendizaje automático (ML) preentrenados a las transcripciones del habla humana para lograr la comprensión del lenguaje natural (NLU) de ese texto. La NLU extrae información sintáctica, como las categorías gramaticales, el reconocimiento de entidades, la identificación de nombres propios y el análisis de sentimientos, clasificando el tono emocional del discurso. La API de Lenguaje Natural se escala automáticamente para adaptarse al texto analizado y es capaz de analizar conversaciones en tiempo real.

AutoML le permite entrenar sus propios modelos de aprendizaje automático para clasificar texto en lenguaje natural y detectar sentimientos. Mientras que el lenguaje natural

La API de lenguaje existe para muchos casos de habla común, y AutoML se puede utilizar para optimizar el lenguaje natural (NLU) para jerga específica, roles de conversación o situaciones sociales.

- Google Cloud Healthcare: Natural Language AI ofrece soluciones de aprendizaje automático que aplican modelos de aprendizaje automático entrenados con diversos conceptos médicos, como enfermedades, medicamentos y procedimientos. El producto puede extraer información sobre estos conceptos del texto no estructurado de historiales médicos y asignarla al vocabulario estándar del sector médico.

Los modelos y la disponibilidad de funciones de la IA en lenguaje natural para el cuidado de la salud dependen de la ubicación en la que se utilizan, tanto por razones regulatorias como por vocabularios y estándares médicos localizados.

- Gemini para Google Cloud: Gemini es el producto de modelo de lenguaje público de gran tamaño de Google, entrenado con grandes cantidades de datos, contra el cual se pueden aplicar entradas de muchos tipos para inferir respuestas típicas o esperadas en función de patrones que coincidan con los datos de entrenamiento del modelo.

Gemini para Google Cloud proporciona una interfaz de chat para modelos entrenados específicamente en temas de Google Cloud, para brindar asesoramiento y sugerencias de mejores prácticas en el uso de los servicios de GCP.

Gemini está presente en muchos lugares de GCP, como Code Assist en las herramientas para desarrolladores de GCP, que ofrece sugerencias en forma de autocompletado e incluso proporciona definiciones de funciones basadas en comentarios de código y el uso y contexto circundantes.

Gemini en los productos de Google Workspace, como Gmail y Docs, proporciona asistencia para la escritura, genera posibles respuestas a mensajes de correo y sugiere fórmulas en hojas de cálculo.

- Vertex AI: GCP incluye varios productos Vertex AI, con la misión común de brindar soporte para la creación y el uso de IA generativa. Vertex AI incluye cientos de modelos preentrenados, incluyendo modelos de Gemini y modelos de terceros y abiertos. Con los productos Vertex, puede personalizar los modelos con entrenamiento adicional, ajustarlos a casos de uso específicos y monitorear su rendimiento.

rendimiento a lo largo del tiempo para detectar sesgos y desviaciones que podrían hacer que un modelo se desvíe de los comportamientos para los cuales fue entrenado y ajustado.

Vertex AI Agent Builder es un generador de agentes interactivo sin código que crea agentes de IA basados en modelos existentes y con acceso a los datos de la organización del cliente. Por ejemplo, un bot de atención al cliente puede consultar el catálogo e inventario de productos de su empresa.

Componentes fuera de los servicios en la nube Con esta exhaustiva lista

de productos disponibles en GCP, es posible que se pregunte qué más necesitará para construir su sistema de aplicaciones.

Abordemos los componentes no GCP de una aplicación distribuida por plataforma: • Código del lado del cliente:

No todo está en la nube: por ejemplo, el usuario. La mayoría de las aplicaciones aún interactúan con un operador humano que utiliza una plataforma informática con uno o más dispositivos de interfaz humana (HID): pantallas táctiles, monitores, teclados, ratones, acelerómetros y brújulas. Estas plataformas de cliente suelen tener menos potencia de procesamiento y almacenamiento que sus contrapartes en la nube/servidor, pero lo que les falta en potencia de procesamiento lo compensan con portabilidad y accesibilidad. Existen varios criterios para determinar si un componente o característica específica de una aplicación es adecuada para la residencia del lado del cliente: o Presentación: Mostrar al usuario información, ya sea visual, auditiva o háptica, requiere un dispositivo de salida adecuado: pantalla, altavoz y actuador de vibración, respectivamente, junto al usuario. El control preciso del dispositivo y la actualización en tiempo real de la salida son requisitos probables, por lo que es mejor ubicar el código responsable de estas salidas en el mismo dispositivo al que están conectados. o Entrada del usuario: la contraparte de las pantallas obtiene cierta información del usuario: pulsaciones de botones, entrada del teclado, micrófono o cámara y la dirección en la que apunta el teléfono.

Todos ellos están estrechamente vinculados a la presencia del usuario. Un controlador

y el recopilador de estos dispositivos que no residen en la plataforma del cliente será un desafío para el desarrollador y una decepción para los usuarios.

o Telemetría: Recopilar telemetría

como ubicación, niveles de luz y otras características del entorno del usuario es un problema similar al de obtener la entrada del usuario y, por extensión, de los argumentos allí proporcionados, se ejecuta mejor en el cliente.

Cálculos locales, de bajo coste pero muy sensibles a la latencia.

Existe toda una clase de cálculos que no requieren una demanda excesiva de computación, pero sí son muy sensibles a la latencia. En estos casos, el proceso de ida y vuelta de una solicitud a la nube puede tardar más que el proceso en el cliente. No está obligado a migrar toda la lógica de su aplicación a la nube. No dedique dos segundos a evitar un segundo de trabajo.

Dependencias de sistemas específicos de la industria o regulatorios: En muchas aplicaciones, se requerirá dependencia de sistemas proporcionados o exigidos por un gobierno u otra autoridad específica de la industria. Es probable que estos estén fuera de GCP y no sean negociables con respecto al protocolo de acceso y uso. Dichas dependencias suelen ser de dos formas: o Servicios externos: Un producto de software como servicio (SaaS) u otro

servicio remoto se puede abstraer detrás de una capa delgada en su arquitectura de GCP que sirve como proxy para el servicio externo real. El servicio de GCP que elija para implementar estos proxies está determinado por las particularidades del servicio: Varias funciones de red, como Google Cloud API Gateway, pueden implementar esto, o en casos más elaborados, se puede colocar una instancia de Cloud Functions o Cloud Run entre su aplicación principal y el servicio externo para realizar cualquier preprocesamiento mecánico y repetido de solicitudes o posprocesamiento de respuestas.

Binarios : Algunos proveedores externos proporcionan un binario compatible con un entorno de ejecución específico. Estos se pueden cargar en el almacenamiento en la nube o en un disco persistente y ejecutarse desde uno.

de los productos de servicio informático de contenedor o máquina virtual de GCP.

Conclusión

Ha llegado al final de este capítulo. Presentó una descripción general de las funciones de la plataforma de servicios en la nube y algunos de los productos de Google Cloud Platform que las ofrecen. Hemos revisado todo el panorama de una aplicación distribuida y analizado las consideraciones a tener en cuenta al decidir qué funciones alojar en la nube y cuáles implementar en el equipo cliente. Finalizamos con algunos casos de uso en los que es necesario incluir componentes que no se pueden implementar en el producto GCP más adecuado y cómo normalizarlos al máximo con el resto de la arquitectura del sistema.

A continuación, en [el capítulo](#) División de características de una aplicación respaldada en la nube, analizaremos en profundidad cómo dividir las características de una aplicación basada en la nube en componentes con un nivel de granularidad que nos permita basar cada uno de ellos en el servicio GCP u otra plataforma más adecuada.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 2

División de las características de una nube Aplicación respaldada

Introducción

Ahora que tenemos una idea de qué servicios nos ofrece Google Cloud Platform, lo que responde a la pregunta de la oferta, veamos qué requisitos presenta una aplicación distribuida, la cuestión de comprender la demanda.

En este capítulo, examinaremos los aspectos técnicos que pueden poseer las características de una aplicación y cómo se traducen o se corresponden con los requisitos técnicos de la infraestructura de soporte, como los servicios de GCP. Para ello, definiremos una gramática que describa las características que pueden aplicarse directamente o adaptarse a cualquier producto de software orientado al usuario.

Se describirán patrones de implementación comunes, como el escalamiento vertical y horizontal de recursos, y un método para determinar tanto su aplicabilidad como su necesidad real. Nuestra gramática, desarrollada para las características de la aplicación, se ampliará para incluir los aspectos relevantes de los proveedores de infraestructura.

Luego aplicaremos esta metodología que hemos desarrollado para seleccionar y designar servicios de GCP adecuados para respaldar las características de un entorno común.

Patrón de aplicación, compras interactivas.

Estructura

En este capítulo se tratarán los siguientes temas:

- Enumeración de responsabilidades y dependencias
- Agrupación por localidad topológica
- Agrupación por afinidad de características
- Agrupación por consideraciones de rendimiento de funciones
- Comprender los requisitos de las funciones
- Definición de la composición de las características de la aplicación
- Descripción de las opciones de infraestructura

Objetivos

El lector aprenderá a descomponer metódicamente las características de una aplicación y luego identificar los aspectos de una aplicación que son relevantes para la evaluación de las características de la plataforma de infraestructura.

Este capítulo desmitifica y define claramente varias características deseables de un marco distribuido, como GCP, brindando al lector una explicación clara de términos que a menudo se venden en términos demasiado vagos o incluso hiperbólicos.

Discutiremos y ejemplificaremos una metodología para diseñar una arquitectura de sistema que maximice el producto de las consideraciones de forma y funcionalidad .

Enumeración de responsabilidades y dependencias

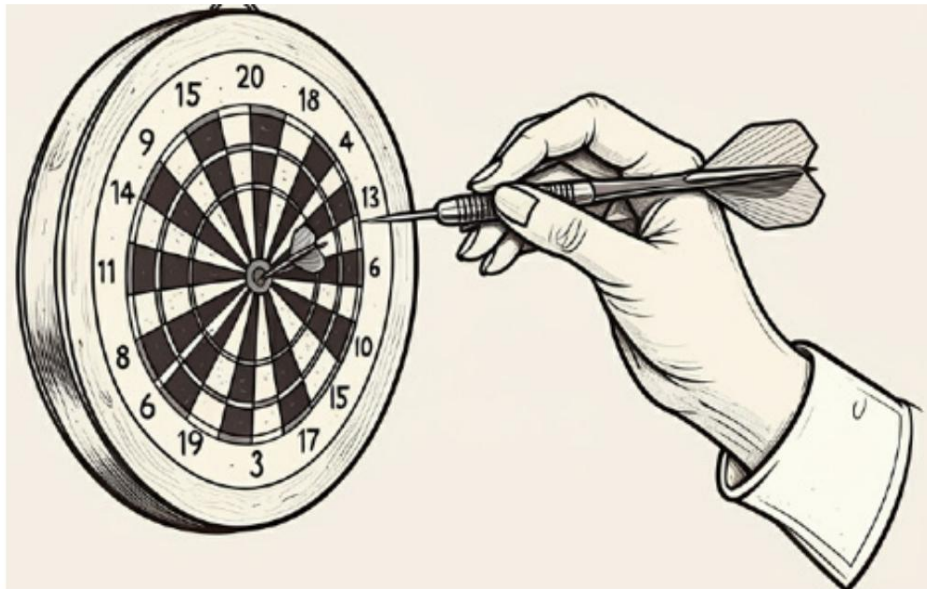
En teoría, cualquier plataforma informática moderna podría ejecutar cualquier código. Si bien es un experimento mental interesante, nuestro producto debe satisfacer a nuestros usuarios, y es cierto que algunas tareas se ejecutan mejor con nuestro teléfono o navegador web, otras con nuestro sistema de recopilación de registros nocturnos, y algunas tareas se realizan con mayor eficacia mediante un enjambre de trabajadores.

Haz el trabajo esta vez y cada vez.

Para decidir dónde realizar cada parte del trabajo general a realizar, estas decisiones de diseño deben fusionar dos conjuntos independientes de factores.

En primer lugar, están las consideraciones de plataforma sobre la forma, la forma o estructura interna que poseen los componentes de nuestro sistema. La segunda restricción de diseño es el alcance de nuestro producto: qué partes deben ser de nuestro sistema. Este es el conjunto de consideraciones funcionales que tenemos en cuenta. El producto de diseño ideal que mejor se adapta tanto a la forma como a la función, y este capítulo ilustrará un método para alcanzar ese resultado óptimo.

Para elegir entre soluciones alternativas para nuestros problemas competitivos de confiabilidad, escala y calidad de las características, nos vienen a la mente las siguientes técnicas históricas, probadas y verdaderas:



Cifra 2.1 Lanzando dardos

Habilidades no relacionadas, como un juego de dardos, o quizás uno de azar, como los dados:



Cifra 2.2 Una tirada de dados

También podríamos probar lo siguiente:



Cifra 2.3 Sesiones de lluvia de ideas

Si bien los dos primeros métodos ciertamente tienen sus puntos fuertes: facilidad de aprendizaje y una ejecución rápida, pero producen una fidelidad bastante baja.

Resultados. Podemos dedicar algo de tiempo, energía y talento para conseguir el trabajo.

Si se hace bien, una colaboración exploratoria basada en hechos es como lo haremos.

Tomamos nuestras decisiones de diseño.

Puntos de vista diferentes y válidos. Un concepto

importante, descuidado en los planes de estudio y la literatura de ingeniería de software, es uno planteado por un experimento mental conocido como el Barco de Teseo. Los detalles del experimento se pueden encontrar aquí:

[:// . . theseus/](#) El problema filosófico que aborda este experimento

es el de la constitución material, un aspecto del cual se puede resumir en la afirmación de que lo que algo es y de qué está hecho son separados y diferentes.

Forma y función: ambas son reales y cada una es crucial, pero cada una puede considerarse y definirse de forma aislada.

Entender cómo podemos definir tanto la forma como la función de una aplicación de software distribuida es el primer paso para definir nuestra metodología de diseño.

Comencemos con un glosario de términos relacionados con las medidas del rendimiento de un sistema, normalmente ligados a consideraciones de forma:

- Latencia: El tiempo que tarda el sistema en mostrar una respuesta significativa a un disparador de demanda
- Sensibilidad

a la latencia: La tolerancia de una aplicación a la demanda perceptible

Latencia en una respuesta. Clasificamos estos principalmente

como:

- o Altamente sensibles: Se nota un aumento en la latencia y se indeseable

- o Baja sensibilidad: Una variación en la latencia no tiene consecuencias.

- o Insensible a la latencia: No se detecta la latencia. Nadie cronometra la respuesta, siempre que la operación finalice.

- Gasto computacional: El número de operaciones, en términos de instrucciones ejecutables, que requiere una operación. Esto es análogo a la notación Big O del coste de tiempo de ejecución. Para nuestros propósitos, utilizaremos valores relativos dentro de nuestro sistema:

 - o Bajo (Trivial) o Medio

o Alto

- Paralelizable: Mide la eficacia con la que una tarea dada puede intercambiar espacio por tiempo. Si un trabajador tarda diez horas, ¿podrían diez trabajadores, cada uno del mismo tamaño que el original, realizar el trabajo en una hora? Algunos trabajos no son divisibles en secciones independientes, y las que sí lo son suelen incurrir en algún costo al dividir el trabajo y fusionar los resultados, por lo que en la mayoría de los casos no se logra un equilibrio directo.

Los grados de paralelización que asignaremos son:

o Ninguno.

o Ganancia baja: reducción sublineal en el tiempo para aumentos en el espacio. o Ganancia alta con techo: rendimientos decrecientes más allá de cierto número de trabajadores.

Alta ganancia, ilimitada: Un equilibrio lineal entre tiempo y espacio. A más trabajadores, menos tiempo.

Ahora, definamos un conjunto similar de términos centrados en aspectos del comportamiento que demuestra un sistema: •

Demanda: El desencadenante o instigador de la actividad en nuestro sistema. • Las clases comunes de demanda son:

o Solicitud de usuario: un usuario solicita que alguna aplicación realice alguna acción en respuesta a esa solicitud específica. o

Actividad acumulada que alcanza un umbral determinado: algunas actividades continuas y repetidas llenan un depósito hasta el punto en que realizamos alguna acción sobre su contenido.

o Un valor acumulativo que alcanza un umbral: un valor que aumenta con el tiempo y alcanza un valor significativo en el cual tomamos acción.

o Periódico (cada hora): Actividad regular programada, independientemente de actividad en el sistema.

o Acción administrativa: Alguna acción de mantenimiento o gobernanza que inician los operadores, no los usuarios finales, del sistema.

- **Afinidad de características:** Las características que exhibe una aplicación están todas conectadas de alguna manera, si realmente forman parte de la misma aplicación. La vinculación entre dos características es más fuerte o más débil que entre otras dos. Las características estrechamente acopladas deben implementarse mediante servicios más accesibles entre sí; en otras palabras, si el botón de votación genera una función "Añadir recuento" al candidato, esta función debe ser invocable desde el código de votación.
- **Proximidad topológica:** Similar a la afinidad de características, pero en cuanto a la plataforma, algunas características tienen los mismos requisitos de ejecución que otras y, por lo tanto, pertenecen a la misma plataforma. Un ejemplo práctico es obtener el voto del usuario y mostrar una notificación del candidato ganador. Ambas requieren la interacción con el usuario final de nuestra aplicación y se implementan mejor en la misma plataforma.
- **Duración del flujo de trabajo:** algunas tareas toman milisegundos y otras pueden ejecutarse durante horas antes de lograr algún progreso observable que pueda conservarse como un hito.

Clasificaremos la duración de una operación con las siguientes designaciones:

Inmediata: Una

- **operación se inicia, se ejecuta y finaliza sin demoras significativas ni bloqueos externos.** Las operaciones sin estado suelen completarse de inmediato.
- **Transacción:** Se debe completar un grupo de suboperaciones, generalmente en secuencia, antes de que se complete una operación. Si falla algún eslabón de la cadena, falla toda la operación, por lo que la vida útil de la operación principal es al menos la suma de la vida útil de las subtareas que la componen.
- **Sesión de usuario:** un usuario utiliza una aplicación para acceder a un conjunto de funciones, cuyo conjunto y secuencia exactos están determinados hasta cierto punto por el usuario individual. De larga
- **duración:** trabajo que puede iniciarse mediante un disparador o una solicitud, y que puede tardar más tiempo en completarse del que un usuario típico esperaría antes de pasar a alguna otra función.

- Tubería de múltiples etapas: una metaoperación que consta de muchas etapas, cada una de las cuales avanza el estado de la operación hacia su finalización.

Los términos restantes definidos son los que categorizan la funcionalidad de las características de una aplicación, con algunos ejemplos para cada uno:

- Conexión del sistema: inicio de sesión, cierre de sesión, conexión a otra plataforma o servicio sobre el que se basa una aplicación.
- Solicitud del usuario: El núcleo de una aplicación interactiva. Realizar una compra, enviar un mensaje y votar son funciones que atraen a los usuarios a nuestro producto y que les permiten solicitar directamente.
- Sesión de usuario: recuperar un perfil de usuario, recuperar el historial de un usuario o la actividad del sistema.
- Telemetría del cliente: Operaciones del cliente no solicitadas explícitamente por el usuario. Mantenimiento de la conectividad, actualización de la ubicación u otros datos del entorno.
- Entrega asincrónica: recepción de actualizaciones de clientes, notificaciones push y respuestas asincrónicas a desencadenantes de demanda.
- Administración y control de usuarios: Cambiar permisos de usuario o recursos.
- Administración y control del sistema: cambiar los recursos o la disponibilidad de todo el sistema.
- Informes de análisis de datos bajo demanda: Visualización del estado del sistema en tiempo real. Número de usuarios, tasa de solicitudes, latencia e inventario restante.
- Informes periódicos de análisis de datos sin conexión : Estado agregado del sistema, calculado en un momento dado. Las ventas trimestrales, el número de suscriptores mensual y la actividad diaria de los usuarios son ejemplos de datos que carecen de sentido y resultan prohibitivamente costosos de procesar bajo demanda en tiempo real.

Con el glosario de términos definido, necesitamos una forma de aplicarlos.

Nota: La siguiente discusión se refiere a un subconjunto de características potenciales de un

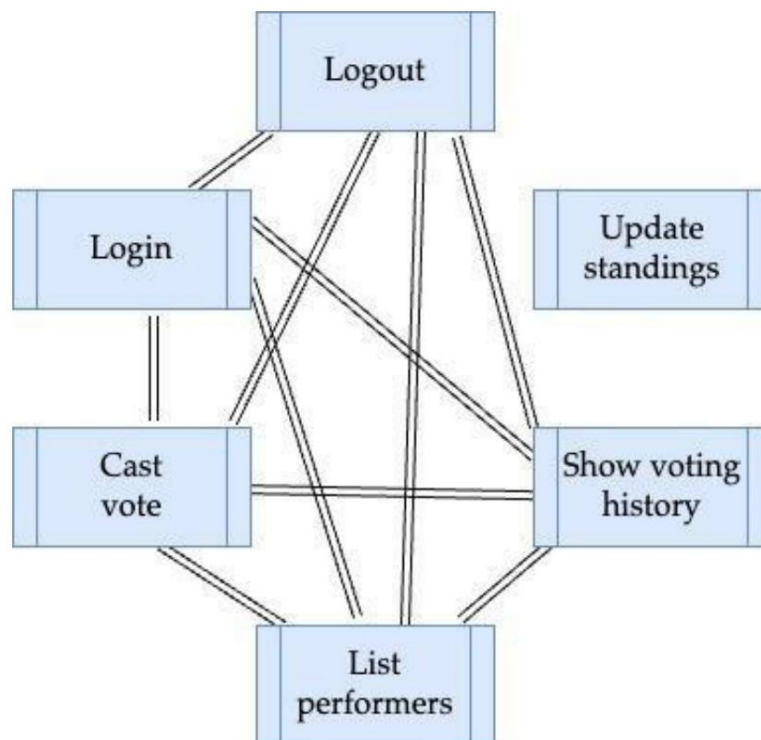
Sistema de votación para ilustrar la metodología explicada. Se realizará una exploración completa de dicha aplicación en capítulos posteriores.

Agrupación por localidad topológica

Los dos aspectos de las características de la aplicación descritos anteriormente, la afinidad y la proximidad, se distinguen de los demás. A diferencia de los demás atributos de las características descritos, estos no son características de ninguna característica en particular, sino relaciones entre ellas. Una forma práctica de visualizar dicha información para un conjunto potencialmente grande de características se utiliza comúnmente en el diseño de bases de datos relacionales.

Un gráfico de red o diagrama de nodos-enlace muestra principalmente las relaciones entre entidades, en lugar de centrarse en las entidades y su composición. Podemos visualizar nuestras características de afinidad como un gráfico de red donde los nodos son las características de la aplicación y los bordes son las relaciones entre ellas.

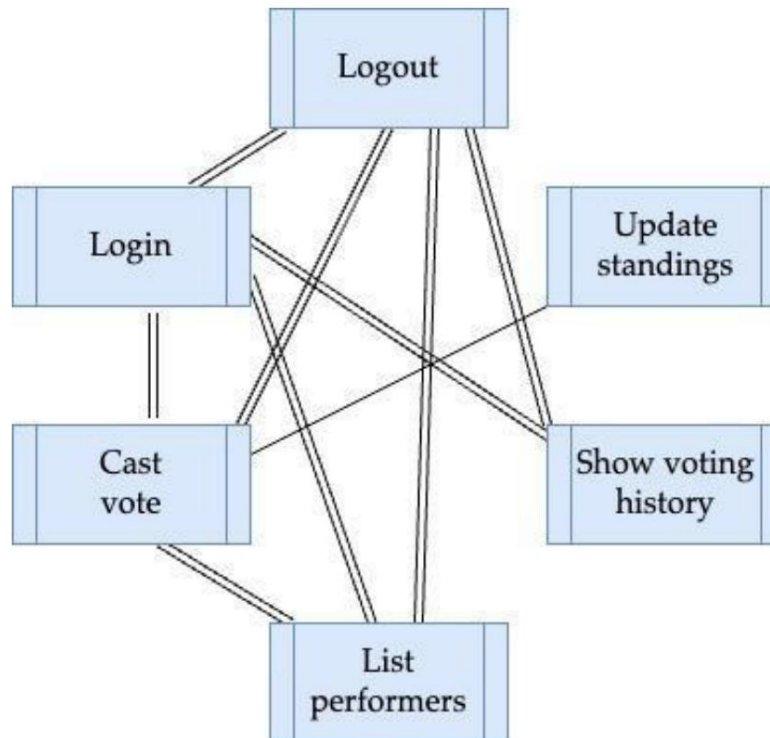
Aquí hay un gráfico de la proximidad topológica de las características de nuestra aplicación:



Cifra 2.4 Gráfico de proximidad de características

Agrupación por afinidad de características

El siguiente gráfico revela la afinidad de características dentro de nuestra aplicación:



Cifra 2.5 Gráfico de afinidad de características

La fuerza de la relación entre los dos nodos se representa mediante el peso relativo o grosor de la línea utilizada para dibujar el Conexión. En la figura anterior, la función de inicio de sesión está estrechamente vinculada En la operación de Voto Emitido , se trata de una acción siguiente frecuente; mientras que Existe una relación tenue entre emitir un voto y actualizar clasificación, que se invoca de forma asincrónica después de una serie de votaciones están echados.

Determinación de la afinidad y proximidad de las características

Al observar la Figura 2.5, vemos que la función de inicio de sesión tiene una gran Afinidad con la función de voto emitido, lo cual es evidente ya que normalmente

Después de iniciar sesión, emita un voto. Además, no podrá votar hasta que haya iniciado sesión correctamente en el mismo dispositivo cliente, como lo demuestra la fuerte proximidad de las funciones "Iniciar sesión" y "Emitir voto" en [la Figura 24](#). No iniciamos sesión en un dispositivo y luego votamos en otro.

Ahora tenemos el siguiente vocabulario que podemos usar para discutir nuestra aplicación desde varios puntos de vista: La

- geografía de la aplicación: Las características que son vecinas en el espacio.
- La afinidad de características: Las características que son vecinas en el tiempo.
- Cuestiones de coste como producto del espacio y el tiempo necesarios para ejecutar una característica.
- La inmediatez de la visibilidad de una función para el disparador o solicitante de activación.
- La atomicidad del funcionamiento de una característica: el ciclo de vida y el efecto persistente de la ejecución de una característica.

Utilizaremos estas herramientas para definir los requisitos de la arquitectura de nuestro sistema, mapeados a un conjunto de capacidades de la plataforma. Es inevitable que se hagan concesiones en cualquier producto que se seleccione, pero siempre habrá una opción que se ajuste mejor a los requisitos. Reducir las características de nuestra aplicación a componentes más discretos y atómicos contribuye al objetivo de poder mapear dichas características de la forma más directa posible a Google Cloud Platform (GCP) y a servicios de infraestructura no basados en GCP.

Agrupación por consideraciones de rendimiento de funciones

El siguiente paso en este proceso es profundizar un poco más en las categorizaciones o etiquetas de características restantes que hemos identificado.

Una tabla de consideraciones de rendimiento centradas en las características nos permite ver la composición general de nuestra aplicación. Definamos una con un subconjunto de características de la aplicación como ejemplo:

Feature	⊖ Demand driver	⊖ Class of functionality	⊖ Latency sensitivity	⊖ Computational expense	⊖ Parallelizable	⊖ Duration
Login	User request	System connection	High	Medium	None	User session
Cast vote	User request	User request	High	Low	None	Transaction
List performers	User request	User request	High	Low	None	Immediate
Update standings	Accumulated activity	Asynchronous delivery	Low	Medium	High, ceiling	Multi-stage pipeline
Show voting history	Admin action	User admin	Low	Medium	High, ceiling	Long running
Logout	User request	Session Management	High	Low	None	Immediate

Cifra 2.6 Características de rendimiento de las funciones

La figura anterior le ayudará a ver y comparar los diferentes valores de cada característica en paralelo. Facilita la realización de afirmaciones necesarias, como: « El factor más común que impulsa las características de nuestra aplicación es la demanda del usuario». Además, afirmaciones más complejas con un dominio más amplio, como: « Nuestra aplicación no puede usar la paralelización para reducir la latencia de las solicitudes de usuario en la función, Emitir Voto, pero sí puede emplearse para escalar la función Actualizar Clasificación».

Comprender los requisitos de las funciones

Imaginemos nuestro sistema en su conjunto, en un momento excepcional en el que nadie ha iniciado sesión. No hay votantes conectados al sistema desde su aplicación móvil o nuestra página web, y actualmente no se están ejecutando funciones impulsadas por solicitudes de los usuarios.

Aunque el sistema no está en absoluto en calma. Los registros de las aplicaciones se están anonimizando y agregando, y nuestro flujo de análisis de datos está recopilando todos los votos emitidos a nivel mundial durante el último día. En nuestra oficina europea, nuestros gerentes de producto están compilando el informe del historial de votaciones para la revisión mensual de negocios de mañana. Gracias a la configuración de nuestro servicio en la nube, la replicación está copiando nuestros almacenes de datos regionales a sus equivalentes globales para garantizar la fiabilidad y un mejor rendimiento de las consultas realizadas en cualquier región.

Todas estas son funciones importantes de nuestro sistema, pero ninguna de ellas es solicitada por nuestros usuarios finales. Por lo tanto, aplazaremos su análisis y analizaremos primero las funciones que se activan a petición de los usuarios.

Comencemos con la función de inicio de sesión. Un usuario final puede comenzar a usar la aplicación iniciando sesión, un proceso que implica identificarse como un usuario conocido y luego autenticar esa identidad. Este proceso...

No se inicia sin que el usuario lo inicie. No conectamos usuarios en teléfonos aleatorios en plena noche.

El inicio de sesión comienza cuando un usuario pulsa un icono o hace clic en un enlace. El usuario solicitó explícitamente que el sistema lo autentificara para poder acceder a otras funciones.

Hay algunas cosas que podemos asumir con seguridad sobre todas las funciones de Solicitud de usuario:

- La cantidad de solicitudes de usuario simultáneas que el sistema debe anticipar es $O(N)$, donde N = número de usuarios conectados, en circunstancias normales.
- Cualquier función activada por una solicitud del usuario debe ser sensible a la latencia.
- La mayoría de las funciones de solicitud de usuario tendrán afinidad con algunas otras funciones de solicitud de usuario; las aplicaciones de usuario final de función única son raras, aunque no inauditas.

Al examinar otra función, emitir voto, la hemos designado con una clase de funcionalidad de solicitud de usuario, a diferencia de la función de inicio de sesión, que pertenece a la clase de conexión del sistema. La distinción radica en la intención: el impulsor de la demanda describe cómo se inicia la acción, mientras que la clase de funcionalidad describe con mayor precisión la razón de ser de la función y su propósito. Si bien iniciar sesión es algo que todos sabemos que debemos hacer, emitir un voto es la función más importante que ofrecemos: emitir votos es la razón de ser de nuestro sistema de votación y, por lo tanto, es una solicitud del usuario en el sentido estricto del término.

La función de emitir voto es muy sensible a la latencia. Sería inquietante y reduciría la confianza en nuestro sistema si los usuarios emitieran su voto y no recibieran una confirmación de su registro.

Este reconocimiento debe darse con bastante rapidez, ya que cuanto más lo demoremos, más probable será que el teléfono pierda su señal, el navegador se bloquee o el usuario cierre la ventana de forma impaciente o accidental.

El gasto computacional de emitir un voto no es mucho para el registro real de un voto. Recordemos el ejemplo del pago con tarjeta de crédito del capítulo anterior: no tenemos que contabilizar todos los votos, actualizar todos

Copias de nuestros registros y calcular la posición de todos los candidatos. Lo que debemos hacer es garantizar que el voto del usuario se haya registrado y que su efecto se note. La respuesta al emitir un voto es reconocer que su elección está en el sistema y no tiene que preocuparse, en lugar de que su voto haya pasado por nuestro sistema y se haya fusionado con todos los demás votos emitidos, y se refleje en todos los datos derivados de las votaciones. Esto último, transferir la solicitud del usuario a un mecanismo duradero en el sistema, puede ser económico si utilizamos el servicio de producto en la nube adecuado.

Por lo tanto, el gasto computacional que supone emitir un voto es bajo.

Consideremos cuán paralelizable es emitir un voto, dado el alcance limitado de la función que hemos establecido. Hay una función en el cliente que se invoca al tocar un ícono: su principal responsabilidad es transferir el voto a otro componente duradero de nuestra aplicación que no esté vinculado a la plataforma cliente, que es relativamente frágil y lenta. Imaginemos algo como esto en la aplicación cliente: `void handoff_selection(int vote_index)`

```
{ if (vote_index <= 0 || vote_index > this-
  >candidate_count) { throw std::invalid_argument(" Voto
    recibido para una opción no válida"); } int errno = this-

  >vote_recorder->accept_vote(vote_index); if (errno)
  { throw
    std::system_error("Error al inyectar. Voto no registrado."); } }
```

Hay una operación crítica para el propósito de este método: la invocación de `accept_vote()`. Este método no tiene múltiples pasos de procesamiento independientes que se distribuyan entre varios trabajadores para su ejecución simultánea. Al analizar la instrucción que hemos identificado como crítica (`accept_vote(...)`), solo le pasamos un argumento, un entero escalar, que no presenta ningún método para dividir los datos para su consumo entre más de una instancia.

Trabajador. Por lo tanto, hemos etiquetado el valor paralelizable de esta función como Ninguno, ya que no hay razón ni oportunidad para usar más de un trabajador para reducir el tiempo necesario para ejecutar esta función.

Finalmente, la duración de emitir un voto es el tiempo que nos toma validar la opción que nos fue dada; el tiempo que luego nos toma invocar y obtener una respuesta del método `de nuestro servicio vote_recorder_` y el tiempo que nos lleva confirmar la respuesta.

Mirando nuevamente nuestra descripción de la duración del flujo de trabajo, esto coincide con la descripción de la Transacción.

Definición de la composición de las características de la aplicación

El proceso de dividir la funcionalidad de una aplicación en componentes claramente delimitados con una topografía óptima para el rendimiento consta de una secuencia de tres pasos principales, que ahora describiremos:

1. Enumerar características: primero, debes dedicar algún tiempo a identificar las características de tu producto, aquellas operaciones que pondrá a disposición para satisfacer tus casos de usuario o requisitos funcionales.

A modo de ejemplo, enumeraremos las características de una aplicación de dominio común: un sistema de compras para un comercio minorista. Dicha aplicación podría incluir las siguientes funciones: a. Iniciar sesión

en la cuenta de compras. b. Crear una

nueva cuenta de compras. c. Añadir un

artículo del inventario al carrito. d. Iniciar la

actualización del inventario del artículo. e. Revisar la

actualización del inventario. f.

Solicitar la reposición del inventario del

artículo. g. Pagar el impuesto sobre las ventas.

2. Determine los atributos de rendimiento de cada característica: Esto no es trivial, pero con un experto en la materia y un arquitecto de sistemas presente, puede ser un proceso sencillo. Diseñe una tabla como la siguiente:

uno en la figura 2.4 con una fila para cada una de las características que tienes identificado.

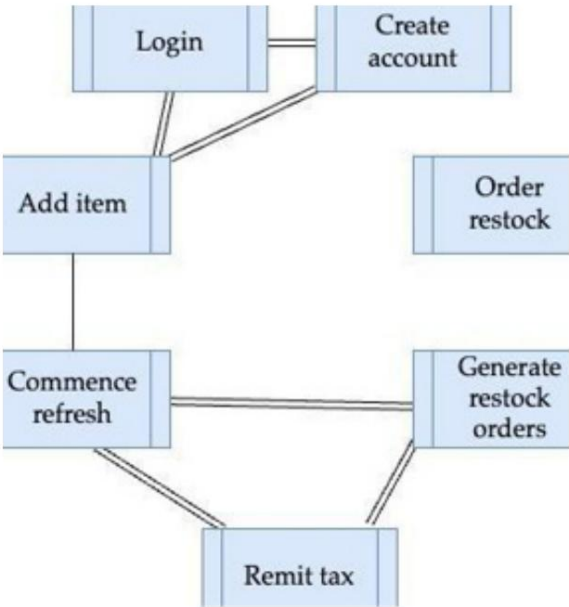
Prepárese para volver atrás y revisar las características a medida que las discute. adicionales; después de describir la característica de actualización inventario, puede darse cuenta de que agregar un artículo al carrito es más caro o de mayor alcance del que inicialmente pensaste.

La siguiente figura describe las siete características que tenemos identificados para nuestro sistema de compras minoristas:

Feature	⊙ Demand driver	⊙ Class of functionality	⊙ Latency sensitivity	⊙ Computational expense	⊙ Parallelizable	⊙ Duration
Shopping Login	User request	System connection	High	Medium	None	User session
Create new account	User request	User request	High	Medium	None	Transaction
Add item from inventory	User request	User request	High	Medium	Low	Transaction
Commence inventory item refresh	Accumulated activity	Asynchronous delivery	Insensitive	Low	None	Transaction
Generate inventory restock orders	Cumulative value	Offline batch	Insensitive	Medium	Low	Multi-stage pipeline
Order inventory item restock	Admin action	User request	Low	Medium	High, ceiling	Long running
Remit sales tax	Periodic	Offline batch	Insensitive	Low	Low	Long running

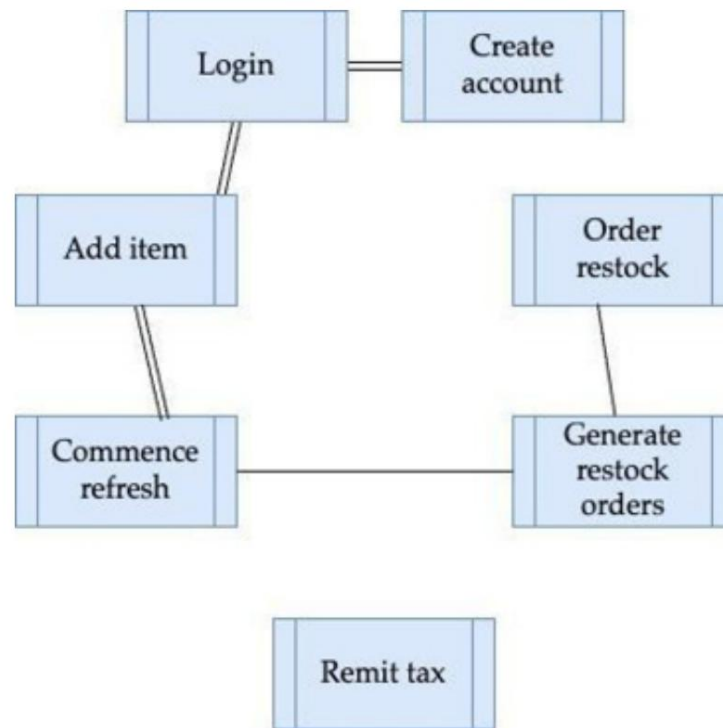
Cifra 2.7 Características de la aplicación de compras

3. Colocar las características: Con las características, su alcance y su costo descritos; a continuación ilustramos su adyacencia en el espacio y en el tiempo. La proximidad de estas siete características se muestra en la siguiente red diagrama:



Cifra 2.8 Proximidad de las funciones de la aplicación de compras

El siguiente diagrama de red ilustra la afinidad de las mismas siete características:



Cifra 2.9 Afinidad de las características de la aplicación de compras

En este punto, comprendemos bien los requisitos técnicos de las características descritas. Podemos considerar esto como la demanda de nuestra infraestructura. Ahora que podemos definir la demanda, el siguiente paso es establecer un vocabulario y una gramática equivalentes para la oferta: la infraestructura que esperamos que satisfaga dichos requisitos técnicos.

Cualidades pertinentes de un servicio o componente de infraestructura. Existe una gran cantidad de

componentes básicos de infraestructura disponibles para que construyamos nuestras diversas funciones, y rara vez hay una opción absolutamente correcta o incorrecta: se podría usar una máquina virtual de larga duración para ejecutar un servidor de solicitudes o se podría implementar un conjunto de funciones discretas en la nube. La selección no es binaria, sino de grado, en

Varios ejes. Repasemos ahora algunas de esas dimensiones:

- **Rendimiento:** El tiempo transcurrido que una plataforma informática determinada requiere para completar la ejecución de nuestra carga de trabajo. No todo requiere una supercomputadora. Una máxima popular al comprar una laptop nueva es "no voy a plegar proteínas en esta". Las plataformas informáticas sin servidor comparten el espacio físico de proceso y la potencia entre (muchos) pares, y normalmente no se obtendrá la misma velocidad con una décima parte de un núcleo de procesador que con tres núcleos dedicados a su proceso. Esto puede ser suficiente, o puede que tenga algunas tareas de computación intensivas que hagan que cada milisegundo de tiempo de espera cuente. Algunos servicios pueden tener una garantía de calidad de servicio, y otros pueden ser de máximo esfuerzo. Conocer la garantía de rendimiento de cada opción nos permite ser flexibles con las funciones disponibles y dedicar las opciones de mayor rendimiento a las cargas de trabajo que las necesitamos.
- **Escalabilidad horizontal y vertical:** El esfuerzo necesario para iniciar un aumento o disminución de la capacidad de un servicio y la velocidad con la que dicho cambio se hace efectivo. Activar nuevas máquinas virtuales, registrarlas con balanceadores de carga y retirarlas del servicio no es algo que se pueda lograr en segundos, mientras que las solicitudes más concurrentes a funciones en la nube o aplicaciones gestionadas sin servidor pueden gestionarse con la suficiente rapidez como para atender de forma transparente picos repentinos de tráfico.

Por otro lado, cada una de esas muchas funciones en la nube solo tendrá cierta capacidad por sí sola, mientras que una máquina virtual con un procesador más potente y más RAM puede asumir tareas más grandes.

- **Costo:** El costo material de una plataforma por unidad de potencia de cómputo efectiva. Cuantos más recursos reales consuma un servicio de su proveedor de nube, más le cobrarán por él.
Al final, no puede ignorar las cuestiones prácticas por mucho tiempo; el producto más económico que satisfaga sus necesidades debe considerarse la mejor opción. Recuerde considerar los costos adicionales: un producto más económico que requiere añadir otro.

El producto complementario puede no ser la opción más barata en comparación con un alternativa costosa pero completamente autónoma.

- Soporte a largo plazo: El período durante el cual su inversión en un El producto será rentable en comparación con otras alternativas. productos. La obsolescencia es lo peor, inesperado pero inevitable. desafío que puede interrumpir el ciclo de vida del desarrollo de un producto. Después meses de descubrimiento e investigación, semanas de formación y trimestres Durante el desarrollo y la implementación, un equipo de producto espera centrarse para satisfacer las solicitudes de funciones de los usuarios y analistas del mercado. No Se acoge con satisfacción un plan de migración forzada anunciado incluso en el términos más alegres y alentadores. Considere lo siguiente comunicado como plantilla de un anuncio que podría ser enviado por un producto del cual su aplicación tiene una fuerte dependencia En. Alguna versión de este aviso genérico ha sido una noticia no deseada. Una sorpresa para muchos desarrolladores de aplicaciones es que sustituir cualquier productos y empresa para gadgets, widgets y todo lo demás. anuncio de desuso de un servicio, ya sea relacionado con la nube o no, infraestructura o industria específica, generalmente sigue el formato de esta carta ficticia para el servicio de gadget genérico, y da como con poco aviso y pocas opciones:

: usuarios de gadgets@

ParaDeSoporte@todo-lo-que-quiera

AsuntoAnuncio de nuestro nuevo producto de implementación de componentes

—Widget diseñado para el !

futuroEstimado cliente

Nos complace presentarles Widget-service, un reemplazo , nuestro nuevo, emocionante y revolucionario para Gadget-service. .

Durante los próximos seis meses brindaremos soporte para su

Migración a esta cosa brillante, moderna y llena de características.: Todo

Se recomienda a los clientes de Gadget-service migrar a Widget-service

en marzo, momento 2025, que trasladaremos el soporte de Gadget.

al máximo esfuerzo durante un mes.

...

La rotación de infraestructura puede considerarse un costo asociado al producto. Una vida útil más corta implica un mayor costo indirecto: factores como la capacitación, la reingeniería y la redistribución, inevitables en las migraciones de infraestructura más transparentes y fluidas, que se amortizan en menos tiempo.

Nota: Por esta razón, muchas distribuciones de Linux ofrecen versiones con la etiqueta de Soporte a Largo Plazo (LTS). Muchos clientes importantes de estas distribuciones solo utilizan versiones LTS, ya que una actualización del sistema operativo es costosa, compleja y puede afectar negativamente a toda la operación. Por lo tanto, conviene realizarla con la menor frecuencia posible y solo después de que la versión haya estado en uso y se haya revisado durante un período de tiempo considerable.

Descripción de las opciones de infraestructura

Con estas consideraciones principales de un producto de infraestructura identificadas, describamos un proceso para comparar las alternativas que ofrece nuestra plataforma seleccionada: 1. Enumerar

las opciones: Al igual que el paso de identificar nuestras características anterior, esto simplemente consiste en enumerar los productos que estamos evaluando.

Para abreviar, restringiremos nuestras opciones a un subconjunto de los productos de GCP y las plataformas que no son de nube que revisamos en el capítulo anterior.

a. Google Cloud Run b.

Cloud Functions

c. Pub/Sub d.

Lote

e. Marco de aplicación de cliente móvil. f.

Desarrollo nativo de plataforma de cliente móvil. g.

Gestión de identidad y acceso.

2. Describe las cualidades de cada producto: Usemos el mismo formato de tabla que tenemos para las características: se adapta bien a la tarea, proporciona consistencia en nuestros procesos y ofrece la

conveniencia de la familiaridad:

Product	⊙ Performance	⊙ Horizontal scalability	⊙ Vertical Scalability	⊙ Cost	⊙ Long Term Support
Google Cloud Run	Medium	Highly scalable with demand	Configurable performance options	Relatively inexpensive	Long future roadmap
Cloud Functions	Medium	Highly scalable with demand	Fixed or limited performance options	Relatively inexpensive	Long future roadmap
Batch	Medium	Scalable, not transparent	Configurable performance options	Relatively inexpensive	Long future roadmap
Pub/Sub	Highly performant	Highly scalable with demand	Fixed or limited performance options	Medium cost	Long future roadmap
Mobile client platform native platform	Medium	Low scalability	Fixed or limited performance options	Relatively inexpensive	Newer options exist and promoted
Mobile client app framework	Low / Best effort	Low scalability	Fixed or limited performance options	Relatively inexpensive	Long future roadmap
Identity and Access Management	Highly performant	Highly scalable with demand	Fixed or limited performance options	Relatively inexpensive	Long future roadmap

Cifra **2.10** Tabla de calidades de productos

Es importante reconocer que la figura anterior es una comparación de peras y manzanas: IAM y lote son productos muy diferentes, lo cual se considerará en el siguiente paso de nuestro proceso.

- Adaptación de las características a la infraestructura adecuada: Con este análisis de la oferta, lo que ofrece cada producto, junto con el de la demanda y los requisitos de cada característica, podemos tomar decisiones razonablemente acertadas sobre la infraestructura que respalda cada una de nuestras características. El valor de un método como este, basado en la microeconomía, reside en que permite tomar decisiones de infraestructura basadas en las necesidades del sistema, en lugar de en un sesgo particular del arquitecto o del proveedor de la plataforma.

Dado que nuestras características se definen con los límites de la responsabilidad funcional, no con los límites de ningún componente técnico en mente, es probable (o seguro) que no habrá una correspondencia uno a uno entre el producto de infraestructura y la característica, sino una combinación de productos de infraestructura que serán útiles para implementar una característica determinada.

Ahora, repasaremos el proceso de emparejamiento: combinar un par de características con productos de infraestructura y buscar la combinación de requisitos y capacidades que tenga el menor costo posible.

¿Cuál es una buena plataforma para la función de inicio de sesión del usuario? La [Figura 2.7](#), muestra que tiene bajos requisitos de rendimiento computacional, proximidad a la plataforma cliente y es sensible a la latencia. Un framework de aplicación cliente cumple con los requisitos de proximidad, latencia y computacional. Podemos intuir con seguridad que también se necesita IAM para soportar una función de inicio de sesión.

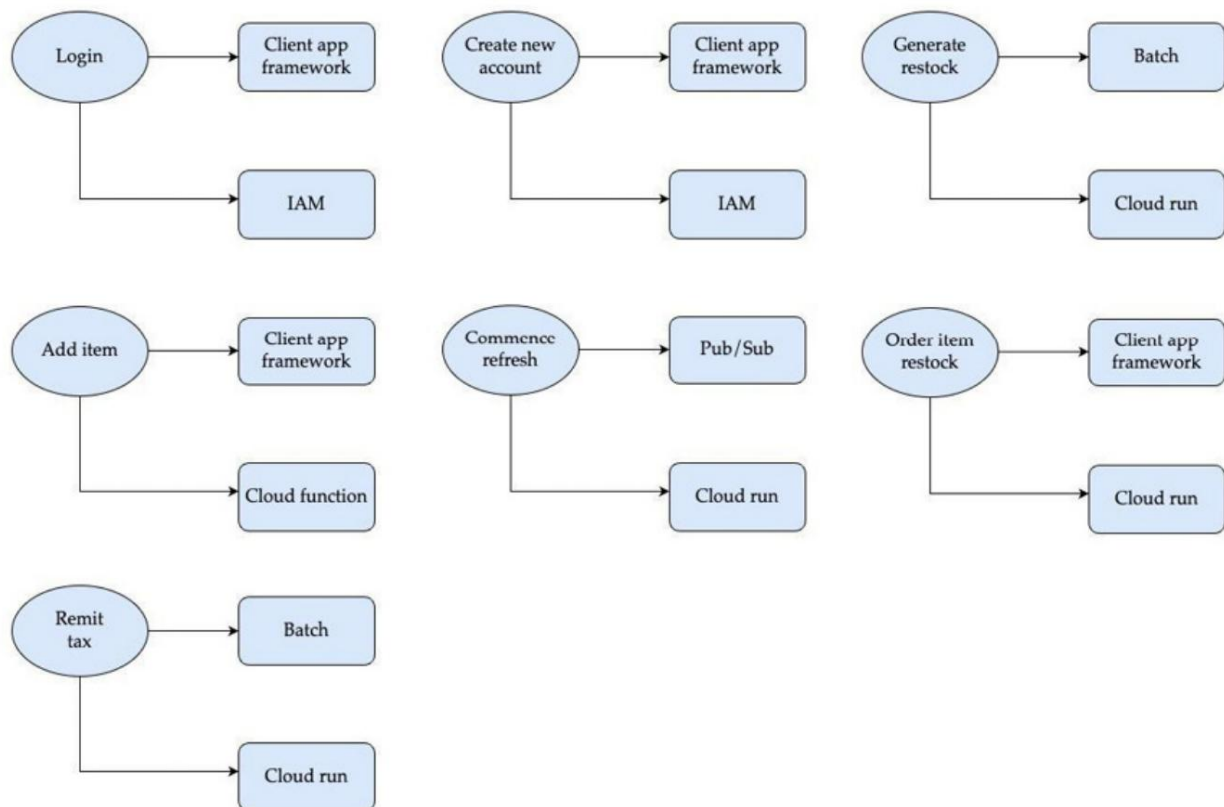
Seleccionemos un producto que sea adecuado para proporcionar potencia informática para

Añadir un artículo al carrito de un usuario. Esta función es sensible a la latencia, no requiere una gran potencia de procesamiento y se basa en las solicitudes de los usuarios. Estas solicitudes son propensas a picos de demanda concurrente, lo que a su vez requiere un escalado horizontal para adaptarse a dichos picos y caídas.

Cloud Run es altamente escalable horizontalmente, proporciona un rendimiento medio en relación con los productos de cómputo de GCP y el rendimiento del trabajador se puede configurar para satisfacer la demanda de procesamiento esperada.

Añadir artículos al carrito debería tener aproximadamente los mismos requisitos de rendimiento en cualquier ocasión, por lo que la escalabilidad vertical configurable de Cloud Run es una buena opción. No es la única opción posible, pero es la opción óptima entre los productos que hemos considerado en este ejemplo.

Después de realizar este ejercicio para nuestras siete características, la siguiente figura muestra el gráfico de dependencia que surge:



Cifra **2.11** Gráfico de dependencias de infraestructura de características

Este gráfico deja claro un punto importante: es raro que un solo

El producto ofrece todo lo que una función puede necesitar. Dos ejemplos muy diferentes de la figura anterior ilustran este punto:

- Una función controlada por el usuario, como Inicio de sesión, requiere una aplicación cliente para proporcionar una interfaz de usuario, pero utiliza un servicio en la nube para proporcionar la funcionalidad de respaldo que sería difícil de implementar bien en un teléfono (por ejemplo).
- La función de remisión de impuestos utiliza el servicio por lotes, que debe estar acoplado con un producto de potencia computacional para realizar el trabajo que envía el servicio por lotes.

Esta falta de un producto monolítico que lo maneje todo es intencional.

Recuerde la descripción y los argumentos en contra de la clase Dios en el capítulo anterior. Este antipatrón es claramente respetado y evitado por los arquitectos de GCP.

Conclusión

Este capítulo abordó cómo evaluar las características de su aplicación en términos de requisitos funcionales y considerar las exigencias técnicas que dichas características impondrán a su implementación. También analizamos varios productos de infraestructura con una perspectiva crítica sobre en qué dimensiones se pueden evaluar y dónde se ubican cada uno en dichos ejes. Esto culminó en una selección fundamentada y fundamentada de productos de infraestructura para nuestra aplicación, basada en su idoneidad para el problema y el costo.

[El capítulo 5](#) Determinación de la infraestructura sobre la cual construir características , continuará con este proceso de descomposición de características y asignación de infraestructura, ya que analizaremos cómo optimizar los componentes de infraestructura de las características: agrupándolos y colapsándolos o fusionándolos en función de la demanda, la afinidad y la proximidad.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com>



CAPÍTULO 3

Determinar la infraestructura sobre la que se construirán las características

Introducción

Con una definición sólida de los principales componentes o subsistemas de una aplicación, es importante saber en qué plataforma es más adecuado cada uno. El lector comprenderá las ventajas de los servicios en la nube y las plataformas cliente, y cómo asignar de forma óptima cada subsistema a la infraestructura adecuada. Los factores determinantes de la escalabilidad, las consideraciones de latencia y la afinidad de características son factores que influyen en la elección de la ubicación más adecuada para cada componente.

Este capítulo es una guía para el proceso de identificar estos factores, evaluar las opciones y realizar selecciones basadas en principios entre los candidatos para cada subsistema o característica.

Estructura

En este capítulo se tratarán los siguientes temas: Canalización

- de solicitudes para escalar a $O(N)!$
- Garantizar la capacidad de respuesta visible para el
- usuario Seguridad, privacidad y permanencia

- Seguridad en Google Cloud Platform

Objetivos

Este capítulo explorará algunos requisitos técnicos comunes y a menudo malinterpretados de las características de la aplicación, como la latencia y los límites de transacción. Analizar los requisitos para determinar sus necesidades reales permitirá examinar los efectos realmente necesarios para alcanzar dichos objetivos.

Tener requisitos con un nivel de granularidad más preciso aumenta la flexibilidad que ofrece al arquitecto para cumplirlos. A continuación, analizamos los servicios en la nube que ofrecen soluciones a estos requisitos descompuestos y las maneras de organizarlos para alcanzar conjuntamente los objetivos generales.

¡Canalización de solicitudes para escalar a $O(N)$!

Uno de los mayores desafíos, quizás el más significativo, al diseñar una aplicación confiable y de alto rendimiento es escalar las capacidades del sistema para satisfacer la demanda. La notación matemática conocida como notación Big O se utiliza para clasificar la complejidad en tiempo de ejecución o el costo de un recurso. Ante un aumento en la demanda de $O(N)$, donde N es el número de solicitudes, buscamos satisfacer esta demanda con un aumento correspondiente menor que $O(N)$, pero no mayor que $O(N)$. Esta sección describe técnicas para procesar solicitudes a través de un embudo de recursos organizados, es decir, mediante una serie de recursos cada vez más limitada.

Urgencia versus importancia Nuestro sistema

de votación para concursos de talentos debe garantizar que cada voto en una elección para el mejor artista de la temporada se cuente de manera precisa y confiable.

La palabra garantía es el punto más importante de esa declaración. Un usuario debe tener motivos para confiar en que la aplicación realmente hará lo que...

Dice que sí. Si a un usuario se le informa que su voto ha sido registrado, es importante que su voto se cuente para el candidato de su elección.

El recuento de los votos de cada candidato es de importancia crucial, pero no es urgente.

Reconocer la distinción entre urgencia e importancia es clave para comprender el valor y la confianza adecuada en la consistencia final, un modelo de transacción que se explicará y examinará en detalle en esta sección. Antes de profundizar en dicho modelo, establezcamos una comprensión común de los principios subyacentes, a menudo confundidos, de urgencia e importancia.

Las definiciones disociadas de urgencia e importancia son:

- Importante: Si debemos completar algo para lograr nuestro objetivo, esa cosa es importante.
- Urgente: Si esperar a hacer algo hasta un futuro próximo lo hace más costoso o menos valioso, entonces esa tarea es urgente. En términos más directos: ¿Es mucho mejor hacerlo ahora que mañana? Si la respuesta es sí, entonces es urgente.

?

Una matriz de Eisenhower, también conocida como matriz de prioridad, popularizada por Dwight D. Eisenhower, quien la atribuyó a un mentor anónimo, es la ilustración más popular de estas dos dimensiones distintas. El uso principal de una matriz de Eisenhower es decidir, de la manera más pragmática, cómo priorizar la programación de tareas individuales dentro de un conjunto dado de tareas que compiten por la atención. Tras examinar una situación común en la que se puede aplicar este método, veremos cómo se aplica al diseño de características en un producto de software.

Un conjunto de tareas que un propietario podría enfrentar y que debe priorizar es:

- Apagar un incendio en la cocina.
- Compre un extintor de incendios.
- Saque los contenedores de basura la noche de recogida de basura.
- Limpie el vidrio de la ventana del ático.

Con estas tareas en una lista de tareas pendientes, luego asignamos cada posición en cada una de las dos dimensiones en una matriz de Eisenhower, urgencia y

importancia, utilizando los criterios de calificación de los términos anteriores:

	Urgent	Not Urgent
<u>Important</u>	Put out fire	Buy fire extinguisher
<u>Not Important</u>	Put out garbage bins	Clean attic window

Cifra **3.1** Matriz de Eisenhower para propietarios de viviendas

El orden en que deben llevarse a cabo las tareas en estos cuadrantes es claro para el trabajo de mayor y menor prioridad en un momento dado:

- Las tareas urgentes e importantes vienen primero.
- Las tareas que no son urgentes ni importantes quedan para el final.

Los otros dos son menos claros, pero están en medio de la prioridad máxima y mínima: ocuparse de tareas importantes, no

- urgentes, antes de que generen tareas urgentes (como tener que lidiar con incendios futuros sin un extintor en este ejemplo).
- Si no puedes realizar tareas urgentes, aunque no sean importantes, no es un problema grave. Por lo tanto, solo hazlas cuando no haya trabajo en ningún otro cuadrante que compita por tu atención.

El mayor valor de organizar y priorizar el trabajo con este método reside en la separación entre inmediatez e impacto: urgencia e importancia. A menudo se confunden las cosas importantes con las urgentes.

cuando, de hecho, los dos aspectos suelen ser mutuamente excluyentes. Eisenhower resumió esto en una famosa declaración que más tarde se conocería como el Principio de Eisenhower:

Tengo dos tipos de problemas: el urgente y el importante. El urgente, no son importantes y, lo importante nunca es urgente-Eisenhower ·

Consistencia eventual en sistemas distribuidos

El reconocimiento de la inmediatez y la finalización como cosas separadas. Los requisitos se materializan en el ámbito de la computación distribuida y popularizado por Bigtable de Google.

Tomaremos una definición comúnmente aceptada del concepto de consistencia final:

La consistencia eventual es la garantía de que todos los datos leídos serán consistentes con todos los datos escritos eventualmente, que es algo impreciso pero el tiempo futuro previsible. Esto no implica que no sea determinista.

caos podrías saber cuando el sistema haya logrado

consistencia después de un tiempo entre la escritura de los datos y todas las escrituras ser fusionadas, y los efectos de esas escrituras siendo resueltas ·

Ser capaz de desenredar las partes de una escritura a gran escala o amplia operación en aquellas partes que son urgentes y aquellas que son

En realidad, los efectos secundarios no urgentes son un paso clave para agilizar nuestra Diseño de aplicaciones para optimizar la capacidad de respuesta. Una pregunta para Los arquitectos de aplicaciones deben tener en cuenta lo siguiente al diseñar una función sensible a la latencia: ¿El usuario realmente necesita esperar??

Un estudio de caso sobre la consistencia eventual

Un breve examen de un caso de uso en nuestro sistema de votación ilustrará la ventaja de separar importancia y urgencia, introduciendo una

Finalmente, se creará un modelo consistente para implementar esto.

Una aplicación de votación tiene una función básica: permitir que un votante emita un voto para una Candidato. Cualquier otra funcionalidad es la guinda del pastel. Así que, cuando... En lo que respecta a la ejecución confiable y receptiva de la función de votación, no

estropearlo

He aquí un requisito simple que un usuario podría tener:

- Requisito: Contar los votos rápidamente, en 2 segundos. (latencia sensible)

Esto podría ser interpretado por un equipo de ingeniería como Tenemos que agregar su voto al número total de votos para el elegido

Candidato confirma que el total del candidato se incrementó y

Luego responde al usuario Tu voto ha sido contado en segundos

2

Es fácil decirlo y la lógica no parece muy compleja, pero en

En este caso, los problemas son en gran medida consecuencias de escala.

Examinemos dos cuestiones que afrontaremos.

En una competencia mundial de talentos, los votantes pueden participar desde
En cualquier lugar y en todas partes. Un viaje de ida y vuelta de red a un sistema centralizado.

No se garantiza la carga de la base de datos en 2 segundos desde ningún punto final
potencial. Puede emplear una red rápida y confiable en su backend, como
como la estructura de red de GCP, pero hay una parte de la ruta
entre el dispositivo del usuario y su aplicación que está fuera del alcance de GCP
alcance y fuera de su control.

Es probable que decenas de miles de votantes emitan su voto en cualquier momento, por lo que
Confiar en una semántica de transacciones sencilla en tiempo real es...
provocar largos retrasos, tiempos muertos, malas condiciones de carrera y fallos.

Para que 50.000 votos casi simultáneos sean procesados por una única central
tarea en 2 segundos (también conocidos como 2000 milisegundos), esperamos cada voto
No tomar más de 40 microsegundos. $\frac{2000}{50000} = 0.04$

Un método demasiado simple para calcular la carga, pero el tema es...

ilustra es cierto. Es poco probable que un flujo simple y serial de
Las transacciones con estas restricciones de rendimiento tendrán éxito.

No todo está perdido; sólo que nuestras expectativas necesitan ser refinadas.

Tal como lo hemos definido, la función de votación tiene requisitos estrictos.

que podemos graficar en una matriz de Eisenhower de la siguiente manera:

		Urgent	Not Urgent
<u>Important</u>	Vote		
<u>Not Important</u>			

Cifra 3.2 Matriz de Eisenhower para la función de voto

Estas son malas noticias, teniendo en cuenta nuestro breve estudio de viabilidad anterior. La función de votación es importante, como se mencionó, y es la razón de ser de nuestra aplicación. La función de votación, tal como la hemos definido, también es urgente: el usuario quiere ver que su voto se emitió y no esperaría más de un par de segundos, un tiempo de respuesta típico.

Sin embargo, recuerden el principio de Eisenhower: pocas cosas, si es que hay alguna, son a la vez urgentes e importantes. Reflexionemos sobre lo que implica nuestra función de votación.

Un usuario emite su voto por uno de varios candidatos. Su selección debe quedar registrada, y dicha grabación debe ser duradera y debe confirmarse con el usuario rápidamente. No podemos confirmarlo hasta que la grabación sea realmente duradera; de lo contrario, estaríamos mintiendo, por lo que el resultado final de la confirmación del voto registrado es importante y urgente.

Sin embargo, no ocurre lo mismo con el recuento de votos, es decir, la suma de todos los votos emitidos a los totales de cada candidato. Es importante contar todos los votos registrados, ya que esto determina el resultado de la elección. Aunque no es urgente contar todos los votos registrados para

los candidatos, ya que no afecta el resultado, sino que sólo retrasa nuestro anuncio del ganador.

¿Qué pasa con los totales acumulados durante la elección, es decir, los resultados hasta el momento... es algo arbitrario, ya sea que informe sobre los resultados de hace una hora o de hace cinco minutos, siempre y cuando sea transparente sobre cuál es.

Ahora, dividamos la función de votación en dos: emitir voto y actualizar la clasificación: la primera es el registro y confirmación del voto de un usuario, y la segunda es el recuento de todos los votos emitidos para determinar el total de cada candidato.

Estas características fueron descritas en el capítulo anterior, en [la Figura 2.6](#).

Lo que hemos hecho nos permite reposicionar estas dos características en cuadrantes diferentes en la matriz de Eisenhower:

	Urgent	Not Urgent
<u>Important</u>	Cast vote	Update standings
<u>Not Important</u>		

Cifra 3.3 Matriz de Eisenhower de la característica del voto descompuesto

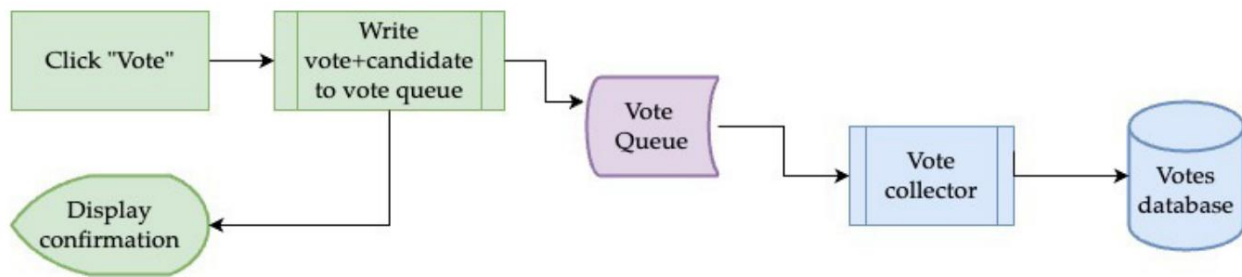
El beneficio obvio de esta descomposición es que hay menos trabajo etiquetado como urgente, menos trabajo lleva menos tiempo y, por lo tanto, hay una mayor oportunidad de cumplir con nuestras restricciones de desempeño.

Sin embargo, hay más para tener en cuenta: recuerde que los desafíos para ofrecer una función de votación monolítica confiable y sensible a la latencia se vieron aumentados en gran medida por el acoplamiento de la función a un único punto de estrangulamiento en la implementación propuesta, el almacén de datos al que se deben aplicar todos los votos.

Lo descrito brevemente anteriormente desvincula la función de voto emitido visible para el usuario de cualquier procesador de transacciones en serie. Se pueden registrar los votos en tantos lugares como se desee, siempre que cada voto se registre en uno de ellos, la intención del usuario estará a salvo.

No hay magia, todos esos votos tendrán que aplicarse eventualmente a algún almacén de datos global; pero ese es un tema completamente aparte.

Veamos una imagen de un primer paso para ver cómo se podría implementar esta función de votación descompuesta:



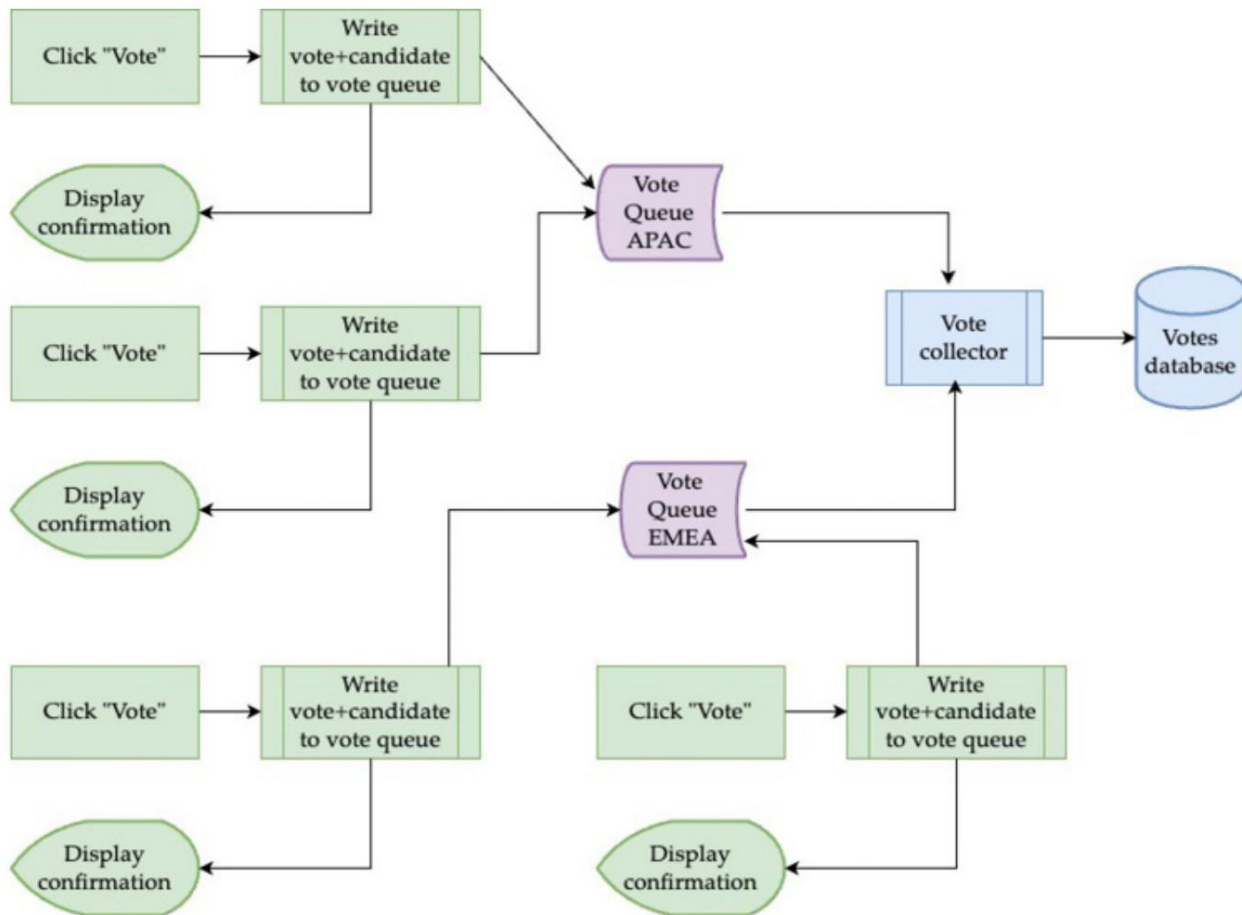
Cifra 3.4 Arquitectura de la función de voto descompuesto

Esta es una mejora con respecto a la función de primera votación, ya que se reduce el procesamiento entre el clic en el voto y la confirmación. Sin embargo, como se ilustra aquí, solo hemos pospuesto lo inevitable. La nueva Cola de Votos es ahora un cuello de botella, con sus propias limitaciones de rendimiento, y es inevitable que algún candidato sorprendente genere suficientes votos a la vez como para sobrepasar la tasa de ingesta de la Cola de Votos. Además, el problema de rendimiento del tiempo de ida y vuelta de la red a la Cola de Votos es tan real como el tiempo para llegar a una base de datos; la Cola de Votos debe residir en algún lugar, y este estará lejos de algunos usuarios de una aplicación disponible globalmente.

Sin embargo, nos hemos dado una salida al introducir la posibilidad de escalamiento horizontal en ese nuevo cuello de botella. No hay razón para que todos usen la misma cola de votación. Podríamos haber...

Colas de votación separadas para cada continente, país/provincia o incluso para cada usuario.

A continuación se muestra una versión ampliada horizontalmente de la arquitectura anterior con múltiples colas de votación para la localidad geográfica:



Cifra 3.5 Arquitectura de la función de votación descompuesta a mayor escala

Esto tiene el efecto de mover el punto de estrangulamiento o cuello de botella de esta función al proceso de recopilación de votos por lotes, que no es sensible a la latencia; que podemos colocar lo más cerca posible de la base de datos de votos a la que está estrechamente vinculado.

¡Lo hemos logrado! A un alto nivel, hemos diseñado una arquitectura para esta función que permite aplicar escalado horizontal para cumplir con los requisitos de latencia. Podemos aplicar escalado horizontal y vertical según sea necesario al diseñar el recopilador de votos en detalle. La durabilidad de los votos emitidos depende de la implementación de la cola de votos.

pero existen servicios GCP conocidos que se destacan por brindar tales garantías en este escenario.

Recordemos ahora el título de esta sección: Canalización de solicitudes. Esta es una descripción acertada de la estructura del diseño: las solicitudes de los usuarios se reducen desde un conjunto muy amplio de puntos finales a un número reducido de trabajadores intermedios, y luego se reducen aún más hasta el punto final, el menos escalable horizontalmente, de nuestro diseño: una única base de datos lógica.

Garantizar una capacidad de respuesta visible para el usuario

Con el concepto de canalización de solicitudes ilustrado en el ejemplo anterior, esta sección describirá el uso de los productos de GCP para proporcionar una implementación concreta del ejemplo y proyectará las limitaciones reales en la capacidad de respuesta que esta técnica supera.

Escalado horizontal mediante

El usuario

final de la aplicación de votación desconoce esta compleja y bien diseñada máquina, y no debería. Lo único que realmente le preocupa al emitir su voto es la respuesta: debe ser rápida, segura y tranquilizadora. Aunque posiblemente sea mejor que nada, si la respuesta frecuente al presionar el botón de votación es «Mmm... algo salió mal» o su sinónimo común: «Error del servidor», la gente no tendrá mucha paciencia para el concurso de talentos ni confiará en la fidelidad de los resultados.

Con esto en mente, un objetivo de diseño debería ser minimizar el procesamiento entre la solicitud y la respuesta del usuario. El mínimo indispensable para garantizar la durabilidad del voto emitido es ideal: aplazar lo que sea posible.

El servicio GCP, ideal para dar soporte al usuario de la cola de votos, Pub/Sub, está diseñado para este caso de uso común y generalmente crítico. Según la descripción del producto Pub/Sub de Google: Pub/Sub es un servicio de mensajería asíncrono y escalable que desacopla los servicios que producen mensajes de los que los procesan.

esos mensajes.

Pub/Sub permite que los servicios se comuniquen de forma asincrónica, con latencias del orden de 100 milisegundos.

Encontrarás la descripción en <https://cloud.google.com/pubsub/docs/overview> aquí:

Esto equivale a 100 milisegundos en cada cliente en paralelo y completamente desacoplado de otros clientes concurrentes. A diferencia de la arquitectura anterior, en la que las respuestas de nuestros clientes competían por una porción de tiempo, terminando con 40 microsegundos cada una, este modelo tiene un costo fijo de 100 milisegundos por cliente, prácticamente independientemente de cuántos clientes haya; no compiten. Con un objetivo de 2000 milisegundos para el voto emitido, quedan aproximadamente 1,9 segundos para ejecutar la interfaz de usuario y el procesamiento periférico del lado del cliente que nuestra aplicación implemente. El registro duradero del voto del usuario ha pasado de ser un posible cuello de botella en la ejecución de nuestra función a una pequeña fracción del tiempo de procesamiento.

Pub/Sub ofrece gestión automática de la capacidad para escalar los recursos de procesamiento y satisfacer la demanda. La replicación automática de todos los datos de Pub/Sub, en al menos dos zonas de red, garantiza la durabilidad y disponibilidad de los datos.

Se han eliminado dos preocupaciones principales mediante el diseño de un conjunto de características diseñado para satisfacer nuestros requisitos reales y la selección de una infraestructura que se adapta bien para respaldar los requisitos de esas características.

Garantías de transacción. Una

transacción, tal como la definimos en informática, es un conjunto de operaciones que deben completarse todas para que cualquier otra se considere completa. Es decir, todo o nada. El ejemplo clásico es la transferencia de fondos de una cuenta a otra, que en realidad consiste en dos operaciones: retiro de la cuenta de origen y depósito en la cuenta de destino.

Si primero registra el retiro, reduciendo el monto de la cuenta de origen y luego el depósito a la cuenta de destino, pero la cuenta de destino del depósito falla debido a una excepción de tiempo de ejecución en el proceso,

Entonces tendrías un pagador que no recibe \$100 y un beneficiario que aún espera un pago. Si ajustas las dos cuentas en el orden inverso, primero agregando dinero a la cuenta del beneficiario y luego fallas al reducir la cuenta de origen, acabas de ganar dinero de la nada. Estas dos operaciones deben encapsularse en una transacción: retirar los fondos, agregarlos y, solo cuando ambas operaciones sean exitosas, confirmar los cambios y hacerlos permanentes. Sin esa confirmación, que cierra el trato, las operaciones pendientes eventualmente se deshacen o se revierten; ninguna de las dos ocurrió.

Un sistema de transacciones confiable exhibe cuatro propiedades básicas, resumidas por el acrónimo ACID:

- Atomicidad: Todos los cambios dentro de una transacción son atómicos, ocurren todos o no ocurre ninguno, no hay subcomponentes.
- Consistencia: Al final de una transacción, se conservan todas las invariantes o garantías de integridad del sistema, pero puede que no ocurra lo mismo durante los cambios de estado internos dentro de una transacción.
- Aislamiento: independientemente del tiempo real, cada transacción parece ejecutarse antes o después de todas las demás transacciones sin ninguna conciencia o dependencia de la concurrencia entre ellas.
- Durabilidad: Una vez que una transacción se completa y se confirma, se garantiza la persistencia de sus efectos. No hay necesidad de preocuparse ni de realizar comprobaciones posteriores: una transacción confirmada se realizó y no hay fallos ni lagunas en el protocolo que provoquen su reversión.

Más allá del modelo de transacción base

Existen varios modelos de transacción que amplían el modelo ACID básico, generalmente para satisfacer las necesidades de sistemas altamente distribuidos con altos niveles de concurrencia o aquellos con actualizaciones a muchos conjuntos de almacenes de datos acoplados. El protocolo de confirmación en dos fases es uno de estos algoritmos; otros son implementaciones del mismo.

concepto de consistencia eventual descrito anteriormente en este capítulo.

Gran parte del gran valor de los servicios de GCP reside en que proporcionan y respetan los límites transaccionales. Por proporcionar, se entiende que un producto proporciona los medios para definir una transacción y, posteriormente, garantiza su integridad. Además, en su API y patrones de uso, proporciona al desarrollador de aplicaciones puntos claros en los que deben realizarse las operaciones para participar en los límites bien definidos de la transacción. En este caso, la adherencia significa que los propios productos de GCP respetan y participan en las transacciones definidas en torno a su uso: si se inserta una fila en una tabla de GoogleSQL y se publica un evento en un tema de Pub/Sub dentro de una transacción que falla, GoogleSQL y Pub/Sub realizan la acción correcta para la transacción.

En el ejemplo que desarrollamos previamente, al descomponer la función de votación en operaciones independientes, tomamos una transacción muy amplia que abarcaba la votación y la actualización del recuento de votos de un candidato, y la dividimos en transacciones independientes: registrar un voto y aplicarlo a la base de datos electoral. Reconocer y definir el alcance adecuado de una transacción es un factor clave para optimizar el rendimiento de un sistema de software. El rendimiento, en esta declaración, incluye explícitamente considerar la integridad del estado de un sistema, no solo la capacidad de respuesta del comportamiento visible para el usuario.

Seguridad, privacidad y permanencia

La seguridad y la privacidad están relacionadas, pero no son lo mismo. Mientras que la privacidad consiste en establecer límites acordados en torno a la visibilidad de la información, la seguridad es la garantía de que el comportamiento de un sistema, incluidos esos límites, no pueda ser alterado ni alterado fuera de su diseño.

Ciberseguridad. La

definición de seguridad es bastante sencilla y restringida: el estado de estar libre de peligro o amenaza. Usamos el término ciberseguridad para referirnos a la seguridad de los sistemas informáticos, pero en un sistema informático, ¿de qué amenazas queremos estar libres?

Enumeremos los desafíos abordados o centrémonos en la práctica de la ciberseguridad:
Operación no

- **autorizada:** Todo sistema relevante tiene reglas que rigen quién puede acceder a él y cómo. Si nos registramos para votar, podemos votar. Si un votante registrado puede votar por sus tres artistas favoritos, no podemos votar por un cuarto candidato, por muy buen guitarrista que sea. Un sistema seguro garantiza que solo pueda operar dentro de las reglas declaradas e implementadas para su uso.
- **Corrupción:** Si votamos por el acto acrobático del perro, ese es el voto registrado y mi voto no puede ser alterado por ninguna parte maliciosa. Si autorizamos un pago de 20 USD a una tienda de té de burbujas, el importe del cargo debe mantenerse en 20 USD y no debe aumentarse a 50 ni a 130 USD. Un sistema seguro está a salvo de la alteración de su estado por fuerzas externas o por el uso indebido de sus funciones.
- **Robo:** La información es un activo, tiene valor. Cuando las personas usan un sistema, suelen dejar alguna huella: su información de contacto, perfil bancario, historial de votación o preferencias de compra. Un sistema seguro es aquel del que esta información no puede extraerse ni utilizarse para nada que no sea el propósito acordado. Las filtraciones de datos son la manifestación más frecuente de sistemas inseguros.
- **Denegación de servicio:** Se trata de un robo de servicio común y forzado. No se trata del robo de los activos de un sistema, sino del robo de la disponibilidad del propio sistema. Es como si un delincuente te impidiera entrar a la fuerza en tu coche o en tu casa. Por supuesto, bloquear todas las rutas a un destino requiere mucha energía, y los perpetradores deben obtener algún beneficio. Los ataques de denegación de servicio se utilizan a menudo para obtener un rescate y detener el ataque, para perjudicar a una empresa obligando a los usuarios a usar servicios alternativos o para algún propósito político o social en contra del servicio. Un sistema seguro es altamente resistente y puede recuperarse de un ataque de denegación de servicio.

Seguridad en Google Cloud Platform

GCP ofrece un alto grado de seguridad de dos maneras. En primer lugar, como conjunto de productos, los propios servicios de GCP se adhieren a las mejores prácticas y son resistentes a los tipos de ataques y amenazas descritos anteriormente, por lo que GCP es seguro.

En segundo lugar, como proveedor de infraestructura, GCP incluye numerosas funciones que ofrecen protección contra ciberamenazas. El sistema de detección de intrusiones en la nube detecta y alerta cuando se detectan patrones de acceso o artefactos considerados maliciosos o peligrosos. Varios productos ofrecen un cifrado sofisticado tanto para datos en reposo como en tránsito. El firewall en la nube se puede habilitar a través de la API de seguridad de red complementaria y protege sus recursos en la nube tanto de amenazas globales emergentes como de amenazas específicas para su sistema o usuarios.

Privacidad de datos La

privacidad en los sistemas informáticos se denomina más específicamente privacidad de datos. La privacidad de datos es la capacidad de cada usuario de un sistema informático para determinar quién puede acceder a sus datos personales. Los datos personales son información exclusiva de cada individuo, no común a todos los usuarios de un sistema. Mi nombre de usuario, mis credenciales de inicio de sesión y mi historial de votaciones son personales, mientras que las horas en que el sistema aceptó votos son impersonales: no son específicas de ningún individuo y no pueden utilizarse para determinar nada sustancial sobre un usuario individual.

Un sistema que proporciona privacidad de datos es aquel que protege los datos personales y sensibles del acceso no autorizado, la manipulación y el uso indebido. Este es solo un aspecto de la protección de la privacidad: el aspecto técnico. Para garantizar la privacidad de sus usuarios, un servicio también debe contar con una política de gestión de datos bien diseñada y transparente que deje claro el control que tiene el usuario sobre el uso de sus datos personales y cómo ejercerlo.

El diseño y la publicación de una política adecuada de gestión de datos implica una serie de decisiones empresariales y de producto. Una vez implementada la política, se pueden emplear varios productos de GCP para garantizar que el sistema se mantenga en...

cumplimiento de dichas políticas, supervisar su cumplimiento, detectar cualquier regresión del incumplimiento y ayudar a remediar sus efectos.

La suite de protección de datos sensibles de GCP ayuda a definir, aplicar y supervisar el cumplimiento de las políticas de privacidad, tanto externas (como los requisitos regulatorios) como internas, adoptadas y publicadas para los usuarios. El producto de descubrimiento de datos sensibles puede detectar la presencia de datos sensibles en un sistema. Se pueden definir requisitos de anonimización y redacción para eliminar la información de identificación personal (PII) de los datos.

Existe una serie de vulnerabilidades fuera de los sistemas respaldados por GCP: las fuentes y destinos externos de datos representan posibles fugas de información confidencial. Las opciones de Protección de Datos Confidencial permiten inspeccionar los puntos de entrada y salida para detectar fuentes inesperadas de información personal identificable (PII) y posibles filtraciones no planificadas o manipuladas de información confidencial a sistemas externos, y detenerlas cuando se detectan.

[Permanencia de los datos.](#) La

información que se lee del sistema debe ser idéntica a la que se escribió. Los datos no deben modificarse, transformarse, degradarse ni mutar sin un diseño ni una intención. Este concepto de que lo que se lee es lo que se escribió se conoce como permanencia de los datos.

Este no fue un problema común durante la mayor parte de la historia de la humanidad. Las tablillas de piedra, los rollos de papiro, los periódicos y los cheques inscritos con tinta indeleble son bastante resistentes a alteraciones no detectadas.

En las últimas décadas, la cantidad de datos que registra la civilización se ha multiplicado exponencialmente, aumentando en muchos órdenes de magnitud.

Esto ha sido acompañado por un cambio hacia medios de almacenamiento cada vez más compactos y volátiles: las unidades de estado sólido y la memoria flash son todas vulnerables a alteraciones, sin evidencia real de tal manipulación.

No todas las alteraciones no intencionadas son resultado de una mala fe. Los defectos de software y hardware (errores), los eventos externos, como las sobrecargas o los pulsos electromagnéticos, y el paso del tiempo pueden hacer que los datos...

ilegible o peor aún, legible como un valor alterado.

Los productos de almacenamiento y base de datos de Google Cloud Platform proporcionan varios medios para garantizar la permanencia de los datos en su producto. Las réplicas de datos protegen contra la pérdida de datos debido a eventos físicos locales, como una inundación en un centro de datos. El registro de auditoría permite comprobar la ausencia de alteraciones inesperadas. La función Bucket Lock permite proporcionar almacenamiento inmutable en la nube y garantizar un archivo auténtico, una instantánea fiel de sus datos en un momento específico.

Conclusión

La definición de las características de la aplicación debe abordar los requisitos del usuario, pero no debe hacerse de forma aislada. Tanto el mundo real como el digital imponen restricciones, y lo técnicamente factible debe guiar nuestra definición de las características que satisfacen los requisitos del producto.

Este capítulo presentó motivaciones convincentes para definir cuidadosamente el alcance de las características de la aplicación, considerando la viabilidad técnica junto con la intención del producto. Se examinó la confusión frecuente entre las características de urgencia e importancia, junto con algunos antecedentes históricos.

La consideración de tareas urgentes e importantes se aplicó como metáfora a sus contrapartes técnicas: capacidad de respuesta versus integridad de los datos. Aplicamos la técnica de Eisenhower a un caso práctico de nuestro producto e identificamos los diferentes requisitos que podrían utilizarse para separar una función monolítica en componentes distintos.

Se describieron en detalle el procesamiento de transacciones y las variaciones de un tema, ya que estos conceptos constituyen la base necesaria para tomar decisiones de implementación informadas que satisfagan los requisitos de nuestra función. Tras explorar las consideraciones de escalabilidad, el capítulo se centró en otro tema interrelacionado y a menudo erróneamente confundido: la seguridad y la privacidad. Se enumeraron los requisitos de un sistema seguro e identificamos algunos servicios y funciones de GCP que pueden utilizarse para satisfacerlos.

La privacidad del usuario es una decisión tanto empresarial como técnica, por lo que identificamos la necesidad de una política de privacidad de datos y describimos la función de varios productos de GCP utilizados para supervisar y garantizar su cumplimiento. El capítulo tres concluyó con una breve discusión sobre la permanencia de los datos y los productos de GCP que respaldan su integridad y disponibilidad.

En el próximo capítulo, nos centraremos en las alternativas en algunas áreas principales de la arquitectura de aplicaciones, cómo diferenciarlas y determinar la opción más adecuada para un caso de uso particular.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 4

Elegir entre las opciones de GCP

Introducción

Google Cloud Platform (GCP) contiene múltiples servicios que ofrecen utilidad y funcionalidad similares. Al momento de escribir este artículo, el dominio general de la potencia computacional incluye 15 productos GCP diferentes; y existen 6 productos de base de datos diferentes en la plataforma GCP que podrían usarse para almacenar filas de datos indexadas.

Si bien cada uno de estos productos posee fortalezas específicas que lo distinguen de los demás, existe suficiente superposición en sus capacidades y descripciones como para dificultar la selección de un producto para un caso de uso específico. Los arquitectos de aplicaciones a menudo dudan o se arrepienten de su elección inicial de un producto de infraestructura. A menudo, un producto demuestra ser suficientemente bueno, pero se omite alguna característica de otro, o algunos aspectos del producto elegido son subóptimos. Este capítulo presentará varias comparaciones que aumentarán la idoneidad de la satisfacción a largo plazo con las selecciones de infraestructura.

Estructura

En este capítulo se tratarán los siguientes temas:

- Elegir una plataforma de ejecución del lado del servidor
- Cómo determinar dónde es mejor almacenar sus datos
- Identidad y autenticación
- Manejo asíncrono de mensajes

Objetivos

Este capítulo se centrará en las fortalezas de algunos productos GCP comunes y difíciles de distinguir en cuatro áreas principales: potencia de cómputo, almacenamiento de datos, identidad y autenticación, y procesamiento asíncrono. El lector comprenderá por qué un producto aparentemente equivalente en cada una de estas áreas sería la opción adecuada para sus necesidades, tanto por sus fortalezas como por sus posibles limitaciones.

Elegir una plataforma de ejecución del lado del servidor

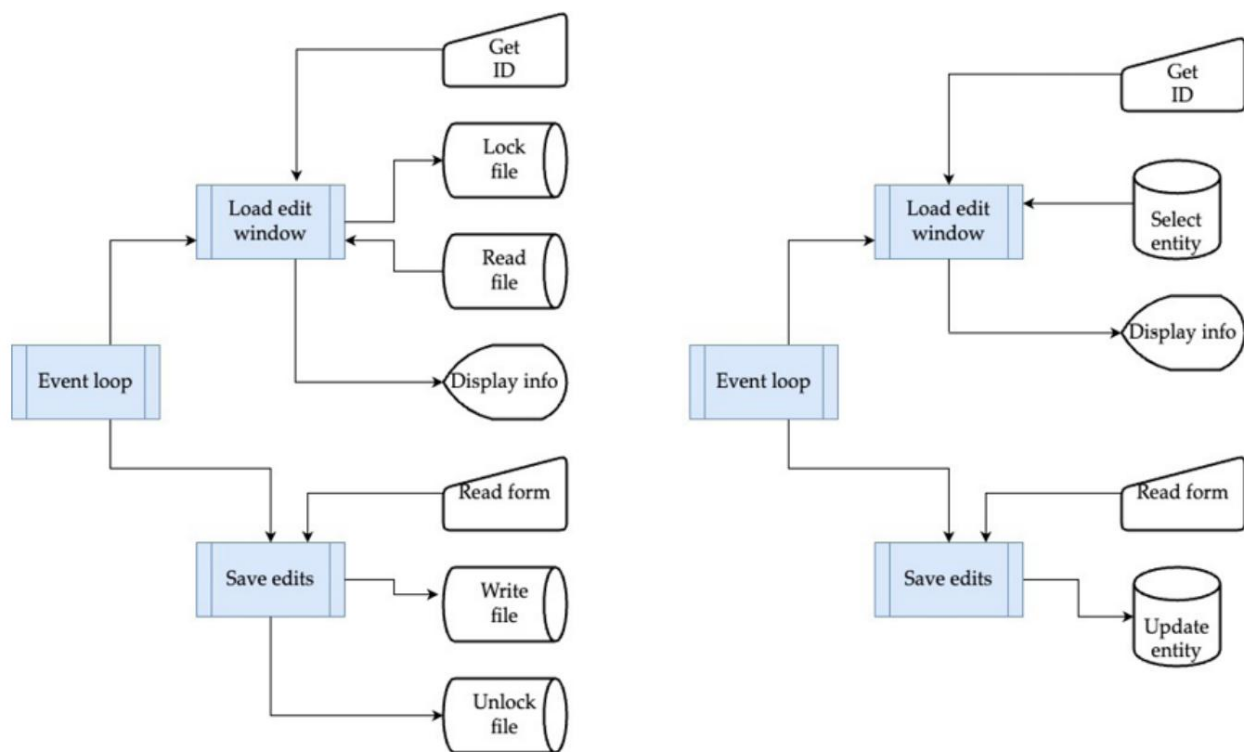
GCP ofrece numerosas opciones de plataforma para ejecutar código de aplicación y, en la mayoría de los casos, varias de ellas podrían funcionar; en otras palabras, no hay una elección incorrecta. En esta sección, analizaremos las ventajas que se deben buscar al seleccionar una plataforma de cómputo y los casos de uso para los que varios de estos productos son ideales.

El impacto de la selección de la plataforma de cómputo. La elección de una plataforma de cómputo en tiempo de ejecución es el factor más significativo y determinante de la configuración general de un sistema de software. Habrá momentos en que se requerirá guardar un registro, leer un registro o mostrar un valor en una aplicación, pero cualquiera de los marcos de base de datos o interfaz de usuario relevantes proporcionará alguna función para integrar en su aplicación. En este caso, el cómo de una operación puede variar, pero qué y dónde hacer se mantendrá constante. Por el contrario, el mecanismo utilizado para ejecutar su código, las funciones sin servidor, un marco de aplicación basado en eventos y una biblioteca de abstracción de aplicaciones.

dictará los puntos en el tiempo y la contención modular de las operaciones que su aplicación ejecuta para cumplir con sus requisitos.

La siguiente figura muestra el contraste, con cierto grado de abstracción.

En la siguiente figura, el diagrama de la izquierda representa una aplicación cliente basada en eventos que almacena datos de la aplicación mediante almacenamiento basado en archivos, como los buckets de almacenamiento de Google. El diagrama de la derecha muestra la misma aplicación, pero utilizando una base de datos relacional de Lenguaje de Consulta Estructurado (SQL) en la nube para almacenar los datos de la aplicación:



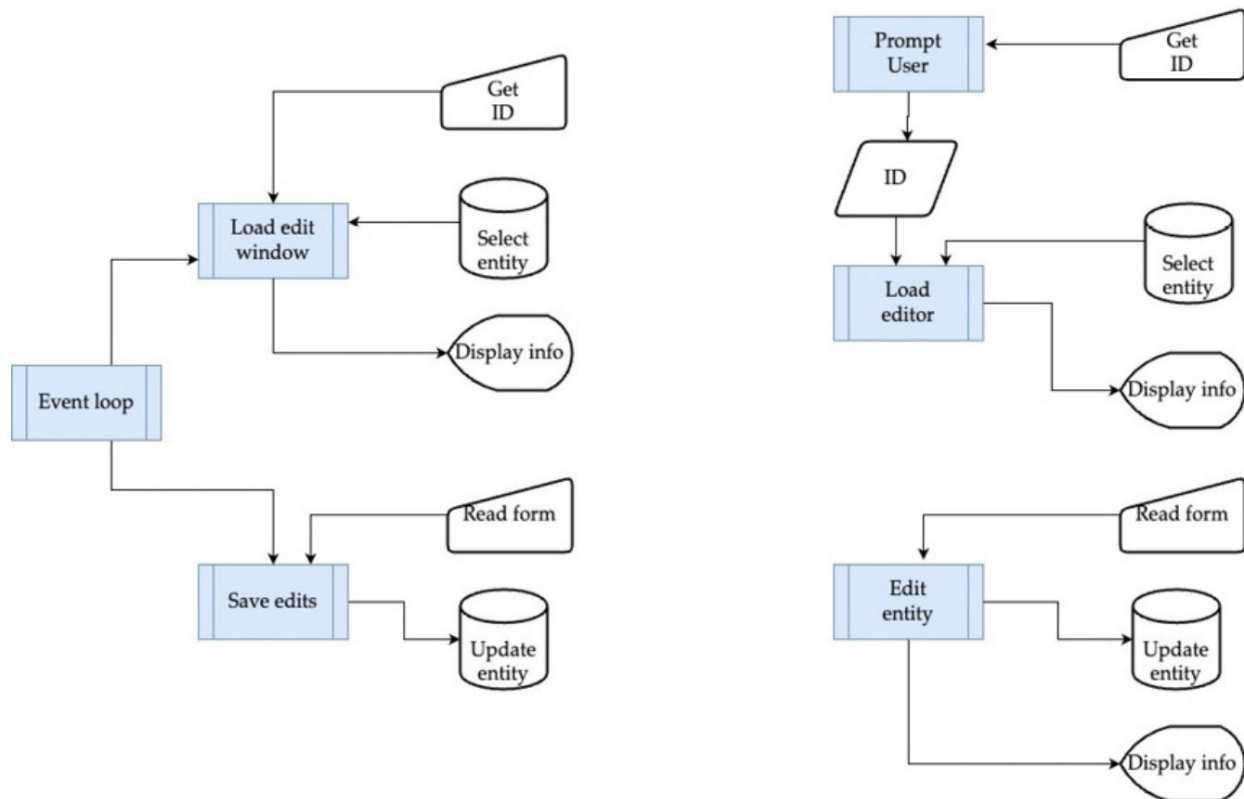
Cifra 4.1 Comparación lado a lado de las plataformas de almacenamiento

Observe la gran similitud entre las dos topologías de aplicación. El flujo general de la aplicación no ha cambiado; el número de componentes de procesamiento y su interacción con las funciones de almacenamiento e interfaz de usuario es el mismo; solo varían las características específicas y su uso para adaptarse a la semántica de cada producto de almacenamiento.

Ahora, examinemos una comparación similar de esta aplicación, pero en lugar de variar la plataforma de almacenamiento, diseñaremos

Dos productos computacionales diferentes con su aplicación distinta paradigmas.

El diagrama de la izquierda representa nuestra aplicación utilizando el mismo sistema controlado por eventos. Marco de aplicación y base de datos relacional. El diagrama correcto muestra cómo se podría diseñar la aplicación utilizando un API de transferencia de estado representacional (REST) , mientras se utiliza la misma Producto de base de datos para almacenamiento:



Cifra 4.2 Comparación lado a lado del bucle de eventos y la arquitectura REST aplicaciones

Las diferencias entre las aplicaciones del lado izquierdo y derecho en la Figura 4.2 son más significativas que las diferencias en la Figura No hay 4.1.

Correspondencia 1:1 de cuándo se envían diferentes operaciones en el último diagrama. En el diseño REST, las distintas funciones no tienen bucle de eventos central o unificador desde el cual se coordinan y enviado, cualquier estado debe ser establecido y publicado por el funciones en sí mismas, como con el valor de ID que se muestra en la Figura 4.2.

El efecto de esta distinción entre plataformas de cómputo y otras categorías de infraestructura es lo suficientemente impactante como para mencionarlo explícitamente. Si descubre que algún producto de almacenamiento X tiene alguna ventaja sobre otro producto de almacenamiento Y que utiliza actualmente en su sistema, la migración de su aplicación puede no ser trivial, pero estará bien integrada y localizada dentro de su código base. Por otro lado, decidir usar una arquitectura de microservicios, como una API REST, en lugar del marco de cliente basado en eventos que utiliza actualmente, requerirá una reescritura de su aplicación; es probable que los cambios importantes resulten en una nueva base de código; quizás reutilice algunos módulos o el alcance de los paquetes, pero los cambios serán significativos de arriba a abajo.

Teniendo esto en cuenta, vale la pena invertir tiempo y energía en comprender a fondo y elegir cuidadosamente las opciones de cómputo disponibles en GCP. A continuación, describiremos cada uno de los tres productos de cómputo de GCP de forma independiente antes de realizar un análisis comparativo.

Motor de aplicaciones de Google

Google Application Engine, comúnmente conocido como App Engine, es un servicio que ejecuta sitios web monolíticos renderizados en el servidor. En este caso, "monolítico" significa que, en la superficie, todo el sitio web está contenido y definido por el código y los recursos administrados dentro de la instancia de App Engine. Un sitio web renderizado en el servidor es aquel en el que la presentación del sitio web, es decir, el contenido del navegador, se renderiza en el servidor, en el código ejecutado dentro de App Engine, y luego se envía al navegador del cliente.

App Engine es una plataforma sin servidor: el desarrollador desarrolla e implementa su aplicación sin tener en cuenta los recursos del equipo subyacente en el que se ejecuta en tiempo de ejecución. Las opciones de configuración, especificadas por el desarrollador, se utilizan para guiar a App Engine en el inicio y cierre de las instancias de la aplicación administrada en respuesta a los cambios en la demanda y los patrones de tráfico.

App Engine ofrece dos niveles de servicio o entornos: el entorno estándar de App Engine y el entorno flexible de App Engine. La principal diferencia entre ambos entornos reside en el grado de control que ofrecen sobre el escalado de la aplicación en tiempo de ejecución en función del número de solicitudes y la cantidad de datos que debe gestionar.

Primero, veamos algunas características generales del producto App Engine, que proporcionarán contexto para un examen de los dos entornos.

App Engine admite aplicaciones web escritas en varios lenguajes de programación. Las aplicaciones que desarrollas también suelen utilizar un framework o herramientas web específicas para el lenguaje de programación que elijas, como Flask para Python, Apache Maven para Java y Ruby on Rails para Ruby.

La administración de las aplicaciones de App Engine se puede realizar mediante la interfaz de línea de comandos (CLI) de gcloud o la consola de GCP. Para usuarios con más experiencia o para gestionar eficientemente un gran número de aplicaciones de App Engine, GCP también ofrece interfaces de programación de aplicaciones (API) de App Engine para la ejecución programática y con scripts de las tareas administrativas que se realizarían manualmente con la CLI o la consola. Esta API administrativa de App Engine está disponible como interfaz REST y como interfaz de llamada a procedimiento remoto (RPC) para bibliotecas cliente específicas del lenguaje.

App Engine proporciona recursos de cómputo, un motor para ejecutar tu código y no se adentra en otras categorías de productos, como almacenamiento, seguridad o mensajería. Las aplicaciones de App Engine pueden interactuar con otros servicios de GCP, como Firestore, Pub/Sub y Cloud Storage.

El desarrollo de una aplicación de App Engine está bien descrito, y Google y la comunidad ofrecen numerosos ejemplos y soluciones recomendadas para muchos problemas comunes. Por ello, App Engine es una buena opción para la primera incursión de un desarrollador en GCP, si la aplicación en sí es adecuada: una aplicación web sin necesidad específica de renderizado del lado del cliente ni otro procesamiento local. La naturaleza sin servidor de la plataforma App Engine ofrece al desarrollador la oportunidad de familiarizarse con...

las herramientas de GCP, como el comando `gcloud` y la consola para desarrolladores, sin dividir su atención entre esas herramientas para desarrolladores, su aplicación en sí y la mecánica de gestión de la implementación en tiempo de ejecución.

El entorno estándar de App Engine es compatible con aplicaciones desarrolladas en Go, Java, Node.js, PHP, Python y Ruby. Se admiten varias versiones recientes de cada lenguaje, con un plazo de soporte típico de varios años para cada versión ofrecida. El soporte para una versión específica de un lenguaje en App Engine suele estar vinculado al soporte ofrecido por el propio proveedor del lenguaje y se ofrece en App Engine durante un tiempo limitado después de que el proveedor haya dejado de usar dicha versión.

Las aplicaciones implementadas en el entorno estándar de App Engine se ejecutan en un entorno Sandbox, similar a un entorno virtual de Python o una cárcel chroot de Linux. La aplicación está aislada de la interacción accidental con otras instancias o aplicaciones y solo tiene acceso a los servicios de App Engine y GCP definidos y ofrecidos explícitamente. Una aplicación no puede realizar llamadas arbitrarias a los servicios del sistema operativo con la expectativa de que tengan un efecto persistente fuera de su propia instancia de App Engine. En términos más directos, si su aplicación del entorno estándar usara el paquete del sistema operativo Python para eliminar archivos arbitrarios en la ruta raíz, dichas eliminaciones no serían visibles ni tendrían ningún efecto en otras instancias concurrentes o futuras de su aplicación ni en las aplicaciones de otros desarrolladores. Los cambios persistentes en los datos de usuario y de la aplicación deben realizarse a través de los servicios adecuados, como productos de almacenamiento en la nube o bases de datos en la nube.

Las opciones y el control limitados sobre las aplicaciones del entorno estándar permiten que este entorno ofrezca importantes ventajas consecuentes sobre el entorno flexible. Gracias a este aislamiento de instancias, App Engine puede tratar todas las aplicaciones e instancias como si fueran iguales, es decir, intercambiables. Esta visión independiente de las instancias, a su vez, permite a App Engine iniciar nuevas instancias de la aplicación con gran rapidez, como

Se requiere una limpieza mínima posterior a la instancia o una personalización previa a la carga del entorno de ejecución. Por lo tanto, el entorno estándar de App Engine es ideal para aplicaciones que necesitan escalar horizontalmente con gran rapidez, adaptándose a picos extremos de tráfico de solicitudes. Gracias a esta misma genericidad de las instancias, las aplicaciones del entorno estándar pueden ejecutarse a un coste muy bajo, reduciendo la escala de la aplicación a cero instancias durante periodos de baja demanda.

Muchos desarrolladores de aplicaciones consideran inicialmente que las limitaciones del entorno estándar son menores que su cartera actual o preconcebida de tecnologías y servicios, y lo descartan como una opción inviable. Esto merece una reflexión más profunda. El entorno estándar debe evaluarse desde una perspectiva nueva, en función de sus propios costes y beneficios, en lugar de compararlo con una lista de compatibilidad con diseños diseñados para otras plataformas. Muchos requisitos preconcebidos de un sistema de aplicaciones para su infraestructura son, de hecho, irrelevantes ante las características sin servidor y de bajo mantenimiento del entorno estándar de App Engine; mientras que las ventajas de escalabilidad y costes son difíciles de igualar en otros servicios.

El entorno flexible de App Engine ofrece las ventajas en términos de material y esfuerzo del entorno estándar; no todas las aplicaciones pueden desarrollarse con sus opciones de menú predeterminadas, ni todos los sistemas pueden aprovechar el escalado y el ahorro de costes que ofrece. El entorno flexible de App Engine ofrece una mayor variedad de herramientas de desarrollo y mayor libertad para elegir las opciones de entorno de ejecución, de ahí su nombre.

La primera categoría en la que el entorno flexible ofrece más opciones es en la elección del lenguaje de desarrollo y el soporte en tiempo de ejecución.

El entorno flexible permite el desarrollo en los siguientes lenguajes, en **negrita** los que son exclusivos del entorno flexible: Go, Java, Node.js, PHP, Python, Ruby, .NET y runtimes personalizados.

Además de admitir una selección más amplia de idiomas, el entorno de ejecución de las instancias de aplicación de su entorno flexible es

Bajo un control mucho mayor. Las instancias de entornos flexibles se ejecutan en máquinas virtuales (VM) de Google Compute Engine , para las cuales puede configurar bibliotecas de software personalizadas y conectar sus propios contenedores Docker. Incluso puede acceder a estas máquinas virtuales mientras se ejecutan mediante Secure Shell (SSH) para observar y depurar una instancia de aplicación en ejecución.

Si bien el entorno flexible no administra su aplicación hasta el punto de poder brindar el mismo nivel de implementación de aplicaciones sin intervención que el entorno estándar, sí brinda una experiencia y una ventaja similares para la administración de las instancias de VM en las que se ejecuta su aplicación, en lugar de las instancias de la aplicación en sí.

Las instancias de máquina virtual del entorno flexible de App Engine se pueden configurar con casi la misma libertad que cualquier otra máquina. Esto se debe a que existen ciertos antipatrones, como configuraciones de software que presentan riesgos de seguridad conocidos, que no son compatibles.

Las características que distinguen una máquina virtual de entorno flexible de una máquina propia o una máquina virtual alquilada directamente del servicio subyacente, GCP Compute Engine, residen en la gestión de las instancias del entorno flexible. Estas se gestionan de dos maneras: escalado para satisfacer la demanda y administración del sistema.

El número de instancias simultáneas de una máquina virtual (VM) en un entorno flexible se escala automáticamente, de forma similar a como se gestionan las instancias de aplicación en el entorno estándar. A medida que aumenta la demanda de su aplicación en un entorno flexible, este iniciará más máquinas virtuales para procesar el tráfico adicional. Si bien el escalado puede parecer equivalente al del entorno estándar, la instanciación y el apagado de las VM consumen muchos más recursos y tiempo que los recursos sin servidor en dicho entorno. No se garantiza que el escalado automático del entorno flexible se mantenga al día si una aplicación experimenta picos de demanda repentinos, pronunciados e impredecibles. En tal caso, se deben emplear otros mecanismos para evitar déficits de capacidad.

Una ventaja importante del escalado automático es su sensibilidad a la demanda local del usuario. Dentro de las restricciones configuradas, el entorno flexible inicia instancias en las regiones de la red más cercanas al origen del tráfico, lo que minimiza la latencia del usuario y los costos de los servicios de red.

El segundo aspecto de la gestión de máquinas virtuales que maneja el entorno flexible es el de la administración del sistema: el mantenimiento de su entorno de máquinas virtuales.

Describamos las tareas administrativas comunes que se realizan en un entorno flexible.

Las actualizaciones del sistema operativo y de seguridad de los paquetes instalados se aplican automáticamente a la imagen de su máquina virtual. Las nuevas instancias ejecutarán el software actualizado y las instancias en ejecución se reiniciarán automáticamente hasta una vez por semana para garantizar que el software antiguo no permanezca en producción durante un período prolongado. Si una actualización del sistema operativo se considera crítica y se sabe que es compatible con la versión anterior, se aplica automáticamente cuando está disponible.

Las instancias en ejecución se someten a una comprobación de estado; es decir, se les envían solicitudes para confirmar su actividad y disponibilidad para atender el tráfico de usuarios. Si una instancia no responde para confirmar su estado, no se considera disponible para el enrutamiento de solicitudes de usuarios y se reiniciará si el estado no es correcto.

Además de preservar la disponibilidad del sistema, las comprobaciones del estado del servidor ayudan a prevenir respuestas lentas y solicitudes canceladas. Por ejemplo, una causa común de bloqueo del servidor, que provoca solicitudes canceladas, es un sistema de archivos raíz lleno o casi lleno. Esta condición se detecta y soluciona mediante las comprobaciones del estado del entorno flexible, generalmente antes de que se manifieste.

En este punto del texto, hemos explicado en profundidad el producto Google App Engine y sus dos variantes: el entorno estándar y el entorno flexible de App Engine. App Engine existe desde 2008 y fue el primer servicio de computación en la nube ofrecido por Google. No hay motivos para preocuparse por la obsolescencia.

o desaprobación, el servicio App Engine ha evolucionado y avanzado mucho desde su introducción y ha cambiado para integrarse con el ecosistema de GCP que ha crecido a su alrededor.

App Engine es una buena opción para alojar un sitio web cuyo control y administración se pueden delegar en las funciones y herramientas administrativas de la infraestructura. Sin embargo, existen componentes distribuidos que no se ajustan al modelo de App Engine de un sitio web único e indivisible, y hay sitios web que requieren un control más preciso del entorno. A continuación, examinaremos otros servicios de computación de GCP para abordar estas situaciones: Google Cloud Run y Google Compute Engine.

Google Cloud Run

Google Cloud Platform ofrece una gama de servicios, desde los más abstractos, como Cloud Functions y App Engine, hasta los más concretos, conocidos como hardware. Google Cloud Run se encuentra en un punto intermedio de este espectro: ofrece un servicio totalmente gestionado que puede ejecutar cualquier tipo de carga de trabajo que configure.

Contenedores, paquetes de software herméticos. El concepto de contenedores es fundamental para muchos servicios de Google Cloud Platform. Un contenedor es, conceptualmente, un paquete de artefactos de software: ejecutables, especificaciones de configuración y datos, que es autocontenido, de modo que puede ejecutarse de forma aislada de otros contenedores que puedan estar presentes y ejecutarse en el mismo sistema host. Esto es similar al entorno de pruebas descrito en la sección anterior, Google Application Engine para el entorno estándar de App Engine, pero a un nivel inferior: el de los archivos y procesos del sistema operativo.

Cloud Run utiliza contenedores como unidad básica de implementación de usuarios. Usted, o las herramientas de GCP en su nombre, agrupan todos los archivos necesarios para ejecutar su aplicación dentro de los límites de un entorno de sistema operativo base conocido. El contenedor es el entorno para su aplicación; las rutas de directorio, los archivos y los procesos son

todo presente en el contenedor sin referencias externas sin resolver.

Esto permite que el sistema host cargue y ejecute su aplicación en contenedor como una tarea hermética: sellada y hermética, sin efectos secundarios en el entorno fuera del contenedor y sin preocupaciones por los efectos secundarios de otros hermanos en contenedores similares.

Veamos algunos lenguajes y marcos disponibles.

Dado que usted configura completamente el entorno de contenedores donde se ejecutará su aplicación, tiene total libertad para usar cualquier lenguaje de desarrollo, marco de trabajo y herramientas que prefiera, siempre que estén disponibles para una de las imágenes de sistema operativo base que ofrece Cloud Run. Esta es una ventaja de los contenedores: al estar aislado, puede explorar y usar opciones sin preocupaciones, ya que la hermeticidad de su entorno de ejecución evita cualquier preocupación de terceros sobre incompatibilidad o efectos secundarios.

Si bien cualquier suite de desarrollo concebible permite desarrollar e implementar en Cloud Run, existe un alto grado de automatización y herramientas para herramientas de desarrollo comunes y bien definidas. Las aplicaciones desarrolladas siguiendo las mejores prácticas publicadas en Go, Node.js, Python, Java, .NET Core o Ruby pueden compilarse automáticamente desde el código fuente e implementarse en producción con un solo comando. La selección de artefactos de soporte y otras configuraciones del contenedor en tiempo de ejecución se realiza mediante el comando `gcloud`, con las opciones de línea de comandos correspondientes.

Tipos de aplicaciones de Cloud Run: Cloud Run admite dos modelos para ejecutar aplicaciones en contenedores implementadas: servicios y trabajos.

Un servicio de Cloud Run está diseñado para ser un proceso en ejecución continua, como un servidor web o un servidor RPC, que responde a solicitudes entrantes esporádicas. Estos son el tipo de trabajos que responden a las consultas o entradas de los usuarios y suelen escalarse horizontalmente a medida que aumenta o disminuye el número de usuarios simultáneos.

Un trabajo de Cloud Run es un modelo de proceso comúnmente conocido como lote. No debe confundirse con el servicio GCP Batch; se refiere genéricamente a un flujo de trabajo con una entrada conocida, etapas de procesamiento predeterminadas y un punto en el que se considera completo. Esto no significa que...

El número, la programación y las entradas para cada instancia de trabajo de Cloud Run no están predeterminados o incluso predichos, sino que cuando se inicia una instancia específica, sus entradas son conocidas y estarán completas cuando se completen todas las etapas del procesamiento del trabajo.

Ahora, veamos el uso del escalamiento automático del servicio Cloud Run para satisfacer la demanda de los usuarios.

El escalado automático de un servicio de Google Run es muy similar al caso de uso paralelo de App Engine. A medida que se enrutan nuevas solicitudes a tu trabajo, se entregan a una instancia disponible o se inicia una nueva para procesarla. Tras un tiempo de inactividad, una instancia se retira del servicio y se liberan sus recursos. Dado que iniciar nuevas instancias lleva tiempo, puedes configurar tu trabajo para que siempre mantenga un número mínimo de instancias, incluso si no hay solicitudes recientes. Esto evita una latencia excesiva al procesar las primeras solicitudes tras un periodo de inactividad. Si bien Cloud Run puede escalar automáticamente hasta 1000 instancias de servicio en respuesta al tráfico, puedes configurar un límite para el número de instancias simultáneas de tu trabajo para evitar sorpresas en tu factura de uso. En el siguiente ejercicio, exploraremos cómo usar el escalado de trabajos de Cloud Run para adaptarlos a los perfiles de solicitud.

Un trabajo por lotes típico tiene una vida útil mucho más larga y se supone que experimenta una demanda con una configuración más predecible. Existen tres modelos para ejecutar un trabajo de Cloud Run, que corresponden a los casos de uso comunes para procesos de larga duración que no están vinculados a un usuario individual: herramientas, trabajos programados y trabajos de matriz. Describiremos cada uno de ellos y los compararemos con un caso de uso ajeno a GCP, del que son un sustituto:

- Trabajos de herramientas o scripts: Un requisito común es ejecutar una serie de comandos o un script que los encapsule bajo demanda de forma única. Esto puede ser para realizar alguna tarea con un disparador asíncrono o impredecible, como archivar todos los archivos de registro de esta región (se realizará una auditoría regulatoria) o convertir esta tabla de base de datos a archivos CSV (la base de datos se está migrando a un nuevo servidor). Estos se ejecutarían cuando lo indiquemos.

durante un período de tiempo considerable. Esos se conocen como trabajos de herramientas, o trabajos de script y son el tipo de tareas que se iniciarían en un Sesión de terminal en un entorno de computación que no es en la nube. En GCP, Estos se ejecutarían como un trabajo de herramienta con los artefactos de salida y registros de ejecución dirigidos a un producto GCP persistente, como un Cubo de almacenamiento de GCP.

- Trabajos programados: La necesidad de algunas tareas es predecible de manera confiable, Proverbialmente como un reloj. Todos los días a medianoche, un contador diario debe reiniciarse; cada mañana debe aparecer un mensaje del día actualizado; y cada noche, archivos temporales de ese día Se limpian los procesos. Este es un requisito muy común. que cada sistema operativo tiene algún mecanismo para implementar es: cron, trabajos y tareas programadas, por nombrar algunos.

Google Cloud proporciona esto a través del servicio Cloud Scheduler.

Puede configurar la región y la especificación de la hora, que incluye la frecuencia con la que se ejecutará un trabajo específico de Cloud Run y

El programador iniciará automáticamente instancias de su Cloud Run

trabajo para cumplir con esos criterios de programación. Los trabajos pueden ser opcionalmente configurado con una política de reintento, que especifica un tiempo máximo que un

Se debe permitir que el trabajo programado se ejecute sin éxito.

finalización, después de lo cual el programador detendrá la ejecución

trabajo e iniciar una nueva instancia.

- Trabajos de matriz: muchas tareas son fácilmente divisibles entre muchos trabajadores, de modo que el producto del número de trabajadores y el tiempo de pared necesario Completar la tarea es una constante. Para citar un proverbio popular:

Muchas manos hacen el trabajo ligero. La capacidad de ejecutar un

El comando xargs de Linux para una serie de entradas

comando, y este comando es el modelo en el que se basa Cloud Run

Los trabajos de matriz se basan en

Podemos establecer la compensación entre el tiempo de pared y el número de trabajadores.

aproximadamente en términos algebraicos como sigue:

$$\text{CPUtime to completion total} = \frac{\text{seconds needed of number workers}}{\text{se necesitan 100 segundos}} \quad /$$

de CPU para convertir 20 imágenes en color a

escala de grises, debería tomar 100 segundos para que un trabajador la complete o cinco segundos en el reloj si tiene 20 tareas de trabajador, cada una invirtiendo simultáneamente una imagen: walltime

$$= 100 = 100/1$$

$$\text{tiempo de pared} = 5 = 100/20$$

El costo real del tiempo de CPU del trabajo (100 segundos) no ha cambiado, pero el tiempo de pared se ha dividido por la cantidad de segundos de CPU que se gastaron simultáneamente.

Muchas tareas se pueden dividir fácilmente en unidades de trabajo más pequeñas e independientes. A esta división y distribución del trabajo la llamamos fragmentación. Otro ejemplo es la copia de archivos: una tarea podría configurarse para copiar 1000 archivos. Diez tareas podrían copiar 100 archivos cada una, o 100 tareas podrían asignarse a diez archivos cada una. Claro que no todos los archivos tienen el mismo tamaño, pero la mayoría de los algoritmos de fragmentación asumen que los datos de entrada pueden dividirse en fragmentos de tamaño equivalente y cuentan con algún mecanismo para hacerlo.

Esto resulta en un equilibrio lineal fácil de entender entre el tiempo transcurrido y el espacio ocupado. Al decidir cómo configurar el escalado de esta función, podemos investigar el coste de tener menos empleos de larga duración frente a un mayor número de empleos de corta duración para fundamentar nuestra decisión.

Una vez que hemos decidido un método de fragmentación adecuado, como el número de archivos en un depósito de almacenamiento o las filas en una imagen digital, y el número máximo de tareas que deseamos escalar, configuramos un trabajo de Cloud Run que ejecuta código de fragmentación personalizado. Este código selecciona una porción o fragmento único de los datos de entrada según el índice o el número de tarea de una instancia de tarea determinada. En nuestro ejemplo de copia de archivos, fragmentaremos los datos de entrada calculando el número total de archivos y luego asignaremos a cada fragmento una porción igual de esos archivos, como se muestra en el siguiente código.

Nota: Se supone que los siguientes números de tareas e índices de archivos comienzan en cero, como es la convención para las matrices en la mayoría de los lenguajes de programación y bibliotecas:

```
recuento_total_de_archivos = len(lista_de_archivos_de_entrada);
primer_número_de_archivo_para_esta_tarea = recuento_total_de_archivos /
```

```

    recuento_total_de_tareas * este_número_de_tarea;
    último_número_de_archivo_para_esta_tarea = recuento_total_de_archivos /
    recuento_total_de_tareas * (este_número_de_tarea + 1) - 1;

```

Los límites calculados para cada fragmento se muestran en la siguiente figura:

total files	total task count	task number	first_file_number	last_file_number
1000	20	0	0	49
1000	20	1	50	99
1000	20	2	100	149
1000	20	3	150	199
1000	20	19	950	999

Cifra 4.3 Límites de fragmentos de trabajo de matriz

Luego, en el código de nuestro trabajo, copiaríamos el rango de archivos en el fragmento de la tarea actual de la siguiente manera:

```

para (int índice_de_archivo = primer_número_de_archivo_para_esta_tarea;
    índice_de_archivo <= último_número_de_archivo_para_esta_tarea;
    índice_de_archivo++)
{ copiar_archivo_a_salida(lista_de_archivos_de_entrada[índice_de_archivo]); }

```

Para ejecutar este trabajo de matriz, debe configurar el trabajo de Cloud Run para que ejecute el código anterior en un número específico de tareas, en este caso, 20. Cloud Run iniciará las 20 tareas en paralelo y, como cada tarea está restringida a su propio conjunto de archivos por el código anterior, se ejecutarán simultáneamente e independientemente. Una vez completadas las 20 tareas, Cloud Run considera que el trabajo de matriz está completo.

Trabajos de larga duración en Cloud Run.

Otra ventaja a destacar sobre los trabajos de Cloud Run es que pueden ejecutarse durante más tiempo del que se esperaría de su contraparte no GCP identificada anteriormente, es decir, un comando en una sesión de terminal interactiva. Las sesiones de terminal se consideran poco fiables, un riesgo de seguridad o ambas cosas. El sistema operativo Linux ofrece...

Hay muchas formas de dar cabida a comandos de larga duración: los comandos `screen` y `tmux`, `at jobs` y el comando `nohup` son algunos de ellos.

Windows ofrece menos opciones, pero comúnmente se usa el comando de inicio para lograr esto.

Los trabajos de Cloud Run pueden ejecutarse hasta 24 horas y no están conectados a una terminal interactiva. Cloud Run supervisa su finalización, guarda su estado de salida y puede configurarse para reintentar automáticamente si se detecta un fallo. Además, la salida del comando se conserva para su posterior análisis. Esto reemplaza las funciones de las soluciones alternativas para comandos de larga duración, con la ventaja adicional de una semántica de reintento bien definida.

En resumen, Google Cloud Run es un servicio que ejecuta un ejecutable personalizado de tu propia creación con un alto grado de hermeticidad mediante la contenedorización. Estos trabajos pueden ejecutarse con varios modelos: ejecución inmediata, ejecución programada y periódica, y ejecución paralela fragmentada. Se supervisa el estado de finalización de los trabajos y Cloud Run puede configurarse para reintentar los trabajos fallidos con varios parámetros que rigen los reintentos.

Compute Engine Ahora,

analizaremos un servicio que se encuentra más alejado del espectro de la implementación de aplicaciones administradas y sin servidor y que aporta la fortaleza de Google Cloud Platform a la gestión de máquinas: Compute Engine.

Compute Engine ofrece una gestión similar a la de los dos servicios que hemos analizado hasta ahora: App Engine y Cloud Run. Cada uno puede iniciar instancias, monitorizarlas para detectar fallos, aumentar o disminuir el número de instancias activas según la demanda y reintentar las instancias fallidas. Este nivel de gestión es fundamental para los productos de cómputo de GCP. La principal diferencia entre los tres productos de cómputo analizados en este capítulo radica en la naturaleza de lo que gestionan y el tipo de recurso de las instancias que controlan.

Máquinas virtuales

App Engine gestiona aplicaciones, Cloud Run gestiona entornos sandbox y Compute Engine gestiona máquinas virtuales. Para quienes no estén familiarizados con el concepto, una máquina virtual es la emulación de un sistema informático completo: CPU, memoria, almacenamiento en disco, conexiones de red y, a veces, incluso dispositivos de interfaz humana (HID), como teclados y ratones. La emulación de estos componentes de hardware se realiza íntegramente mediante software: no se trata de un disco donde se escriben datos, sino de un búfer en un programa que se comporta como un disco a la vista del sistema operativo. Esto ofrece numerosas ventajas sobre el uso directo de un sistema informático físico, principalmente la hermeticidad, la compatibilidad entre sistemas y el uso compartido de recursos. Para obtener explicaciones detalladas, se recomienda al lector buscar recursos sobre el tema en Google.

Para una experiencia práctica inicial, el proyecto de código abierto Qemu en <https://www.qemu.org/> Es un excelente recurso. Para una mayor exploración de productos de calidad de producción, se recomienda consultar el proyecto KVM más avanzado en <https://linux-kvm.org/> que es el hipervisor utilizado dentro de GCP. /

Compute Engine permite al usuario configurar una imagen de máquina virtual (VM) , especificando el tipo de procesador, la cantidad de memoria virtual, la cantidad y el tamaño de los discos conectados y las conexiones de red. El contenido de los discos conectados se puede cargar con el software deseado, tal como se instalaría software en cualquier servidor o estación de trabajo.

Aquí radica la primera ventaja de las máquinas virtuales: su imagen inicial se guarda en un conjunto de archivos en algún lugar de un disco duro real. Al iniciar o instanciar una copia en ejecución de su máquina virtual, esta se carga desde la imagen original de su equipo. Si algo falla, se borran accidentalmente directorios necesarios o el sistema se corrompe debido a un error, el estado inicial deseado del sistema permanece presente en esos archivos de imagen. La máquina dañada puede apagarse y se puede iniciar una nueva instancia, sin tener en cuenta los eventos que afectaron a la instancia anterior. El software que administra estas imágenes subyacentes y las utiliza para iniciar instancias de máquinas virtuales se conoce como monitor de máquina virtual (VMM) o hipervisor.

Muchos hipervisores, incluido Compute Engine, amplían la gestión de imágenes para ofrecer compatibilidad con instantáneas. Una instantánea permite guardar como imagen una imagen del estado actual de una máquina virtual en ejecución, que puede utilizarse para iniciar instancias posteriores, al igual que la imagen base original. Esto, en efecto, proporciona una máquina del tiempo para las máquinas virtuales: una forma de retroceder en el tiempo hasta el momento en que se tomó la instantánea.

Comparación de productos de cómputo. Los tres

productos de cómputo de GCP que se han analizado tienen alcances muy diferentes, pero comparten un tema común: todos gestionan aspectos específicos de algo. Si bien el contenido que gestionan difiere, todos ofrecen capacidades de gestión y automatización similares para los recursos que ponen a disposición e implementan. Primero, resumiremos los aspectos comunes de estos productos y luego revisaremos el factor diferenciador: la unidad de potencia de cómputo que cada producto proporciona a sus usuarios.

Gestión de recursos informáticos de GCP

Administrar un recurso significa, en este contexto, configurarlo; aprovisionarlo y desplegarlo en servicio activo; monitorear su ejecución; y finalmente responder a su terminación de alguna manera:

- **Configurar:** una descripción completa del recurso, tanto su contenido como los parámetros que rigen las etapas posteriores de su ciclo de vida son colectivamente la configuración del recurso.
- **Aprovisionamiento e implementación:** Reservar o asignar las dependencias de tiempo de ejecución necesarias para instanciar el recurso se denomina aprovisionamiento del recurso. Algunos ejemplos de dependencias de tiempo de ejecución son las máquinas reales donde ejecutar una máquina virtual, una dirección IP para el punto final de un servicio y una cuota suficiente en el presupuesto del usuario.
- **Implementar un recurso:** este es el acto definitivo de hacerlo realidad. Iniciar un servidor web en App Engine para aceptar solicitudes, arrancar una máquina virtual; implementar un recurso hace que una instancia del mismo sea real,

Ya no es sólo una descripción de la aplicación que deseamos ejecutar.

- **Monitoreo:** El propósito de monitorear un recurso es conocer su estado. Esto permite saber cuánto tráfico de red se recibe, cuánto espacio de disco se utiliza, el uso de CPU del binario de una aplicación y si un proceso se está ejecutando o ha finalizado.
- **Respuesta a la terminación:** Cuando un recurso ha sido terminado, ya sea por agentes externos o por algún mecanismo interno, es necesario poder tomar la acción prescrita para un sistema altamente disponible y confiable. Reiniciar un servidor web que ha salido, reiniciar una máquina virtual que se ha congelado y guardar los registros de una tarea por lotes de larga duración son ejemplos de algunas acciones de terminación comunes.

Recursos computacionales ofrecidos

Los servicios que describimos ofrecen recursos con un espectro de abstracción. Los resumiremos desde el más abstracto hasta el más concreto.

Por un lado, el entorno estándar de App Engine ofrece la gestión de aplicaciones, generalmente servidores web, sin transparencia sobre los recursos informáticos de nivel inferior utilizados para proporcionar dichas aplicaciones. El código existe de forma independiente; las dependencias o recursos de respaldo son ocultos y opacos, y se gestionan en segundo plano.

El entorno flexible de App Engine proporciona máquinas virtuales configuradas con una aplicación y el software de soporte especificado por el usuario. Si bien el entorno flexible no oculta las múltiples capas de software y hardware de las que depende una aplicación, sí automatiza la creación y gestión de esas dependencias de nivel inferior.

Cloud Run administra contenedores; entornos de ejecución herméticos y autónomos que ejecutan cualquier código que usted configure, pero aislados para que no afecten ni se vean afectados por entornos de contenedores vecinos, incluidas otras instancias del mismo contenedor.

Compute Engine gestiona máquinas virtuales completas: sistemas informáticos idealizados. Estas máquinas pueden ejecutar cualquiera de los numerosos sistemas operativos disponibles y contener conjuntos arbitrarios de dispositivos, como discos, adaptadores de red y procesadores.

En un mundo ideal, cuanto más abstracta sea una especificación, mejor será no requerir ninguna dependencia específica, ya que otorga la mayor libertad para adaptarse al entorno y la situación. Sin embargo, la abstracción conlleva una pérdida necesaria de precisión y control, algo que algunos sistemas no pueden evitar. Por lo tanto, como regla general, que debería considerarse un principio rector más que una receta para el éxito, cuanto más se acerque al extremo abstracto, mejor.

Tenga en cuenta que existen otros productos de GCP que son aún más abstracciones, como Cloud Functions; y productos más concretos que las máquinas virtuales de Compute Engine, como la solución GCP Bare Metal. Los productos descritos aquí se encuentran en la banda media del espectro de abstracción y cubren los requisitos de la mayor parte de las aplicaciones distribuidas.

La siguiente figura posiciona los tres productos computacionales analizados en el espectro de abstracción y representa aproximadamente la porción relativa de sistemas de aplicación a los que se dirigen:



Cifra 4.4 Productos en el espectro de la abstracción de recursos

Cómo determinar dónde es mejor almacenar sus datos

Al seleccionar una nueva estación de trabajo o computadora portátil, la primera característica que busca un consumidor después de la computadora en sí son generalmente las especificaciones del disco duro o, más precisamente, la unidad de estado sólido.

De la misma manera, después de seleccionar el servicio de cómputo para una característica de aplicación determinada, el medio por el cual esa característica persiste en su estado o se almacena

Sus datos son la siguiente consideración habitual.

Utilizando un enfoque similar al de los productos informáticos, examinaremos varios productos que proporcionan almacenamiento persistente y duradero para datos de aplicaciones. Los servicios que analizaremos abarcan una amplia gama de casos de uso y paradigmas de almacenamiento, y se describirán las características distintivas, tanto conceptuales como pragmáticas, de estos productos.

Firestore en la nube

Cloud Firestore no es una base de datos relacional; es una base de datos NoSQL especializada que se utiliza para almacenar y recuperar datos en forma de documentos. A diferencia de las bases de datos relacionales o las bases de datos no relacionales centradas en filas, la unidad de almacenamiento es bastante grande: un documento, como un libro, un formulario o un currículum.

Los documentos de Firestore están estructurados. Un documento contiene campos que Firestore representa en formato JSON. Un ejemplo rápido de un libro representado como documento JSON compatible con Firestore podría ser similar al siguiente: { Libro: { título: "El Hobbit",

ISBN:

"9780345445605",

autor: "J. R. R. Tolkien",

resumen: "Hobbits y enanos

buscan oro con un mago y elfos...", capítulos: { "El comienzo": "Era una noche lluviosa

en la

madriguera del hobbit...", "El medio": "El valle élfico fue divertido... Las

minas son oscuras..." , "El final": "Ese dragón era enorme y aterrador... Qué bien estar en casa." } } }

Se pueden buscar colecciones de documentos mediante valores de filtro para varios campos. Cloud Firestore garantiza un rendimiento óptimo de las consultas mediante índices para localizar rápidamente los documentos según los valores de sus campos. Firestore crea automáticamente un conjunto de índices básicos para su colección de documentos; puede agregar índices para admitir consultas específicas y complejas que su aplicación ejecutará. Si se ejecuta una consulta para la que no existe un índice adecuado, se mostrará un mensaje de error con un enlace para crear un índice sugerido en la consola para desarrolladores de Firebase.

El mecanismo de consulta de Cloud Firestore ofrece varios operadores y admite consultas compuestas y anidadas, pero no permite consultas tan complejas como las de una base de datos relacional con SQL. Para solucionar esto, existen varios servicios de búsqueda de terceros dedicados.

Una consulta de Firestore para libros de otros autores que contienen referencias a hobbits en la descripción se podría escribir en Python de la siguiente manera:

```
from google.cloud import firestore

db = firestore.Client()
consulta_biblioteca =
db.collection("libros") .where("autor", "no
incluido", "Tolkien")) .where("resumen, "incluido", "Hobbit")

libros = consulta.stream()
para libro en libros:

    print(f"{libro.isbn} => {libro.título}")
```

Los documentos se pueden crear, actualizar y eliminar como JSON mediante funciones de Firebase, con compatibilidad total con transacciones de atomicidad, consistencia, aislamiento, y durabilidad (ACID), como se describió en el capítulo anterior. La semántica de transacciones de Python garantiza la integridad de los datos tanto en documentos individuales como en la colección general de documentos.

Otra característica distintiva de Cloud Firestore se deriva del hecho de que es una rama de otro producto, Firebase. Firebase es

Una plataforma de desarrollo para aplicaciones móviles, principalmente Android e iOS, y aplicaciones cliente web. Siendo este el objetivo de Firebase, Cloud Firestore cuenta con una sólida compatibilidad con bibliotecas cliente. Las funciones de Firestore se pueden ejecutar directamente desde el código que se ejecuta en el cliente, no solo desde los componentes de infraestructura de GCP.

Los Firestore son los siguientes:

- Firestore se basa en documentos; en lugar de filas de datos que se parecen a una hoja de cálculo, Firebase almacena datos estructurados cuyo contenido se asemeja más a un documento de texto con secciones definidas, como encabezado, párrafo y nota al pie.
- Firestore tiene una accesibilidad excepcional desde el código del cliente.

Sus desventajas son:

- Las consultas de Firestore no son tan capaces como las de otros productos de bases de datos, como las bases de datos relacionales.

Lenguaje de Consulta Estructurado en la Nube (GCP) Cloud

SQL es un producto que actúa como abstracción para varios productos de bases de datos relacionales, como PostgreSQL, MySQL o SQL Server. El uso de Cloud SQL como proxy para cualquiera de estos sistemas de bases de datos ofrece dos ventajas principales: la independencia de la plataforma en su aplicación y la gestión completa de los recursos de su base de datos.

- Aplicaciones independientes de la plataforma: si bien todos los sistemas de gestión de bases de datos relacionales (DBMS) populares utilizan SQL para realizar operaciones sobre los datos, ahí es donde termina la similitud.

La administración de los objetos de datos, como tablas, índices y entidades de soporte, varía considerablemente de un sistema de base de datos a otro. Aún más problemático, el lenguaje SQL compatible con los productos de base de datos presenta variaciones tanto en la sintaxis como en las capacidades. Esto puede tener consecuencias no deseadas: frustración para los desarrolladores que están familiarizados con el SQL compatible con un proveedor diferente o que dependen de él, y dificultades para portar o migrar de un producto de base de datos a otro.

Cloud SQL se posiciona entre una aplicación y cualquiera de las

Sistemas de bases de datos que soporta. Los desarrolladores y administradores escriben código de aplicación, especificaciones de bases de datos y configuración administrativa sin tener en cuenta ni conocer el sistema de gestión de bases de datos (SGBD) de respaldo. Las capacidades que ofrece Cloud SQL están bien definidas y son compatibles con un producto de base de datos relacional ideal . Mediante una combinación de traducción en tiempo de compilación y ejecución, Cloud SQL convierte las operaciones de base de datos ejecutadas por una aplicación en las operaciones correspondientes, nativas del producto de base de datos que se esté utilizando en tiempo de ejecución.

- Administración de bases de datos: Cloud SQL es coherente con el modelo de administración presentado en los productos de computación descritos anteriormente en este capítulo. Las tareas regulares y críticas de administración de bases de datos, como las copias de seguridad, la replicación y la gestión de la capacidad de almacenamiento, son importantes de planificar y urgentes de ejecutar cuando se requieren. Cloud SQL configura estas tareas, al igual que su interfaz de aplicación, con herramientas y una gramática independientes del producto de base de datos compatible en el que se ejecutarán. La administración de bases de datos es un campo complejo con poco margen de error, y eliminar gran parte de la complejidad específica del producto de la función del administrador de bases de datos (DBA) supone una reducción del estrés para el DBA y un aumento de la fiabilidad y la estabilidad de la base de datos y los productos que dependen de ella. Una vez configurada la replicación de las instancias de base de datos mediante la semántica de Cloud SQL, dicha replicación se realizará en PostGreSQ, MySQL o SQL Server, utilizando la expresión local de dicha semántica.

La fiabilidad de estas tareas es tan importante como su correcta configuración. Cloud SQL se beneficia de la alta disponibilidad de todos los productos de GCP. En un servidor de base de datos local, la programación de una copia de seguridad o una expansión de almacenamiento suele depender de la disponibilidad de un recurso local: un proceso del servidor que inicia las tareas de administración. Si algo falla con esa dependencia de programación, la tarea no se ejecutará y el sistema estará en riesgo. GCP replica la ejecución de la administración en diferentes regiones, de modo que incluso una interrupción...

La distribución de recursos en una configuración regional no afecta la integridad ni la disponibilidad del sistema.

Bigtable es la

primera base de datos NoSQL popular que ofrece una alternativa al modelo de datos relacional, base de la mayoría de los SGBD de uso generalizado anteriores. Mientras que las bases de datos relacionales contienen elementos de datos no estructurados e imponen estructura mediante relaciones definidas entre ellos, las columnas de datos de Bigtable pueden ser no estructuradas o poseer su propia estructura inherente, sin necesidad de operadores relacionales sólidos que se apliquen a ellas. A diferencia de las bases de datos relacionales, que admiten el uso de cualquier columna para identificar filas de datos, las tablas de Bigtable tienen una clave predeterminada, un ID de fila, y los valores de las demás columnas son atributos de esa fila.

Rendimiento de Bigtable

El modelo de datos no relacional contribuye a dos ventajas principales de rendimiento de Bigtable: es rápido y, como su nombre indica, grande. Las filas de datos se pueden escribir y leer en Bigtable a mayor velocidad que con una base de datos relacional, y la ventaja de rendimiento se acentúa a medida que aumenta el tamaño de las filas y el número de operaciones.

Bigtable está optimizado para superar en rendimiento a la mayoría de las bases de datos relacionales, con filas de datos mucho más grandes. Esta optimización tiene sentido en un modelo no relacional, ya que los datos de una base de datos relacional suelen dividirse en varias tablas normalizadas. Por lo tanto, una sola fila grande es menos frecuente en el diseño de una base de datos relacional. Escribir filas correlacionadas en muchas tablas lleva tiempo, lo cual contradice los objetivos de diseño de Bigtable: acceder a grandes cantidades de datos lo más rápido posible. Por eso, el modelo de datos no relacional, desnormalizado o plano de Bigtable es necesario para lograr su alta tasa de lecturas y escrituras. Los datos de Bigtable pueden ser grandes tanto en amplitud como en profundidad: una sola fila puede tener hasta 256 megabytes antes de que se produzca una degradación del rendimiento, y una tabla puede contener

miles de millones de filas y tener varios petabytes de tamaño.

Existen otros detalles de implementación de Bigtable que le permiten gestionar bases de datos tan grandes con la misma eficacia. En resumen, Bigtable accede a los datos de las tablas mediante servidores Tablet; nodos de procesamiento independientes que pueden escalarse horizontalmente para satisfacer los requisitos de tablas individuales y patrones de tráfico. Por lo tanto, si los datos aumentan o las consultas a ciertas porciones de sus datos aumentan, Bigtable puede escalar los procesos utilizados para satisfacer esa demanda.

Las transacciones de Bigtable tienen un alcance limitado. Una consecuencia del cambio respecto al modelo de base de datos relacional es el modelo de transacciones compatible con Bigtable. Dado que se espera que una fila de datos en Bigtable no se relacione ni se una con otra fila de ninguna tabla, las transacciones solo se admiten dentro de una sola fila.

Esto fomenta aún más la simplificación de las filas de datos en Bigtable, hasta el punto en que todos los datos necesarios para representar una unidad de trabajo lógica comprometida están contenidos dentro de una sola fila.

Al evaluar Bigtable frente a bases de datos relacionales, se debe considerar su rendimiento y la semántica específica de los datos. Si los datos de una aplicación no requieren un alto grado de normalización o correlación entre tablas independientes y su tamaño aumentará considerablemente, Bigtable es una buena opción para proporcionar servicios de persistencia para la aplicación.

El modelo de datos único de Bigtable, los datos de columnas estructuradas y repetitivas, la preferencia por filas de datos autocontenidas desnormalizadas y el alcance transaccional más limitado exigen replantear las prácticas arraigadas en los desarrolladores al trabajar con bases de datos relacionales. Al evaluar un servicio de almacenamiento, se debe considerar el costo en tiempo y esfuerzo del desarrollador para migrar a Bigtable. Si una aplicación no requiere ni utiliza las ventajas de escalabilidad y velocidad de Bigtable, una base de datos más tradicional puede ser una opción eficaz y más práctica.

Spanner

Spanner es el producto de base de datos más ambicioso en cuanto a alcance en GCP y, probablemente, del mundo. Dado que esta es una afirmación audaz, permítannos...

Lo justifico aquí describiendo tanto la amplitud como la profundidad de las características de Spanner:

- Amplitud de los modelos de datos de Spanner: Spanner admite datos almacenados tanto en modelos relacionales como clave-valor, como sistemas de gestión de bases de datos (SGBD) basados en SQL y sistemas NoSQL. Spanner también permite a los desarrolladores representar datos en dos paradigmas adicionales: modelos de datos gráficos y vectoriales.

El soporte de Spanner para datos relacionales se ofrece tanto como una interfaz nativa de GoogleSQL como a través de una interfaz PostgreSQL, que permite acceder a las tablas relacionales de Spanner mediante comandos PostgreSQL, para permitir una migración rápida de aplicaciones de PostgreSQL a Spanner, sin una curva de aprendizaje significativa o reescritura de la aplicación.

La compatibilidad de tablas clave-valor de Spanner no es equivalente al almacén de clave-valor de Bigtable. Spanner introduce el concepto de tablas intercaladas y filas secundarias, y ofrece una mayor consistencia entre filas, en lugar de la compatibilidad con columnas anchas y la filosofía de fila única de Bigtable.

Spanner también permite representar y recorrer datos como un gráfico gracias al soporte de un estándar publicado recientemente, Graph Query Language; para el cual siempre hay información actualizada disponible en <https://www.gqlstandards.org/>.

:// . . /

Spanner también ofrece soporte nativo para un cuarto paradigma de datos: una base de datos vectorial. También conocida como vector de búsqueda, los datos vectoriales se almacenan como una representación matemática y su objetivo es buscar similitudes en los datos, en lugar de coincidencias exactas. La asociación por similitud hace que la búsqueda vectorial sea ideal para aplicaciones de aprendizaje automático e inteligencia artificial generativa, que suelen gestionar grandes conjuntos de datos con relaciones difusas o estadísticas.

- Rendimiento de Spanner en tiempo de ejecución: Spanner se ha desarrollado con la experiencia de varias generaciones de bases de datos, almacenamiento y sistemas distribuidos predecesores de Google. Algunas de sus características de confiabilidad y escalabilidad se deben a los mejores...

de las características de estos productos anteriores, mientras que algunas características de Spanner se han diseñado para abordar las lecciones aprendidas a lo largo de años de ejecución de estos servicios anteriores a escala.

Todas las bases de datos Spanner son distribuidas: existen numerosas copias de datos almacenadas en diferentes ubicaciones y procesos de servidor asociados a cada una de estas copias. Estas réplicas procesan las lecturas y escrituras en sus datos locales y cooperan en segundo plano para aplicar los cambios locales a sus contrapartes distribuidas. Esto es similar a la consistencia final en Bigtable, pero ofrece garantías de transacción mucho más sólidas, como se describirá en breve.

Spanner separa la potencia computacional necesaria para atender solicitudes de base de datos de los datos almacenados, lo que permite un escalamiento horizontal transparente de los recursos computacionales para que diferentes patrones de acceso, muchas lecturas en un subconjunto estrecho de datos, lecturas o escrituras de una gran cantidad de filas y consultas complejas que hacen referencia cruzada a diferentes porciones de datos, se puedan aprovisionar y atender de manera eficiente.

La replicación, las copias de seguridad e incluso la propagación de cambios en el esquema de una base de datos se realizan automáticamente, según las opciones de configuración especificadas por el usuario, en segundo plano. Las tareas de mantenimiento, como la conversión y migración de esquemas, que impiden el acceso del usuario a los datos durante la migración, se ejecutan en réplicas que están temporalmente protegidas del acceso del usuario. En conjunto, estas funciones permiten que una base de datos Spanner no tenga tiempo de inactividad en cuanto a la disponibilidad de los datos.

Spanner ofrece garantías de transacciones Strong-ACID, que son al menos tan sólidas como las de cualquier base de datos relacional, pero debido a la naturaleza distribuida inherente de Spanner, Spanner debe ir un paso más allá.

Spanner implementa la ordenación TrueTime de las transacciones que ocurren simultáneamente en una base de datos distribuida. Como resultado, una base de datos Spanner proporciona consistencia externa, la consistencia más sólida definida para sistemas de procesamiento de transacciones. En resumen, esto significa que Spanner ofrece una semántica y reglas bien definidas para ordenar y resolver secuencialmente las transacciones que ocurren en múltiples réplicas.

de una base de datos distribuida y global.

Al evaluar Spanner como una opción de producto, se deben considerar su costo y las fortalezas especializadas de las alternativas competitivas. Dado que Spanner se esfuerza por incluir y superar las capacidades de varias otras clases de bases de datos, ¿por qué sigue habiendo competencia en estas clases? Siempre hay otras consideraciones, y los sistemas especializados pueden optimizarse para casos de uso específicos. A continuación, resumiremos algunos de estos contraargumentos al uso de Spanner: La lectura de un gran número de filas es más rápida en

- Bigtable. En su modelo NoSQL, tablas de pares clave-valor, Spanner ofrece más opciones de indexación e iguala o supera el rendimiento de Bigtable en la recuperación de filas específicas. Sin embargo, este no es el patrón de acceso para el que Bigtable está optimizado. Si se lee una gran serie de filas, en un rango de claves, la menor sobrecarga de los mecanismos de indexación simples y más pequeños de Bigtable le otorga una clara ventaja en velocidad.
- El coste del material de una llave inglesa es superior al de una equivalente típica de la competencia. Si bien se puede argumentar que, a largo plazo, su mayor fiabilidad y automatización la convierten en una inversión más rentable, no todos los usuarios necesitan o pueden permitirse un gasto mayor.
- El costo fijo de Spanner puede superar el valor de almacenar un conjunto de datos pequeño. La creación y el mantenimiento de cualquier base de datos de Spanner requieren una cantidad considerable de recursos. Si una base de datos es relativamente pequeña, el costo fijo podría ser mayor que el costo de los datos reales. Acceder a una pequeña tabla de búsqueda de categorías de productos a través de una tabla NoSQL local podría ser más rápido que usar un nodo de Spanner para acceder a estas varias docenas de filas de datos invariables.

Almacenamiento en la

nube. Google Cloud Storage es una categoría de producto diferente a las bases de datos descritas hasta ahora. Las bases de datos se utilizan para almacenar tablas de filas repetidas de datos, cada una de las cuales se ajusta a la misma descripción o diseño de columnas. En diferentes momentos,...

La tabla de facturas puede tener una o 10 000 filas, pero cada una de ellas contendrá el ID de la factura, el importe a pagar, la fecha de vencimiento y el destinatario de la factura. Puede procesar cualquier subconjunto de filas con la misma lógica.

Los datos se estructuran tanto horizontalmente en columnas como verticalmente en filas.

El almacenamiento en la nube, por otro lado, se utiliza para almacenar objetos binarios grandes (BLOB). Además de ser un acrónimo, BLOB también describe adecuadamente una gota o masa de una sustancia líquida. En este sentido, un blob tiene límites, pero su contenido, al igual que el de un líquido, carece de una estructura perceptible desde el exterior.

Esto no significa que no pueda existir una estructura o definición de los datos en un BLOB, sino que dicha estructura y definición se determinan completamente dentro de la aplicación que accede a él. El propio sistema de almacenamiento, en nuestro caso Cloud Storage, controla los límites de un BLOB: su tamaño, ubicación y vida útil. Cloud Storage gestiona el BLOB como una unidad atómica completa. Una aplicación que procesa un BLOB conoce el significado de las diversas secuencias de bytes que contiene.

La mayoría de los desarrolladores de software, y de hecho la mayoría de los usuarios de computadoras, conocen los blobs por su nombre histórico común: archivos. En la práctica, Google Cloud Storage es un sistema de archivos, y los blobs son archivos.

Cloud Storage almacena datos en la unidad de un BLOB, que, como ya hemos descrito, es la implementación de un archivo. La analogía de Cloud Storage con un sistema de archivos típico basado en un sistema operativo se extiende al uso de un bucket, que es el equivalente funcional de un sistema de archivos. Los buckets son los contenedores donde se almacenan los blobs; cada bucket se configura con opciones de rendimiento, retención y fiabilidad inherentes a todos los blobs almacenados en él.

Una opción recientemente introducida para los buckets de almacenamiento en la nube es el espacio de nombres jerárquico. Un bucket con un espacio de nombres jerárquico permite almacenar los blobs en una jerarquía de carpetas, como directorios en Linux o carpetas en un sistema Windows.

El almacenamiento en la nube se distingue como servicio de manera similar a todos los demás.

Servicios de GCP: disponibilidad, fiabilidad y escalabilidad. Además, Cloud Storage ofrece opciones para diferentes características y costes de servicio.

A continuación, enumeraremos y describiremos brevemente estas facetas de Cloud Storage:

- **Rendimiento y fiabilidad:** Los buckets se pueden configurar para estar presentes en cualquier número de regiones de red. Los procesos de la misma región pueden escribir y leer en una réplica de un bucket, lo que reduce la latencia y el coste del tráfico de red. Cloud Storage sincroniza automáticamente los datos escritos en todas las copias regionales de un bucket para que los datos de cualquier región sean coherentes con los de cualquier otra copia. Esta replicación no solo reduce la latencia, sino que también proporciona fiabilidad y disponibilidad de los datos. Si los datos de una región se dañan o se pierden, Cloud Storage enrutará automáticamente las solicitudes de lectura y escritura de esa región a una región similar cercana, mientras repara la réplica dañada utilizando los datos de las otras copias.
- **Opciones de servicio:** Cloud Storage admite un nivel de servicio configurado para un depósito, denominado clase de almacenamiento del depósito.
La clase de almacenamiento determina el tiempo máximo de retención de los datos almacenados en el depósito y la latencia de lectura y escritura para el depósito. Además, la Clase de Almacenamiento dirige la durabilidad de los datos, es decir, la garantía de permanencia de los datos.

El costo del almacenamiento en la nube se basa en el uso. Se paga por los bytes almacenados y accedidos, sin tarifas fijas por el servicio en sí. Las tarifas específicas que se aplican al usuario por el uso se basan en la clase de almacenamiento utilizada. Las clases de almacenamiento más comunes y algunos aspectos clave de su uso previsto se enumeran en la siguiente figura:

Storage Class	Retention limit	Relative durability	Access patterns	Relative cost
Standard	short lived	low	frequent access	most costly
Nearline	30 days	highly durable	infrequent access	low
Coldline	90 days	highly durable	infrequent access	very low
Archive	365 days	highly durable	rarely accessed	lowest

Cifra **4.5** Clases de almacenamiento ofrecidas por Cloud Storage

Al evaluar el almacenamiento en la nube como una opción de almacenamiento se deben considerar las siguientes opciones:

Uso, durabilidad necesaria y tamaño de los datos almacenados. El almacenamiento en la nube es una buena opción para almacenar grandes cantidades de datos con una estructura interna específica para su aplicación. Un archivo de imagen es un ejemplo; la cantidad de datos en una imagen puede ser muy grande. Los megapíxeles se refieren a megabytes, y la información dentro de la imagen está codificada en un formato altamente especializado. En teoría, no es imposible almacenar los valores de los píxeles de una imagen en una tabla de base de datos relacional, pero no se obtendrán grandes beneficios, y el tiempo y el almacenamiento adicionales requeridos para dicha implementación serían muy elevados.

La selección de un tipo de almacenamiento en la nube adecuado puede realizarse equilibrando el uso de los objetos almacenados con su coste. Los datos generados en línea y transmitidos por streaming son fáciles de reproducir y no requieren una gran durabilidad, mientras que los historiales médicos de los clientes deberían ser lo más duraderos y duraderos posible.

Identidad y autenticación

Hemos revisado las principales consideraciones de un sistema informático: potencia de cálculo y almacenamiento de datos. Estos son fundamentales para la aplicación, cuya función es almacenar y manipular información sobre su dominio específico. Ahora abordaremos las clases de productos que se centran más en los requisitos del entorno informático y las necesidades técnicas de la aplicación, en lugar de en las inherentes al dominio de la aplicación.

La gestión de identidad y autenticación (IAM) es el primer tipo de producto que analizamos, y su objetivo es satisfacer los requisitos del entorno informático, en lugar de los requisitos directos del usuario de la aplicación. Esto se refleja en el hecho de que, si bien la capacidad de procesamiento y el almacenamiento de datos se han mantenido conceptualmente sin cambios durante la existencia de la industria informática comercial, la IAM ha surgido y evolucionado rápidamente con importantes cambios fundamentales en las últimas décadas. Los primeros usuarios de computadoras dependían de métodos de autenticación simples, ingenuos para los estándares actuales. Uno de los primeros sistemas operativos populares, el Programa de Control /

/ identificaba a los usuarios mediante el Número de Usuario

Sin necesidad de autenticación, usted ingresó su número de usuario y se le confió seguir adelante y utilizar el sistema.

Vivimos en un mundo donde muchas personas están conectadas a sistemas informáticos y pueden acceder a nuestras aplicaciones y datos, por lo que nuestras barreras de acceso son mayores. Cuanto mayor es el valor comercial, político y personal que se atribuye a las computadoras, mayor es el incentivo para el uso malicioso de estos sistemas, así como el impacto de los casos accidentales de identidad errónea. A medida que los intentos de obtener acceso no autorizado se vuelven más sofisticados, los mecanismos para prevenirlo mejoran, lo que resulta en una escalada mutua.

Mantenerse al día con el estado de la situación es algo que un desarrollador de aplicaciones responsable deja en manos de expertos y productos de ciberseguridad. Describiremos y compararemos algunos de los productos que se ofrecen como parte de GCP.

Autenticación de Firebase

Firebase Authentication, al igual que Cloud Firestore, es un producto que forma parte de la suite de desarrollo de aplicaciones cliente de Firebase. Firebase Authentication es un marco integral para gestionar la autenticación de usuarios desde y hacia aplicaciones cliente móviles o web.

Funciones de autenticación de Firebase

En el caso de Firebase, la autenticación de extremo a extremo busca transmitir todo el ciclo de vida de la identidad de un usuario. El registro o la creación de una cuenta de usuario, el inicio y cierre de sesión, y el mantenimiento del perfil y las credenciales del usuario se gestionan mediante Firebase Authentication.

Además de cubrir completamente los requisitos de IAM de una aplicación, Firebase Authentication ofrece varias implementaciones alternativas de la identidad de usuario. Un usuario de Firebase Authentication puede usar una identidad de usuario específica de la aplicación o identidades existentes, como cuentas de Google y Apple, que cumplan con los estándares de la industria.

IAM; y puede utilizar autorización basada en teléfono o correo electrónico para las cuentas. Firebase Authentication también ofrece autenticación anónima, la creación de una cuenta anónima temporal que luego puede actualizarse a una cuenta de usuario conocida preservando la continuidad de la actividad del usuario.

La autenticación de Firebase se actualiza y amplía periódicamente para mantenerse al día con los últimos avances en protección de la identidad del usuario, como su compatibilidad con la autenticación multifactor (MFA), que se puede agregar a las cuentas de usuario ya sea como opción voluntaria o según lo requieran los desarrolladores de una aplicación.

Al evaluar Firebase Authentication como una opción de producto, se debería preferir como opción predeterminada, a menos que requisitos específicos la descarten.

Firebase Authentication está disponible en varios niveles de servicio que ofrecen diferentes funciones y estructuras de costos. El costo del servicio Firebase Authentication se basa en el uso, al igual que los demás servicios de GCP descritos. Los cargos de Firebase se calculan en función del número de usuarios activos diarios (DAU), que corresponde a la cantidad de usuarios únicos (no sesiones de usuario) que iniciaron sesión durante un período de 24 horas.

Si una aplicación incluirá una plataforma de cliente enriquecida, es decir, una aplicación cliente móvil o web, y desea delegar o externalizar el conocimiento específico del dominio sobre identidad y autenticación, se debe considerar seriamente Firebase Authentication. Firebase Authentication se puede configurar e implementar en una aplicación en un tiempo mínimo. Para los desarrolladores que desean controlar la apariencia del flujo de autenticación, Firebase permite personalizar la interfaz de usuario (IU) del flujo de inicio de sesión mediante las opciones de FirebaseUI. Si un desarrollador desea una IU de inicio de sesión verdaderamente única, el código de Firebase UI se puede bifurcar en GitHub y personalizar.

Google [Cloud IAM](#) es un servicio

inherente e incluido en todos los productos de GCP. Cloud IAM permite definir roles y usuarios, así como asignar permisos a los usuarios para diferentes recursos de GCP. A diferencia de Firebase Authentication, Cloud IAM se centra en la nube.

Recursos, como almacenes de datos y máquinas virtuales, y en las abstracciones, como los roles a los que se permiten o deniegan las operaciones en esos recursos en la nube. La experiencia de usuario (UX) de la gestión de identidades, como las páginas de inicio de sesión o cambio de contraseña, se implementa fuera de Cloud IAM; en la autenticación de Firebase o en algún otro producto centrado en el cliente.

Roles versus usuarios

Una consideración importante al diseñar un sistema seguro es la disociación de los permisos de acceso de usuarios específicos. Un ejemplo que ilustra este principio es el de una función en la nube llamada `increase_credit_limit()` en un sistema de compras. Esta función, como su nombre indica, permite al usuario comprar productos sin pagar. El permiso para otorgar crédito a los compradores es una facultad que conviene limitar a miembros internos y de confianza del personal de atención al cliente.

Una forma de administrar estos permisos es agregar cada usuario calificado a una lista de invocadores permitidos del método `increase_credit_limit()`.

Esto en sí no es un problema obvio, pero luego sus desarrolladores se dan cuenta de que necesitaban implementar otro método, `revoke_credit(user)`, y otro llamado `get_all_users_with_credit()`.

— — — — — `()`

Ahora mantiene tres listas de usuarios autorizados, separadas pero idénticas, lo que eventualmente generará inconsistencias si no están sincronizadas. En lugar de este método de registro de permisos con entradas dobles, triples o más, Cloud IAM, al igual que la mayoría de los sistemas de control de acceso, implementa el concepto de rol: una abstracción de usuarios. Si un usuario es un nombre propio, como Larry Page, un rol es un nombre común relacionado, como Director Ejecutivo.

Aplicando este modelo a nuestro ejemplo de crédito para compradores, definiríamos un rol llamado `Credit_granter` y permitiríamos que este rol invocara nuestras tres funciones. Cuando un nuevo representante de atención al cliente (CSR) recibe permiso para gestionar el crédito para compradores, simplemente añadimos el rol.

Credit_granter a los roles asignados al CSR específico. Cuando un CSR registrado intenta invocar `increase_credit_limit()` para un comprador, si el usuario del CSR tiene el rol `( )` Credit_granter , el permiso se le otorga transitivamente y la operación se ejecuta; de lo contrario, se genera un error de permisos.

El concepto de entidad que accede a recursos, como un usuario o un rol, se conoce generalmente como principal. Resulta práctico y generalmente válido considerar un principal como un usuario específico o una entidad que actúa como representante de un usuario.

Veamos ahora las políticas de IAM.

Las políticas de IAM son reglas, especificaciones que rigen cómo se puede llevar a cabo una operación solicitada por un Principal en un recurso determinado. Existen tres tipos de políticas, y se puede definir cualquier número de ellas para un recurso. La autorización del acceso solicitado depende de todas las políticas asociadas al recurso al que se accede.

Los tipos de políticas de IAM son:

- Permitir políticas: especificación de qué principales tienen permitido acceder al recurso.
- Políticas de denegación: especificación de a qué entidades se les niega el acceso al recurso.
- Políticas de límites de acceso principal (PAB) : especificación de los límites de los recursos a los que los principales pueden acceder.

Cloud IAM debe considerarse una base recomendada para la solución integral de gestión de identidades de una aplicación. Google Cloud IAM no es un producto que un desarrollador de aplicaciones opte por incorporar en una aplicación respaldada por GCP. Las funciones de gestión de acceso de GCP IAM son inherentes a todos los demás servicios de GCP y se utilizan, en cierta medida, incluso en las aplicaciones distribuidas más sencillas. Las decisiones que enfrentan los desarrolladores al diseñar una aplicación de GCP son qué funciones específicas de IAM utilizarán y qué tan sofisticada será la jerarquía de políticas que desarrollarán para los recursos de su aplicación.

Hay muchas primitivas predefinidas en el producto IAM que se pueden usar cuando

Construyendo la política de acceso general de su aplicación. Roles predefinidos para cada servicio de GCP, una jerarquía de recursos inherente y un conjunto de herramientas de inteligencia de políticas están disponibles para todos los proyectos y desarrolladores de Google Cloud para ayudar a comprender y controlar los requisitos de control de acceso de las aplicaciones de GCP.

Plataforma de Identidad:

es un conjunto de kits de desarrollo de software (SDK) que se utilizan para proporcionar un servicio de autenticación personalizable para su aplicación basada en GCP. Identity Platform puede considerarse como una contraparte o equivalente a Firebase Authentication, descrita en la sección anterior . Los desarrolladores que utilizan Firebase Authentication pueden optar por usar Identity Platform como backend para la interfaz de usuario de Firebase, en lugar de los componentes propios de Firebase. Identity Platform proporciona la experiencia de usuario para las políticas de control de acceso definidas e implementadas por Cloud IAM.

La Plataforma de Identidad incluye SDK para aplicaciones cliente en iOS, Android y la web (basadas en navegador). Existen SDK de cliente para las tareas de administración de la Plataforma de Identidad en varios idiomas.

Todo el ciclo de vida de la identidad del usuario: el registro de un nuevo usuario, el inicio y cierre de sesión del usuario y la gestión de credenciales, como el cambio de contraseña o la actualización a MFA, se puede realizar con los SDK que proporciona Identity Platform.

Identificar las características de autenticación de la plataforma

La Plataforma de Identidad ofrece la posibilidad de admitir diversos tipos de identidad, a discreción del desarrollador de la aplicación. Algunos de los tipos de identidad admitidos son los siguientes: Correo

- **electrónico/contraseña:** Se utiliza una dirección de correo electrónico como identidad del usuario y como medio de verificación. Se asocia una contraseña segura al usuario y se gestiona mediante procesos seguros.
- **Redes sociales:** Las cuentas de usuario existentes en proveedores de identidad federada, como Google, Apple y GitHub, se utilizan para identificar

el usuario en esta aplicación, y la autenticación se realiza a través de esos proveedores.

Teléfono: Un número de teléfono identifica a un usuario y el acceso a ese teléfono se utiliza para autorizar la identidad de un usuario.

- Lenguaje de Marcado para Aserciones de Seguridad (SAML) y OpenID Connect (OIDC): Son estándares de IAM del sector que utilizan numerosos sistemas corporativos. Identity Platform puede interoperar con sistemas que los utilizan para ampliar las aplicaciones de GCP e incluirlas en el inicio de sesión único (SSO) dentro de un entorno corporativo.
- Sistema de autorización personalizado: si ninguno de los sistemas de gestión de identidad existentes satisface las necesidades particulares de una aplicación, Identity Platform proporciona un conjunto de SDK que permiten a un desarrollador diseñar e implementar su propio modelo de identidad de usuario.

Identity Platform proporciona el soporte necesario para implementar una solución completa para la gestión de identidades orientada al usuario: registro y gestión de perfiles de autoservicio, inicio y cierre de sesión, e incluso algunos niveles de recuperación de cuentas. El producto también incluye bibliotecas para facilitar la administración de usuarios. Tenga en cuenta la elección de palabras en las oraciones anteriores: Identity Platform proporciona el soporte necesario, pero no ofrece la experiencia de usuario de gestión de identidades lista para usar que ofrece Firebase Authentication.

El desarrollador debe seleccionar e incorporar la funcionalidad disponible de Identity Platform en su aplicación cliente. Una analogía útil es la del menú de un restaurante: mientras que Firebase Authentication ofrece una variedad de menús predefinidos, Identity Platform ofrece una gran variedad de platos a la carta entre los que el desarrollador debe elegir un conjunto compatible de opciones para preparar una comida satisfactoria.

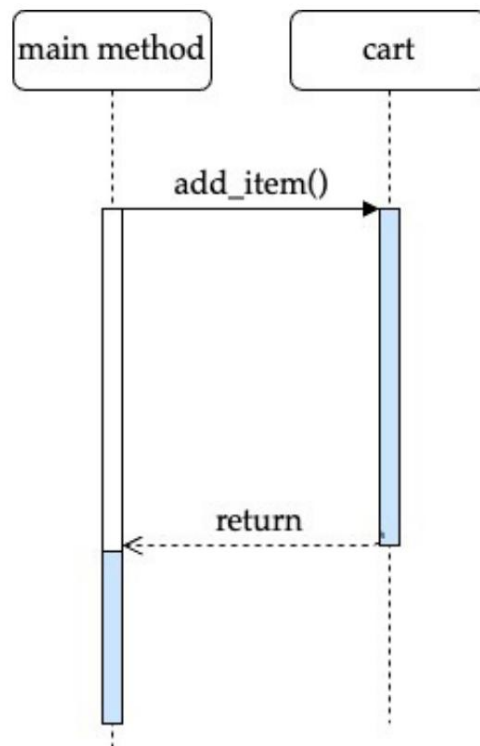
Como ocurre con la mayoría de los productos de Google Cloud Platform, a los desarrolladores se les cobra por Identity Platform según el uso. El uso de Identity Platform se mide en usuarios activos mensuales (MAU), que es la cantidad de usuarios que han iniciado sesión durante un mes determinado, independientemente de cuántas veces hayan accedido al sistema. Algunos métodos de inicio de sesión, como la autenticación telefónica, pueden generar cargos de terceros, por lo que...

El uso de dichos métodos también se cobra por cada mensaje enviado.

Manejo asíncrono de mensajes

Un factor importante en el diseño de una aplicación responsiva y confiable es la distinción entre importancia y urgencia, que se analizó en el capítulo anterior. Una opción de implementación fundamental es la que se establece entre operaciones síncronas y asíncronas, que definiremos a continuación.

La operación síncrona clásica es aquella con la que todo desarrollador de software se ha encontrado: una llamada a una función o método. Esta operación involucra a dos partes: quien invoca o llama a una función y la función invocada. La naturaleza síncrona de una llamada a un método reside en el orden estricto y la secuencia rígida de operaciones, como se muestra en este diagrama de secuencia de una llamada al método `add_item` de un objeto de carrito :



Cifra 4.6 Diagrama de secuencia de llamada de método

La barra azul representa la ubicación de cualquier instrucción que se esté ejecutando.

Ejecutado. Como podemos ver, cuando el método principal invoca el método `add_item`, cede el control del procesador al código del método del carrito, y el método principal solo reanuda la ejecución cuando el método `add_item` recupera el control. El método `add_item` se invoca sincrónicamente y las operaciones entre ambas partes se ejecutan secuencialmente.

Cuando el invocador de una función se encuentra en un proceso del sistema operativo distinto al de la función invocada, se utiliza el término RPC. Aparte del límite entre procesos de la invocación y el retorno, no se modifica lo descrito anteriormente; el invocador espera, en su proceso, mientras se ejecuta la función remota.

Por el contrario, una operación asincrónica es aquella que es invocada por un llamador, que no espera a que la función invocada se complete o devuelva el control; de hecho, el control nunca se cede.

Relacionando estos dos modelos de ejecución con la cuestión de la urgencia y la importancia, se necesita una operación sincrónica para proporcionar una respuesta urgente, mientras que si la respuesta a una solicitud de una operación no es urgente, también se puede utilizar un método de ejecución asincrónico.

Los productos de GCP que describiremos en la siguiente sección son dos que admiten la ejecución asincrónica de operaciones que casi con certeza son importantes pero no urgentes en la forma que acabamos de describir.

Sub es

un servicio de mensajería. Su único propósito es entregar un mensaje del remitente al receptor. Lo primero que analizaremos es la terminología utilizada en los sistemas de mensajería para nombrar a las entidades involucradas:

- Mensajes: Un conjunto de datos, definido por la aplicación, que se entrega de una parte a otra. El mensaje no contiene información ni referencias a un destino específico.
- Editores o productores de mensajes: Un proceso que crea un Mensaje y lo pone en tránsito para ser recibido y posterior

procesados en el futuro por algún otro proceso.

- Suscriptores o consumidores de mensajes: Un proceso que está abierto para recibir mensajes y enviarlos para su posterior procesamiento.

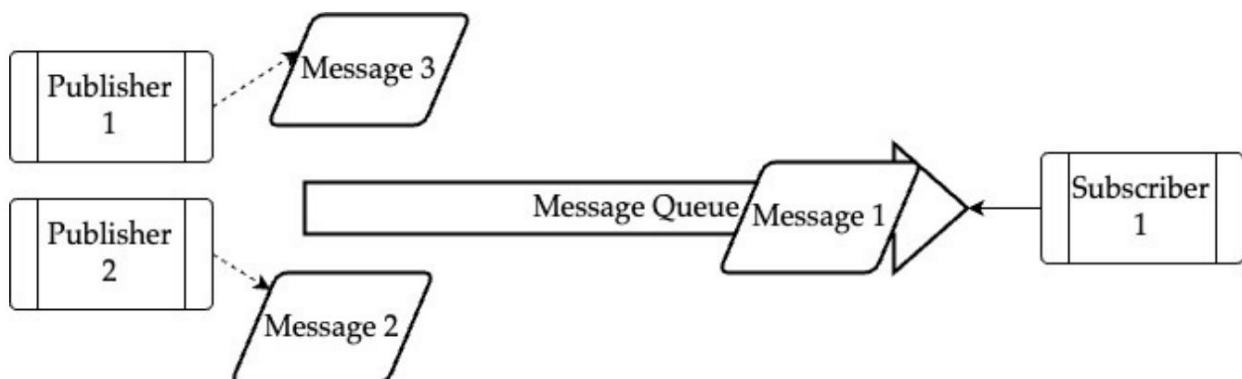
Un punto importante a destacar es que no existe una conexión directa entre un Editor y un Suscriptor. Los Editores no saben nada de los Suscriptores, y estos solo ven los Mensajes que reciben, no a quienes los crearon.

Escalabilidad de una arquitectura de mensajería. Dado que el suscriptor que procesa un mensaje es independiente y, de hecho, desconoce el proceso que lo publicó, un enfoque basado en mensajes permite un control excepcionalmente preciso sobre la escala de la capacidad de procesamiento. Pub/Sub puede configurarse para escalar horizontalmente el número de suscriptores para que coincida con la velocidad de publicación de los mensajes, garantizando así que la cola se mantenga en un número específico de mensajes publicados pendientes de ser consumidos por un suscriptor.

Considere los siguientes diagramas en la discusión que sigue.

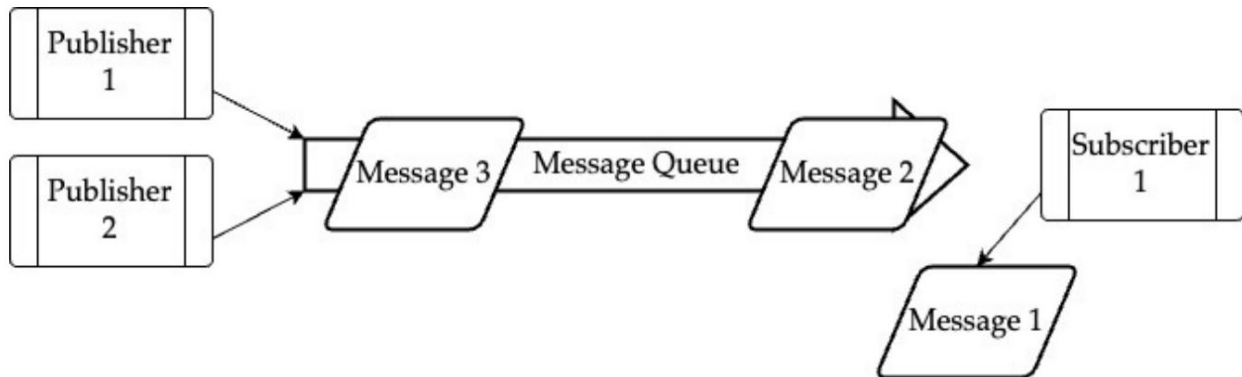
Como nuestra aplicación se ejecuta a una velocidad típica, dos procesos han producido mensajes que deberán procesarse.

En el primer diagrama, se creó previamente un mensaje existente y se colocó en la cola de mensajes, denominada "Tema" en Pub/Sub, para su entrega a los suscriptores. Nuestro proceso de suscriptor revisa este tema para ver si hay mensajes para procesar. Al mismo tiempo, nuestros dos publicadores ponen en cola mensajes adicionales:



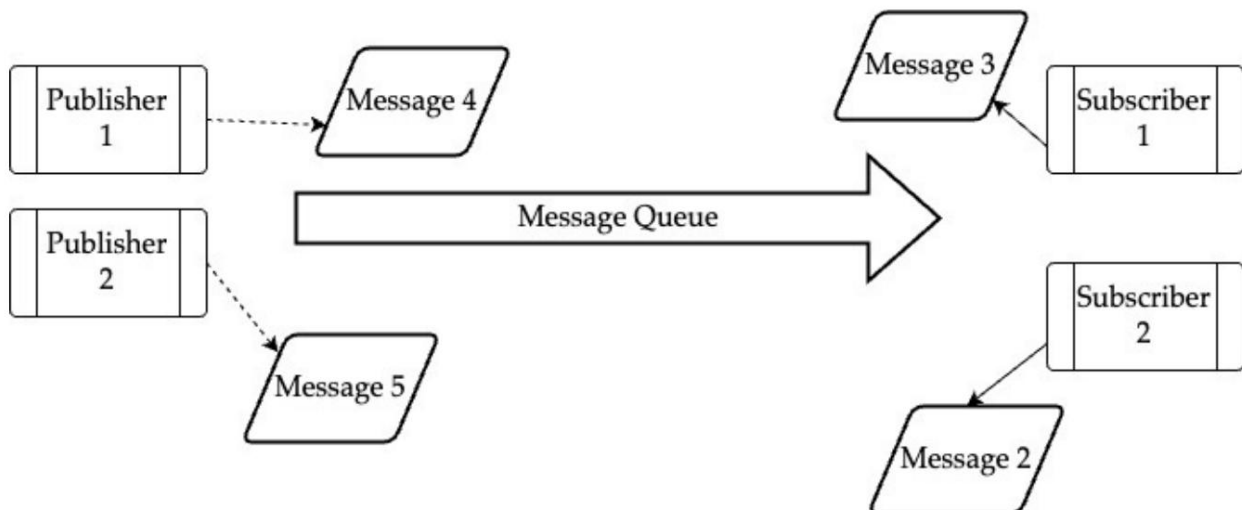
Cifra 4.7 El primer estado de nuestro procesamiento de mensajes

En el siguiente paso, el suscriptor ha tomado el mensaje del cola y comienza a procesarlo. Mientras esto sucede, nuestros editores han colocado sus nuevos mensajes en la cola:



Cifra 4.8 El segundo estado de nuestro procesamiento de mensajes con un retraso en la acumulación

Ahora tenemos más mensajes en la cola que suscriptores y Nuestros editores están creando nuevos mensajes. Podemos aumentar la Número de suscriptores para evitar una acumulación de mensajes en cola, como se muestra en la siguiente figura:



Cifra 4.9 El tercer estado de nuestro procesamiento de mensajes: un estado estable de procesamiento de mensajes.
consumo

Una oportunidad de escalamiento alternativa que podría llevarse a cabo en lugar de,

O, en conjunto con el escalamiento horizontal descrito anteriormente, se realiza el escalamiento vertical de los suscriptores para aumentar su tasa de procesamiento de mensajes. La flexibilidad para realizar este tipo de escalamiento adaptativo sin riesgo de interrupción de los procesos ascendentes en el lado de los editores de la cola de mensajes es posible gracias a la relación asíncrona y desacoplada entre los generadores de mensajes y sus procesadores.

La evaluación de Pub/Sub como opción de producto debe basarse en la consideración de sus características, limitaciones y coste. El uso de Pub/Sub solo aporta valor cuando se combina con un servicio adecuado para implementar las tareas del suscriptor/procesador. En tal contexto, y en una situación en la que una aplicación pueda utilizar este modelo, la incorporación de Pub/Sub puede proporcionar mejoras complementarias en el rendimiento y la fiabilidad de la aplicación. La comunicación unidireccional entre publicadores y suscriptores a través de Pub/Sub se produce en un tiempo del orden de 100 milisegundos, lo que se compensa con la velocidad de procesamiento constante bajo carga que alcanza una arquitectura basada en Pub/Sub.

Pub/Sub se factura con un modelo de costes más complejo que muchos de los productos analizados hasta ahora. En resumen, Pub/Sub se factura según los tres factores siguientes:

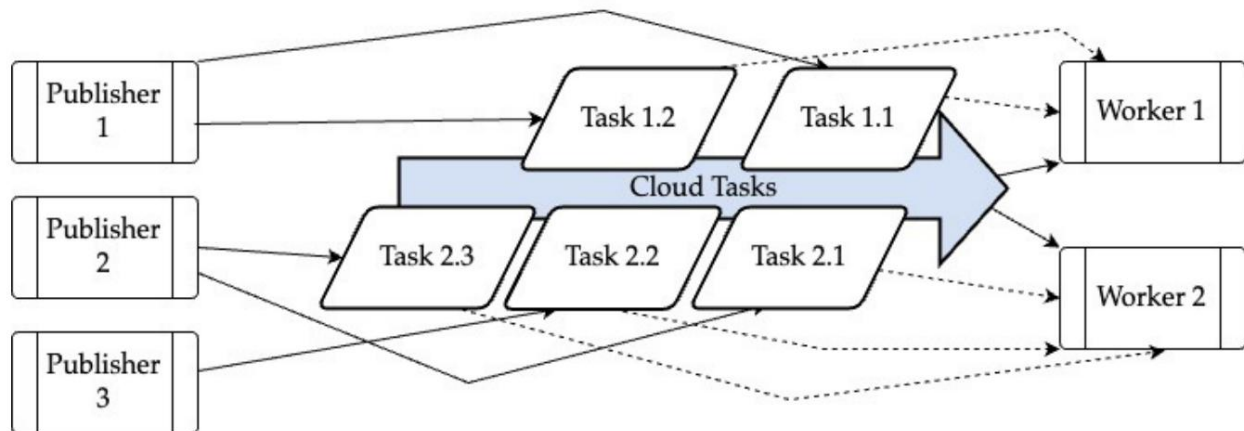
- Costos de rendimiento para el tránsito de mensajes desde los publicadores a los suscriptores.
- Costos de transferencia de datos para mensajes que cruzan los límites regionales de la red.
- Costos de almacenamiento de instantáneas de las colas y mensajes que se conservan después de ser entregados.

Tareas en la nube.

Tareas en la nube es un servicio administrado para el envío y la entrega de trabajo a un gran número de tareas de trabajo. Tareas en la nube no es un proveedor de computación que ejecuta su código; es el despachador de procesos que se ejecutan en uno de los productos de computación de GCP, como App Engine, Cloud Functions o un servicio de Cloud Run. Tareas en la nube es similar a Pub/Sub, ya que ambos son servicios que permiten la ejecución asíncrona del trabajo, en lugar de la ejecución síncrona inmediata. La clave...

La diferencia entre ambos radica en que los publicadores de Cloud Tasks no están desvinculados del endpoint que procesará la tarea; un publicador especifica el trabajador al que se entregará una tarea, pero sin la expectativa de que se ejecute inmediatamente. Cloud Tasks difiere la ejecución de una RPC específica. En contraste con Pub/Sub, donde los publicadores de mensajes desconocen dónde se entregará el mensaje, solo que se ha puesto en cola para estar disponible hasta su entrega.

La siguiente figura ilustra el estado de ejecución de un modelo de ejecución basado en Cloud Tasks:



Cifra **4.10** Tres editores envían tareas a dos suscriptores a través de Cloud Tasks

En este escenario, el Publicador 1 creó dos tareas y las envió al Trabajador 1 mediante Cloud Tasks. Simultáneamente, los Publicadores 2 y 3 crearon tres tareas que se envían al Trabajador 2. Cuando el Trabajador 1 esté disponible, se le enviará la Tarea 1 y, al terminar de procesarla e indicar a Cloud Tasks que está disponible, se le enviará la Tarea 1.2. De igual forma, cuando el Trabajador 2 indique que está disponible, Cloud Tasks le enviará primero la Tarea 2.1 y, posteriormente, las Tarea 2.2 y 2.3. La diferencia con Pub/Sub radica en que al Trabajador 2 nunca se le enviará una tarea destinada al Trabajador 1 y, de igual manera, el Trabajador 1 podría estar inactivo sin que se le envíe una tarea al Trabajador 2.

No se puede asignar trabajo asincrónico a ninguna instancia de trabajador disponible.

Siempre es una opción. En un ejemplo real, considere un sistema que procesa diferentes tipos de pedidos: tarjeta de crédito, cuenta bancaria, etc. Debería habilitar a cada trabajador para procesar todos los tipos de pedidos, lo que podría generar complejidad y problemas de recursos, o contar con trabajadores especializados en el procesamiento de pedidos para cada tipo de pedido y despachar un pedido para su posterior procesamiento por el trabajador correcto.

Al evaluar Cloud Tasks como opción de producto, se deben tener en cuenta sus limitaciones. Cloud Tasks está abstraído para poder entregar tareas a los trabajadores configurados en prácticamente cualquier producto de cómputo de GCP que proporcione un punto final HTTP. Cloud Tasks garantiza que las tareas serán entregadas y confirmadas por el trabajador designado, con reintentos configurables. Cloud Tasks tiene un límite de 500 consultas por segundo (QPS) o 500 tareas enviadas por segundo, a diferencia de Pub/Sub, que se escala a cualquier velocidad de entrega necesaria. Cloud Tasks no escala los trabajadores; gestiona la entrega de las tareas, dejando la gestión de los trabajadores al servicio en el que se implementan.

Las tareas en la nube se cobran por uso, como la mayoría de los productos de GCP. Al momento de escribir este artículo, las tareas en la nube son gratuitas para el primer millón de operaciones al mes y se cobran \$0.40 USD por millón de operaciones hasta 5 mil millones al mes.

Conclusión

Existen varios requisitos técnicos específicos para una aplicación distribuida que deben considerarse, y para cada uno de ellos, GCP ofrece múltiples opciones. La selección de productos puede realizarse analizando la idoneidad de cada opción en relación con la característica de la aplicación en cuestión y comparándola con los demás productos elegidos. Algunos productos informáticos pueden ser más o menos compatibles con servicios de almacenamiento específicos. En lugar de evaluar el uso de cada producto de forma aislada, el objetivo es seleccionar el conjunto de productos que ofrezca el mayor valor colectivo.

Se revisaron los factores de costo como el costo de los materiales, especialmente los servicios.

que puedan ampliarse en órdenes de magnitud, deben considerarse tan fuertemente como las capacidades técnicas de un producto.

Con esto, el lector tiene la información necesaria para evaluar los productos GCP analizados y un marco para investigar y describir los aspectos relevantes de cualquiera de los otros productos que no se trataron explícitamente aquí.

El siguiente capítulo se centra en las partes del sistema más alejadas de GCP: la aplicación de la plataforma cliente. Se describirán las tres plataformas cliente más populares: aplicaciones web basadas en navegador, aplicaciones móviles iOS y Android, junto con las opciones de desarrollo para cada una.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



Sección 2:

Aplicación respaldada por la nube Arquitectura

CAPÍTULO 5

Opciones tecnológicas del lado del cliente

Introducción

La forma en que un usuario accede a su producto es lo que define su aplicación para él. En los últimos años, este punto de interacción se ha materializado en algunos tipos de dispositivos: teléfonos inteligentes o tabletas, y computadoras de escritorio o portátiles. Este capítulo explorará las diferentes tecnologías utilizadas para representar un sistema informático a los actores humanos, que son los usuarios principales y la razón de ser de dicho sistema. Se presentarán al lector las herramientas y técnicas de desarrollo para optimizar el trabajo del desarrollador y, al mismo tiempo, ofrecer la mejor experiencia de usuario en cada plataforma cliente.

Estructura

En este capítulo se tratarán los siguientes temas:

- Aplicaciones móviles nativas
- Marcos móviles multiplataforma
- Aplicaciones web
- Aplicaciones cliente de escritorio
- Aplicaciones cliente en el navegador

- Chatbots y agentes de voz

Objetivos

El desarrollo de aplicaciones móviles, aplicaciones web y aplicaciones de escritorio locales o en el navegador tienen sus propias fortalezas y potencial.

mercados. Este capítulo resume los mercados comunes y emergentes.

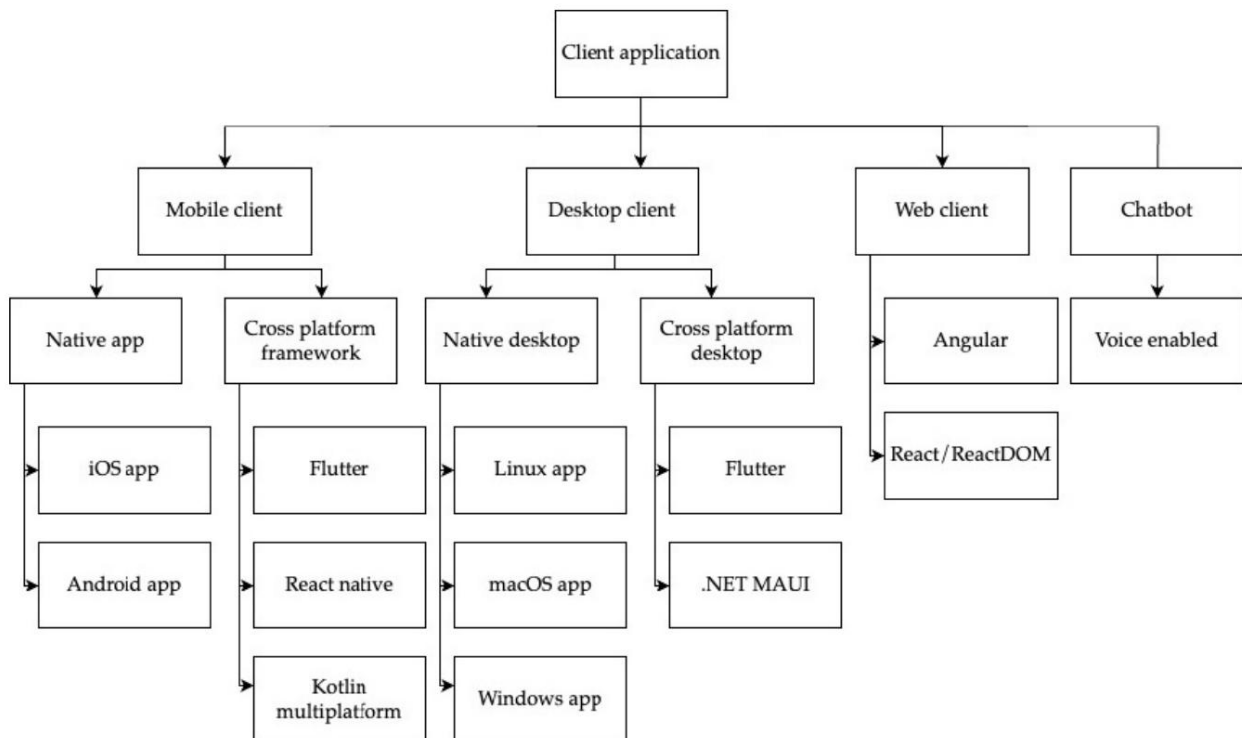
Tecnologías del lado del cliente que actúan como la cara pública de su producto.

Los costos tanto del desarrollo inicial como del mantenimiento de una aplicación

Se explorará en múltiples plataformas. El lector comprenderá la

Consideraciones a la hora de elegir desarrollar su interfaz de usuario como una aplicación cliente nativa de la plataforma, una aplicación cliente multiplataforma, una aplicación web basada en navegador o alguna combinación de estas.

Se ilustra la taxonomía de las plataformas y herramientas de desarrollo de clientes en la siguiente figura:



Cifra 5.1 Taxonomía de desarrollo de aplicaciones cliente

Aplicaciones móviles nativas

Las aplicaciones que usas en tu smartphone o tablet, con las mejores, únicas y novedosas funciones de interfaz de usuario (UI) , casi con seguridad están escritas con los frameworks y bibliotecas de desarrollo nativos y específicos de la plataforma para el sistema operativo de ese teléfono o tablet. Una excelente experiencia de desplazamiento, la gestión de ventanas más fluida y las animaciones de iconos más modernas se consiguen utilizando funciones de UI específicas del hardware y el sistema operativo.

El incentivo para que el desarrollador use estas bibliotecas específicas del proveedor en una aplicación es lograr la aplicación más atractiva y idiomática posible. El incentivo de los proveedores para que usted use sus herramientas de desarrollo es tener aplicaciones disponibles solo en su plataforma, o con la mejor experiencia de usuario en su plataforma, lo que atrae a más usuarios a su hardware.

Por supuesto, existe una desventaja para el desarrollador: la incompatibilidad entre plataformas. Una aplicación de iOS no funcionará en Android y viceversa. Los modelos de aplicación de ambas plataformas son tan diferentes que dificultan la migración del código fuente de una a otra, hasta el punto de resultar impráctico.

A continuación, describiremos brevemente las herramientas principales para el desarrollo de plataformas nativas en Android, iOS y iPadOS. También analizaremos las técnicas para interactuar con los productos de GCP desde dichas aplicaciones nativas.

[Herramientas de desarrollo para Android](#). Las

aplicaciones cliente nativas de Android se desarrollan principalmente con Android Studio, el entorno de desarrollo integrado (IDE) gratuito de Google. Existen otras cadenas de herramientas de desarrollo, otros IDE similares a Android Studio, como IntelliJ IDEA, y conjuntos de herramientas de línea de comandos que se ejecutan desde una ventana de terminal.

Android Studio está disponible para los sistemas operativos Windows, Mac y Linux, y la instalación incluye un conjunto completo de herramientas.

Un editor, un entorno de ejecución de Android, un compilador, un sistema de compilación, un depurador interactivo y simuladores de dispositivos para muchos teléfonos y tabletas Android están todos integrados en Android Studio.

Las aplicaciones de Android se pueden escribir en varios lenguajes de programación. Java, Kotlin, C++, C#, Node.js, Ruby y Go son opciones populares, pero los lenguajes con mayor compatibilidad con las herramientas principales son Kotlin y Java. Kotlin es un derivado, o quizás una evolución, del lenguaje Java, y los programadores de Java reconocen y comprenden la mayoría de sus construcciones. Si bien Google promueve Kotlin para el desarrollo de nuevas aplicaciones, Java es un elemento clave en el ecosistema Android y es el más utilizado en el soporte de la comunidad y en ejemplos de terceros.

Al desarrollar una aplicación Android con las herramientas recomendadas por Google, parece previsible que un desarrollador encuentre un buen soporte para la interoperabilidad con otros productos de Google, como los servicios de GCP. Esta suposición es cierta. Google Cloud proporciona bibliotecas cliente que encapsulan gran parte del código estándar para acceder a los productos de GCP en código idiomático específico del lenguaje. La cantidad de código que el desarrollador de aplicaciones necesita para especificar los metadatos y credenciales de su proyecto de GCP suele ser mayor que la necesaria para realizar cualquier operación específica de un producto de GCP.

Como ejemplo, el código para cargar un archivo local desde una aplicación cliente a un BLOB de Google Cloud Storage es tan simple como este fragmento

```
de Java: String bucketName = "my-unique-  
bucket"; String fileName =  
"readme.txt"; Storage storage =
```

```
StorageOptions.newBuilder().setProjectId(projectId).build().getService();  
BlobId blobId = BlobId.of(bucketName, fileName); BlobInfo blobInfo =  
BlobInfo.newBuilder(blobId).setContentType("text/plain").build();  
storage.createFrom(blobInfo, Paths.get(fileName)); Además de estas bibliotecas  
del lado del cliente en lenguaje nativo para la mayoría de los productos de  
Google Cloud Platform, un desarrollador también puede usar una API de transferencia de esta
```

aplicaciones, independientemente del idioma o la versión.

Herramientas de desarrollo para iOS.

Para desarrollar aplicaciones para iOS y iPadOS (en adelante, iOS), Apple ofrece Xcode. Xcode está disponible para todos los usuarios de macOS y tiene un alcance similar al de Android Studio. Xcode incluye editores de código fuente, un depurador interactivo, simuladores para dispositivos Apple y versiones de SO específicas, y un sistema de compilación.

Si bien se desarrollan aplicaciones iOS en diversos lenguajes, Apple ejerce un control mucho más estricto sobre el ecosistema de desarrollo para iOS que Google para Android. Esto se traduce en una experiencia de usuario más consistente y predecible para los usuarios de dispositivos Apple y un conjunto más reducido de opciones de herramientas para los desarrolladores. Las aplicaciones para iOS se pueden desarrollar en Swift y Objective-C. Si bien se pueden usar otros lenguajes a través de diversas herramientas y wrappers de terceros, estos son los principales lenguajes compatibles con Xcode para iOS. Objective-C es un lenguaje bastante antiguo, y Apple recomienda encarecidamente a los desarrolladores que utilicen Swift para el desarrollo de nuevas aplicaciones.

Google no proporciona el mismo conjunto completo de bibliotecas nativas para iOS que para el desarrollo en Android. Sin embargo, los desarrolladores pueden acceder a todos los productos de GCP a través de sus API REST, que son independientes del lenguaje y del cliente. Además, muchos productos de GCP están disponibles a través de interfaces de Firebase que proporcionan la mayor parte de la funcionalidad del servicio mediante un mediador de Firebase. Por ejemplo, aquí hay un fragmento de código Swift que usa Cloud Storage para Firebase para subir un archivo a un objeto binario grande (BLOB) de Google Cloud Storage: // File on local client let localFile = URL(string:

```
"resources/  
README.txt")!
```

```
// Crea una referencia al BLOB de almacenamiento de destino let
```

```
storageRef = storageRef.child("files/README.txt")
```

```
// Sube el archivo let
```

```
uploadTask = storageRef.putFile(from: localFile, metadata: nil) { metadata, error en
```

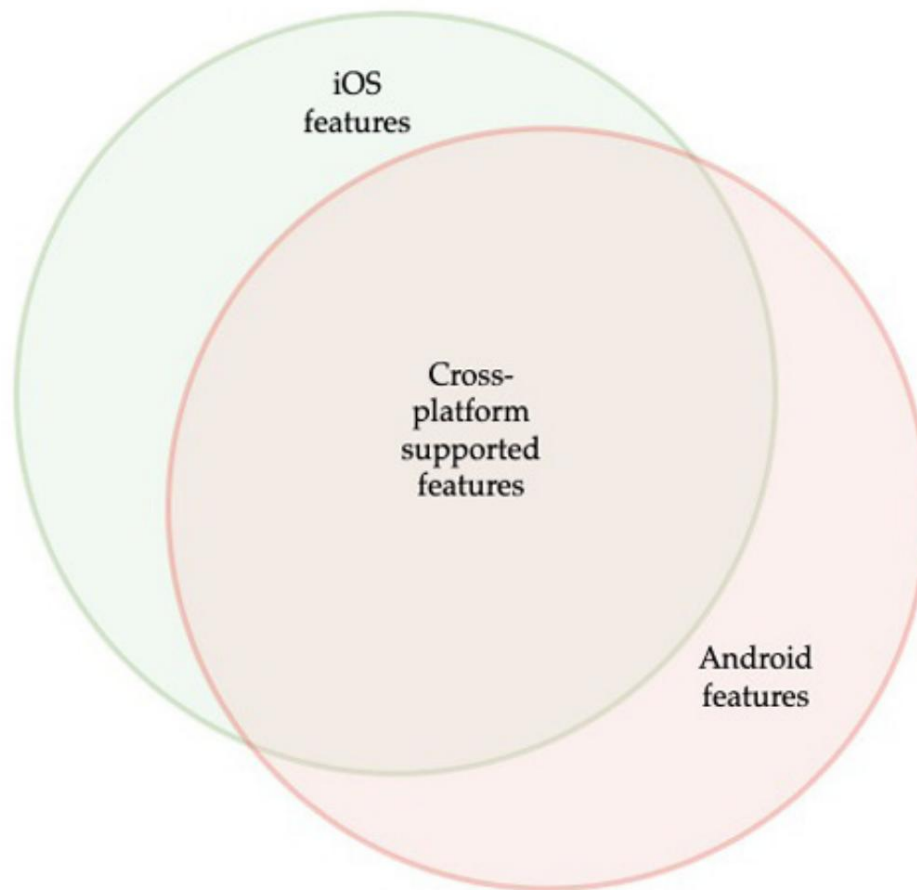
```
guardia deja metadatos = metadatos de lo contrario {  
  // Se produjo un error  
  
  devolver  
}
```

Si bien los productos de GCP difieren en los mecanismos exactos utilizados para el desarrollo de aplicaciones cliente nativas de iOS y Android, la funcionalidad equivalente de GCP está disponible para ambas plataformas. Si un desarrollador decide crear aplicaciones nativas independientes para ambas plataformas móviles, existen diferencias mucho mayores en sus bases de código que en las interfaces de GCP, y esto debería ser una consideración menor.

Marcos móviles multiplataforma

Un paso hacia una base de código común y, por consiguiente, un paso más allá de una experiencia de usuario completamente nativa es el uso de un framework de aplicaciones multiplataforma. Este consiste en un conjunto de clases de código y bibliotecas que ofrecen una vista abstracta o genérica de una aplicación móvil y sus componentes, como paneles, menús, iconos y listas de desplazamiento. En lugar de codificar llamadas a clases de Android o iOS como `LinearLayoutScrollView` y `UITableView`, respectivamente, un desarrollador usaría un `ListView` (en Flutter), que se representaría como una lista de desplazamiento vertical en Android o iOS mediante los widgets nativos de cada plataforma.

La compatibilidad multiplataforma y una aplicación nativa parecen la panacea. De hecho, esto es una buena combinación para la interfaz de usuario y los elementos de experiencia comunes en ambas plataformas, pero existe una brecha entre esto y los frameworks de aplicaciones nativas. El siguiente diagrama de Venn ilustra las deficiencias de un framework multiplataforma:



Cifra 5.2 Funciones multiplataforma

Si bien la mayoría de los conceptos y elementos de la interfaz de usuario son comunes para todas las plataformas móviles, hay una serie de características que son exclusivas de Apple, Android o dispositivos específicos bajo cada una de estas plataformas.

Características de iOS como los botones de acción flotantes no tienen equivalente en Android, mientras que el cajón de navegación de Android no tiene equivalente en iOS.

El control detallado de estas funciones específicas de la plataforma suele requerir que el desarrollador utilice un framework nativo de aplicaciones móviles. Además, el uso directo de la implementación nativa de una función ofrecerá un mejor rendimiento que el uso indirecto o su equivalente genérico, proporcionado mediante una abstracción multiplataforma.

Aleto

Flutter es un framework para crear aplicaciones cliente que se ejecutan de forma nativa en iOS y Android. Además, Flutter permite crear aplicaciones web para

Windows, macOS y Linux.

Flutter es un proyecto de código abierto respaldado por Google. Las aplicaciones de Flutter están escritas en Dart, un lenguaje de programación de código abierto. Google proporciona bibliotecas cliente para varios productos de GCP y otros productos de Google, como Google Translate y Docs. Si bien un desarrollador puede usar las API REST para acceder a todos los servicios de GCP, el uso de bibliotecas nativas, cuando estén disponibles, resultará en un código más simple y menos propenso a errores.

[React Native](#) es un

framework de interfaz de usuario de código abierto desarrollado por Meta. React Native permite crear aplicaciones que se renderizan de forma nativa en iOS y Android. Las aplicaciones React Native están escritas en JavaScript, lo que garantiza una gran cantidad de recursos técnicos.

Una de las fortalezas de React Native es la extensa y comprometida comunidad de desarrolladores y su conocimiento técnico disponible.

Google no ofrece bibliotecas de cliente React Native para productos GCP. Varios productos de GCP se pueden usar desde aplicaciones React Native a través de una biblioteca cliente de Firebase, similar a la compatibilidad descrita anteriormente para iOS. Se puede acceder a todos los productos de GCP mediante sus API REST universales a través de un cliente HTTPS o mediante la interfaz gRPC de cada producto mediante una biblioteca gRPC de JavaScript.

Kotlin [Multiplataforma](#) (KMP) es

una característica del lenguaje Kotlin que adopta un enfoque muy diferente al de Flutter. Kotlin Multiplataforma proporciona un marco para separar la lógica de negocio y el flujo de control de una aplicación del código de la interfaz de usuario (UI), entendiendo que la primera es independiente del dispositivo y la segunda se implementa mediante código específico de la plataforma. Así, mientras Flutter implementa todo el código de la aplicación, incluyendo la representación de la UI, una aplicación KMP aísla el código de la interfaz de usuario en módulos que se desarrollan por separado para iOS y Android utilizando los marcos nativos de cada uno.

El código de la interfaz de usuario (IU) se puede desarrollar con Java o Kotlin en Android y Swift en iOS. JetBrains, creador de KMP, también ofrece herramientas de IU declarativas como Compose Multiplatform, que permiten a los desarrolladores crear IU compartidas en aplicaciones de Android, iOS, de escritorio y web. Las aplicaciones creadas con este framework de IU multiplataforma se asemejan más a la aplicación Flutter; si bien el código cliente se renderiza con frameworks nativos de la plataforma, el nivel de especificación y control no incluye características exclusivas de una plataforma.

La compatibilidad con los productos de Google Cloud Platform en aplicaciones KMP es inherente a la compatibilidad de Google con el lenguaje Kotlin. Los puntos donde una aplicación cliente interactuará con los servicios de GCP se encuentran dentro de los módulos de lógica de negocio de una aplicación KMP, codificados en Kotlin. Google proporciona las bibliotecas del lenguaje Kotlin para varios productos de GCP, como Cloud Storage, Bigtable, Firestore y Pub/Sub, entre otros. Si un producto de GCP aún no ofrece una biblioteca de Kotlin, se puede usar la API REST del producto para acceder a todas sus funciones.

Aplicaciones web

Una aplicación cliente web no se refiere a un sitio web renderizado del lado del servidor, como una página web que utiliza lenguajes de marcado como HTML y CSS. Con la llegada de JavaScript (alrededor de 1996), las páginas web podían descargar código al navegador, que se ejecutaba en lugar de ser simplemente renderizado por el navegador. Cuando una aplicación se denomina aplicación web, consiste en código que se ejecuta en el navegador del cliente o usuario final en su dispositivo, ya sea una computadora de escritorio, una tableta o un teléfono.

JavaScript es el lenguaje de programación que todos los navegadores modernos pueden ejecutar y en el que están escritas las aplicaciones cliente web. Sin embargo, las complejidades de una aplicación web típica llevan a que prácticamente todos los clientes web de gran tamaño se escriban utilizando un marco o bibliotecas de módulos JavaScript.

Describiremos los marcos y las consideraciones de arquitectura en detalle en una sección posterior sobre aplicaciones cliente en el navegador.

Aplicaciones cliente de escritorio

Si bien muchas aplicaciones distribuidas ofrecen interfaces móviles específicas para cada plataforma, durante varios años la tendencia ha sido confiar en clientes basados en navegador para los usuarios de escritorio.

Nota: Escritorio en este uso se refiere no sólo a computadoras de escritorio sino también a computadoras portátiles o, en general, a una computadora con un teclado y un mouse.

La razón de esto es la creciente capacidad de los navegadores de escritorio y la divergencia de los sistemas operativos de escritorio. Desarrollar versiones independientes de una aplicación para diferentes versiones de Windows, macOS y Linux es un trabajo considerable. Los desarrolladores de navegadores como Chrome y Safari se han esforzado por que sus productos representen las mismas páginas web y aplicaciones integradas en estos diferentes sistemas operativos cliente. Por lo tanto, si un desarrollador cliente escribe su aplicación utilizando las funciones que ofrecen estos navegadores, puede aislarse de las diferencias entre plataformas y...

preocupaciones.

Sin embargo, existen algunas funciones no disponibles o requisitos de rendimiento que una aplicación cliente web integrada en el navegador no puede cumplir. En estos casos, la solución es un binario nativo para el sistema operativo de escritorio.

Desarrollo nativo de escritorio. Las

aplicaciones de escritorio se desarrollan en una amplia variedad de lenguajes de programación y marcos y bibliotecas de soporte. Python, Go, C++, Ruby, Java y otros lenguajes están disponibles y son compatibles con cierto grado de compatibilidad en los entornos de desarrollo de aplicaciones de escritorio. Para cualquiera de estos lenguajes, Google ofrece bibliotecas nativas del lado del cliente que permiten acceder a sus productos de GCP desde una aplicación cliente.

La máxima capacidad de respuesta en el cliente y una apariencia familiar se lograrán desarrollando aplicaciones de escritorio en el lenguaje y el framework preferidos para cada sistema operativo. Elegir el lenguaje y el framework ideales para cada sistema operativo es un proceso complejo.

El proceso implica equilibrar las recomendaciones de los proveedores (Apple y Microsoft) con el apoyo de la comunidad y las preferencias de los desarrolladores. Como regla general, si un desarrollador invierte en desarrollo de escritorio nativo, las soluciones recomendadas por los proveedores brindarán acceso a la mayor cantidad de funciones y generarán la menor fricción en la experiencia de desarrollo. Las herramientas de desarrollo recomendadas por estos proveedores para cada sistema operativo cliente se identifican de la siguiente manera:

- **Windows:** Microsoft ofrece Visual Studio, un IDE útil para mucho más que el desarrollo en Windows. El marco de desarrollo y la compatibilidad de lenguajes más potentes que ofrece Visual Studio se destinan, naturalmente, al desarrollo en Windows en PC de escritorio. Este objetivo es compatible con los marcos de Microsoft Foundation Classes y Windows Presentation Foundation, que se pueden usar con varios lenguajes, pero C# es el lenguaje principal para el desarrollo de aplicaciones en Windows. macOS: Apple siempre ha invertido mucho en las herramientas de la comunidad de
- **desarrolladores** y ha proporcionado herramientas de desarrollo y soporte de gran calidad. Xcode es el conjunto de herramientas reunidas en un IDE que Apple proporciona en macOS para desarrollar aplicaciones para todas las plataformas de Apple, incluidas las aplicaciones de escritorio de macOS. Swift es el lenguaje predilecto de Apple para el desarrollo de aplicaciones y, si bien Objective-C es compatible, está diseñado para admitir bases de código antiguas, por lo que el desarrollo de nuevas aplicaciones debería priorizar el uso de Swift. Apple ofrece varios marcos de aplicaciones e interfaces de usuario: SwiftUI y AppKit son creadores de aplicaciones interactivos en los que el desarrollador escribe código personalizado para añadir comportamiento específico a la aplicación y lógica de negocio. El marco de aplicaciones subyacente para macOS, Cocoa, proporciona acceso a todas las funciones de la interfaz de usuario de macOS y puede ser utilizado por un desarrollador sin necesidad de un creador de aplicaciones.
- **Aplicaciones GUI de Linux:** Es más difícil definir un entorno para las aplicaciones de interfaz gráfica de usuario (GUI) de escritorio de Linux que para Windows o macOS, ya que no existe un proveedor oficial cuyo estándar sea canónico. En su lugar, analizaremos el marco popular y con apoyo de la comunidad que ha evolucionado y tiene amplias posibilidades.

Soporte en muchas distribuciones de Linux, GTK.

GTK es un conjunto de herramientas de interfaz de usuario de código abierto mantenido por el equipo de GTK, cuyos colaboradores incluyen a varios de los proveedores más destacados de productos Linux, así como a numerosas personas cuya experiencia combinada permite que GTK sea técnicamente pertinente y agnóstico respecto a cualquier proveedor. GTK también permite crear aplicaciones para macOS y Windows con una apariencia nativa, aunque la compatibilidad de GTK con funciones nuevas o especializadas específicas de cada plataforma es inferior a la de las herramientas de desarrollo de Apple y Microsoft. GTK proporciona una colección de elementos de interfaz de usuario cooperativos, conocidos como widgets. GTK está disponible para numerosos lenguajes de programación y se basa en gran medida en los principios de la programación orientada a objetos para organizar y personalizar los elementos de la interfaz de usuario que codifica. Además de la construcción de la interfaz de usuario de una aplicación mediante código, existen varios constructores y frameworks de aplicaciones que proporcionan una capa superior a la jerarquía de clases de GTK.

[Desarrollo de escritorios multiplataforma](#). Existen numerosos

productos que permiten el desarrollo de aplicaciones para su implementación en múltiples plataformas de clientes de escritorio. Los más populares y con mayor soporte, a continuación, incluyen productos proporcionados por los principales proveedores de plataformas y con soporte explícito de GCP:

- Flutter: Flutter se mencionó anteriormente en este capítulo como una opción para el desarrollo de aplicaciones móviles multiplataforma. La misma compatibilidad multiplataforma de Flutter incluye los tres entornos de escritorio, y el mismo código base utilizado para desarrollar aplicaciones móviles también admite aplicaciones cliente de escritorio. Además de generar componentes nativos de interfaz de usuario para Windows, macOS y Linux, la misma compatibilidad con GCP para aplicaciones móviles está disponible para todas las aplicaciones de Flutter. Por lo tanto, se proporcionan bibliotecas nativas de Dart para muchos productos de GCP, y las API REST de cualquier servicio permiten acceder a su conjunto completo de funciones. Además de los componentes de Flutter dentro del lenguaje Dart, Flutter para escritorio incluye

complementos que son específicos de cada entorno de escritorio y que se pueden utilizar para interactuar con código nativo personalizado escrito en el lenguaje elegido de la plataforma.

- **.NET MAUI:** .NET es el marco multiplataforma de Microsoft para crear aplicaciones y servicios en la nube. .NET es gratuito y de código abierto y cuenta con la contribución de una gran comunidad de desarrolladores, pero con una inclinación hacia los desarrolladores con experiencia en las plataformas de Microsoft. .NET Multi-platform App UI (MAUI) es un framework específico dentro de .NET que facilita el desarrollo de aplicaciones multiplataforma de escritorio y móviles. Existen varios entornos de desarrollo de terceros que permiten desarrollar aplicaciones MAUI, pero Microsoft Visual Studio Code es la opción preferida y más popular. Las aplicaciones .NET pueden escribirse en C# o F#, y ambos pueden combinarse en un mismo proyecto.

Google proporciona bibliotecas cliente .NET para todos los productos de GCP. Además, existen bibliotecas cliente de terceros populares y de valor añadido que permiten la conexión a productos de GCP desde aplicaciones cliente .NET. Por último, las API REST de cualquier producto de GCP se pueden usar desde una aplicación .NET MAUI.

Aplicaciones cliente en el navegador

Una aplicación cliente que se ejecuta completamente dentro, o más precisamente, mediante un navegador web, ofrece numerosas ventajas. Dicha aplicación puede entregarse al cliente al ser invocada, lo que permite un control centralizado sobre la versión que el usuario ejecuta. La compatibilidad multiplataforma se confía a unos pocos desarrolladores de navegadores, en lugar de ser responsabilidad de miles de desarrolladores de aplicaciones individuales. Finalmente, la seguridad de una aplicación web, como se la conoce comúnmente, está en cierta medida garantizada por el entorno de pruebas impuesto por un navegador; el acceso a recursos locales, como los directorios raíz de los discos o los puertos USB, es limitado.

Estas fortalezas del entorno del navegador también pueden verse como

Limitaciones o advertencias desde una perspectiva diferente. El mismo entorno de pruebas, que impide que una aplicación web defectuosa o maliciosa sobrescriba o cargue archivos confidenciales, también limita la capacidad de la aplicación para copiar datos a una memoria USB. La ventaja de descargar la aplicación justo a tiempo también implica que esta no está disponible o es inútil sin conexión a internet.

Finalmente, como se ha visto como un patrón recurrente, cuanto más se aleja una aplicación del sistema operativo nativo y de las características de la interfaz de usuario, menos características únicas de esa plataforma nativa estarán disponibles para su uso en la aplicación.

Teniendo todo esto en cuenta, una aplicación web es una buena opción para una aplicación en la que gran parte de la funcionalidad está en la nube, y una experiencia consistente en todas las plataformas del cliente es más deseable que la apariencia más nativa en cualquier plataforma en particular.

[Lenguajes de programación de aplicaciones web](#). El lenguaje compatible con los principales navegadores es JavaScript, que también incluye su derivado, TypeScript. Existen muchos frameworks y bibliotecas que se utilizan en el desarrollo de aplicaciones web frontend, como React y Angular . Sin embargo, estos son frameworks de JavaScript que presentan una gramática de alto nivel y una jerarquía de componentes, pero son, en realidad, macros para el código JavaScript que ejecuta un navegador. Estos frameworks añaden soporte de alto nivel para los elementos de la interfaz de usuario y la interacción del usuario, pero el código que los integra e implementa la lógica empresarial está escrito en JavaScript. Para trabajar eficazmente con estos frameworks, un desarrollador primero debe comprender JavaScript.

[Angular](#) es

un framework de aplicaciones web de código abierto creado y mantenido activamente por Google, basado en un proyecto anterior, AngularJS. Angular cuenta con la colaboración de una amplia comunidad de colaboradores corporativos e individuales.

Los desarrolladores angulares escriben declaraciones declarativas usando HTML

Plantillas, que se compilan en instrucciones de JavaScript y se optimizan según el análisis estático de las plantillas declaradas. En sentido estricto, Angular compila en TypeScript, un lenguaje de tipado estático derivado de JavaScript. TypeScript es compatible con los principales navegadores como parte de la compatibilidad con JavaScript, por lo que la distinción entre ambos no afecta la portabilidad, pero sí facilita la optimización del código compilado por parte de Angular.

Google no proporciona bibliotecas cliente de Angular específicas para los productos de GCP. Existen varias técnicas para acceder a los servicios de Google Cloud Platform desde una aplicación Angular. La solución más directa consiste en acceder a la biblioteca cliente de Node.js de cualquier producto de GCP desde los componentes declarativos de Angular, y la documentación de GCP ofrece ejemplos de cómo hacerlo. Una solución más compleja consiste en crear un servicio de backend que actúe como proxy del lado del servidor para el acceso a los productos de GCP e invocar dicho servicio desde una aplicación Angular.

[React y ReactDOM](#) React es una

biblioteca para renderizar componentes de UI, escrita en JavaScript.

React extendió JavaScript con la sintaxis Markdown, conocida como JSX.

React es de código abierto, creado y lanzado al público por Facebook después de haber sido utilizado allí internamente.

React no es un framework para crear aplicaciones, por lo que se utiliza con una biblioteca complementaria para crear aplicaciones cliente. Para aplicaciones web, React se utiliza en combinación con el paquete ReactDOM, que proporciona los métodos necesarios para manipular objetos dentro del Modelo de Objetos del Documento (DOM) del navegador.

Google Cloud Platform es compatible con React, pero para los componentes implementados en Google Cloud, en lugar de como cliente local de los servicios de GCP. Hay varias maneras de acceder a la mayoría de los productos de GCP desde un cliente web de React/ReactDOM.

Firebase ofrece servicios de proxy para varios productos de GCP, como Cloud Storage. Esta es la solución descrita para aplicaciones React Native anteriormente en la sección " Móviles multiplataforma".

Marcos. Para un control más directo utilizando las mismas API que se usan en otras plataformas cliente, las API REST JSON y gRPC para productos de GCP se pueden usar desde JavaScript y, por lo tanto, desde React/ReactDOM. Esta solución también es compatible con GCP para aplicaciones móviles React Native.

Herramientas de desarrollo de aplicaciones web. La

necesidad de entornos integrados que integren herramientas de diseño, editores de código, depuradores y control de versiones es tan importante para las aplicaciones web como para cualquier otro tipo de componente. Debido a su naturaleza inherentemente distribuida, las aplicaciones web se desarrollan mejor utilizando herramientas que estén integradas con servicios web y cuenten con sólidas funciones de colaboración. Algunas de las herramientas más destacadas y populares son Visual Studio Code, IntelliJ IDEA, Android Studio, WebStorm y Codeanywhere. De esta lista, Codeanywhere destaca por ser una aplicación web en sí misma, con la compatibilidad multiplataforma y el requisito de acceso a internet que ello implica.

Compatibilidad de GCP con clientes de aplicaciones web. Las

aplicaciones web pueden acceder a los productos de GCP de muchas maneras. Google proporciona una biblioteca cliente de Node.js para todos los productos de GCP, que ofrece el máximo nivel de compatibilidad y funcionalidad para las estructuras de aplicaciones que la utilizan. La biblioteca cliente de la API de Google para JavaScript, también conocida como gapi, proporciona acceso a la API JSON de muchos servicios de Google, incluidos productos de GCP como Cloud Storage y Pub/Sub. También existen bibliotecas de terceros populares que permiten acceder a las API JSON y gRPC de los servicios de GCP.

Chatbots y agentes de voz

Una interfaz de usuario nueva, novedosa y cada vez más popular es la de un chatbot. Un chatbot es una interfaz pseudoconversacional para las funciones de una aplicación, que se basa en lenguaje natural para manipular el estado de la sesión del usuario. Ejemplos: el uso del Asistente de Google para

Controla dispositivos domésticos inteligentes, reproduce música o busca en internet. Otro chatbot con una experiencia de usuario muy diferente es el chatbot de la OMS de la Organización Mundial de la Salud, disponible en Facebook Messenger.

Chatbots y GCP

Existen numerosos frameworks para desarrollar un chatbot, algunos de los cuales se basan en LLM como ChatGPT de OpenAI o Gemini de Google. Estos frameworks suelen incluir componentes frontend y backend escritos en lenguajes adecuados, como React para el frontend y Python para el backend.

La experiencia de vincular un chatbot a los productos de GCP varía considerablemente según el backend y el nivel de personalización deseado. Gemini está integrado en Google Cloud como Gemini para Google Cloud y puede ayudar a los desarrolladores de GCP con la asistencia en la codificación, el análisis de datos y la administración de aplicaciones en la nube. Para integrar Gemini en la funcionalidad de sus aplicaciones, en lugar de administrarlo, se utiliza la API de IA de Vertex para entrenar modelos de aprendizaje automático, y los agentes conversacionales de Google, también conocidos como DialogFlow CX, para crear interfaces conversacionales. Vertex AI Agent Builder es un creador de aplicaciones para agentes conversacionales.

Para acceder a los productos de GCP en casos de uso distintos a los compatibles con estos productos, o para chatbots creados con backends ajenos a Google, como los productos de OpenAI, se recomienda seleccionar lenguajes para los niveles frontend y backend de la implementación del chatbot que sean compatibles con las API de GCP. Por ejemplo, si las consultas del usuario implican recuperar contenido de los buckets de Google Cloud Storage, un backend de chatbot escrito en Python podría usar las bibliotecas cliente de Python de GCP para obtener el contenido relevante solicitado por el usuario.

El uso de un modelo de lenguaje extenso conlleva el riesgo de un uso indebido, ya sea intencional o no, y de la generación de contenido dañino. Las herramientas y los procesos incluidos en el Kit de Herramientas de IA Generativa Responsable de Google deben utilizarse para garantizar la seguridad y la privacidad de los usuarios de un chatbot.

Interfaces de voz para chatbots

El uso de la voz, en lugar de la entrada del teclado, para hablar con un chatbot requiere la transcripción del habla del usuario a texto y la conversión complementaria de la respuesta textual de la computadora a voz.

Además de los factores comunes de compatibilidad de plataformas y lenguajes de programación, existen consideraciones adicionales al seleccionar servicios de voz: compatibilidad con idiomas y dialectos. Un transcriptor de voz debe estar entrenado en los idiomas naturales que los usuarios de una aplicación podrían hablar y debe ser capaz de detectar automáticamente el idioma hablado de forma fiable. De igual manera, el audio generado por una computadora debe ser comprensible para los usuarios de la aplicación, y la pronunciación y la inflexión deben ser apropiadas para el idioma, el dialecto y el tono del texto que se está convirtiendo.

Hay dos tipos de servicios que se requieren de una interfaz de voz:

- **Transcripción:** Los servicios de transcripción se basan en modelos de aprendizaje automático entrenados y consumen muchos recursos de la CPU. Existen numerosos paquetes de transcripción de voz, tanto de código abierto como comerciales, tanto locales como en la nube. La biblioteca Whisper es gratuita en OpenAI y ejecuta modelos de aprendizaje automático localmente para interpretar el habla. Un producto complementario, la API de OpenAI para Whisper, ejecuta los modelos en la nube de OpenAI para su uso con clientes que carecen de la potencia de procesamiento suficiente para ejecutar la transcripción localmente. La API de Google Speech-to-text proporciona un servicio de transcripción distribuida que se basa en componentes locales y en la nube para proporcionar transmisión y transcripción de voz en masa. **Texto a voz:** La conversión de texto a voz es computacionalmente exigente, pero es un problema más sencillo que la transcripción, con muchas soluciones disponibles. Estos servicios se conocen como motores de texto a voz (TTS). Algunos productos conocidos y de alta calidad son eSpeak, Mozilla TTS y MaryTTS. Si bien existen productos con backends basados en la nube, muchos producen voz de buena calidad ejecutándose en el hardware típico de la plataforma del cliente.

Uso de GCP con voz: La introducción de un transcriptor de voz y una interfaz de texto a voz es una capa adicional a la funcionalidad existente de un chatbot de texto y no debería afectar el uso de los servicios de GCP en la aplicación cliente del chatbot. Siempre que una aplicación de chatbot invoque un servicio de Google Cloud, la aplicación debe presentar la entrada del usuario y generar la respuesta, ya sea directamente como texto o convertida a audio.

Conclusión

En este capítulo se revisaron las diferentes plataformas cliente disponibles para proporcionar un portal a los usuarios en un sistema distribuido. Se analizaron los diferentes paradigmas de desarrollo, como el nativo, las capas de compatibilidad y el basado en navegador, junto con una descripción de algunos lenguajes y herramientas utilizados para cada uno.

Para vincular estos modelos de aplicación con nuestro enfoque en Google Cloud, el capítulo identificó las técnicas utilizadas para acceder a los servicios de GCP desde cada uno de estos tipos de aplicaciones, incluido el soporte directo proporcionado por Google.

En el siguiente capítulo, nos centraremos en la otra cara de la barrera entre cliente y servidor, las consideraciones de diseño y la metodología para descomponer la funcionalidad del backend y minimizar las dependencias estrictas entre implementaciones específicas de la aplicación y las características de la infraestructura. Aprenderemos a diseñar componentes con interfaces accesibles desde la mayor cantidad de tecnologías y en el mayor número de contextos posible.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



CAPÍTULO 6

Servicios en la nube en el backend

Borde

Introducción

Google Cloud Platform incluye numerosos servicios completos, con cierta superposición de funcionalidades. Además, algunos de estos productos tienen requisitos que compiten entre sí o son contradictorios. Estos dos factores pueden generar una arquitectura compleja que refleja aspectos e idiosincrasias particulares de los componentes del servicio en la nube seleccionados. Una aplicación robusta y duradera evita estas fugas de abstracción y presenta interfaces públicas bien definidas y eficaces en la frontera entre niveles. Las API públicas deben ofrecer operaciones diferenciadas, buscando la hermeticidad de los componentes que garantice efectos predecibles y visibles de las solicitudes y estados definidos.

Este capítulo presenta consideraciones clave de diseño y soluciones para abordar estas inquietudes en el diseño de sistemas de alto nivel de una aplicación distribuida. Estas prácticas generan aplicaciones que se adaptan a la única constante, el cambio, en la tecnología informática.

Estructura

En este capítulo se tratarán los siguientes temas:

- Mantenimiento de clientes independientes del backend
- Niveles de visibilidad para el cliente
- Planificación de la migración a servicios de nube sustitutos o equivalentes
- Visibilidad entre servicios en la nube
- Identificación y empaquetado de servicios estrechamente acoplados

Mantenimiento de clientes independientes del backend

La línea que separa el código del lado del cliente, que se ejecuta en las plataformas descritas en el [Capítulo 5](#) Opciones de tecnología del lado del cliente, y el código del lado del servidor o de la nube debería ser la línea más difícil en el diseño de una aplicación.

Esto significa que el código a ambos lados de esa línea debe tener la menor visibilidad o consideración posible de la implementación de los componentes del otro lado. Dicho de otro modo, las interfaces visibles entre el cliente y las plataformas en la nube deben ser lo más estrechas y superficiales posible. En este contexto, "estrecha" significa definir la menor cantidad de métodos posible, mientras que "superficial" significa tener parámetros y valores de retorno tan simples como sean necesarios para realizar el trabajo.

Los componentes cliente y servidor deben acceder entre sí únicamente mediante métodos públicos que convergen en parámetros y devuelven valores que contienen únicamente tipos de datos lógicos e independientes de la plataforma. Existe una máxima general en el diseño de sistemas que establece que todas las API deben ser lo más limitadas posible y estar compuestas por operaciones atómicas. Las ventajas de adherirse a este principio se ven magnificadas por algunos aspectos específicos de una interfaz cliente/nube. Revisaremos tanto el principio general como las consideraciones más importantes al utilizar un servicio en la nube.

[Principios generales de diseño de API](#). Existen numerosos

patrones generales para un diseño de API exitoso en ingeniería de software. Lo que sigue no es una lista exhaustiva ni un análisis profundo, sino un breve repaso de los principales puntos relevantes que se ampliarán en las secciones específicas para clientes y nubes.

Para promover una API fácil de entender, utilice formatos estándar y fácilmente digeribles para el intercambio de datos.

Las listas de parámetros, las respuestas y los tipos exportados deben ser legibles y utilizables sin incurrir en impuestos. En este caso, el impuesto es un costo indirecto que debe pagarse para desbloquear el activo adquirido directamente.

Para poner esto en contexto, si adopta un formato de datos nuevo o poco común en sus listas de parámetros que requeriría que la mayoría de los usuarios de su API dediquen tiempo a instalarlo, aprenderlo o depurarlo, esas tareas adicionales suponen un gasto. Por muy novedoso o posiblemente mejor que sea un nuevo formato, sus usuarios no quieren dedicar su valioso tiempo y recursos a este gasto. Además, existen herramientas, documentación y soporte de la comunidad para estándares existentes y conocidos que los usuarios de un formato nuevo no pueden aprovechar. Si su intercambio de datos requiere más de unas pocas instancias de tipos simples, como enteros o cadenas, codifique los tipos complejos con JSON. Sus usuarios ya están familiarizados con JSON o descubrirán que la gran cantidad de documentación y herramientas disponibles facilita la adaptación. Además, aprender estos estándares es una inversión en habilidades técnicas de gran utilidad.

Para mantener un diseño simple, evite listas de parámetros largas y complicadas.

Codificar y decodificar una larga lista de parámetros es un ejercicio complejo. Todos los métodos deben examinarse críticamente para detectar un número excesivo de parámetros. Se pueden aplicar dos pruebas a cada parámetro de un método para determinar si se puede eliminar:

- Marcar el parámetro como un parámetro no utilizado: ¿este parámetro presenta información que es utilizada por el método invocado?
- Identifique el parámetro como un parámetro redundante: ¿es este parámetro necesario para proporcionar la información necesaria y se puede inferir la información de los valores o del estado visible de otros parámetros?

Se puede usar un refinamiento adicional para acortar una lista de parámetros, especialmente si son demasiado precisos. La especificación de parámetros individuales, que en realidad son detalles individuales de un valor, debería refactorizarse.

En tipos compuestos. Por ejemplo, dos parámetros `string, last_name:string` podrían reemplazarse por un parámetro `full_name: string[]`

`__ : []`

Los módulos o paquetes de software se vuelven más breves, comprensibles y robustos gracias a la definición y el uso de tipos de datos personalizados, específicos de cada dominio e implementación. En un ejemplo clásico, este fragmento de C nos permite reemplazar cuatro campos por uno en todo el código base: `typedef struct Rectangle { int x, y; int width; int height; }`

`Rectangle;`

Sin esta definición de tipo, el código que compara dos rectángulos requiere un esfuerzo excesivo para comprender y verificar: `int [] overlap_region;`

...

```
región_superposición = superposición(forma1_izquierda, forma1_superior, forma1_ancho,
forma1_alto,
    forma2_izquierda, forma2_superior, forma2_ancho, forma2_alto);
superposición_izquierda =
región_superposición[0];
superposición_superior = región_superposición[1];
superposición_ancho = región_superposición[2]; superposición_alto = región_superposición[3];
```

Este código, que utiliza nuestro tipo Rectángulo personalizado, hace que la intención sea más clara y es más compacta:

`Rectángulo superpuesto_región;`

...

`overlap_region = find_overlap(shape1, shape2);` El segundo fragmento proporciona varias ventajas, con mayor brevedad, conocimiento del contexto y confiabilidad, mientras que el primer fragmento se basa en límites de matriz no impuestos.

Un ejemplo más complejo involucraría tipos anidados o tipos con una mayor cantidad de campos constituyentes.

Las respuestas deben ser fácilmente entendidas por los clientes de una API.

Las respuestas tienen mayor potencial de generar confusión que los parámetros y, por lo tanto, merecen mayor atención en cuanto a su facilidad de comprensión. Esto se debe a que, si bien los parámetros de un método son elementos que se espera que posea un cliente de la API, tener conocimiento de la respuesta es un artefacto del método invocado. Dado que la respuesta es parte del servicio que proporciona la API, puede ser completamente ajena al usuario que la llama. Una respuesta puede contener un tipo personalizado definido y exportado por la API o un valor relevante en el entorno del proveedor de la API.

Para evitar requerir un conocimiento detallado del servicio invocado, las respuestas de la API del servicio deben estar bien definidas y documentadas, y no generar cargas excesivas como las descritas anteriormente en esta sección. Utilice códigos de respuesta idiomáticos estándar, como los códigos de respuesta HTTP para una API HTTP; devuelva los datos en formato JSON con metadatos descriptivos; y proporcione los encabezados de respuesta necesarios. Por ejemplo, la siguiente respuesta contiene metadatos descriptivos en los nombres que aclaran el significado y facilitan la documentación: { "product_list": [{ "product_id": "S001A",

"quantity_in_stock":

25 }, { "product_id": "Z010A",
"quantity_in_stock": 12 },

{ "product_id": "Z095B",
"quantity_in_stock": 0 }],

```
"página»: 10,  
"máximo_de_elementos_por_página":  
3 }
```

Mantenga las API limitadas para que la intención de los métodos sea clara y el uso de la API sea flexible.

Otro enfoque es utilizar un mecanismo de invocación remota diferente, como gRPC, un marco de llamada a procedimiento remoto de código abierto que cuenta con el respaldo de Google y tiene cierto nivel de soporte de Amazon y otros proveedores de la nube. Las bibliotecas gRPC proporcionan campos de solicitud y respuesta idiomáticos del lenguaje y la plataforma que abstraen los formatos de datos específicos de la plataforma de los parámetros y los valores de retorno de los métodos remotos.

Aunque un servicio debe tener todas las características, contemplando todos los casos de uso potenciales, el deseo de ser todo para todos los usuarios no debería resultar en una API con muchos métodos o con métodos cuyas responsabilidades varíen en función de los valores de los parámetros o de la secuencia de eventos. Un componente dado debería representar una cosa y hacerla bien. Aunque no hay un límite estricto sobre cuántas operaciones son demasiadas para un componente o servicio o qué tan atómico debe ser un método, hay algunas pruebas que un desarrollador de API puede aplicar a un diseño propuesto para detectar métodos posiblemente demasiado precisos o demasiado amplios: Demasiado preciso: Si dos métodos están estrechamente acoplados de modo que una llamada al primer método solo se realiza como precursora del segundo método, y el

- segundo método siempre debe ser precedido por el primero. Lo más probable es que los dos sean una sola operación y se deban combinar en un solo método.
- Demasiado amplio: Un método que procesa diversos datos y opera en diferentes partes de ellos, basándose en algún parámetro o indicador discriminante, podría, en realidad, ser varias operaciones distintas bajo un mismo nombre. Refactorizar estos comportamientos en métodos independientes permitirá una mayor flexibilidad en su aplicación a más casos de uso, menor sobrecarga por invocación y una comprensión más sencilla del propósito de cada método.

Consideraciones especiales para interfaces de cliente o en la nube.

La sección

anterior describió varias buenas prácticas de diseño de API y técnicas para implementarlas, independientemente de la plataforma. Estos principios son beneficiosos en cualquier situación, pero en un caso de uso donde una API se utiliza para cruzar una frontera topográfica importante, como la que existe entre una aplicación cliente local y un servicio en la nube, las ventajas se magnifican, o más precisamente, el precio de no seguirlas se magnifica en comparación con el caso promedio dentro de la frontera. Ahora revisaremos el mismo conjunto de patrones de diseño e identificaremos sus beneficios, que se amplifican al aplicarse a una API multiplataforma.

El uso de formatos estándar y de fácil digestión permite una comprensión común cuando se involucran múltiples plataformas.

Una plataforma en la nube, como GCP, se compone de numerosos servicios relacionados, pero independientes. Es natural que estos servicios compartan tipos y formatos de datos altamente especializados debido a sus puntos en común internos y a la necesidad de interoperar. Estos tipos de datos especializados son idiosincrásicos y poseen atributos y patrones de diseño que evolucionaron para satisfacer los requisitos específicos de la plataforma. Esto representa un problema para los clientes de dicha plataforma; es posible que estos tipos y formatos especializados no sean familiares o no se adapten eficazmente a todas las plataformas de los clientes.

Considere una aplicación móvil que hace un uso directo e intensivo de muchos productos de GCP; la cantidad de memoria o sockets abiertos que es la norma en un componente de GCP probablemente sea prohibitiva en un teléfono.

Otro aspecto de la portabilidad es el gramatical: un formato de paquete de recursos binarios que es estándar en iOS podría no tener equivalente directo en un producto de base de datos en la nube.

Estos desafíos de interoperabilidad se reducen si el cliente codifica los parámetros como JSON y el servicio proporciona las respuestas en JSON. Un formato común, utilizado en ambos lados de una API, evita que ambas partes tengan que adaptarse a formatos y valores externos o incompatibles.

Evitar listas de parámetros largas y complicadas reduce la traducción y el tráfico entre plataformas.

La presencia de muchos parámetros puede dificultar la legibilidad del código y ser una señal de que una función puede tener responsabilidades vagas o múltiples, lo que dificulta la reutilización. Cuando una función se invoca localmente, los parámetros pueden pasarse por valor o por referencia; esta última es relativamente económica: un puntero o identificador es pequeño y fácil de obtener. Cuando una función se invoca remotamente, el lujo de las referencias de bajo coste ya no es un lujo.

Una referencia remota se puede implementar de diferentes maneras, pero al final, siempre hay una dirección, aunque lógica en un contexto, en lugar de un desplazamiento en la memoria virtual, que se pasa como identificador remoto. Para utilizar esta referencia remota, debe existir algún medio para traducir el valor de dicho identificador a la ubicación del objeto real en la memoria de la plataforma de origen. Este tipo de resolución está altamente optimizado en los mecanismos modernos de llamada a procedimiento remoto (RPC), pero no es de coste cero, a diferencia de una dirección de memoria local. En el caso de un objeto complejo, una referencia remota suele implementarse como un objeto proxy que, a su vez, cuenta con lógica interna para recuperar los atributos del objeto remoto según se necesiten.

A medida que aumenta el número de parámetros de un método remoto, cada parámetro tiene una de dos operaciones preparatorias: se serializa su valor o se construye una referencia remota y luego se serializa.

Cada uno de estos parámetros serializados se envía a través de una red, lo que supone un coste adicional respecto a las llamadas a funciones locales, y finalmente, el receptor lo deserializa. Al utilizar uno de los proxies remotos descritos anteriormente, puede generar más tráfico de red al obtener los valores de los atributos almacenados remotamente en la plataforma de origen.

Estas consideraciones están enmascaradas por el mecanismo RPC utilizado, y dichos mecanismos están optimizados hasta el punto en que la sobrecarga de uno o unos pocos parámetros no es una consideración en el diseño de la API.

Sin embargo, las listas de parámetros excesivamente largas aumentarán esta sobrecarga y, en algún momento, se notará el impacto. En algún momento posterior,

El impacto será significativo y, en un punto aún más tardío, perjudicial. Mantener las listas de parámetros simples y concisas es la mejor manera de evitar que se alcancen esos puntos.

Mantener las respuestas fáciles de entender reduce los cambios de paradigma impuestos entre plataformas.

Una llamada a un método remoto es costosa y conlleva mayor complejidad arquitectónica y mayor tiempo de ejecución que una función local. Si un diseño ha sido revisado y un método remoto pasa la revisión, debe haber buenas razones para justificar su existencia. La justificación más común y obvia es acceder a estados o capacidades que no están prácticamente disponibles en la plataforma que lo invoca.

Analicemos esta afirmación en las dos partes siguientes: Primero:

- "estado no prácticamente disponible" se refiere a datos que simplemente se encuentran en otro lugar. Podría ser un valor en una caché de clave-valor en memoria, el resultado de una consulta compleja en un conjunto de tablas de bases de datos o el valor actual del contador global de un sistema de procesamiento de transacciones. Sea cual sea la información, si está disponible en otra plataforma remota, esta debería publicar una API para que esté disponible donde se necesite.
- En cuanto a la otra parte de la justificación, las capacidades no disponibles en la práctica son aspectos en los que algunas plataformas informáticas son eficaces y otras no. Delegar operaciones a una plataforma donde sean más prácticas es otra razón válida para una llamada a procedimiento remoto. Las operaciones complejas y paralelizables, como la detección de bordes en muchas imágenes digitales y el entrenamiento de modelos de aprendizaje automático, son prohibitivamente caras de ejecutar en cualquier plataforma cliente. Un servicio en la nube puede aportar terabytes de RAM y cientos de procesadores para abordar un problema, ofreciendo capacidades que un teléfono o un portátil no podrían igualar.

Con esto establecido, existe una desconexión implícita entre los conceptos y las estructuras que son idiomáticos en el invocador y en las plataformas del proveedor remoto. Habiendo establecido que los dos lados de un

Los RPC son fundamentalmente diferentes, sus prácticas comunes y patrones de implementación también son diferentes.

Con esta tendencia inherente a detalles de implementación contraintuitivos en diferentes plataformas, la necesidad de simplificar las respuestas de los métodos es aún mayor.

El cliente de una API remota tiene un trabajo considerable que realizar para comprender los conceptos, patrones de diseño y detalles de implementación de la plataforma externa, por lo que los desarrolladores de API deben hacer todo lo posible para reducir la carga cognitiva general de su API. Menos campos, una buena nomenclatura y una mejor documentación pueden contrarrestar los desafíos de comprensión que presenta una plataforma remota.

Mantener las API limitadas aumenta las ventajas de otras buenas prácticas. Una buena API se centra en las necesidades del cliente. Proporcionar métodos que hagan lo que los usuarios necesitan para realizar su trabajo es la máxima funcionalidad que cualquier API debería ofrecer. No existe una consideración especial para esto en una API remota, pero un menor número de métodos tiene un efecto de ramificación, lo que resulta en menos listas de parámetros que descartar y menos respuestas que optimizar y construir representaciones JSON (o similares). Este último principio, cuya recompensa se materializa por su función como multiplicador de fuerza, aumenta las ventajas de las otras prácticas mencionadas.

Niveles de visibilidad para el cliente

El código más seguro, confiable y resiliente ante el cambio no tiene métodos públicos y solo un estado privado. Considere una instancia de la clase Obelisk definida aquí en C++:

```
#include <iostream>

usando el espacio de
nombres std; clase
Obelisco
{ privado: int
    contador; unsigned long int
objetivo; protegido:
```

```
void increment() { if
(counter >= this->target) { cout
<< "objetivo " << this->target " se ha cumplido para " << contador; }
else
```

```
{ contador+=1; } } public: Obelisk(unsigned
long int objetivo=32000)
{ this->target
```

```
=
```

```
objetivo;
contador = 0; } }; int main()
{ Obelisk *rock = new
Obelisk(); cout << "la roca es firme"; }
```

Sería difícil para cualquier cliente directo de la clase Obelisco corromper un Obelisco o comportarse fuera de sus especificaciones. Esto se debe a la dificultad de lograr que un Obelisco haga algo . Este es un ejemplo de un cliché común en las tecnologías de la información; otro ejemplo es que el sistema más seguro es aquel que está apagado.

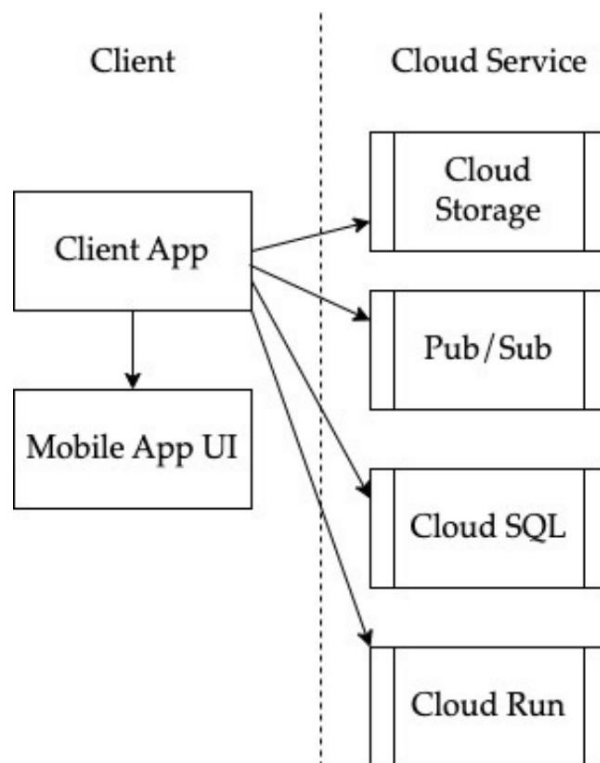
Aunque esta clase es demasiado estable para ser útil, sirve como punto de partida. Todos los componentes de software buscan un equilibrio entre la riqueza de funciones y la fiabilidad. Si un objeto (o método) que, en realidad, no ofrece nada es el más fiable, un objeto en el otro extremo, uno que revela cada operación interna y parte del estado, es capaz de ofrecer la mayor funcionalidad, pero también es el menos fiable.

En resumen, la funcionalidad potencial suele tener una relación recíproca con la confiabilidad.

Un principio que se menciona a menudo en el diseño de software es: comenzar con todo privado y ampliar el acceso tan poco y tan raramente como sea necesario para obtener el resultado deseado.

trabajo · Si bien esta máxima se recomienda para el diseño de software en realizado en general, tiene especial importancia al diseñar interfaces entre plataformas. La API proporcionada por los componentes de la nube debe proporcionar una Se recomienda utilizar los métodos más sencillos y mínimos posibles para minimizar tanto el diseño como el Dependencias y dependencias de tiempo de ejecución del cliente en la nube plataforma. Existen varias prácticas de diseño que se pueden aplicar a Para lograrlo, hay que tener en cuenta desde el principio un mediador: el uso de Proxies del lado del servidor para el cliente que recopilan una cantidad de datos en la nube dependencias y las presenta como una cara mínima y unificada al cliente.

A modo de ejemplo, se podría proponer inicialmente una arquitectura como ésta:



Cifra 6.1: Cliente que accede directamente a varias API en la nube

En este diseño arquitectónico, la aplicación cliente tiene conocimiento directo de varios servicios en la nube. Hay llamadas a varios servicios de GCP.

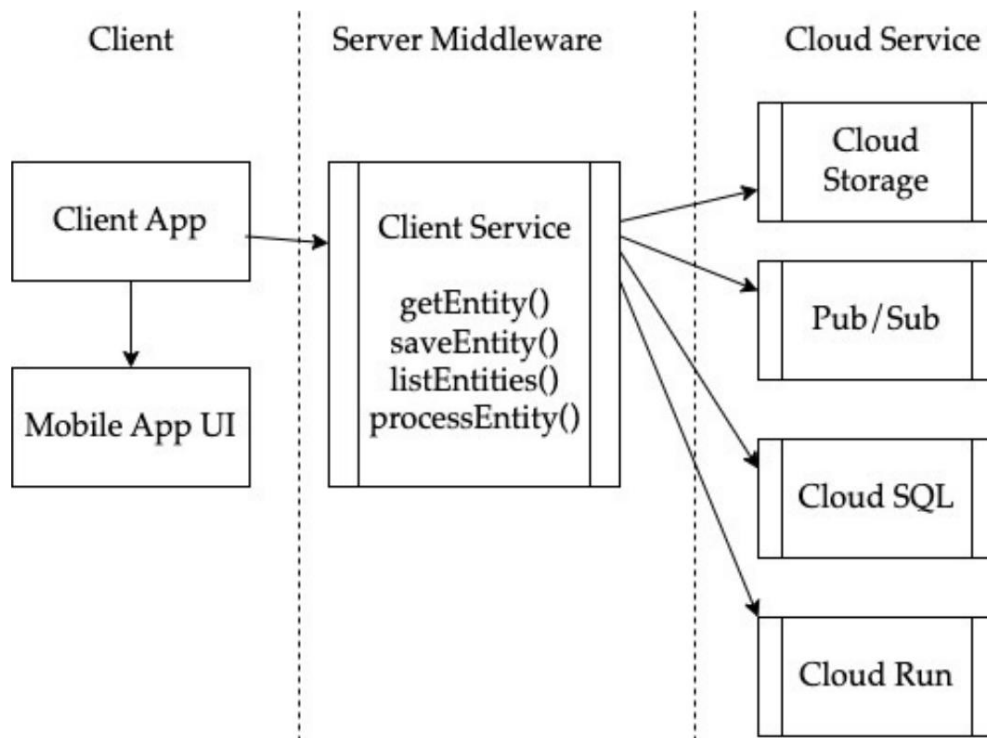
métodos dispersos en los módulos de la aplicación cliente. Como

Como resultado, es probable que se produzcan cambios en las API del servicio en la nube o en su uso. requerir refactorización, revisión y nuevas pruebas del cliente. Si hay

Si hay múltiples clientes, como diferentes aplicaciones móviles, de escritorio y web, el impacto de dicho cambio se multiplicará.

Además, existen otros métodos publicados por estos servicios en la nube que no son necesarios para las características del Cliente, pero su accesibilidad puede llevar a un uso erróneo debido a confusión o mal uso intencional por parte de un actor malintencionado en cualquiera de esos clientes.

Considere refactorizar la arquitectura anterior, que se muestra en [la Figura , 6.1](#), en este equivalente funcional:



Cifra 6.2 Cliente que pasa por un proxy del lado de la nube

Este diseño ha reunido todas las operaciones en la nube, reduciendo la visibilidad del cliente a un solo componente con pocas operaciones necesarias. Esta técnica ofrece varias ventajas, como:

- La aplicación cliente no necesita importar ni invocar las API completas de los cuatro servicios en la nube utilizados, por lo que la curva de aprendizaje es menor para los desarrolladores que trabajan en aplicaciones cliente.
- El uso no intencionado de las funciones de los servicios en la nube se evita ocultando cualquier método en la nube irrelevante o innecesario.

cliente.

- Cualquier cambio en las API o el uso del servicio en la nube se contiene en la nube; los clientes están aislados de cualquier efecto.

El servicio de cliente proxy se puede implementar utilizando un servicio en la nube lo más liviano posible, como App Engine o un servicio Cloud Run.

Planificación de la migración a servicios de nube sustitutos o equivalentes

Minimizar las dependencias rígidas es una buena práctica en la arquitectura de software que se promueve constantemente y ampliamente, rara vez se cuestiona, pero rara vez se explora. Lo haremos ahora, como punto de partida para esta sección.

Primero, debemos definir la dependencia estricta frente a la dependencia blanda. Una dependencia estricta se refiere comúnmente a un requisito existencial; es decir, si X tiene una dependencia estricta de Y, implica que sin Y, X no puede funcionar o quizás existir. Por el contrario, una dependencia blanda es aquella en la que aún se podría cumplir la función principal, pero en un grado reducido o con soluciones alternativas significativas. Una buena prueba de fuego para determinar si algo es una dependencia estricta es que, si la presencia de un recurso es crítica, ese recurso es una dependencia estricta.

Estar en deuda con un tercero es una posición indeseable; todos deseamos ser dueños de nuestro propio destino. Esta sección explorará algunos factores que impulsan la migración de un servicio en la nube a otro y las prácticas que un desarrollador puede seguir para que dichas migraciones sean lo más sencillas y económicas posible.

Bloqueo del proveedor

Analicemos este concepto en la selección de una plataforma en la nube. Un proveedor de plataforma en la nube suele proporcionar varios servicios de infraestructura importantes a una aplicación, como el almacenamiento y la identidad del usuario.

Gestión, cada una de las cuales es una dependencia estricta. Si bien es fundamental contar con algún método para almacenar el estado de la aplicación, así como para registrar y autenticar usuarios, debido a las conexiones y al contexto compartido entre los diferentes servicios que componen una plataforma en la nube, un proveedor específico, como GCP, se convierte rápidamente en una dependencia estricta. En lugar de requerir una base de datos SQL, observamos que se requiere explícitamente GoogleSQL, lo que conlleva dependencias de Google IAM y Google Cloud Storage. Esta situación se conoce como dependencia del proveedor. La dependencia del proveedor es una situación problemática en la que un cliente no puede cambiar de un producto específico a uno de la competencia. ¿Cuándo es esto un problema? Quizás nunca para algunos usuarios. Si su proveedor de nube ofrece la mejor relación calidad-precio, siempre a la vanguardia en cuanto a funciones útiles y nunca supera su presupuesto, no tiene motivos para dejarlo; realmente no tiene competencia.

Eso, por supuesto, es una situación ficticia. Por muy dedicado e ingenioso que sea un proveedor de tecnología, existen otros con buenas ideas. Durante décadas, la industria informática ha vivido un estado en el que las empresas se adelantan unas a otras con nuevas tecnologías, apoderándose de la posición de líder y luego perdiéndola ante el siguiente innovador. Más constante que esta serie de productos revolucionarios es un proceso lento pero constante de mejora, optimización y especialización por parte de los proveedores consolidados. Un proveedor puede decidir centrarse en dar soporte a clientes de ciertas industrias o a escalas operativas específicas, dejando a otros clientes mejor atendidos por otros proveedores. Por supuesto, es inevitable quedar atrapado en un proveedor determinado en algún momento. Para mitigar el costo de la dependencia del proveedor, se pueden adoptar principios de diseño y prácticas de ingeniería que faciliten el proceso de migración y minimicen el impacto (incluido el costo de los materiales) de sustituir el producto de un proveedor por el de otro.

Obsolescencia del producto

Una dependencia que queda obsoleta y sin soporte demasiado rápido es una

Gran gasto. Incluso sin discontinuar un producto, este podría simplemente estar obsoleto. Pueden surgir soluciones basadas en nuevos paradigmas o que ofrezcan ventajas únicas de rendimiento, y su producto actual ya no será la mejor opción. No está roto, pero existen opciones más atractivas en el mercado.

Cualquiera de estos casos se refiere a la obsolescencia del producto, pero con diferentes grados de fuerza que obligan al cliente a actuar. En el caso de un producto discontinuado, se ve obligado a identificar y migrar a un sustituto, mientras que en el caso de un producto obsoleto, se pierde una ventaja potencial que ofrece una alternativa. Llamaremos al primer caso migración forzada y al segundo caso, miedo a perderse algo (FOMO, por sus siglas en inglés) o el costo de una oportunidad perdida. El concepto de un modelo de Soporte a Largo Plazo (LTS, por sus siglas en inglés) se introdujo y se analizó brevemente en el capítulo dos, pero este es, en el mejor de los casos, un amortiguador o una prórroga contra una migración forzada. Es posible que aún tenga un FOMO muy válido, y confiar en un producto LTS no lo solucionará.

Diseño para la migración No hay

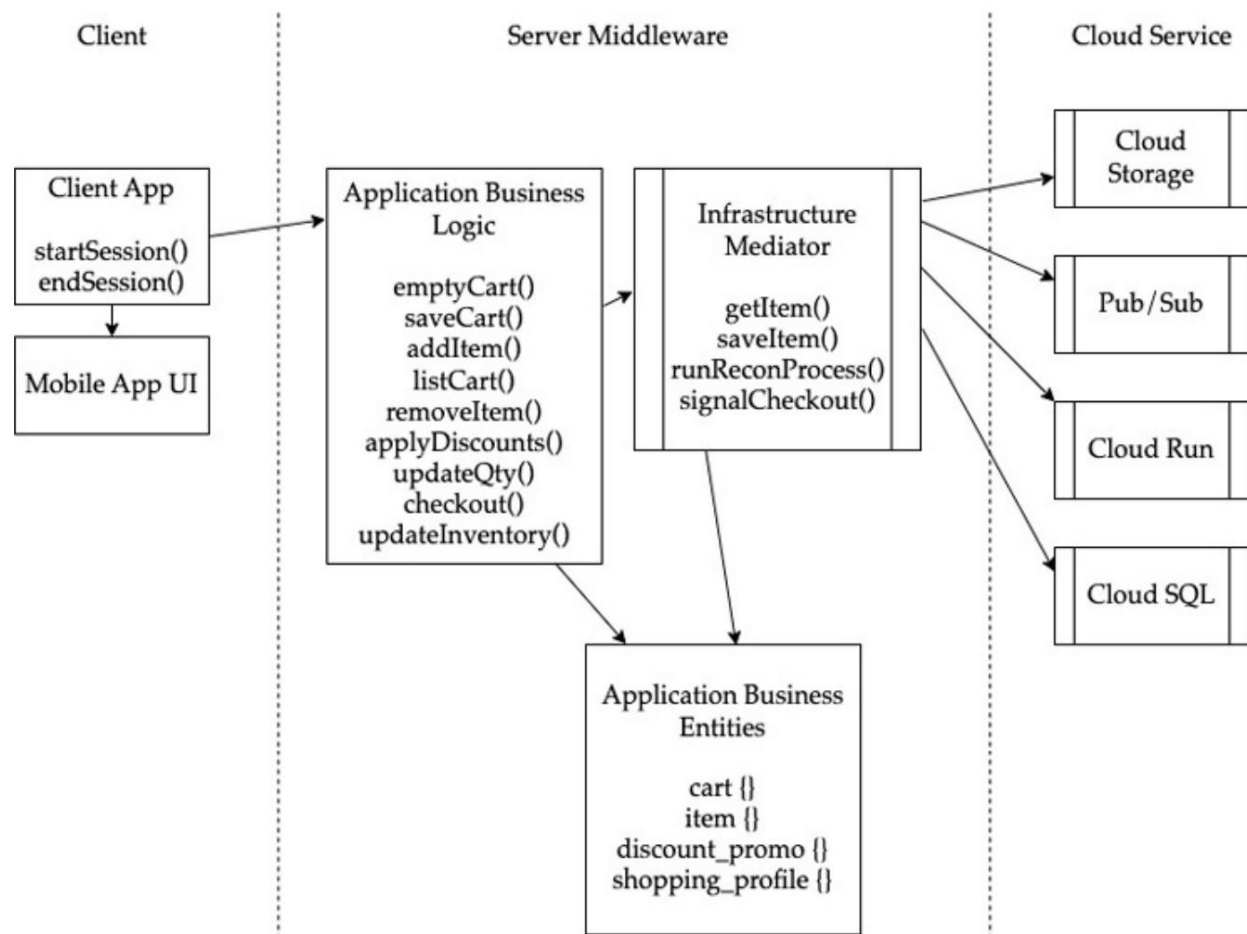
forma de eliminar por completo las referencias específicas y duras a sus dependencias. Puede introducir capas de abstracción y refactorizar jerarquías de clases y archivos de configuración para reubicar la gramática específica del producto o los recursos importados, pero no hay forma de reducirlos por debajo de la cantidad necesaria para configurar y controlar el producto en cuestión. Lo que puede hacer es usar las técnicas mencionadas anteriormente para reducir el número de módulos, ya sean clases, paquetes o archivos, que dependen explícitamente de recursos externos. Llamaremos a esto reducir componentes dependientes, y si bien no podemos eliminar las dependencias funcionales de estos recursos en los componentes posteriores, podemos hacer que aquellos no específicos del producto externo en efecto, podemos concentrar todas las dependencias con alcance externo en la menor cantidad posible de lugares.

La introducción de componentes proxy permite al desarrollador concentrarse en las dependencias externas.

Consulte la [figura 6.2](#), que presenta una solución a un problema similar,

Cómo minimizar el acoplamiento entre plataformas. La solución descrita introduce un mediador entre el cliente y varios servicios en la nube, reuniendo el conocimiento de estos servicios en un lugar topográficamente más cercano a ellos y lógicamente más cercano al cliente. Este patrón de diseño tiene como efecto secundario reducir la amplitud de las dependencias de servicios externos, y no hay mejor validación de buenas prácticas que la repetición de beneficios. Al emplear este patrón para reducir la influencia del producto de un proveedor en nuestra arquitectura, haremos que el proxy sea menos específico para las cuatro operaciones requeridas por el cliente y, en su lugar, proporcionaremos abstracciones no específicas del producto para las operaciones proporcionadas por cada uno de los productos de la plataforma en la nube.

Estos componentes sirven como sustitutos de los productos en la nube que son sus homólogos. Aprovechada al máximo, esta práctica puede aislar grandes partes de una aplicación de las consideraciones de infraestructura, dividiendo la aplicación en presentación al cliente, lógica de negocio, entidades, mediadores de infraestructura y componentes de infraestructura. La siguiente figura ilustra una posible disposición de este patrón arquitectónico en el contexto de una aplicación de compras en línea:



Cifra 6.3 Aplicación de compras que utiliza un mediador de infraestructura

Algunas descripciones de los cuatro niveles en este diagrama lo señalarán explícitamente. Facetas y ventajas del patrón.

La lógica de negocio de la aplicación, que codifica los procesos y el estado de la sesión de un usuario, está contenido en un módulo que tenemos Etiquetado como Lógica de Negocio de la Aplicación. Este código es presumiblemente requerido en cada cliente: móvil, web o de escritorio. De acuerdo con las buenas Diseño de software, eliminamos la codificación redundante en cada cliente y extraerlo a un servicio, compartido y llamable desde todos los clientes aplicaciones.

El módulo de aplicación cliente se reduce al código que está estrechamente vinculado a El cliente y el módulo complementario, la interfaz de usuario de la aplicación móvil, son solo ese código que es altamente específico de la interfaz de usuario de la plataforma del cliente individual. Este nivel está muy alejado de cualquier noción de proveedor de servicios en la nube y

Se comunica idealmente con el nivel de lógica empresarial de la aplicación como su único proveedor de funcionalidad, utilizando los tipos definidos en el módulo Entidades empresariales de la aplicación.

El componente mediador de infraestructura es el foco de este patrón de diseño. Este código abstrae los productos de infraestructura en sus roles funcionales y busca liberar al resto de la aplicación de dependencias estrictas de nuestra plataforma en la nube. Es probable que exista una correspondencia directa entre los métodos definidos en el mediador y los métodos ofrecidos por uno de los servicios en la nube, lo cual es previsible, ya que el objetivo no es la brevedad, sino la traducción. Las estructuras de datos que representan las entidades en nuestra aplicación se definen de forma centrada en el negocio, en lugar de específica para la infraestructura, en el módulo complementario Entidades de Negocio de la Aplicación . Este módulo define y publica los datos que necesita la lógica de negocio de nuestra aplicación, pero sin referencias ni tipos específicos de la infraestructura. El mediador de infraestructura abstrae las funciones de nuestra plataforma en la nube, y el módulo Entidades de Negocio de la Aplicación abstrae los datos proporcionados por o enviados a nuestra plataforma en la nube.

El uso de un mediador de infraestructura de esta manera aísla gran parte de la dependencia de un proveedor de nube específico a un solo componente, pero proporciona mayor protección y no protege completamente al resto del sistema del conocimiento y las dependencias específicas del proveedor. Analizaremos ambos puntos aquí.

Un mediador bien implementado encapsula una dependencia en su totalidad, no solo a su proveedor. Dado que un proveedor de nube puede ofrecer varios productos con funcionalidades superpuestas, como Google App Engine y Cloud Run, el mediador permite al desarrollador experimentar o alternar entre estas posibles contrapartes con relativa...
facilidad.

Si bien un mediador sirve como proxy para muchas funciones basadas en la nube dentro del sistema, existe un problema de arranque que no soluciona. Se utiliza una plataforma del lado del servidor para ejecutar el mediador, así como los componentes que utilizan sus servicios. La mejor práctica es mantener todos los componentes del lado del servidor que dependen del mediador lo más genéricos y compatibles con la plataforma posible.

Implementación y despliegue.

Puede haber casos en los que una dependencia explícita entre un servicio en la nube y un componente específico sea difícil o poco práctica de romper; en tales casos, resulta pragmático documentar y aceptar dicha relación. Veremos algunos de estos escenarios en las siguientes secciones. Es importante recordar que el objetivo es maximizar el aislamiento de las dependencias externas, en lugar de garantizarlo en un grado o número de módulos específicos.

Finalmente, los Servicios de la Plataforma en la Nube constituyen un nivel más alejado de nuestro control y experiencia. Se trata de servicios, como los productos de almacenamiento y computación de GCP, que se enumeran y describen en los capítulos «Determinación de la Infraestructura sobre la que se Construyen las Características» y «Elección entre las Opciones de GCP». Estos servicios ofrecen enormes beneficios a una aplicación distribuida, pero también requieren una inversión significativa en infraestructura y experiencia específica del proveedor.

Limitar su exposición a menos módulos de nuestra aplicación permite a nuestro equipo de desarrollo centrarse en las especialidades: un desarrollador de aplicaciones móviles no necesita convertirse en un experto en administración de GCP o en prácticas de API de GCP.

Una vez implementada una arquitectura de este tipo, podemos separar completamente los ciclos de desarrollo impulsados por preocupaciones internas, como el coste o la escalabilidad de nuestros servicios en la nube, de los ciclos de lanzamiento impulsados por las fuerzas del mercado, como las nuevas funcionalidades. Supongamos que un desarrollador diseña una nueva estructura para los artículos del inventario que elimina el almacenamiento de las descripciones de los artículos como Objetos Binarios Grandes (BLOB) en el Almacenamiento en la Nube y los almacena completamente en una tabla `Spanner`. Podemos buscar y reemplazar el código de las operaciones `getItem()` y `saveItem()` del mediador de infraestructura y reestructurar nuestras dependencias para refactorizar este nivel sin afectar la lógica de negocio ni a los clientes.

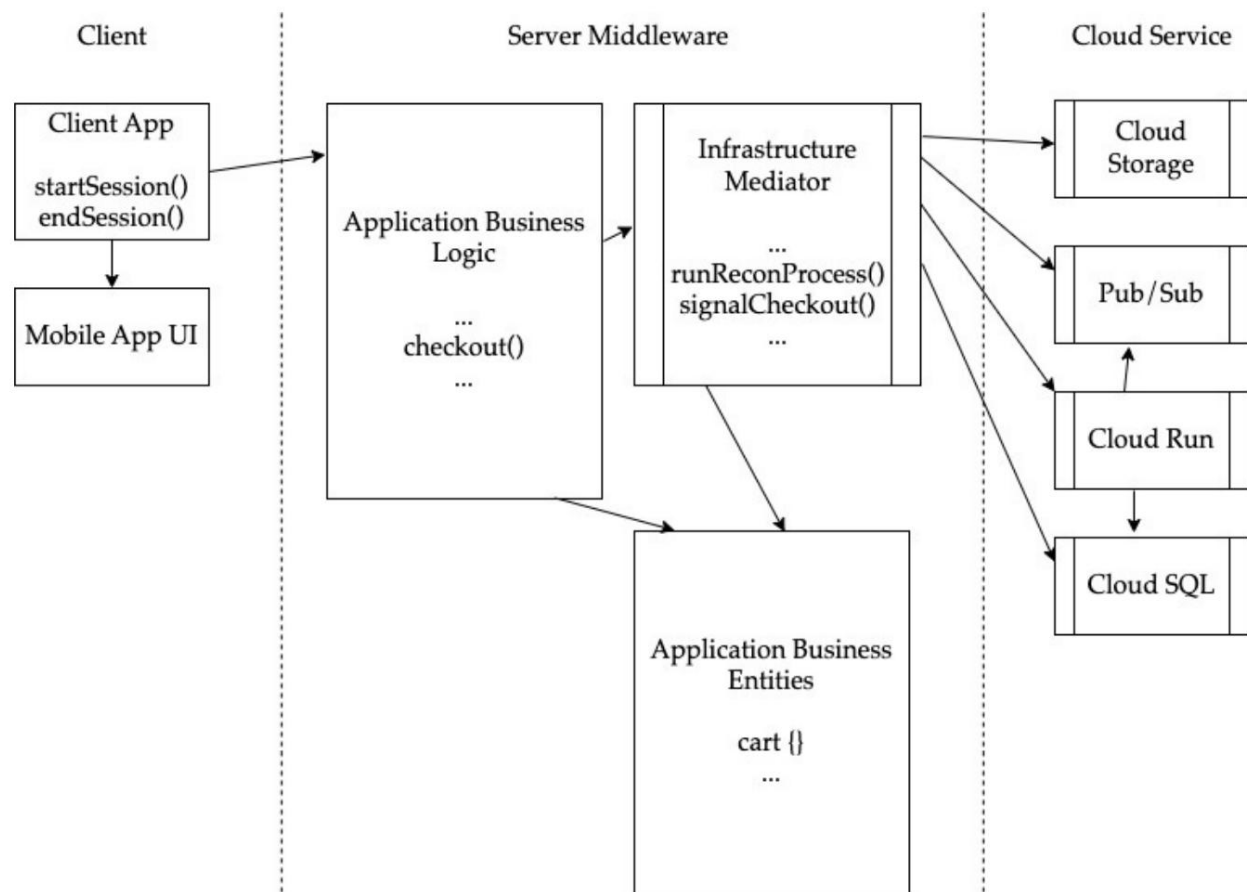
De forma similar, y más drástica, algún día podríamos migrar nuestra aplicación por completo a otro proveedor de servicios, como Amazon Web Services o Microsoft Azure. El mediador podría tener que asumir una complejidad adicional para compensar la disparidad de características entre...

productos o plataformas, pero el mediador está bajo nuestro control y nos permite sopesar esa complejidad adicional frente a las ventajas que ofrece una nueva plataforma de infraestructura.

Visibilidad entre servicios en la nube

Queda por abordar un tipo más de dependencia externa: las dependencias entre productos hermanos dentro de la plataforma en la nube. El problema es menos obvio y la solución más compleja que en los casos abordados hasta ahora, pero el potencial de mejora es real y merece la pena explorarlo.

Dependencias entre productos de servicios en la nube. Examinemos las soluciones presentadas en las secciones anteriores. Una ventaja es la posibilidad de migrar características de la infraestructura, como la persistencia de los datos de las aplicaciones, a nuevas representaciones. Cambiar los productos de infraestructura utilizados para estas funciones permite al desarrollador cambiar de una representación BLOB a una basada en filas de la base de datos, o de un proceso de larga duración a uno gestionado sin servidor. Otro escenario que este patrón permite al desarrollador es migrar de un producto de plataforma a otro proveedor, limitando al mismo tiempo los cambios en el diseño y el código base de la aplicación. La técnica común utilizada fue la refactorización del código para limitar las referencias directas a productos específicos a módulos designados. Podemos aplicar esta misma estrategia general a algunos casos dentro y entre productos en la nube, eliminando así sus dependencias directas entre sí. Utilizaremos un caso práctico específico como ejemplo ilustrativo. Tomemos el caso de un servicio ejecutado en Google Cloud Run: un proceso de conciliación de inventario de larga duración. Dicho proceso podría indicarse al final de la sesión de usuario, cuando el comprador haya completado el pago. Las dependencias relacionadas con esta función se verían como en la siguiente figura:



Cifra 6.4 Aplicación de compras con dependencias de infraestructura

En esta implementación, el mediador de infraestructura inicia inicialmente un servicio de conciliación de larga duración en Cloud Run, que espera a que se ejecute. Cuando un comprador realiza el pago, el mediador publica el evento mediante Pub/Sub. El mediador ha aislado correctamente la capa de la plataforma en la nube de la lógica de negocio; no existen dependencias entre ambos.

Sin embargo, ¿qué ocurre con el servicio que se ejecuta en Cloud Run? Se suscribe a un tema de Pub/Sub y recibe notificaciones de eventos de pago cuando los recibe. Su función es actualizar el inventario y marcar los artículos que no alcanzan un cierto límite de cantidad para su reordenamiento. Para ello, consulta y actualiza el inventario y, posiblemente, inserta filas en nuestra tabla de pedidos. Por lo tanto, el servidor que se ejecuta en Cloud Run tiene dependencias estrictas específicas de los servicios de Pub/Sub y Cloud SQL.

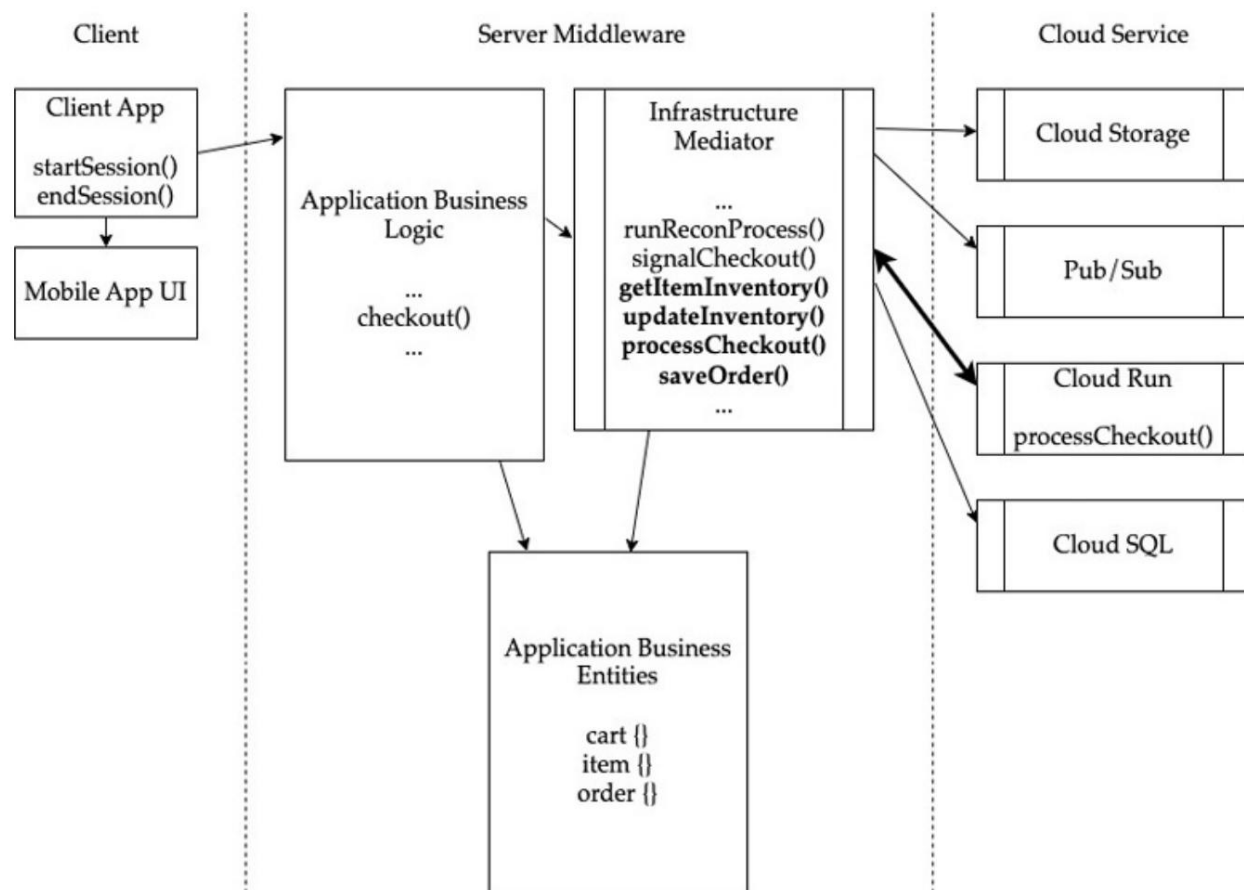
Este no es un problema tan obvio como tener dependencias en

La infraestructura dentro de nuestro cliente de aplicaciones y la lógica eran: la migración a otra plataforma tendría aproximadamente el mismo impacto y efecto, independientemente de estas dependencias. Sin embargo, la otra razón del aislamiento, migrar de un producto a un sustituto dentro de la misma plataforma, sigue siendo un problema. La experiencia especializada en la plataforma ya no es tan problemática como antes de introducir el mediador de infraestructura, pero migrar de los buckets de Google SQL a Google Cloud Storage aún requiere que busquemos referencias en el código y realicemos cambios en varios módulos.

[Aislamiento de productos de servicios en la nube](#) Las

dependencias entre los servicios en la nube se pueden abordar fácilmente (en teoría) ampliando las responsabilidades del mediador para que actúe como proxy de los servicios en nuestro nivel de servicio en la nube, así como también como proxy de nuestros módulos de lógica de negocios.

En lugar de que nuestro proceso de conciliación en Cloud Run acceda a nuestras bases de datos y se suscriba directamente a Pub/Sub, podemos añadir métodos al mediador para proporcionar estas funciones al proceso de conciliación. Estos métodos pueden publicarse mediante una API independiente si es necesario para evitar una dependencia circular. Las dependencias de esta función se verían así:



Cifra 6.5 Aplicación de compras con mediador de infraestructura bilateral

La principal diferencia a destacar es la conexión bidireccional entre el mediador y el servicio Cloud Run. Nuestro proceso de conciliación ahora llama al mismo mediador que lo inició y le asigna trabajo. Esto permite al mediador aceptar solicitudes para obtener y almacenar datos de nuestro servicio Cloud Run y, posteriormente, ejecutar operaciones de Cloud SQL para satisfacer dichas solicitudes. Para eliminar las dependencias entre nuestro servicio de conciliación y Pub/Sub, el mediador ahora envía eventos a Pub/Sub y extrae eventos de la cola de forma asíncrona, invocando un método del servicio de conciliación para procesarlos.

Con este paso, ahora hemos contenido las dependencias de los productos de Cloud Platform dentro del módulo mediador de infraestructura.

Identificación y empaquetado de servicios estrechamente acoplados

Hemos identificado la motivación para desvincular al máximo nuestra arquitectura de las dependencias externas y hemos descrito una estrategia para lograrlo. A continuación, profundizaremos en un caso práctico que incluye el caso problemático de módulos con dependencias explícitas difíciles de romper.

En primer lugar, recordemos el mediador, creado para ser el único dependiente de todos los productos de infraestructura de Cloud Platform. El principio básico de diseño de este mediador se puede resumir así: si la característica

Z de la aplicación necesita la característica X de un producto en la nube Y, se añade una característica al mediador que recupera X de Y y proporciona una versión de X sin tipos de datos ni referencias específicos de Y.

Analicemos esa afirmación y asignemos algunos valores reales para los marcadores de posición: Y es

- un producto de plataforma en la nube, en este caso, Pub/Sub.
- X es una operación de Pub/Sub, como publicar un mensaje extraído en el tema `pending_reconciliations` . —
- La característica de aplicación Z es una operación de lógica empresarial denominada `applyCheckoutToInventory` que se ejecuta después de que un usuario haya completado el pago.
 - El contenido del carrito completado se pasa a esta operación.

Con nuestro primer mediador instalado, la arquitectura de nuestra aplicación de compras se ilustra en [la Figura 6.5](#) . El análisis y el proceso se limitaron deliberadamente a la función de pago, ignorando convenientemente otras funciones de nuestra aplicación por el momento. A continuación, analizaremos una de estas funciones ignoradas que rompe el mecanismo de aislamiento de nuestro mediador de infraestructura: el inicio de sesión y la autenticación de usuarios.

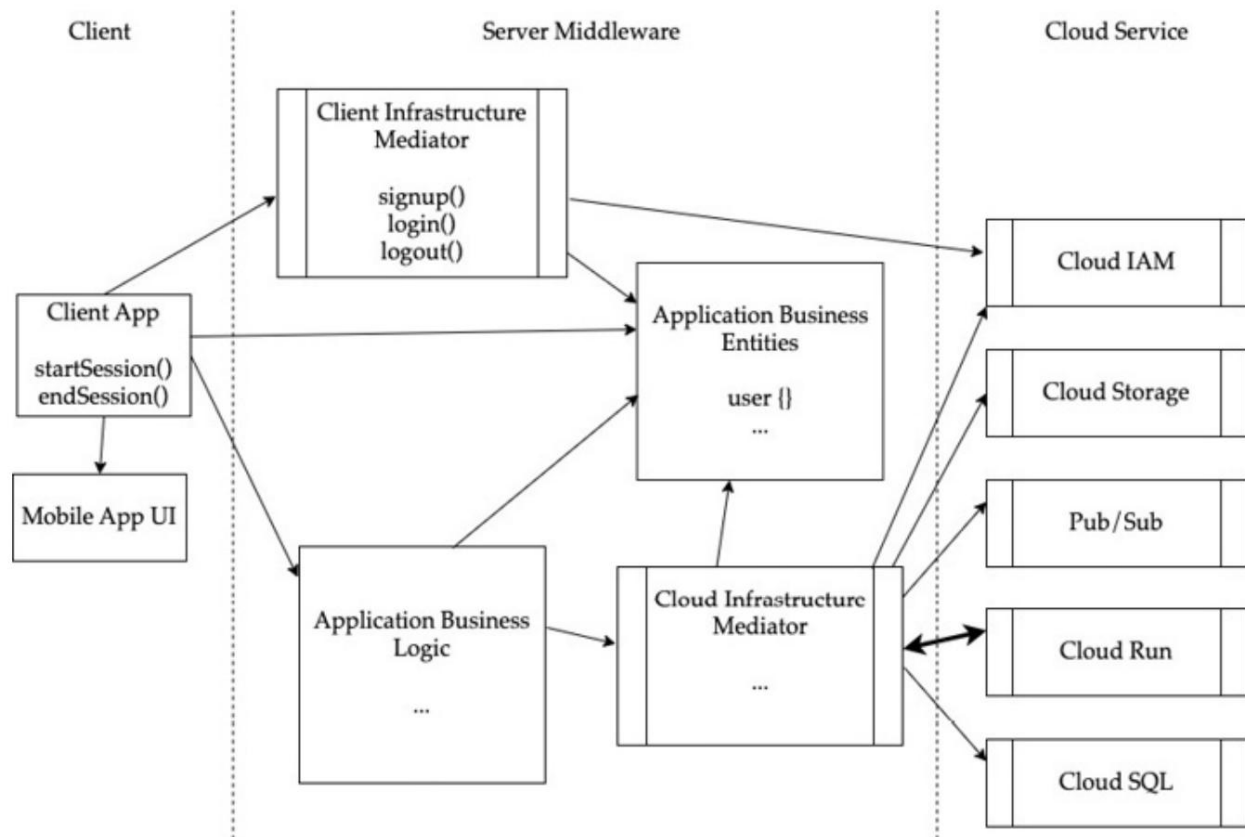
Los usuarios se conectan a nuestro sistema distribuido desde la plataforma cliente. Para limitar la posibilidad de uso indebido o malicioso de nuestro sistema, debemos proporcionar la menor cantidad posible de accesos a nuestro sistema principal a terceros no autorizados y no confiables. Esto presenta el clásico problema del huevo y la gallina al emplear la autenticación de usuarios en la nube.

Servicio. Si confiamos en nuestro mediador de infraestructura, que reside en la nube, para que actúe como proxy para la autenticación de usuarios, esto implica que nos conectamos a dicho mediador antes de que nuestra identidad se haya autenticado como usuario autorizado de la aplicación.

Aquí es importante recordar el objetivo de aislar y limitar los componentes dependientes en un diseño. No es viable no tener componentes con dependencias externas estrictas, suponiendo que las características de estas sean realmente necesarias. Un solo componente dependiente en un sistema es ideal. Sin embargo, dos componentes dependientes son mejores que un número ilimitado y podrían ser la mejor opción para una aplicación distribuida multinivel.

Este número dos no es arbitrario: un mediador para los componentes basados en la nube y un mediador para los componentes del lado del cliente que están vinculados a su plataforma cliente.

La introducción de un mediador de infraestructura del lado del cliente aún ofrece las ventajas de contener las dependencias específicas del producto en módulos específicos y nos libera del dilema del huevo y la gallina que se presentaba en el diseño de la autenticación de usuarios, descrito anteriormente en esta sección. Añadir un mediador para los servicios de cliente requeridos da como resultado la siguiente arquitectura:



Cifra 6.6 Aplicación de compras con mediador de infraestructura local

Tenga en cuenta que los componentes involucrados en esta refactorización (Aplicación Cliente, Mediador de Infraestructura de Cliente, Entidades de Negocio de Aplicaciones e IAM en la Nube) son idénticos a sus contrapartes en la refactorización anterior que introdujo el Mediador de Infraestructura en la Nube, respectivamente: Lógica de Negocio de Aplicaciones, Mediador de Infraestructura en la Nube, Entidades de Negocio de Aplicaciones y varios productos de GCP. Esto demuestra la consistencia del patrón de diseño empleado; los roles son constantes y los componentes específicos asignados a dichos roles varían según la situación en la que se aplique el patrón.

En general, el resultado de los patrones de diseño que hemos empleado en esta sección es la separación del código centrado en la aplicación del código centrado en la infraestructura de la nube. Esta es una extensión lógica de la separación clásica entre la lógica de negocio y los marcos y controladores de la interfaz de usuario. La principal ventaja es proteger el código impulsado por una fuerza externa, como el conjunto de herramientas de una plataforma de cliente móvil específica, del código configurado por una fuerza completamente diferente.

Fuerza externa, como un servicio específico de la plataforma en la nube. Estas dos dependencias externas son influencias independientes, y aislarlas nos permite adaptar los cambios dictados por cualquiera de ellas con el mínimo impacto posible en nuestra aplicación. Además, separar la lógica de negocio de cualquiera de estas dependencias externas garantiza que podamos codificar y validar el conjunto de características críticas de una aplicación sin preocuparnos por los efectos secundarios perjudiciales para nuestra interfaz de usuario o infraestructura de backend.

Conclusión

Al diseñar cualquier sistema de software importante, han surgido ciertos patrones y prácticas de diseño para maximizar la robustez, la mantenibilidad y el rendimiento. En una aplicación altamente distribuida y multinivel, como una respaldada por una plataforma de servicios en la nube, el beneficio de estos patrones se magnifica debido a una mayor demanda y a las bases de código, generalmente más extensas.

En este capítulo se revisaron los principios de diseño de API, especialmente pertinentes para la construcción de API remotas. Se destacaron consideraciones específicas para aplicaciones distribuidas, que actúan como un factor multiplicador de los beneficios de varias de estas prácticas al diseñar una aplicación en la nube.

Se guió al lector a través de las motivaciones y prácticas para aislar la presencia de cualquier producto específico de la nube dentro de la arquitectura y el código base de una aplicación. Se utilizaron casos prácticos para ilustrar las prácticas analizadas y reforzar la relevancia de estos patrones de diseño.

El siguiente capítulo profundiza en el diseño de un cliente independiente del backend, que define e implementa todo el comportamiento específico de la plataforma del cliente, en lugar de estar especificado por la lógica de negocio de la aplicación que lo abarca. Se considerará y planificará la robustez ante interacciones inesperadas del servidor, así como el aislamiento del backend de la aplicación ante estados inesperados en el cliente.

CAPÍTULO 7

Frontends en la frontera del cliente

Introducción

El frontend es la parte pública, visible de inmediato y sujeta a un riguroso análisis de un sistema distribuido. El backend y la infraestructura de una aplicación pueden contener mucha más información, pero estos son invisibles para el usuario típico, que solo ve los controles y experimenta la latencia y el flujo de los componentes del cliente que residen en su teléfono, tableta u ordenador local.

Para controlar bien la experiencia del usuario, un cliente no puede simplemente enviar solicitudes a un backend subordinado e ignorante del usuario. El cliente debe tener un plan interno y un conjunto de garantías para lo que maneja internamente, lo que delega al backend y las respuestas que emitirá a las solicitudes del usuario. Además de responder al cliente, el backend puede presentar sus propias solicitudes al cliente y el frontend debe definir cómo servirá como representante del propio estado del sistema, así como responder a las

acciones del usuario. Este capítulo presenta principios de diseño y recetas para abordar estas consideraciones en el diseño de sistema de alto nivel de una aplicación distribuida. Seguir estas prácticas produce aplicaciones que brindan una experiencia de usuario consistente que gana y mantiene la confianza del usuario, al tiempo que trabaja con las capacidades y limitaciones de los niveles globales basados en la nube de la aplicación.

Estructura

En este capítulo se tratarán los siguientes temas:

- Lógica del lado del cliente
- Definición de solicitudes de clientes al backend
- Respuestas de solicitud bloqueantes y no bloqueantes
- Rechazo y manejo de respuestas inesperadas
- Definición y gestión de acciones no iniciadas por el usuario

Objetivos

En este capítulo, el lector se familiarizará con la estructura general de la plataforma de una aplicación distribuida visible para el usuario. Aprenderá los criterios para separar la funcionalidad y asignarla a un nivel específico dentro de la arquitectura, así como las razones que lo justifican.

Se exponen los riesgos para el rendimiento y la integridad de las aplicaciones dentro de cada capa y en sus límites, y se explorarán y explicarán técnicas para prevenir o mitigar esos riesgos.

Lógica del lado del cliente

Hay consideraciones que deben tenerse en cuenta al determinar si el código se ubicará en una plataforma del lado del servidor, incluyendo servicios en la nube, o como parte de la aplicación cliente local. Esta sección analiza los principios generales para aislar la funcionalidad en una plataforma específica y, a continuación, describe varios casos de uso que requieren una implementación del lado del cliente o del servidor.

Evite múltiples fuentes de verdad. El comportamiento

significativo debe codificarse en un solo lugar. Por ejemplo, si la lógica que define cómo determinar qué oferta puede recibir un comprador está codificada en el método `eligiblePromotions()` de una clase llamada `PricingAdjustor`, ese conjunto de reglas se codifica allí y solo allí en el código base. La razón es simple: confiar en más de una fuente de verdad genera discrepancias entre ellas.

Esto resulta en una fuente de verdad inexistente . Esto se relaciona con una práctica algebraica básica: simplificar expresiones polinómicas con un factor común. En este caso, el código redundante que implementa la misma operación en múltiples ubicaciones puede considerarse un factor común, y la refactorización realizada para extraer y aislar la operación equivale a simplificar la expresión.

En una aplicación con un único entorno de ejecución, dondequiera que se necesite esta función, el código que la invoca puede simplemente importar el módulo PricingAdjustor y luego realizar una simple llamada a un método para utilizarlo. La ubicación estática o en tiempo de compilación del código es la consideración principal, ya que solo existe una ubicación dinámica en tiempo de ejecución. Todas las invocaciones utilizarán la misma lógica y tendrán un rendimiento similar.

En un sistema distribuido multinivel, es necesario considerar adicionalmente la ubicación física, o más precisamente, la ubicación topológica , del código en tiempo de ejecución. Además de la ubicación estática, el paquete, el módulo y la clase en los que se codifica una operación, el desarrollador también debe decidir en qué plataformas de ejecución instanciar las instancias de ese código.

Los diseños comunes y fáciles de entender son la decisión de implementar el código como un componente de servidor común no específico del cliente o ejecutarlo localmente en todos y cada uno de los clientes que ejecutan el frontend de la aplicación.

Hay varios factores a considerar al decidir en qué lado de la división cliente-servidor debe ubicarse un componente. El resto de esta sección enumera y describe algunos de los principales casos de uso donde esta decisión es relevante, así como las consideraciones pertinentes para cada uno de ellos.

Presentación de la interfaz de usuario La salida que el

usuario humano ve, oye y siente se presenta en la plataforma de hardware del cliente y mediante el software que se ejecuta en esa plataforma. Esto es inevitable.

La proporción de la preparación de ese contenido visual y de audio que se realiza en el cliente, así como la renderización de la salida visible para el usuario, puede variar. La acción predeterminada típica es renderizar en el cliente, pero esto no debería ser una decisión binaria: todo o nada, cliente o servidor. Analicemos las ventajas.

de realizar más o menos la representación de la interfaz de usuario en el cliente versus el servidor.

La renderización del lado del cliente proporciona una interfaz de usuario que aprovecha al máximo los elementos locales de la interfaz de usuario. Los cambios más ágiles en el diseño y la disposición de los componentes de la interfaz de usuario se producen cuando esta es observada y controlada por el código local.

Sin embargo, el uso de recursos en la nube introduce factores que ofrecen contrapesos u oportunidades que no existen para el cliente. A continuación, examinaremos un par de ejemplos para ilustrar estos casos de uso.

En el primer ejemplo, se visualiza una imagen de una galería almacenada en la nube en cualquiera de las aplicaciones de la plataforma cliente. Es necesario escalar la imagen y ajustar la profundidad y la paleta de colores para que coincidan con el panel de visualización y las características de visualización de cada cliente específico (por ejemplo, un smartphone).

Un enfoque obvio es que el servidor proporcione la imagen en un formato de calidad razonablemente alta, independientemente de la plataforma del cliente, y permita que cada cliente la ajuste a sus propias capacidades de presentación. Esta técnica mantiene los límites más estrictos entre las plataformas. Solo el cliente conoce sus capacidades de visualización y el uso previsto de la imagen, es decir, una miniatura, un icono o pantalla completa.

Existen costos y preocupaciones prácticas que hacen que este enfoque basado en principios sea menos atractivo.

Una imagen de mayor calidad consume más ancho de banda de red, lo que hace que su transferencia tarde más. El ancho de banda de red no es gratuito ni infinito; tanto las redes celulares como las fijas tienen una capacidad limitada y generan costos, tanto en tarifas como en degradación del rendimiento.

Es probable que el cliente tarde más en redimensionar y remuestrear la imagen de alta calidad que el servidor, lo que aumenta aún más la latencia y el consumo de energía del cliente. El tiempo adicional para transferir la imagen de mayor calidad fue un verdadero desperdicio, ya que perdimos parte de esa calidad de imagen.

En este caso, la filtración de información del cliente al servidor es un compromiso razonable. El cliente puede, en el ámbito de una sesión o al solicitar una imagen, anunciar un perfil de capacidades, incluyendo

Tamaño y resolución de imagen, profundidad de color y calidad de audio preferidos. El servidor puede entonces proporcionar contenido adecuado a las capacidades del cliente, eliminando la transferencia excesiva de datos y el costoso y redundante remuestreo en cada cliente.

Otra técnica utilizada en aplicaciones web se conoce como renderizado del lado del servidor (SSR). Una aplicación web que utiliza SSR ejecuta cualquier código JavaScript en el servidor y envía una página HTML completamente renderizada al cliente. Las ventajas de SSR residen principalmente en el rendimiento del lado del cliente; el navegador solo muestra HTML sin retrasos en la ejecución de JavaScript o CSS en el cliente. Esto es especialmente beneficioso para usuarios con hardware cliente antiguo o menos potente, y ofrece importantes ventajas adicionales para quienes utilizan tecnología de asistencia, como lectores de pantalla. Los lectores de pantalla suelen tener dificultades para interpretar correctamente JavaScript y CSS, pero pueden analizar correctamente una página HTML completamente renderizada.

SSR se ofrece o se utiliza de forma transparente en muchos marcos de JavaScript, incluidos Angular y React.

Captura de entrada, selección y entrada del usuario De manera

similar al uso y control más precisos sobre los elementos de visualización, los controles de la interfaz de usuario para obtener entrada del usuario tienen argumentos a favor y en contra de un mayor aislamiento dentro de la plataforma del cliente.

Por un lado, la validación de datos, como garantizar que se ingresen todos los campos obligatorios o correquisitos, es muy adecuada para el código que se ejecuta en el cliente. No se gana mucho enviando campos individuales en sobres de ida y vuelta a la nube para su validación y construcción de mensajes de error, y el retraso sería notable e inaceptable en la práctica.

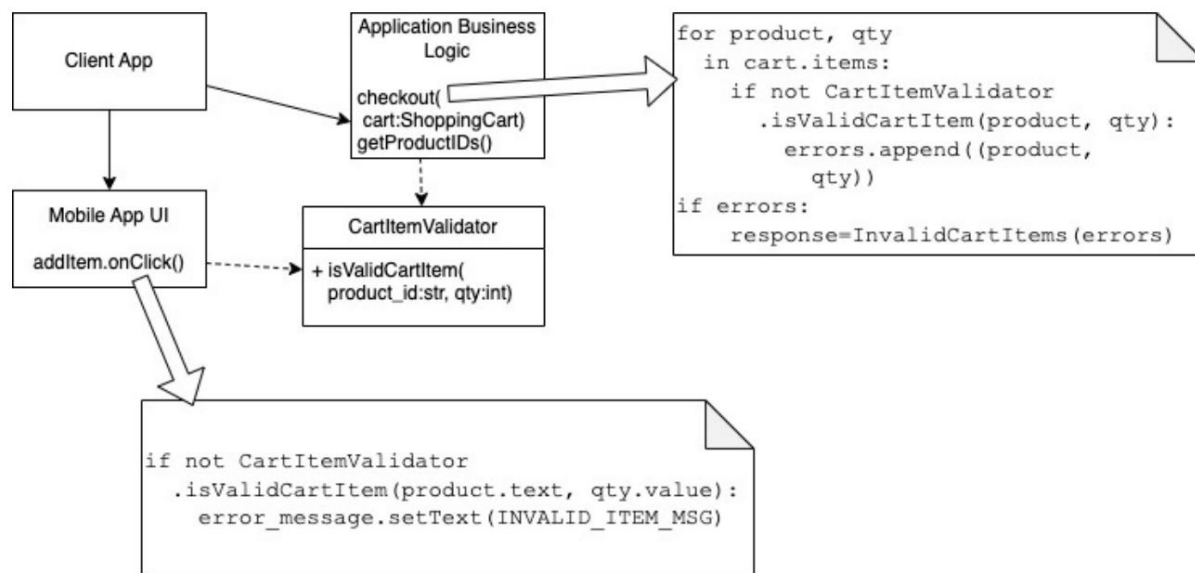
Por otro lado, considere un caso en el que el usuario introduce un campo de texto para buscar un conjunto conocido de valores, como el ID de un producto. Si hay pocos ID de producto disponibles, la lista podría descargarse al cliente para que el código local la use para validar la entrada del usuario.

Sin embargo, ¿qué ocurre si hay una cantidad muy grande o ilimitada de IDs, quizás 200 000, cada uno de hasta 50 caracteres? Descargar 10 megabytes de IDs de producto generará un retraso considerable y ocupará gran parte de la memoria disponible para una aplicación móvil. En este caso, es más viable algún tipo de validación del lado del servidor o un enfoque híbrido.

donde se obtiene una lista estrecha de identificadores de productos candidatos del servidor a medida que el usuario escribe en el campo de texto.

La validación del lado del cliente puede proporcionar una experiencia de usuario más completa y rápida, pero depender de ella conlleva un riesgo. Si los componentes del lado del servidor dependen del cliente para garantizar que todos los datos sean de los tipos y rangos esperados, se corre el riesgo de vulnerabilidades de seguridad. Una aplicación cliente que no valide completamente los valores enviados como parámetros a una API de un servicio en la nube podría provocar que dicho servicio se corrompa o sea vulnerable a una vulnerabilidad de seguridad intencionada. En un escenario más insidioso, un actor malicioso podría aplicar ingeniería inversa al protocolo cliente/servidor e inyectar su propio tráfico, omitiendo la validación de datos de la aplicación cliente.

La mejor práctica es un enfoque de validación de dos niveles. En este caso, la aplicación cliente emplea una validación de entrada interactiva y, a continuación, los métodos de la API del servidor realizan su propia validación de toda la entrada enviada a la nube. De esta forma, la aplicación puede ofrecer una validación detallada y de baja latencia, además de avisos al usuario, la red se libera del tráfico de ida y vuelta para el envío de campos o formularios intermedios, y los componentes del servidor se protegen de entradas inesperadas y potencialmente corruptas. Para garantizar la coherencia, tanto la validación del cliente como la del servidor deben utilizar las mismas reglas y valores de validación, pero instanciados e invocados desde los componentes en sus respectivas plataformas. El diseño para este doble uso de reglas de validación comunes se ilustra en la siguiente figura de clase, con algunos fragmentos de código que ilustran su uso en cada ubicación:



Cifra 7.1 Uso de código de validación común por parte del cliente y del servidor

El código que determina si una cantidad específica de un producto se escribe solo una vez forma parte de la clase `CartItemValidator`. Esta validación se realiza en la interfaz de usuario de la aplicación móvil cuando el usuario intenta añadir cada artículo a su carrito, lo que le proporciona una respuesta inmediata y clara si hay un error en su selección.

Cuando la aplicación móvil está lista para cerrar un carrito, invoca el método `checkout()` de nuestra lógica de negocio del lado del servidor. La operación de pago aplica las mismas reglas de validación a cada artículo del carrito a medida que se añaden al cliente, acumulando todos los errores de validación. Si se detectan errores de validación, estos se devuelven en la respuesta al cliente, quien tomará las medidas necesarias, probablemente mostrando la lista de errores y quizás retirando esos artículos del carrito activo.

Con este conjunto de validaciones de entrada, hemos cumplido nuestro objetivo de ofrecer retroalimentación ágil y una lógica de negocio protegida contra clientes erróneos o maliciosos. Además, hemos mantenido una única fuente de confianza: una codificación canónica de nuestras reglas de validación de datos. El conjunto de estas reglas debería ser responsabilidad de expertos en lógica de negocio y de cualquier requisito regulatorio o contractual, en lugar de dejarse en manos de ingenieros frontend y backend para su interpretación paralela.

Ocultar la lógica sensible a la observación

La sección anterior presentó la separación del conocimiento empresarial de la experiencia específica de la plataforma y la experiencia de usuario. Si bien un equipo de desarrollo pequeño o muy integrado puede dar lugar a que la misma persona sea el ingeniero y quizás también el gerente de producto, aislar estas especializaciones en la ubicación del código beneficia la capacidad del producto y del equipo para escalar, así como la estabilidad y la corrección de la aplicación.

Las implicaciones de la regulación a menudo no se comprenden hasta que un problema introducido por un equipo de ingeniería inconsciente conduce al incumplimiento.

De igual manera, existen decisiones de marketing y estrategia de producto detrás de las políticas que pueden ser desconocidas para los ingenieros que implementan las características de un conjunto de especificaciones. Recopilar toda la lógica de negocio en módulos o paquetes dedicados permite que la implementación de las políticas sea revisada por sus expertos en la materia: especialistas legales, de marketing de producto o del sector.

Estas razones anteriores suelen ser consideraciones estratégicas acertadas al diseñar la arquitectura de una aplicación. Existen beneficios más inmediatos y prácticos al encapsular con precisión el comportamiento clave de la aplicación, que son más fáciles de lograr con un sistema distribuido basado en la nube: la protección de la ventaja de mercado y la propiedad intelectual.

Considere una aplicación con una característica única e innovadora. Puede ser cualquier cosa: un algoritmo de posicionamiento, una técnica de transformación de datos o un método de clasificación. La clave es que el código de su aplicación haga algo diferente que le dé una ventaja sobre la competencia. La imitación es la forma más sincera de adulación, pero sería mejor para su negocio que esta ventaja única se mantuviera durante el mayor tiempo posible.

En una aplicación monoplataforma de arquitectura simple, el código que implementa esta nueva función se instala en todos los clientes móviles y de escritorio. Esto ofrece la oportunidad de analizar minuciosamente la aplicación en tiempo de ejecución, analizar iterativamente los módulos internos identificados y, finalmente, aplicar ingeniería inversa al comportamiento de esa función única.

Al transferir la lógica empresarial sensible a módulos dedicados, presentamos maneras de mitigar esta vulnerabilidad. Es poco probable que su algoritmo o proceso interno permanezca en secreto para siempre, pero, al igual que con el uso de candados de bicicleta, le resultará beneficioso colocar tantos reductores de velocidad como sea posible.

camino del ladrón.

Para reducir estos riesgos para nuestra aplicación y el negocio, podemos tomar ventaja de los patrones de diseño que ya se han discutido. A

El patrón de diseño relevante se describió anteriormente en este capítulo en el Sección: Captura de la entrada y selección del usuario. Aislamiento de la lógica de negocio.

Los módulos de políticas específicos o los motores de políticas pueden aislar el código para un dominio específico, como la colocación de anuncios en una página o

Mandatos regulatorios para ciertas combinaciones de artículos. Con el código

Ahora aislado, otro patrón de diseño, discutido en [el Capítulo Nube](#) 6,

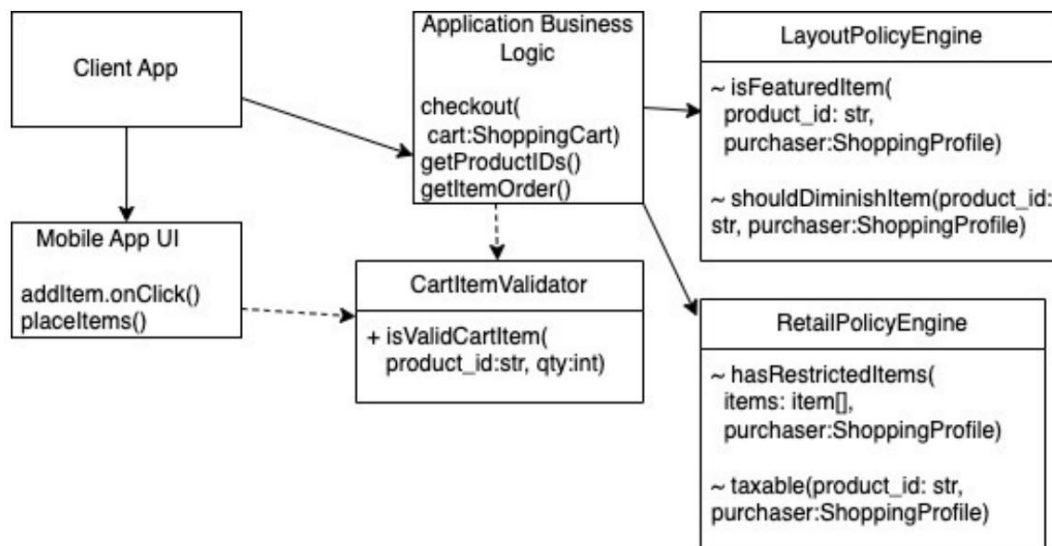
Servicios en el Backend Border en la sección Niveles de visibilidad al

Se puede contratar al cliente para trasladar esta política a un servicio basado en la nube.

que presenta una imagen pública lo más opaca posible. La siguiente figura

muestra un diseño que incluye dichos motores de políticas para dos dominios: elemento

Normativa de distribución y venta al por menor:



Cifra 7.2 Uso de motores de políticas específicos del dominio

Además de la concentración del conocimiento del dominio empresarial, este patrón

Nos permite colocar barreras en el camino de los esfuerzos de ingeniería inversa.

Nuestra aplicación. El código en componentes basados en la nube se puede proteger de Diseción o decodificación directa restringiendo el acceso a API estrechas.

El servicio en la nube se puede colocar detrás de balanceadores de carga y servidores proxy que Detectar y bloquear patrones de acceso irregulares o sospechosos, como invocaciones de métodos repetidas automatizadas.

Facilitar la implementación de código que se modifica con frecuencia. La estabilidad de una aplicación no solo se basa en un comportamiento consistente y esperado durante su uso, sino también en el ciclo de lanzamiento visible del software. Un usuario se cansará rápidamente de que se le solicite o se le obligue a instalar nuevas versiones a diario. Cuando alguien inicia su aplicación, quiere usarla y no esperar a que su sistema haga su trabajo.

Es inevitable que algunos cambios requieran una actualización del software del cliente, pero en una aplicación distribuida bien diseñada, la funcionalidad que se espera que cambie con frecuencia debe eliminarse del cliente en la medida de lo posible. Esto permitirá al equipo de lanzamiento implementar nuevas versiones y actualizaciones exclusivamente en los componentes basados en la nube. Para maximizar la oportunidad de hacerlo, mantenga al cliente centrado en el trabajo del lado del cliente y mantenga las API visibles para el cliente de los componentes del lado del servidor lo más limitadas e independientes de la plataforma posible.

En el diseño más sencillo, esta práctica genera una comunicación constante entre el cliente y el servidor, ya que el cliente invoca la lógica de negocio del servidor para guiar al usuario en su navegación por las funciones de la aplicación. El conflicto entre la distribución dinámica del software y la minimización del tráfico de red durante la sesión se aborda mediante otro patrón de diseño: la descarga de componentes internos para su uso sin conexión.

Este patrón de diseño se utiliza en aplicaciones sensibles a la latencia, como la selección dinámica de idioma en productos de reconocimiento de voz como el servicio Google Speech to Text . Esto permite que un cliente use un reconocedor instalado o descargue uno nuevo si se detecta un idioma previamente desinstalado.

En esta aplicación del patrón de diseño, existe una interfaz bien definida para el componente de lógica de negocio; en este caso, una clase abstracta de reconocedor de voz. Puede haber varias implementaciones de esta interfaz disponibles en la nube, pero una instalación de cliente determinada no puede incluir reconocedores para todos los idiomas y dialectos. Existen varias razones por las que esto no es práctico: no hay suficiente espacio en un teléfono para incluirlos todos, la lista de reconocedores se amplía periódicamente y se introducen mejoras en los existentes.

En lugar de que la instalación de un cliente se incluya con un reconocedor específico del idioma, la aplicación se instala con un pequeño conjunto de valores predeterminados.

reconocedores y un clasificador mínimo para gestionar los reconocedores instalados. Si no se reconoce el habla o se solicita específicamente un nuevo idioma, el clasificador descargará de la nube el reconocedor que se determine que es el más adecuado para el nuevo idioma encontrado.

Dado que los componentes del lado del cliente, ya codificados o integrados en la aplicación, solo requieren la implementación de la interfaz del reconocedor abstracto, la aplicación cliente existente continúa funcionando tras un breve retraso para descargar e instalar la compatibilidad con el nuevo idioma. La presencia de un servicio local de reconocimiento de voz minimiza el acceso a la nube durante una sesión de usuario, y el aislamiento y la instalación dinámica de los motores de reconocimiento evitan la necesidad de actualizar toda la aplicación cliente para acceder a la compatibilidad con el nuevo idioma.

Este patrón se puede emplear en diversos casos de uso en el ámbito empresarial: una aplicación de navegación podría descargar las normas de tráfico locales, y una aplicación de compras podría descargar un motor de reglas para la gestión de sesiones de compra. En ambos casos, la presencia y la aplicación de las reglas son fijas dentro de la sesión del usuario, pero las propias reglas codificadas están sujetas a cambios con el tiempo en función del cliente o de factores externos.

Definición de solicitudes de clientes al backend

El conjunto de solicitudes de cliente a los servicios backend no debe aparecer como una colección de operaciones generadas durante la implementación de funciones frontend. Un backend debe definirse intencionalmente, basándose en un conjunto de requisitos funcionales bien definidos, y la aplicación frontend debe utilizar estos servicios backend publicados.

La planificación es tan importante que nunca termina. Es probable que alguna mejora o deficiencia en el backend se haga evidente durante el desarrollo. Esto está bien y funciona según lo previsto; modifique el plan, no lo abandone.

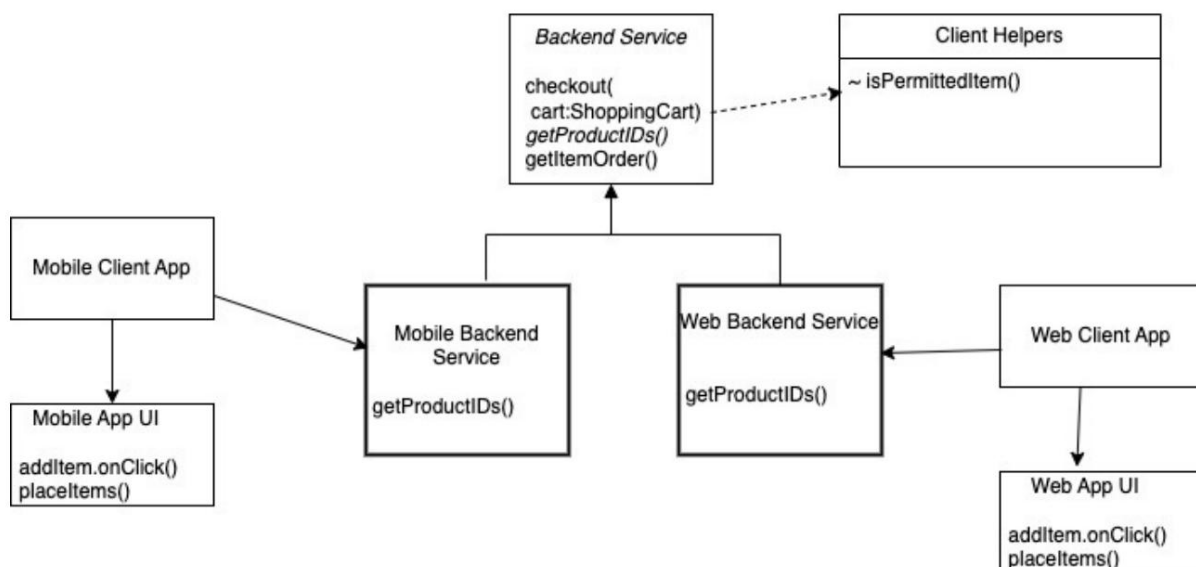
Backends para frontends

El objetivo de un patrón de diseño de nube popular, Backends for Frontends, es

Evite enturbiar el propósito de un servicio backend para que se adapte a múltiples clientes con diferentes requisitos. En este patrón, se desarrolla un servicio backend específico para distintas aplicaciones cliente frontend. La principal ventaja de esta práctica reside en una arquitectura donde múltiples clientes tienen diferentes requisitos de backend. Un único servicio backend se volvería difícil de manejar al converger en ofrecer todo para todos los clientes. La complejidad de este backend monolítico aumenta, lo que ralentiza el desarrollo, aumenta la cantidad de pruebas necesarias y crea un cuello de botella común para el desarrollo de aplicaciones cliente que, de otro modo, serían independientes. El desarrollo del cliente web puede requerir cambios en el servicio backend común, lo que requerirá cambios o, al menos, pruebas y verificación por parte del equipo de desarrollo del cliente móvil.

La desventaja a considerar al implementar este patrón de diseño es la posible explosión del número total de componentes y la posibilidad de código redundante entre los servicios backend. La primera preocupación es una consecuencia inevitable del patrón, pero su efecto se limita al número de aplicaciones cliente. La segunda preocupación puede mitigarse, y a menudo eliminarse por completo, mediante una herencia y descomposición de funcionalidades comunes bien diseñadas.

La siguiente figura del componente presenta un uso ideal, aunque algo abstracto, de este patrón en el ejemplo de ejecución de una aplicación de compras:



Cifra 7.3 Uso del patrón backends para frontends

Implemente servicios de backend en un servicio en la nube lo más liviano y flexible posible.

Los servicios de backend son una pieza útil de la infraestructura y, dado que se centran en aplicaciones cliente específicas, se pueden configurar para que satisfagan mejor la cantidad de usuarios esperados y los patrones de uso de cada plataforma. Es recomendable usar el mismo producto de Google Cloud Platform (GCP) para todos los servicios de backend de la aplicación. El uso de servicios de backend introduce complejidad arquitectónica, y los arquitectos deben evitar agravarla con diferentes modelos de implementación dentro del nivel.

Un servicio backend ideal es sin estado, donde cada llamada a un método no tiene efectos secundarios en el propio servicio. Si bien Cloud Run se recomienda como plataforma para un servicio que requiere gran escalabilidad y flexibilidad en las herramientas de desarrollo, el entorno estándar de App Engine es una buena opción para empezar, ya que ofrece la misma compatibilidad de implementación con gestión de versiones, escalado rápido, balanceo de carga y división del tráfico al implementar una nueva versión de un servicio. El entorno estándar, totalmente administrado, permite a los desarrolladores implementar servidores sin estado y sin servidor de forma sencilla y sin problemas. Si su caso supera las capacidades de App Engine, existen procesos bien documentados para migrar de App Engine a Google Cloud Run.

Respuestas de solicitud bloqueantes y no bloqueantes

Las llamadas a métodos bloqueantes o síncronas son las llamadas de función básicas que todos los programadores aprenden desde el principio. En [el capítulo "Opciones de tecnología del lado del cliente"](#), la sección "Manejo de mensajes asíncronos" describe brevemente la llamada a método síncrono, en la que el invocador de un método se suspende y entra en un estado de espera mientras se ejecuta el método invocado. Sólo cuando la operación invocada devuelve el control a quien la llamó, el método que la invocó reanuda su ejecución.

El modelo asíncrono es aquel en el que el invocador envía un método, pero luego reanuda su ejecución. En este modelo, la llamada al método inicia el método invocado, mientras que en el modelo de ejecución síncrona, la llamada al método lo ejecuta en su totalidad.

El patrón de diseño para esto es bastante simple y bien entendido, y hay una multitud de implementaciones para diferentes casos de uso.

Identificación de métodos asincrónicos

La necesidad de métodos asincrónicos se determina mediante una prueba de fuego de dos partes determinada por las dos preguntas siguientes:

- ¿El método invocado tarda más de lo aceptable para el hilo del cliente, invocándolo para que no haga nada más que esperar?
- ¿Hay algo más que el cliente pueda hacer para mejorar la experiencia del usuario mientras espera que se complete el método invocado?

Nota: Esto podría incluir la actualización de un indicador de progreso, si es útil.

Si la respuesta a la primera pregunta es afirmativa, la operación en cuestión debe examinarse críticamente con el fin de mejorar su rendimiento. Si esto no es práctico o la respuesta a la segunda pregunta es afirmativa, la invocación asincrónica de métodos es una buena opción.

Ahora discutiremos y contrastaremos diferentes métodos para implementar dicha ejecución asincrónica.

GCP proporciona soporte para el procesamiento asincrónico distribuido y escalable con varias características que respaldan el manejo de errores en el procesamiento asincrónico, como las funciones en la nube activadas por la actividad de registro y las herramientas de análisis de seguimiento de Cloud Observability.

Google Cloud Pub/Sub es un producto que permite implementar una forma de esto a gran escala, con invocadores completamente desacoplados de los métodos invocados. GCP también proporciona Google Cloud Tasks, un servicio de colas para GCP que puede invocar controladores de App Engine o solicitudes HTTP a URL de destino.

Existen técnicas específicas del lenguaje y del marco del cliente para invocar métodos de forma asincrónica.

La decisión de usar un servicio dedicado al procesamiento asíncrono o implementar llamadas a métodos asíncronos en el cliente es una decisión de criterio en la fase de diseño, basada en el caso de uso específico. El primero, al usar un servicio como Cloud Tasks, ofrece la escalabilidad y la conmutación por error de un servicio administrado por GCP, pero añade dependencias estrictas de productos de proveedores específicos, una curva de aprendizaje y la continua...

Administración de un producto adicional. El uso de un mecanismo local del cliente conlleva ventajas y desventajas complementarias: menor escalado automático y conmutación por error, pero sin dependencia adicional del proveedor ni administración del producto. Estas consideraciones deben considerarse en el caso de uso de una aplicación donde se determine que se requiere procesamiento asíncrono.

En el caso de uso específico de backends para frontends descrito anteriormente en esta sección, el servicio backend invocado, que se ejecutará de forma asíncrona, ya se encuentra en un servicio escalable y administrado, como Google App Engine. El servicio de respaldo se encuentra detrás de un balanceador de carga. Si los picos de tráfico se encuentran dentro de las capacidades del entorno estándar, no hay razón para añadir una capa adicional de complejidad en el servidor.

Tecnologías de cliente para procesamiento asíncrono Los clientes pueden elegir entre mecanismos específicos del lenguaje, el marco y la plataforma para la ejecución asíncrona.

JavaScript proporciona funciones asíncronas que se utilizan con objetos Promise para enviar y esperar la finalización de los métodos. Estas características de JavaScript se utilizan dentro de las funciones de procesamiento asíncrono propias de los frameworks de JavaScript, como los ganchos `useEffect` en React.

El lenguaje Go incluye goroutines y canales como primitivas para la programación concurrente (asíncrona), pero con un modelo de programación diferente al de JavaScript. C# admite objetos de tarea tipificados con las palabras clave `async` y `await`.

Los desarrolladores de dispositivos móviles Android pueden usar la clase `AsyncTask` para implementar cualquier tarea, incluyendo llamadas a métodos del servidor, en segundo plano en un hilo dedicado. La tarea en segundo plano puede indicar su estado de finalización, lo que genera memoria compartida con el hilo de ejecución principal.

Los desarrolladores de Swift iOS pueden usar un bloque de tarea, funciones asíncronas y la palabra clave `await` para crear funciones en segundo plano y sondear, o esperar, a que se completen.

Si los puntos finales en el servicio backend son de larga duración, se pueden enviar desde el cliente y vincularlos a promesas, goroutines, tareas u otra técnica del lado del cliente para la ejecución asíncrona de métodos.

La elección debe realizarse en función de los entornos de desarrollo específicos y las herramientas utilizadas y la reutilización deseada del código en las plataformas del cliente.

Rechazo y manejo de respuestas inesperadas

La respuesta de un método de servidor suele considerarse binaria: éxito o fracaso. Desde esta perspectiva, el éxito es el resultado deseado. Solicitamos una lista de ID de producto y recibimos una, se presentó un nombre de usuario para la autenticación y se confirmó, o la aplicación solicitó que se añadiera un artículo al carrito de la compra del usuario, y así se hizo. El fracaso se define como un comodín para todo lo demás. Las deficiencias de este enfoque pueden describirse desde dos perspectivas: la oportunidad de robustez del usuario y la de la aplicación.

En este escenario genérico de gestión de errores, el usuario recibe un mensaje de error con apenas información sobre un fallo. Esto lo desconcierta y mina su confianza en la fiabilidad de la aplicación.

La aplicación ignora internamente que existen dos tipos de fallos: operaciones que no permitió debido a reglas o restricciones internas, y estados inesperados como un error de red, un fallo de escritura debido a la falta de espacio en el dispositivo o un error de desbordamiento no detectado. Si la aplicación discriminara entre estos, podría recuperarse o proporcionar soluciones alternativas en algunos casos, mejorando la robustez general.

La primera categoría de fallo (la infracción detectada de las reglas de la aplicación) se denominará retroceso, y la segunda categoría se denominará inesperada. Analizaremos cómo gestionarnos de forma diferente, salvando al usuario o, al menos, manteniendo su confianza en la fiabilidad de nuestro sistema.

A menudo, es posible anticipar y evitar el rechazo. Es posible evitar añadir más artículos al carrito de los que hay en inventario o que permiten las regulaciones comerciales antes de solicitar el pago al servidor.

Sin embargo, en un sistema multiusuario, la posibilidad de una condición de competencia, donde dos usuarios compiten, podrían provocar un fallo en la caja debido a un inventario insuficiente. Esto es previsible y se necesita una explicación.

Se puede ejecutar un mensaje, a veces con una solución alternativa propuesta o una acción alternativa como reducir los artículos a la cantidad disponible y solicitar la aprobación del usuario.

La respuesta inesperada es más difícil de gestionar eficazmente. Presentar los detalles de un fallo de infraestructura al usuario no es útil, y no hay ninguna acción que pueda tomar basándose en la información. También existe el riesgo de revelar detalles de la infraestructura; esta información es útil para los atacantes.

Aunque no se muestran detalles de un fallo inesperado al usuario, la aplicación puede tomar medidas según la naturaleza de los fallos. En caso de un fallo transitorio, como un tiempo de espera de la red u otros recursos, se pueden realizar varios reintentos. En caso de escasez de recursos en la nube, guardar el estado actual del cliente localmente en el cliente y ofrecer la opción de reintentar la operación fallida posteriormente puede ser útil para el usuario, siempre que se presente una descripción clara de la situación y le dé motivos para creer que se resolverá. Si un almacén de datos se queda sin espacio en disco, guardar el estado actual en el cliente, enviar una alerta interna al equipo de operaciones o a los administradores e informar al usuario de que se han realizado ambas acciones es una buena respuesta. Que la aplicación cliente reciba una notificación cuando la situación se haya resuelto y le indique al usuario que reanude la ejecución es una comodidad adicional que infunde confianza en la solidez de nuestro sistema, la aplicación y la organización que la respalda. Para encontrar un equilibrio entre mantener privados los detalles de la infraestructura del sistema y sus fallas y brindar suficiente información para que sea útil para el cliente, la respuesta del servidor debe contener tanta información como sea necesaria para informar al cliente sobre la naturaleza de la falla.

Considere algunos de los errores que pueden encontrarse en la nube al ejecutar una solicitud de cliente. Un fallo transitorio de infraestructura podría ser específico, como un error de red, un tiempo de espera agotado en una llamada a un servicio u operación externa, o un error que GCP haya etiquetado con su `TransientError` genérico. La aplicación cliente realizaría la misma acción en respuesta a cualquiera de estos, por lo que el único nivel de detalle necesario en la respuesta del servidor es el de la detección de un `TransientServerError`, con una indicación opcional sobre cuándo reintentar la solicitud.

Para las fallas del método del servidor descritas anteriormente en esta sección, el siguiente conjunto de respuestas proporciona la precisión necesaria al cliente sin filtrar detalles de implementación del servidor:

Response code	Payload	Retriable by client
OK	Method's expected return value	Not applicable
TransientServerError	Delay for retry	Yes
ServerError	Handle to resolution event	No
InsufficientInventory	List of items and available quantities	User action required
InvalidContents	List of items, quantity affected and messages	User action required

Cifra 7.4 Respuestas del método del servidor

La asignación de respuestas del servidor, como en un método del backend para el frontend, como se explicó anteriormente en este capítulo, a una de estas cinco clases de respuesta permite que el método del servidor más cercano al cliente específico proporcione los detalles con la mayor precisión posible. Por ejemplo, conviene especificar duraciones diferentes para el valor de "Retraso de reintento" para teléfonos y computadoras de escritorio.

Definición y gestión de acciones no iniciadas por el usuario

El flujo normal de eventos en una aplicación informática es bastante simple. El usuario realiza una solicitud y la aplicación responde. En una aplicación distribuida multicapa, como una respaldada por Google Cloud Platform, la respuesta puede generarse mediante una secuencia compleja de interacciones y cambios de estado en diversas plataformas, pero al final la experiencia del usuario permanece: " Solicito que la aplicación haga algo y la aplicación indica que lo ha hecho". Ya hemos visto algunos escenarios en los que la práctica real difiere de este modelo. Si el usuario solicita una acción no permitida, como comprar más de un artículo de lo permitido, la aplicación no realiza la acción solicitada, pero aun así genera una respuesta. De igual manera, si algún fallo interno interfiere con el cumplimiento de la solicitud del usuario, la respuesta de la aplicación lo indica. La interacción solicitud/respuesta se mantiene, pero ahora se generaliza a " Solicito que la aplicación haga algo y la aplicación indica qué se ha hecho".

Casos de uso para acciones iniciadas por el servidor

El usuario individual no es el único agente activo en un sistema distribuido. Otros usuarios concurrentes, procesos no iniciados por una solicitud específica del usuario y efectos secundarios de procesos y sistemas externos pueden cambiar el estado de nuestro sistema. Los cambios en el estado compartido del sistema pueden tener consecuencias directas para los usuarios, y una aplicación debe tener una forma de incorporar, resolver e informar al usuario sobre dichos cambios de estado impuestos.

Antes de entrar en detalles sobre cómo se puede implementar este tráfico sin respuesta, primero categorizaremos y describiremos los tipos de tráfico que se entregan al cliente de esta manera y algunos ejemplos.

Un servidor que inicia una interacción con el cliente, invirtiendo así los roles esperados, se encuentra fuera de la interacción más típica, pero no es infrecuente. Una aplicación incapaz de gestionar las solicitudes del servidor, en efecto, ignora que existe una máquina real y compleja detrás de las llamadas a la API y el estado persistente que respalda nuestra aplicación cliente. Esta visión limitada de un sistema distribuido puede funcionar para algunas aplicaciones, pero es probable que presente situaciones que resulten en problemas para los usuarios. Por lo tanto, los siguientes casos de uso deben considerarse como parte de los requisitos de una aplicación:

- Disponibilidad limitada o inexistente de las aplicaciones: El conjunto de servicios en la nube podría no estar disponible o funcionar solo parcialmente. El servidor debería poder informar a las aplicaciones cliente de esto, si es posible (es decir, suponiendo que la conexión a la nube funcione). La alternativa sería una combinación de operaciones individuales exitosas o fallidas, lo que dejaría a los usuarios confundidos y frustrados, y podría comprometer el estado almacenado de la aplicación.
- Alertas de difusión: algún mensaje, estado de característica especial o evento inminente está presente en la nube y no hay ningún conocimiento codificado preexistente de esto en el cliente.
- Resultado de una llamada u operación de método asíncrono: El procesamiento asíncrono descrito en la sección anterior, Respuestas a solicitudes bloqueantes y no bloqueantes, desacopló eficazmente la solicitud del cliente de la respuesta del servidor. Esto deja la respuesta del servidor como un agente libre, que puede presentarse en cualquier momento, sin restricciones.

Asociación con la solicitud que la activó. Este es un caso especial de tráfico generado por el servidor. Si bien el cliente la solicitó, esta ya se realizó en el pasado y la respuesta se entrega independientemente de la operación solicitante.

Mecanismos para recibir tráfico generado por el servidor. El software puede recibir una notificación de dos maneras, también conocidas como mensaje, señal o evento. El receptor puede extraer una notificación o el emisor puede enviarla. Otros términos que se utilizan al describir este tema son sondeo, sinónimo de notificaciones de extracción, y devoluciones de llamada, sinónimo de notificaciones de inserción.

El cliente recupera las notificaciones de extracción mediante una solicitud de sondeo específica. Este modelo se describe mediante la siguiente secuencia de eventos:

- El servidor agrega notificaciones a una cola de notificaciones y continúa con otro trabajo.
- Los clientes envían una solicitud de extracción al servidor, lo que equivale a devolver una notificación en cola.
- El servidor devuelve una notificación de la cola, eliminándolo de la cola:
 - Si no hay notificaciones en cola, el servidor devuelve nulo o Ninguno.

Las notificaciones pull son, de hecho, el mecanismo tradicional de solicitud-respuesta del cliente. La característica distintiva es que la solicitud no es una invocación imperativa de `doSomething()` por parte del cliente, sino una interrogativa de `isThereAnything?()` a la que el servidor puede responder simplemente, con precisión y como era de esperar, que no.

Las notificaciones push invierten esta relación, permitiendo que el servidor controle el tiempo de entrega de las notificaciones. En efecto, el cliente actúa como servidor, proporcionando un destino al que este puede enviar notificaciones. Este destino puede ser un punto final HTTP accesible o una conexión o canal abierto continuamente, como un WebSocket. También existen protocolos específicos para clientes móviles: Notificaciones Push para iOS y Firebase Cloud Messaging de Google, compatible con diversas plataformas.

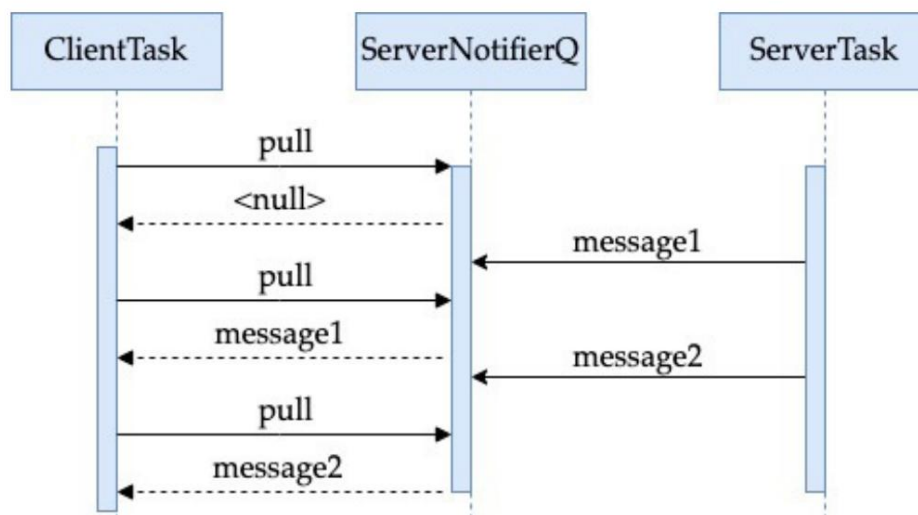
La secuencia de eventos de una notificación Push es la siguiente:

- El cliente registra un canal de notificación o punto final con el servidor y continúa con otro trabajo. El servidor envía una
- notificación al medio de notificación registrado. El cliente recibe la notificación como
- una invocación de método del punto final o un mensaje entrante con una carga útil en el canal abierto.

Invertir el control o la iniciación de la entrega de notificaciones implica un equilibrio de costos. En el caso de las notificaciones pull, muchas solicitudes pueden generar respuestas nulas. Las notificaciones push solo generan un mensaje multiplataforma cuando es necesario. En contrapartida, las notificaciones push requieren una conexión constantemente abierta, que puede estar inactiva durante gran parte del tiempo. El sondeo solo requiere que haya una conexión disponible cuando el cliente envía una solicitud pull.

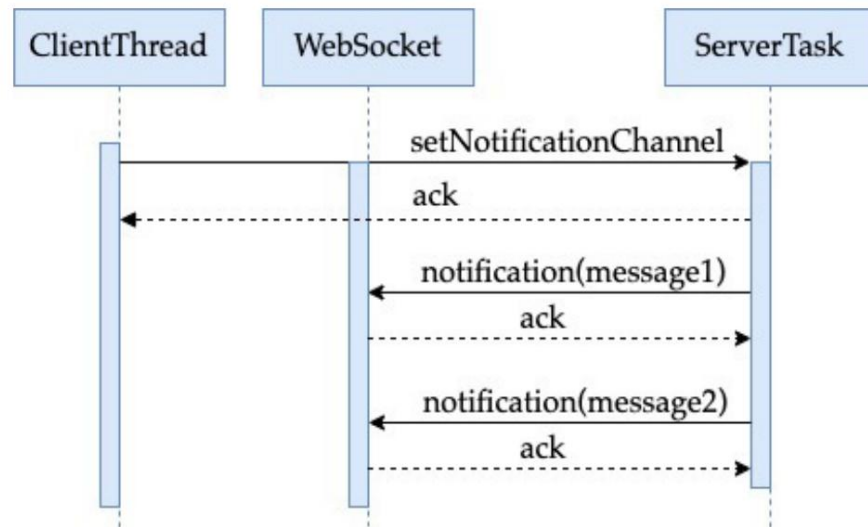
La secuencia de eventos entre el cliente y el servidor está bien definida para los modelos de interacción push y pull. La implementación de cualquiera de estos debe seguir el patrón básico de interacción.

Las siguientes figuras ilustran el patrón de interacción en los modelos push y pull al entregar dos notificaciones de servidor desde una tarea de servidor a un cliente:



Cifra 7.5 Votación/Notificaciones pull

Consulte la siguiente figura para obtener más detalles:



Cifra 7.6 Notificaciones push

Un conjunto de diferencias clave entre ambos es la presencia de operaciones de extracción adicionales en la implementación de extracción y el desacoplamiento en el tiempo de la creación de mensajes por parte de `ServerTask` de la recepción de los mismos por parte de `ClientTask`. Esto se puede resumir en la falta de eficiencia e inmediatez del modelo de sondeo. Una diferencia que sirve como contraargumento es el requisito adicional del `WebSocket` de cliente persistente en la implementación de push. Este objeto de larga duración implica una conexión de red de larga duración entre los dos puntos finales.

Consideraciones especiales para interfaces de cliente o de nube Las compensaciones entre las notificaciones push y pull descritas anteriormente son válidas para todos los casos de uso, pero hay consideraciones que deben tenerse en cuenta y que merecen especial interés al diseñar la arquitectura de una aplicación cliente que se basa en una infraestructura respaldada por la nube.

Mantener una conexión abierta para el canal de notificaciones en un modelo push consume recursos en el cliente y en la nube, y depende de una conexión de red estable y confiable entre ambos. A continuación, evaluaremos el costo de cada uno.

Los recursos de conexión local en el cliente no son grandes, por lo que no son un factor significativo. Un `WebSocket` o un framework de plataforma reserva alguna referencia o identificador a la conexión de capa inferior junto con código de biblioteca: estos identificadores son pequeños, y asumiremos que el código está bien...

optimizado.

Los recursos de conexión en la nube no son necesariamente grandes por cliente, pero la plataforma en la nube debe proporcionar una conexión para cada cliente activo simultáneamente. Es probable que los recursos necesarios para mantener esta conexión sean escasos, junto con otros recursos ya reservados para el cliente, por lo que los recursos en la nube tampoco representan un costo significativo.

La conexión de red en sí, la capa inferior a la que tanto el cliente como la nube se refieren, es un factor diferente. A diferencia del coste de los recursos del cliente y la nube, que se asignan y luego están disponibles, la conexión de red es un recurso con constante demanda, que depende de otros sistemas fuera de nuestro control. Los teléfonos entran y salen del servicio, las rutas intermedias de red se congestionan y los proveedores de servicios de Internet (ISP) sufren interrupciones. Obtener una conexión para notificaciones push siempre disponibles no garantiza que la conexión esté disponible cuando el servidor intente enviar un mensaje a un cliente.

Cuando un canal de push está inactivo, el cliente puede o no perder intentos de push desde el servidor. Un mayor riesgo para la integridad del estado de la aplicación reside en que el cliente puede no ser consciente de la caída del canal en algún momento.

Las notificaciones pull o los modelos de sondeo son tan vulnerables a la fragilidad de una ruta de red como las notificaciones push; es inevitable. Sin embargo, los modelos de sondeo tienen la ventaja de que su naturaleza permite conocer el estado de la conexión: un cliente intenta explícitamente comunicarse con el servidor con una solicitud de sondeo, y se sabe si el intento falla. Si bien los modelos push pueden implementar sondas de mantenimiento de la conexión o de latido para emular esta ventaja, es inherente a un modelo de sondeo, y el estado de la conexión se conoce en el momento del sondeo, en lugar de en el latido más reciente, que está desconectado de cualquier intento de envío desde el servidor.

Conclusión

Este capítulo exploró el diseño de aplicaciones cliente como interfaz o portal de usuario y como parte integrada de la funcionalidad del sistema.

Se dieron las razones para ejecutar la lógica en la plataforma del cliente, junto con

Las desventajas de esto. Se presentó la práctica de codificar módulos de políticas independientes de la plataforma incluidos estáticamente para abordar la posible introducción de múltiples fuentes de verdad, lo que daría lugar a implementaciones desincronizadas de la lógica común.

Se identificaron partes de la interfaz de usuario que pueden prepararse en la nube, con ejemplos que ilustran las razones para ello. A continuación, se describió la aceptación y validación de la entrada del usuario, así como el uso de la doble validación para proporcionar retroalimentación ágil y eficaz, a la vez que se proporciona validación de datos en componentes del servidor, coherente con la experiencia del usuario.

La opacidad de los componentes del servidor es crucial tanto para la reutilización como para la protección contra actores maliciosos. El nivel de abstracción de las respuestas del servidor es un factor importante para aislar la infraestructura del servidor y, al mismo tiempo, proporcionar suficiente información al cliente. Se presentó un modelo para lograr este equilibrio.

Finalmente, el capítulo describió los dos modelos para gestionar las solicitudes del lado del servidor: notificaciones push y pull. Se describieron las ventajas y los casos de uso adecuados de cada uno, junto con sus desventajas.

En los siguientes capítulos, aplicaremos las prácticas y patrones presentados en el desarrollo de una aplicación real. Desarrollaremos un sistema de votación para concursos de talento televisados con un frontend móvil y un backend escalable, fiable y basado en la nube. El siguiente capítulo describirá el sistema de votación en detalle y analizará sus características y desafíos críticos.

Únete a nuestro espacio de Discord

Únase a nuestro espacio de trabajo de Discord para obtener las últimas actualizaciones, ofertas, eventos tecnológicos en todo el mundo, nuevos lanzamientos y sesiones con los autores:

<https://discord.bpbonline.com> .



Sección 3:

De extremo a extremo: una votación global
Sistema

CAPÍTULO 8

Creación de un sistema de votación Disponible y confiable

Introducción

Este capítulo inicia la siguiente sección del libro y presenta nuestro caso práctico: un sistema de votación en línea en tiempo real. Las democracias digitales más grandes del mundo y los concursos de talento televisados son excelentes candidatos para un caso práctico útil de las prácticas descritas en los capítulos anteriores. Los altos picos de demanda de los usuarios, la estricta exigencia de integridad de los datos y la accesibilidad de las aplicaciones frontend son características cruciales de un sistema de votación popular y ponen en práctica las fortalezas de un sistema basado en la nube. Este capítulo presenta las características de nuestro sistema de votación y las principales preocupaciones y desafíos que presenta cada una de ellas.

Estructura

En este capítulo se tratarán los siguientes temas:

- Descripción general del sistema de votación digital
- Descripción general de un concurso de talentos en línea
- Requisitos de rendimiento del sistema de votación

- Modelos de uso
- Agregación de datos
- Accesibilidad
- Creación del paquete y la estructura de directorios del codelab

Objetivos

Este capítulo explorará las fuerzas del mundo real que desafían la fiabilidad y la disponibilidad de un sistema distribuido, específicamente en el contexto de nuestra aplicación de votación. Identificaremos aspectos de estos desafíos, como la distribución geográfica y temporal de la demanda.

Luego realizaremos análisis detallados de la demanda e identificaremos formas de mitigar el riesgo a la integridad del sistema que plantean los factores aislados y examinados.

El laboratorio de código de este capítulo configurará la jerarquía inicial de paquetes y componentes de alto nivel para la base de código del sistema de votación.

Descripción general del sistema de votación digital

Antes de profundizar en el diseño de nuestro sistema, es importante definir claramente el problema que estamos resolviendo. Por lo tanto, primero, repasemos el escenario, el proceso y luego la escala.

Descripción general de un concurso de talentos en línea

Se utiliza una serie de rondas para representar cada nivel de la competencia, con menos rondas que componen cada uno de los tres niveles sucesivos: cuartos de final, semifinales y finales.

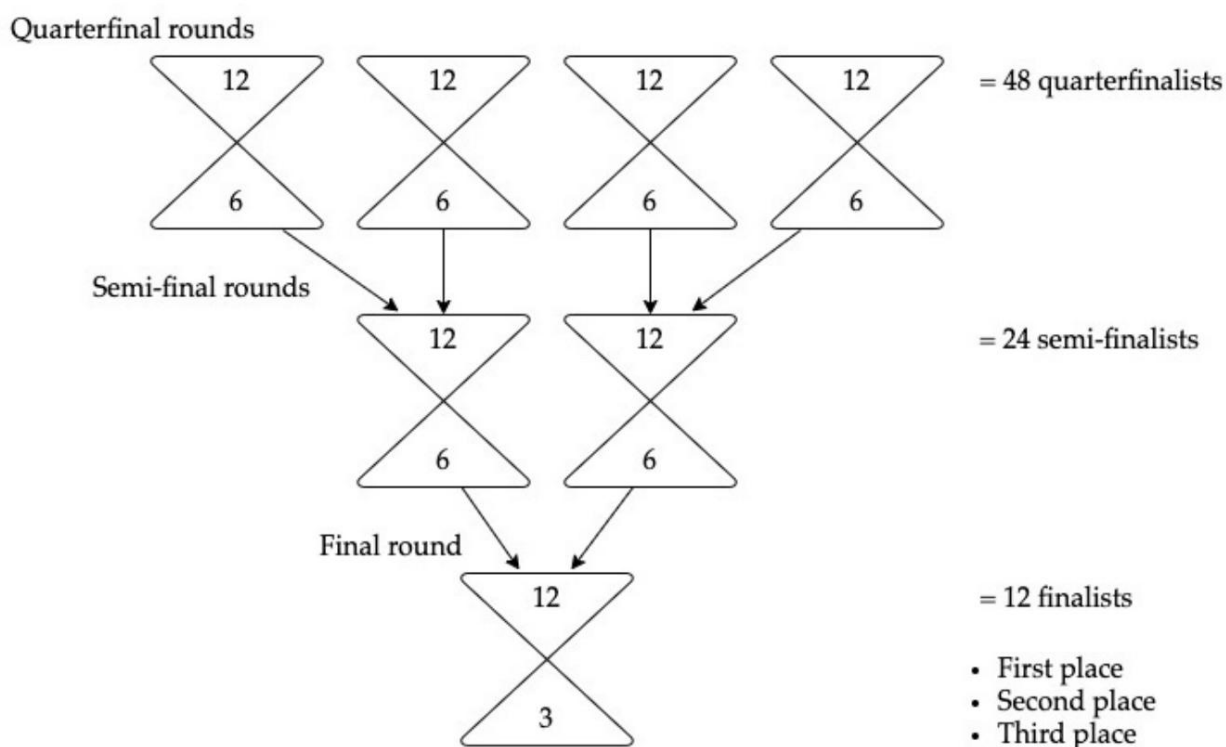
Antes de los cuartos de final se realizan audiciones, en las que participa un número no revelado de concursantes. La selección de los cuartofinalistas en las audiciones la realizan un jurado, no una votación popular. A diferencia del proceso de votación, la selección de las audiciones considera el tipo de artista para garantizar, por ejemplo, que entre los concursantes iniciales elegidos se incluyan cantantes, atletas, comediantes, etc.

Doce concursantes participan en cada ronda de votación, y una audiencia global vota por sus favoritos. Cada voto lo emite un usuario registrado o votante. Un votante puede emitir seis votos por ronda, lo que equivale a la mitad del número de concursantes.

La mitad superior de los concursantes de cada ronda, clasificados según la cantidad de votos que reciben, son seleccionados para pasar al siguiente nivel, y los concursantes restantes son eliminados de la competencia.

Las rondas avanzan, enfrentando a diferentes ganadores de un nivel anterior hasta la ronda final. En el nivel final de una ronda, se eligen los 3 mejores artistas y se designan como ganadores del primer, segundo y tercer lugar de la competencia.

Contando hacia atrás desde la ronda final, determinamos que 48 actos participan en un total de siete rondas en tres niveles, ilustrados en la siguiente figura, que representa un diagrama de árbol:



Cifra 8.1 Niveles y rondas de la competencia

Proceso de votación

El proceso de votación sigue reglas y restricciones específicas que el sistema aplica. Estas reglas son políticas que fundamentan la confianza del público, por lo que es fundamental que la aplicación las respete visiblemente y esté protegida contra vulnerabilidades y usos indebidos que puedan permitir la violación de las reglas de votación publicadas.

Al emitirse un acto, se espera que los espectadores lo vean. Esto se garantiza permitiendo que solo se vote por un acto determinado una vez que los administradores del programa lo hayan marcado como elegible, mediante una aplicación independiente. Los administradores realizan esto para cada acto durante su primera emisión oficial.

Al finalizar una ronda, los administradores dejan los actos elegibles y abiertos a votación durante un tiempo determinado, tras el cual los marcan como no elegibles. Esto establece un límite de tiempo para la elección de la ronda.

Cada votante recibe un total de seis votos para emitir en cada ronda, de los cuales puede emitir cualquier parte: puede votar por ningún acto en una ronda, por un acto y hasta seis. Solo puede votar una vez por un acto específico en una ronda; no se permite emitir varios votos por un mismo acto en una ronda.

Una vez emitido un voto, no se puede revocar ni anular: el votante no tiene forma de reconsiderarlo en caso de arrepentimiento. Un efecto secundario esperado y observado de esta regla es que muchos votantes posponen su voto hasta que se hayan visto un gran número, o todos, de los actos de una ronda.

Requisitos de escalabilidad. Hay varios

aspectos de la escalabilidad que se deben considerar en el diseño y la configuración de la implementación de una aplicación distribuida. El primer paso para diseñar la escalabilidad es identificar y describir cada aspecto relevante en el contexto de nuestro sistema de votación para cada característica relevante. En este caso, la función "Emitir Voto" es la única solicitud que nos ocupará.

Los aspectos de escalabilidad debidos a la consideración de la emisión de votos son los siguientes:

- La tasa de solicitud de piso o inactiva es la cantidad de votos que se intentan cuando hay menos personas votando.

Esta es la tasa normal o sostenida cuando no hay demanda excepcional. La meseta a partir de la cual se desvían los picos de demanda. Esto es útil para planificar una capacidad mínima en línea para el servidor.
recursos.

- La tasa máxima de solicitud es la rapidez con la que se emiten los votos cuando aumenta la participación electoral.

Esta es la demanda máxima teórica esperada del usuario, la cantidad de solicitudes por segundo que se podrían realizar si todos los usuarios elegibles estuvieran activos y votaran por la misma acción.

- La pendiente máxima del tráfico, también conocida como tasa de aceleración, indica la rapidez con la que la comunidad de votantes comienza a votar cuando aumenta la participación.

Así de rápido empiezan a votar los usuarios en situaciones de alta demanda o picos de tráfico. ¿Aumentamos la potencia desde el estado inactivo hasta el máximo en 10 segundos, en un minuto o en 5 minutos?

- El detonante de un pico de tráfico es el evento que precede a un aumento en la demanda de los usuarios.

Este es el evento que provoca que la mayoría de los usuarios comiencen a votar. ¿Es al final de un acto, cuando más personas votan por un acto de inmediato, o al final de un episodio de una ronda, cuando más personas esperan para votar hasta que todos los actos hayan terminado?

- La ventana de tasa máxima sostenida es el tiempo que duran los aumentos repentinos de participación electoral.

Existe una duración para la tasa máxima de participación en la votación.

¿La mayoría de la gente vota dentro del minuto siguiente al pico de la votación o lo hace, distribuyéndose en el transcurso de varios minutos?

¿Hasta cuándo se mantendrá la tasa máxima de votación?

- La distribución del origen del tráfico refleja desde dónde votan las personas al mismo tiempo.

Cuando un gran número de personas vota, puede ser de una región, de un conjunto de regiones o de todo el mundo a la vez. Esto puede ser una función

de una tendencia al sesgo regional hacia los actos o diferentes horarios de transmisión local para los episodios.

Requisitos de rendimiento del sistema de votación

Tras enumerar los seis aspectos mencionados de la demanda de los usuarios con respecto a la función de votación, podemos asignar valores fundamentados a cada uno. A medida que se adquiera experiencia de uso, se espera que estos cambien debido a información nueva o más precisa o, mejor aún, debido al crecimiento orgánico u otros cambios en el comportamiento del usuario.

Además de la funcionalidad del usuario, es importante considerar la fiabilidad y el rendimiento como requisitos de primera clase. El diseño de una aplicación debe tenerlos en cuenta, junto con la funcionalidad especificada en los requisitos del usuario, y no después.

Los aspectos del comportamiento del usuario que impulsan el rendimiento del sistema son los siguientes:

- La tasa de solicitudes inactivas mide la demanda de los usuarios en periodos de descanso relativo: los episodios se emiten semanalmente y, tras la emisión de una ronda y la votación, se asume que no se emitirán más votos. Esto indicaría una tasa de solicitudes inactivas de cero. Puede haber fuentes de tráfico no relacionadas con el usuario, como tareas de monitorización y mantenimiento, pero estas serán de escala nominal y fáciles de identificar.

Sin embargo, el mundo real es más complejo que la situación ideal.

Incluso después de la última emisión y de que los administradores del concurso hayan terminado de votar por una ronda, existen otras fuentes de espectadores a considerar.

Algunas emisoras pueden repetir la emisión de un episodio. Si los episodios o actos individuales de nuestro concurso están disponibles en un servicio de streaming, es imposible predecir cuándo un espectador podría intentar votar por error por un acto que ve a través de ese servicio. Se pueden usar grabadoras de video personales para grabar un programa transmitido y verlo más tarde. En cualquiera de estos casos, aunque el sistema no aceptará ni contabilizará los votos emitidos fuera del horario establecido, deberá responder a la

solicitar de alguna manera, como con un código de estado de ~~429~~ demasiadas solicitudes, por lo que el frontend debe estar listo para aceptar esos intentos de votación.

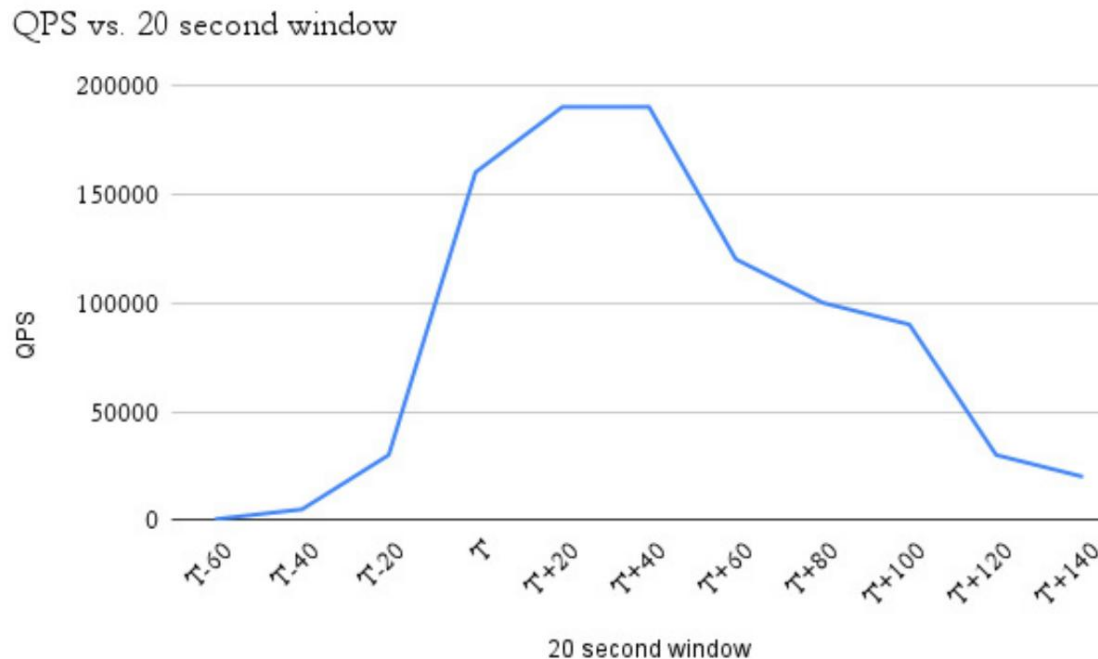
Dada la audiencia global y la importancia que nuestro equipo de producto da a la confianza y la capacidad de respuesta del usuario, siempre debemos poder atender una pequeña cantidad de intentos de votación. Nuestro equipo de producto ha establecido la expectativa de esperar al menos 500 votos por hora, o aproximadamente 0,2 consultas por segundo (QPS).

- Tasa máxima de solicitudes: El número máximo de votos emitidos simultáneamente es un valor fácil de determinar. Podemos asumir que un acto sumamente atractivo haría que todos los votantes registrados emitieran su voto. Por lo tanto, la tasa máxima de votos emitidos se alcanza cuando todos los espectadores elegibles votan en el momento culminante de la actuación. Con una audiencia global, esto es menor que el número total de votantes potenciales; es el número de votantes en la audiencia regional más grande, más el número de espectadores fuera de esa región que podrían ver el acto a través de otros medios, como transmisiones en vivo, servicios de transmisión asíncrona o servicios de acceso remoto. Los investigadores de mercado estiman que esta cifra podría alcanzar los 20 millones de espectadores simultáneos.

Tenemos una suposición de trabajo de una ventana de tiempo de dos minutos durante la cual los votantes inmediatos emitirán sus votos, lo que significa que nuestras interfaces deben poder escalar para manejar 20 millones de solicitudes en dos minutos, lo que es un promedio de 167K QPS.

- Pendiente de pico de tráfico: Un pico de dos minutos tendrá una pendiente pronunciada tanto en el lado ascendente como descendente. Sin embargo, existe un aumento gradual de la demanda; los votos se emitirán en cuanto se presente una actuación, ya sea por la anticipación y predisposición del concursante o por las reacciones positivas tempranas a la actuación. Existe una cola de votos: los votantes que debaten con familiares y amigos o no desvían su atención de la actuación en sí.

El análisis de mercado y el examen de la técnica anterior arrojan un modelo de demanda de votación en torno a la emisión de un acto popular que se asemeja a la siguiente figura:



Cifra 8.2 Forma de un pico de tráfico

Es necesario explicar brevemente el gráfico mostrado en la figura anterior, ya que lo utilizaremos en varios de los siguientes análisis: El eje X representa una ventana de tiempo

- de 3 minutos y 20 segundos, anclada en T, que es la conclusión de un acto. Cada marca en el eje X marca el comienzo de una porción de 20 segundos de esa ventana observada de 200 segundos.
- El eje Y representa el promedio de consultas por segundo durante un intervalo de 20 segundos. Para un valor gráfico de 150 000 QPS, el intervalo de 20 segundos incluyó un total de 3 millones de consultas.

No se muestra en el gráfico, pero cabe destacar el número total de consultas recibidas en este período, que asciende a 18,7 millones. También cabe destacar la tasa promedio general de solicitudes durante este período, que es de 93 500 QPS.

- Desencadenante de un pico de tráfico: Se espera que los espectadores emitan su voto tras un momento culminante o cautivador de un acto, cerca del final de la función. Dado que estos actos son breves, de unos pocos minutos de duración, los espectadores que emiten su voto...

Las votaciones inmediatamente posteriores a la conclusión de una función se superpondrán y se fusionarán con las del momento culminante. Esto se muestra en el gráfico de demanda como el pronunciado ascenso que comienza en T-20, o 20 segundos antes del final de la función, y continúa hasta el momento final, representado por T en el eje X.

Por lo tanto, existe un detonante para un pico de demanda, que es un momento no específico cercano a la conclusión de una actuación. En el caso mostrado en [la Figura](#), el detonante ocurre durante los 20 segundos previos a la conclusión, durante el segmento que comienza con T-20.

- Ventana de tasa pico sostenida: La tasa más alta de votos emitidos comienza, como se mencionó, poco antes de la finalización de una función y se observa que continúa durante casi un minuto después. En [la Figura](#), el pico de demanda corresponde al período comprendido entre T y la ventana de 20 segundos, que comienza en T+40, para una ventana de pico efectiva de 1 minuto. No consideramos que el pico incluya únicamente el apogeo literal del gráfico; la tasa de 160 000 QPS en T es lo suficientemente cercana al pico absoluto como para ser considerada.
- Distribución del origen del tráfico: Es necesario considerar la distribución de la demanda entre nuestra audiencia global. Si bien el número total de votos emitidos es la suma de todos los votantes a nivel mundial, no todos los espectadores ven el evento y emiten sus votos al mismo tiempo. Por lo tanto, cabe esperar múltiples períodos de máxima demanda, centrados en diferentes regiones, a medida que una ronda se transmite desde la sede principal de cada región. Existen mercados más grandes y más pequeños, y diferentes perfiles de espectadores en cada región, lo cual es fundamental para predecir la demanda y asignar recursos para mantener la disponibilidad del sistema en todas las regiones.

Los recursos de red, los balanceadores de carga, los servidores frontend y los procesadores backend son recursos que Google Cloud Platform nos permite configurar a nivel regional. La cantidad de instancias, la estrategia de escalado y la cantidad de instancias que se mantendrán en espera son relevantes para la forma y el tamaño de los picos de demanda descritos anteriormente en esta sección. El análisis ilustrado en [la Figura](#) debería ser...

Se realizaron para cada audiencia regional principal. Es probable que la forma de las pendientes difiera según la región: por ejemplo, en algunos mercados donde la audiencia se compone principalmente de televidentes, la demanda tendrá los picos pronunciados y bien definidos que se muestran en la [Figura 8.2](#). En un mercado donde la mayoría de los espectadores utilizan servicios de streaming, la demanda será más plana, ya que los espectadores ven los espectáculos según su propio horario, sin que sus acciones colectivas estén sincronizadas por su proveedor.

Modelos de uso

La sección anterior analizó en profundidad la demanda máxima de nuestro sistema de votación, la tasa más alta de solicitudes de voto de toda la comunidad global. Nuestros episodios se transmiten en vivo en Norteamérica a través de un único proveedor y concentran la mayor respuesta de votación individual de todas las regiones. Se trata de los aproximadamente 20 millones de votantes que se analizaron en la sección anterior.

[Límites regionales de Google Cloud Platform](#) Ahora enumeraremos las principales regiones geográficas a las que presta servicios GCP y los perfiles de tráfico y participación de votantes esperados dentro de cada una de esas regiones.

GCP representa la geografía en una jerarquía de grupos de continentes, regiones y zonas. Los grupos de continentes son las principales masas geográficas del planeta. Las regiones son ubicaciones donde se alojan los servicios de GCP. Cada región contiene varias zonas, que son sitios de servicio específicos, como centros de datos. No todos los productos están disponibles en todas las regiones, pero todos los productos de infraestructura de ejecución de GCP se ofrecen en todos los continentes y en más de una región en la mayoría de los continentes.

Existen otras entidades geográficas, como los puntos de presencia de red, pero están fuera de la consideración de la mayoría de los desarrolladores al configurar la implementación de una aplicación.

Los grupos continentales y una breve descripción de sus ofertas son los siguientes:

- Américas (América del Norte, Central y del Sur): GCP tiene muchos

Regiones dentro del continente americano, como Canadá, EE. UU. y Sudamérica. Varias de las Zonas son ubicaciones con bajas emisiones de carbono, lo que significa que tienen una baja densidad de carbono en la red. Para obtener información específica sobre el valor de bajas emisiones de carbono, consulte la documentación actual de Google Cloud.

- Europa (Reino Unido, Europa del Norte, Europa del Sur y Europa del Este): la cobertura del GCP de Europa, incluido el Reino Unido, es similar a la de América del Norte, y la mayoría de las zonas son sitios con bajas emisiones de carbono.
- Asia Pacífico (APAC) —Australia, Asia Meridional y Septentrional, Asia Oriental: GCP ofrece múltiples zonas dentro de cada masa continental principal de esta región. La conectividad de red es la más limitada entre las zonas de esta región debido a la gran cantidad de masas continentales disjuntas y distantes que incluye (por ejemplo, Australia). Ninguna de las zonas de la región APAC es un sitio con bajas emisiones de carbono al momento de redactar este documento.
- Medio Oriente, Norte de África y Sudeste Asiático: GCP ofrece algunas regiones en Medio Oriente, y una de ellas se ofrece exclusivamente a través de una asociación regional con un distribuidor local.

Existen restricciones tanto en la ubicación de la cuenta como en la disponibilidad de los servicios de GCP desde esa región. Algunos productos, como Bare Metal y Firebase Storage, no están disponibles en Oriente Medio.

- África (África Central y Meridional): GCP ofrece actualmente una región en el continente africano. No todos los servicios están disponibles en la región africana, lo que depende principalmente de la demanda local de los clientes.

[Demanda del sistema de votación por continente](#). Para diseñar nuestra

aplicación de votación y que se adapte a la demanda, debemos comprender dónde y cuándo se generará. La selección de técnicas en la nube, como la canalización de solicitudes asíncronas, requiere conocer dónde se ubicarán los puntos de tensión.

Podemos asumir con seguridad que el uso en cada uno de los grupos continentales puede considerarse de forma independiente. Con esa suposición,

La siguiente tabla de demanda resume el análisis de uso máximo realizado en la sección anterior para cada uno de los cinco grupos de continentes identificados por GCP y descritos previamente en los límites regionales de Google Cloud:

Continent	Peak window	Max QPS	Total requests per voting window	Average voting QPS
Africa	100 seconds	70K	7M	34K
Americas	60 seconds	190K	18.7M	93K
APAC	45 seconds	110K	11.1M	27K
Europe	150 seconds	90K	15M	40K
Middle east	120 seconds	80K	5M	16K

Cifra **8.3** Pico de demanda de votación por continente

La tabla anterior revela algunos puntos importantes:

- El mayor número de votos se concentra en América, Europa y Asia-Pacífico (APAC). Al diseñar la ubicación del mecanismo de agregación de votos, lo mejor será mantener ese servicio local en esos continentes.
- La tasa más alta de consultas por segundo (QPS) se registra en América, con una ráfaga de un minuto. La segunda región de Asia-Pacífico no se acerca, pero sigue siendo muy alta y con una ráfaga más corta. Esto orienta la escalabilidad de los gestores de solicitudes de votación.

Estos conocimientos son valiosos para optimizar la implementación de nuestro sistema, y nuestro diseño debe ser compatible con dicho modelo. En la siguiente sección, analizaremos en detalle el diseño de nuestro procesamiento de solicitudes de voto y cómo incorpora este conocimiento de la demanda regional.

Agregación de datos

El número de votos requiere un número total igual de solicitudes para registrarlos. Esta demanda de recursos es para los trabajadores, pero a menos que registremos permanentemente cada voto individual (no lo hacemos), no afecta el tamaño de nuestro almacén de datos: un voto o 20 millones de votos aún caben.

Dentro de un campo entero largo. Sin embargo, estos votos, aunque efímeros, requieren recursos para representarlos hasta que se suman al total. Estos recursos pueden ser controladores de solicitudes, mensajes asíncronos en una cola u otro artefacto, pero deben ser robustos y rápidos de crear, dado que nuestro sistema creará 190 000 de ellos por segundo en una instancia. Deberíamos diseñar el ciclo de vida de un voto para que sea lo más simple y eficiente posible, procurando acortar la vida útil de cada voto entregándolo rápidamente al proceso que lo aplica al total de votos, momento en el que ya no necesitamos cada uno de estos millones de votos.

Ahora debemos aplicar algunas consideraciones topográficas a nuestro diseño. Las solicitudes interregionales son más costosas que las solicitudes intrarregionales, y las que cruzan continentes son aún más costosas. El gasto no se limita al costo del material o los cargos por servicio, sino que también se refleja en una mayor latencia y una menor confiabilidad. Por lo tanto, mantener la mayor cantidad de tráfico local posible es una prioridad para nuestra aplicación.

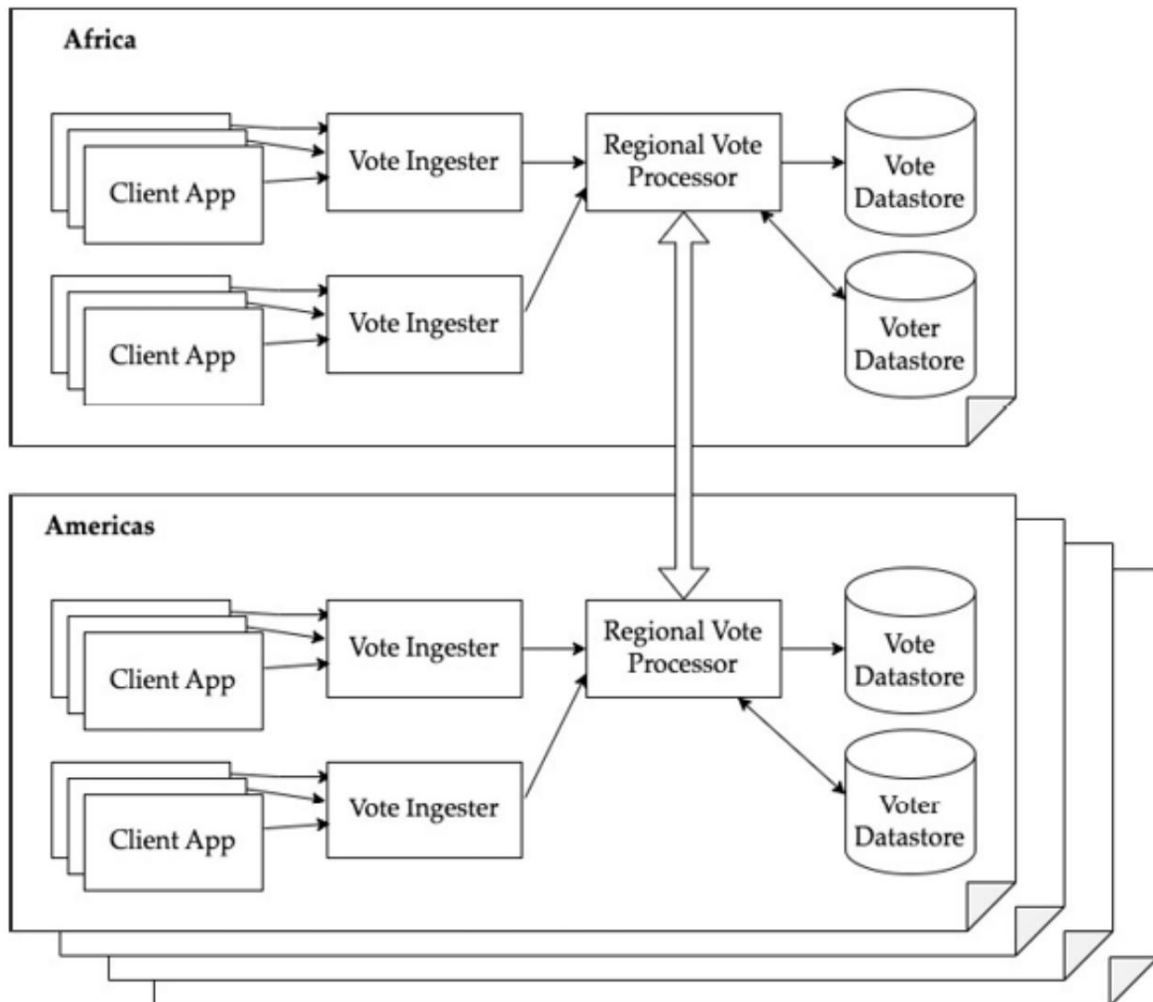
[Agregación de votos distribuidos en un sistema global](#). Nuestro sistema de votación no requiere que contabilicemos todos los votos en tiempo real, ni siquiera durante una transmisión. Esto no sería útil, dada la amplia distribución espacial y temporal de nuestra comunidad electoral. Recordemos la matriz de Eisenhower del [capítulo 3](#) «Determinación de la infraestructura sobre la que se construirán las características» y la separación entre importancia y urgencia. No es importante contabilizar todos los votos, ni urgente. Por lo tanto, podemos introducir cierta latencia en el proceso de conteo de todos los votos, siempre que garanticemos que se contabilicen todos.

Teniendo esto en cuenta, no existe un requisito estricto de cruzar las fronteras continentales para las solicitudes con la granularidad y la gran escala de los votos individuales. Sumar los votos dentro de un conjunto de dominios locales o regionales nos permite evitar solicitudes costosas (como se define en la sección anterior), y cada uno de nuestros dominios regionales puede entonces sincronizar su pequeño conjunto de votos contabilizados en un resultado electoral global fusionado.

Con este precepto arquitectónico en su lugar, nuestro diseño se centra en cómo

Diseñar un flujo de trabajo regional para el recuento de votos, priorizando la fiabilidad y la capacidad de respuesta del usuario durante el período de recuento. Considerando este proceso regional como una caja negra que se definirá más adelante, la arquitectura de nuestro sistema distribuido se ilustra en la siguiente figura:

Nota: Se han omitido varias regiones, son duplicados de las que se muestran explícitamente.



Cifra 8.4 Arquitectura de procesamiento de votos distribuidos

Esta arquitectura introduce niveles que pueden escalarse de forma independiente y se adaptan a la demanda bajo carga. Los clientes son los impulsores finales de la demanda y están sujetos al escalamiento horizontal más amplio y con menos restricciones. El número de usuarios no solo escapa a nuestro control, sino que también es el recurso que no queremos limitar: cuanto más...

Votar, mejor para nosotros. Queremos procesar la solicitud de voto del cliente con la mayor rapidez y eficiencia posible, por lo que conectamos a los clientes a un frontend de ingesta optimizado para procesar la votación, procesarla y responder al usuario con la mayor rapidez posible. Esta optimización nos permitirá gestionar el mayor número de clientes posible, con cada frontend de ingesta capaz de mantener cientos de conexiones simultáneas y miles de QPS. Al permitir que GCP escale horizontalmente el frontend de ingesta a cientos de instancias, se atienden cientos de miles de solicitudes de cliente por segundo.

El Procesador de Votos Regional no tiene requisitos de latencia en tiempo real y la cantidad de clientes a los que presta servicio está bajo nuestro control: se trata de la cantidad de frontends de ingesta de votos que iniciamos. Esta cantidad será mucho menor que la cantidad de clientes y está sujeta a un límite configurado. Siempre que el mecanismo para enviar las solicitudes de voto desde los ingestadores al Procesador de Votos sea confiable, puede ser asíncrono. Pub/Sub es una excelente opción para desacoplar estos dos procesos en el tiempo. Puede tomar votos de los frontends de ingesta en decenas de milisegundos, colocándolos en una cola de mensajes duradera para que un Procesador de Votos los recupere cuando tenga ciclos disponibles. La lógica de nuestra aplicación backend detecta y previene el recuento de votos duplicados mediante la aplicación de presupuestos de votos.

Una rápida comprobación de confianza resulta útil en este punto para determinar la viabilidad de este enfoque en el escenario de mayor demanda identificado en [la Figura 8.3](#). Los votos emitidos como acto popular se transmiten en Norteamérica. Esto no garantizará un rendimiento satisfactorio en todas las regiones y escenarios, pero es el que tiene más probabilidades de sobrecargar la capacidad de nuestra infraestructura y es el mejor punto de partida para detectar y eliminar fallos.

La siguiente tabla se basa en suposiciones razonables sobre el rendimiento del servidor frontend:

Solicitudes de clientes concurrentes	190,000	Dado
Latencia de la interfaz de entrada de votos: 50 milisegundos	(se asume)	
instancia de frontend concurrente 20		Ficticio

solicitudes		
QPS de instancia de interfaz	400	(1 segundo/50 ms) * 20 concurrentes)
Se necesita un recuento de instancias de frontend de 475		190.000/400

Mesa **8.1** Rendimiento y escalabilidad en escenarios de demanda máxima. Los valores clave a evaluar en esta tabla son la latencia del frontend y el número de instancias del frontend. Las siguientes preguntas son relevantes para determinar si se pueden cumplir nuestros requisitos de rendimiento:

- ¿Es alcanzable una latencia de 50 ms en el frontend?
- ¿Es aceptable una latencia de 50 ms del frontend para nuestra aplicación cliente?
- ¿Podemos escalar a 475 instancias frontend en el grupo de regiones de América del Norte?

Accesibilidad

La disponibilidad de recursos suficientes para atender la demanda regional es conveniente, pero no se puede dar por sentado con seguridad. Si bien Google Cloud Platform revisa constantemente la oferta y la disponibilidad de servicios en función de la demanda de los clientes, existen limitaciones externas, como la disponibilidad de espacio y energía, y una demanda general insuficiente, que podrían provocar que un servicio necesario no esté disponible en ninguna región cercana a una zona de alta demanda de usuarios.

¿Cómo aborda un arquitecto de aplicaciones esta situación?

En un nivel alto, hay algunos enfoques que se discutirán en detalle después de enumerarlos aquí: reubicar estratégicamente

- algunos recursos a la ubicación disponible más cercana de una manera que preserve mejor la latencia del usuario y la integridad de los datos.
- Intente optimizar aún más la aplicación para que funcione de manera aceptable con respecto a los SLO de respuesta dentro de los recursos disponibles en ubicaciones críticas.

- Rediseñar la aplicación para que se ajuste a los servicios disponibles en ubicaciones críticas.

Estos están ordenados desde el enfoque menos disruptivo hasta el más disruptivo, que son los cambios más o menos deseables que un arquitecto de aplicaciones debe realizar.

Preferiríamos que los recursos de infraestructura se ajustaran a la aplicación, en lugar de que esta se desarrollara para acomodar los recursos disponibles de la plataforma. El razonamiento que sustenta esta preferencia es doble: la aplicación debe servir al usuario y al producto, y la infraestructura es idealmente un medio para lograrlo; y el proveedor de la plataforma cambiará la distribución regional de los recursos, y los cambios que realicemos para adaptar la aplicación a los recursos de infraestructura pueden quedar obsoletos con rapidez y frecuencia.

Con esa directiva en mente, describiremos estos tres enfoques para abordar las deficiencias en los servicios regionales.

[Reubicación estratégica de recursos](#). Veamos la [Figura](#) de

la arquitectura de nuestro sistema de votación de alto nivel. Esta se basa en la presencia regional de nuestro almacén de datos. Si hemos decidido que Spanner es nuestro servicio de almacenamiento ideal por diversas razones, observamos el mapa de servicios de GCP y descubrimos que hay varias regiones en el sur de Asia, parte de APAC, que ofrecen el servicio Spanner para abordar la situación en la que Spanner no se ofrece en ninguna región de (por ejemplo) el grupo de regiones de Oriente Medio. Observamos el mapa de servicios de GCP y descubrimos que hay varias regiones en el sur de Asia, parte de APAC, que ofrecen el servicio Spanner. En una primera reacción ingenua, simplemente configuraríamos el procesador de votación regional en Oriente Medio para usar una instancia de Spanner dedicada en esas regiones de APAC.

Esta no es una decisión estratégica ya que la latencia entre el procesador de votos y el almacén de datos Spanner es alta e introduciría una latencia inaceptable y RPC fallidos en el procesador de votos.

Una reacción más meditada es trasladar tanto el procesador de votos como el almacén de datos de Spanner para Oriente Medio a la región APAC. Esto desplaza el tráfico intercontinental a la conexión entre el procesador de votos.

Interfaces de ingesta y el procesador de votos. Esto no es problemático. Recordemos que la conexión entre esos componentes es a través de Pub/Sub, cuya naturaleza asíncrona lo convierte en un búfer ideal. Debido a la mayor latencia de este flujo de tráfico, esta consideración de dónde... Introducir distancia y latencia es lo que se entiende por estrategia. reubicación de recursos.

Optimizar la aplicación para que se ajuste a los recursos disponibles recursos

Cuando implementamos una aplicación, la primera preocupación debe ser Satisfacer los requisitos funcionales: las necesidades de los usuarios. Una computadora El sistema existe para cumplir una función comercial o de entretenimiento, no una necesidad inherente y autodefinida de la propia computadora.

Este punto de vista práctico y centrado en el ser humano nos lleva a una de las Los dichos más famosos entre los desarrolladores de software:

La optimización prematura es la raíz de todos los males. Su fuente se atribuye a Donald Knuth. Aquí está la cita completa de su... libro seminal El arte de la programación informática:

El verdadero problema es que los programadores han pasado demasiado tiempo Preocuparse por la eficiencia en los lugares equivocados y en los momentos equivocados: la ; optimización prematura es la raíz de todos los males o al menos de la mayor parte de ellos). programación .

La razón de esto es que centrarse en la optimización de un tema específico El algoritmo o módulo puede acelerar esa parte de una aplicación, pero El tiempo dedicado a hacerlo puede distraer o interrumpir el software. desarrollador de preocupaciones más importantes, como la finalización de un característica crítica del software

Nota: Los desarrolladores pueden utilizar herramientas de creación de perfiles para identificar objetivos rentables de optimización del rendimiento, cuando la necesidad de tales optimizaciones sea clara. .

Por ejemplo, supongamos que la lista de artículos en un carrito de compras ocupa 200 milisegundos para actualizar. Un ingeniero de software inteligente se da cuenta de que... Puede reescribir toda la representación del carrito en la base de datos y en la memoria caché en solo una semana y acelerar esto hasta 10 milisegundos.

Retraso de actualización. Esto es excepcional: una aceleración de 20x.

El contraargumento a este trabajo es que a nadie le importa ni se da cuenta. La mayoría

La gente no percibirá esta diferencia de menos de una quinta parte de segundo,

Y con esa semana de intenso esfuerzo, el ingeniero podría haber sido

agregando la función Mover todos los elementos a la lista de deseos desde la solicitud de función cola. Los usuarios se habrían dado cuenta de eso.

El tiempo para optimizar las consideraciones de tiempo de ejecución, como el tiempo (latencia de respuesta) o espacio (memoria o número de trabajadores), es cuando son necesarios para los usuarios o para la empresa. Esa es la situación en la que nos encontramos. podemos encontrarnos al implementar una aplicación distribuida en un región con recursos insuficientes para la demanda de nuestros usuarios regionales.

No se trata de una optimización prematura, sino de una optimización en el momento adecuado. y en el lugar correcto. Supongamos que deseamos implementar nuestro sistema de votación en las regiones continentales africanas. La siguiente tabla resume nuestra

Requisitos de recursos basados en el desglose de la demanda global

dado en [la figura 8.3](#) :

Solicitudes de clientes concurrentes	70,000	Dado
Latencia de la interfaz de entrada de votos: 50 milisegundos (se asume)		
Instancia de frontend concurrente solicitudes	20	Ficticio
QPS de instancia de frontend	400	(1 segundo/50 ms) * 20 concurrente)
Se necesita un recuento de instancias de frontend de 175		70.000/400

Mesa 8.2 Rendimiento y escalamiento de la demanda en África

Esto es sencillo, pero supongamos que cuando configuramos nuestro

En su aplicación en las regiones africanas, encontramos que existe una restricción: una

Se pueden crear un máximo de 100 instancias de nuestros frontends Ingestor aprovisionado. Eso puede cambiar, pero por ahora, es un límite estricto.

Necesitamos servir 70 000 QPS desde 100 instancias frontend, un 42 % reducción de recursos.

Podríamos hacer esto a través de varios medios: reduciendo el frontend la latencia en un 42% a 29 milisegundos, aumentando el número de

Las solicitudes concurrentes se redujeron del 42 % a 28, lo que aumenta la tolerancia a la latencia de nuestro cliente, o cualquier combinación de estas. La mejora objetivo puede ser alcanzable o no, pero hay un par de factores a nuestro favor: la mayoría del software tiene margen de mejora, y la falta de optimización temprana implica una optimización latente a la espera de ser implementada si es necesario.

Rediseño de la aplicación . Analicemos un

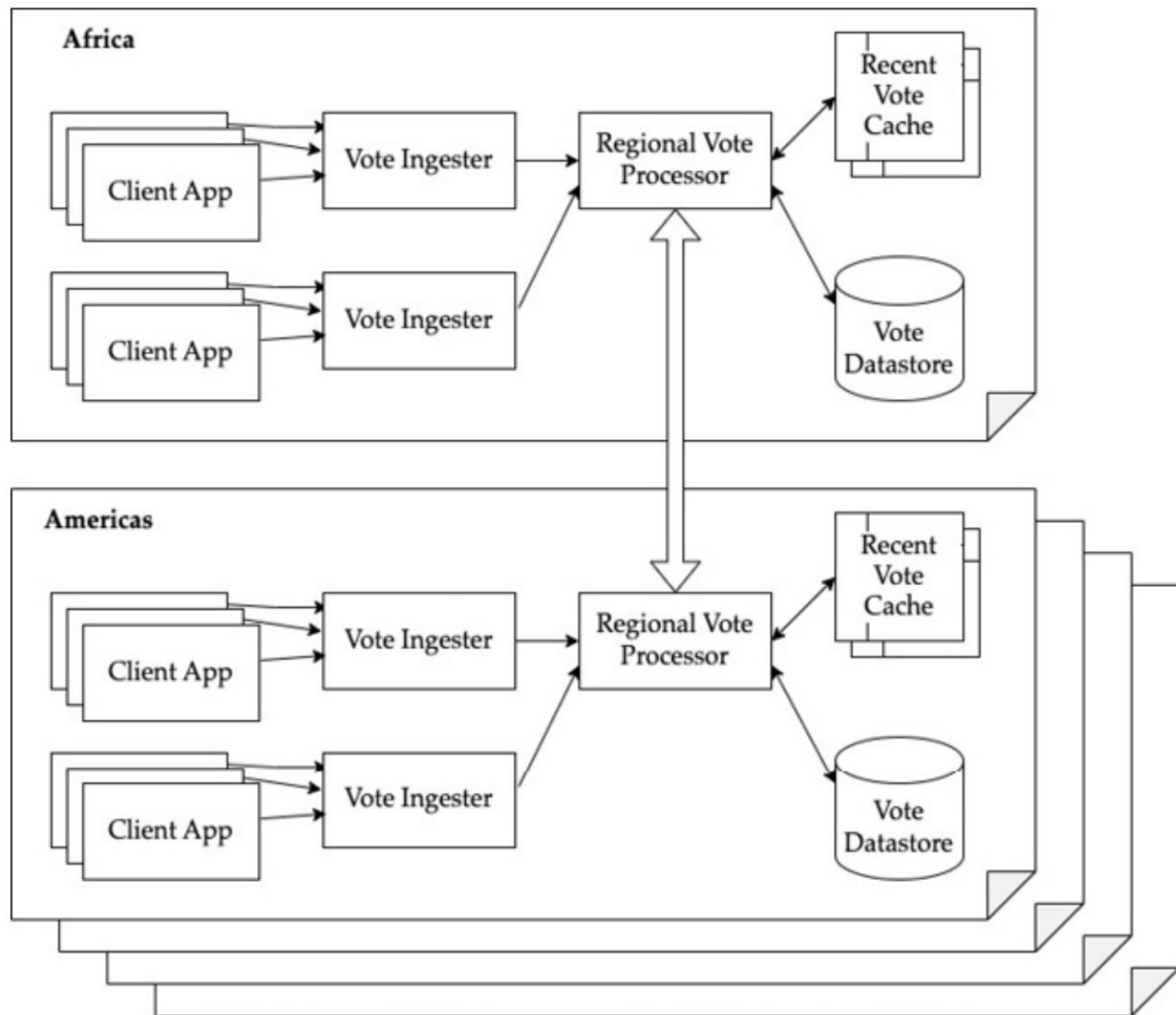
escenario: Se encuentra con un problema de rendimiento que no se puede solucionar (primera estrategia) ni mejorar (segunda). Es posible que haya descubierto que la arquitectura propuesta es fundamentalmente incompatible con la infraestructura elegida y los requisitos de rendimiento.

Supongamos una situación como esta, en la que el almacén de datos, Spanner, no puede procesar las actualizaciones de la actividad electoral del Procesador Regional de Votos a la tasa máxima esperada. Primero examinamos otros productos de persistencia capaces de alcanzar una alta tasa, como Bigtable, pero ninguno ofrece la consistencia global entre los almacenes de datos regionales de la que depende nuestro diseño. Hemos detectado una falla de diseño fundamental y debemos volver a empezar desde cero.

Este es un problema, cuya solución se encuentra en el estado de la técnica. Una solución común consiste en añadir una caché en memoria delante de la base de datos de Spanner, lo que permitiría que el rendimiento del procesador de votos se ajustara a los requisitos. Al analizar los efectos secundarios, determinamos que las escrituras directas relativamente poco frecuentes desde la caché a Spanner cumplen con creces las garantías de rendimiento de Spanner. Dado que la sincronización global entre regiones no es sensible a la latencia, el retraso en la propagación de la actividad de los votantes entre regiones no supone un problema.

La última preocupación es la durabilidad. Dado que Memcached es un almacén de datos en memoria, utilizaremos Memorystore para Memcached, el producto de GCP , que proporciona clústeres de Memcached totalmente administrados en múltiples zonas para garantizar una alta disponibilidad.

La arquitectura de alto nivel con esta adición se muestra en la siguiente figura:



Cifra 8.5 Arquitectura de procesamiento de votos distribuidos con almacenamiento en caché

Describimos tres posibles desafíos que podrían surgir al implementar nuestra aplicación para nuestra audiencia global y abordamos cada uno con una estrategia diferente para comprender la aplicabilidad de cada técnica. En general, las prácticas arquitectónicas presentadas pueden emplearse para abordar cualquier escenario, siempre que se disponga de la infraestructura mínima necesaria.

Creación del paquete y la estructura de directorios del codelab

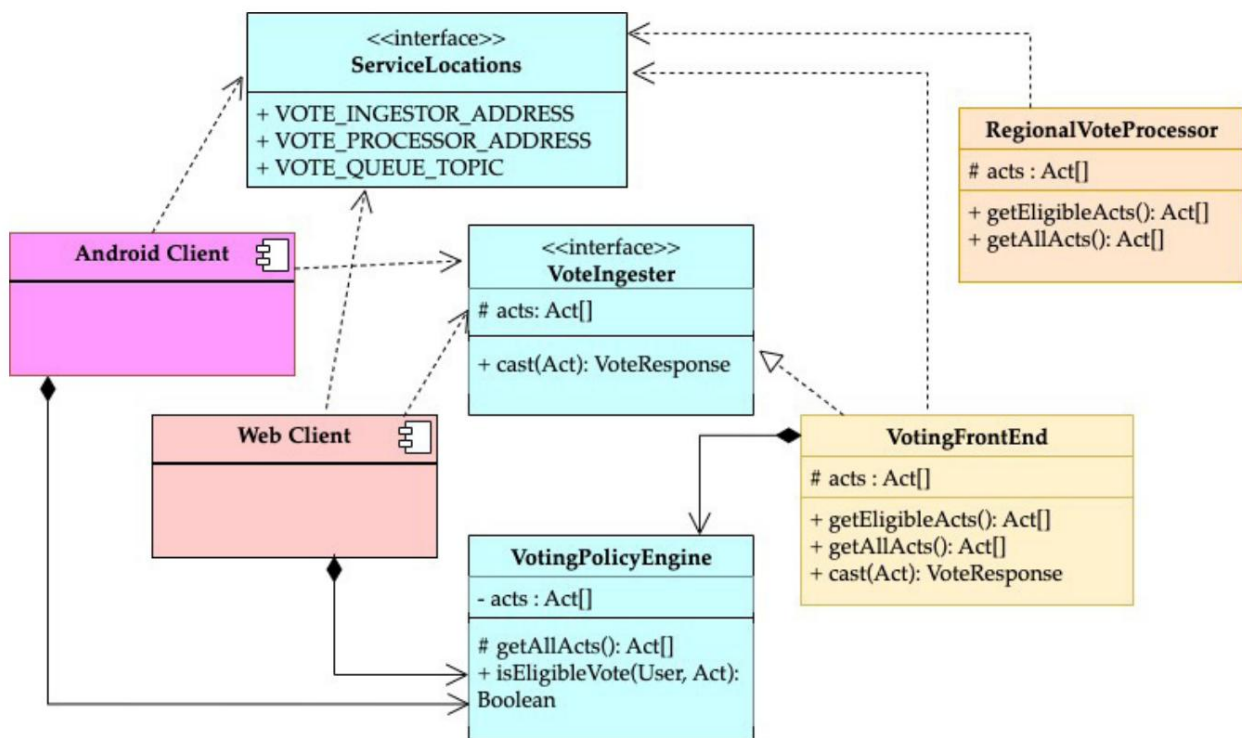
La aplicación de votación se ha descrito con respecto a la funcionalidad.

y nuestro plan de implementación distribuida. Esta sección del libro presentará laboratorios de código en cada capítulo, que ilustrarán partes de la implementación de la aplicación de votación. Estos laboratorios de código presentarán la configuración de GCP y el código del programa que implementa el tema de ese capítulo.

Este laboratorio de código establece la estructura del directorio y del paquete, permitiendo que el código del lado del cliente y del servidor hagan referencia entre sí y a definiciones declarativas y de bibliotecas comunes.

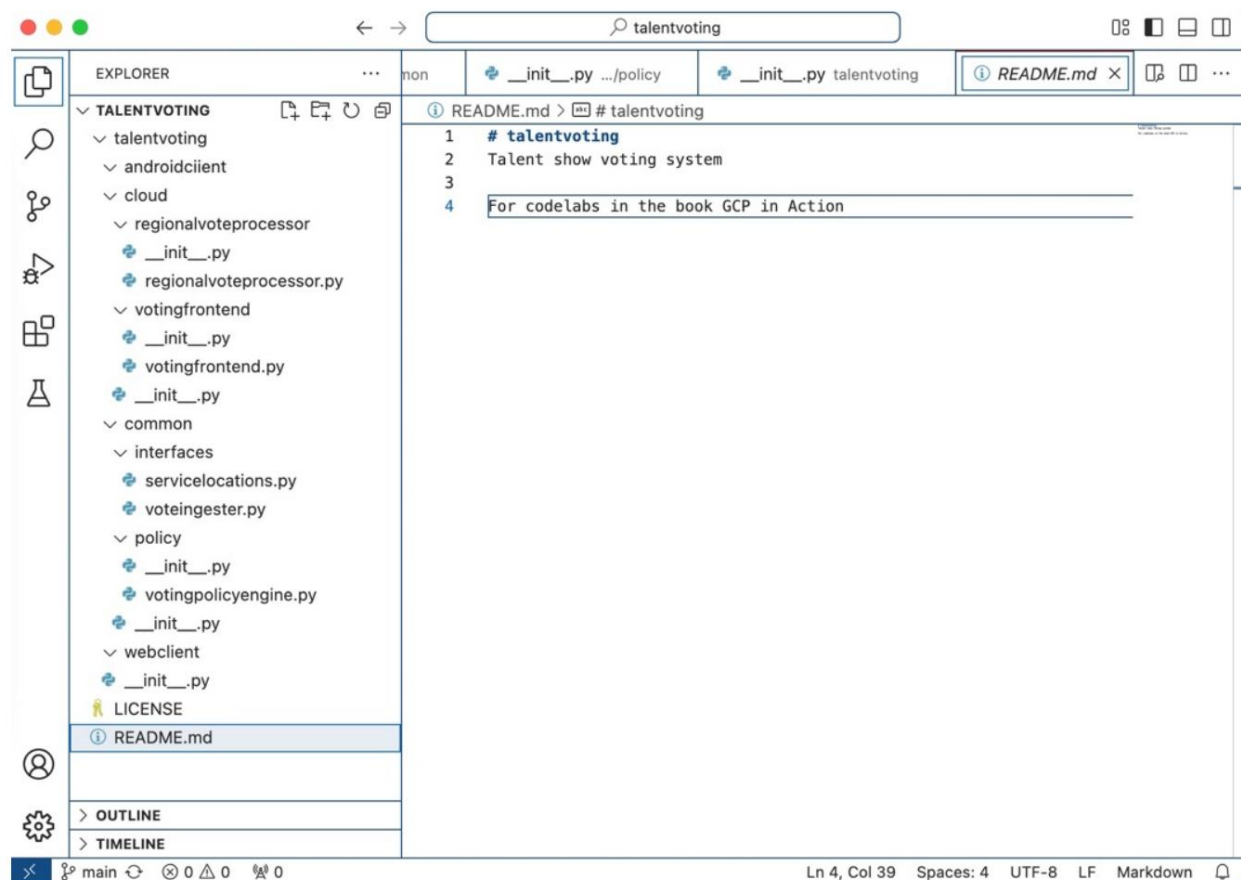
Componentes de la plataforma y archivos iniciales . Estos son los

componentes que se declararán en este laboratorio de código. La codificación por colores separa los componentes por plataforma, lo cual procuraremos reflejar en la jerarquía de paquetes de la aplicación, tal como lo admite nuestro lenguaje de programación, en este caso, Python 3.



Cifra 8.6 Paquetes de componentes y dependencias

Los laboratorios de código utilizarán Visual Studio Code para editar un repositorio de GitHub. El panel del Explorador del proyecto VS Code que muestra los archivos utilizados es el implementar Figura 8.7 siguiente:



Cifra 8.7: Explorador de VS Code para nuestros archivos de paquete principal

Los Codelabs de los próximos capítulos agregarán archivos e ilustrarán el código contenidas dentro de ellas.

Backends para frontends

El patrón de diseño de sistemas distribuidos backends para frontends fue descrito en el Capítulo Es 7, Frontends en la frontera del cliente.

Es apropiado considerar emplear esto en la frontera de un cliente.

Plataforma y servidor o componente en la nube para aislar no específicos del cliente código de varios clientes, concentrando las dependencias del cliente en

Tantas ubicaciones como sea posible.

Este patrón está incorporado en la interfaz VoteIngester que es implementado por el servidor VotingFrontEnd. Los distintos clientes plataformas, incluidas aquellas definidas dentro del cliente web y

Los paquetes de androidclient dependerán de la implementación remota de

Métodos VoteIngestor en lugar de implementar de forma independiente la lógica `cast()` real . `()`

API de votación.

La API de nuestro servicio de votación se define de la forma más abstracta posible, desvinculando a los clientes de esta API del proveedor del servicio. Esta es una medida común y económica para asegurar el futuro de nuestro sistema, limitando las dependencias estrictas de cualquier componente concreto al mínimo posible. En este caso, en lugar de que cada cliente codifique una dependencia estricta en nuestra implementación del frontend de votación, esta dependencia se ha concentrado en los archivos de la familia de paquetes `talentvoting.common` .

.

Conclusión

Nuestro sistema de votación se utiliza para seleccionar a un ganador entre un gran número de concursantes, basándose únicamente en el voto popular mayoritario. Este capítulo ilustró el proceso de análisis y documentación del uso de un sistema distribuido, desde el modelo de negocio/producto.

Se guió al lector en la planificación de la implementación de los componentes principales, en función de su afinidad regional con los usuarios. Se identificaron posibles deficiencias en los recursos de la plataforma regional y se exploraron diferentes técnicas para mitigarlas.

El primer laboratorio de código presentó la disposición de nuestra base de código de función de votación con una descripción del diseño de la API y lo relacionó con las mejores prácticas de diseño que se describieron anteriormente en el libro.

El siguiente capítulo identifica los puntos más estrechos del sistema de votación, donde el sistema es más susceptible a fallas bajo carga. Se describirán las formas establecidas de prevenir y, aún más importante, detectar y mitigar dichas fallas. Este es el objetivo de la ingeniería de confiabilidad del sitio (SRE), un componente crucial de cualquier cultura y organización de ingeniería.